

1

课程简介

理论课程



廈門大學
XIAMEN UNIVERSITY



信息学院 黃 煒
(特色化示范性软件学院) 博士, 副教授
School of Informatics Wei Huang

内容要点

- 课程简介
- 入门须知
- 课程要求
- 授课内容

目录

	1	课程简介
	2	入门须知
	3	课程要求
	4	授课内容
	5	总结

课程简介

课程中文名称

数据结构与算法

课程英文名称

Data Structures and Algorithms

课程类型

软件工程专业基础课，必修

课程简介

课程目的

培养学生形成从思路到程序的能力。

培养学生用数据结构和算法解决问题的能力。

培养学生对编程和软件工程专业的兴趣。

主讲教师

厦门大学信息学院软件工程系 黄炜 副教授

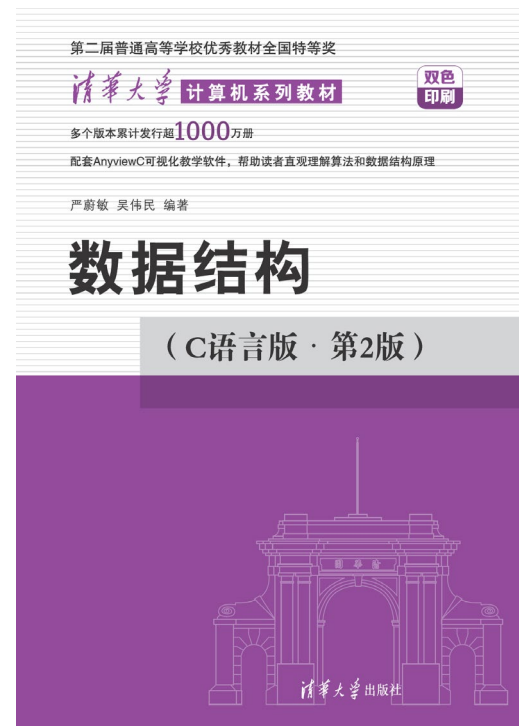
课程教材、参考书

官方教材

严蔚敏, 吴伟民, 数据结构 (C语言版), 清华大学出版社, 2021. (ISBN: 9000302001485)

课程类型

双语教学



课程教材、参考书

教学目的

- 熟练掌握数据结构的基本知识
- 掌握程序设计和实现的方法
- 掌握程序测试的方法
- 了解计算机相关专业术语

课程特色

阅读、测试

主讲教师介绍

- 黄炜，博士，厦门大学信息学院副教授
 - 2003年，厦门大学软件学院软件工程系，工学学士
 - 2007年，中国科学院大学，信息工程研究所，工学博士
 - 2013年，厦门大学，助理教授、副教授，硕士生导师
- 从事《C程序设计》教学已有10余年
- 研究方向：图像处理、人工智能、信息隐藏
- 联系方式
 - E-mail: whuang@xmu.edu.cn

课程定位

- 课程定位《数据结构与算法》是软件工程类专业的学科通修课程。也是研究生考试的必考科目。
 - 正规企业求职，或研究生复试，很可能有上机考试。
- 数据结构与算法不是一门语言，它是一种思维方式。
 - 讲授优化程序，使之运算更快、占用资源更少。
- 本课程以C语言为例，讲解数据结构。
 - 提供可以运行的代码（以课件为准）
- 硬件速度的提高，不是不重视算法性能的理由，而是我们追求高性能算法的动力。

应用需求

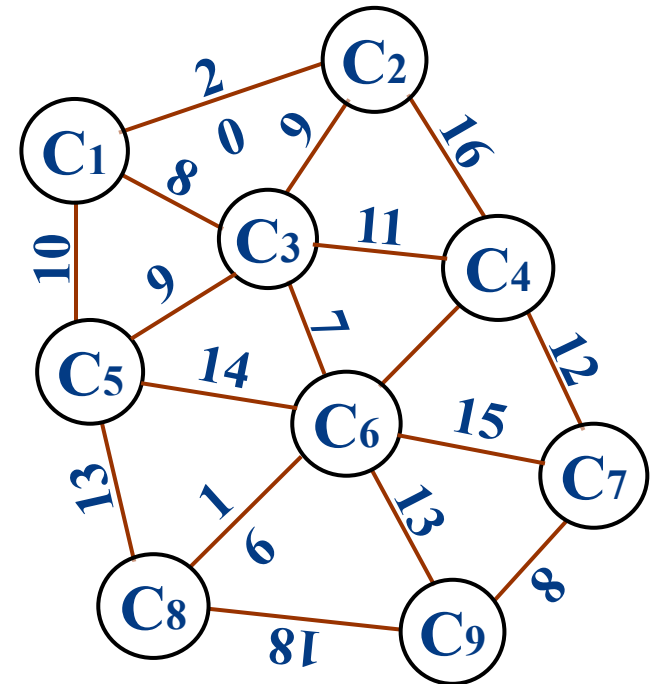
- 例1-7 最短路径问题

- 已知 n 个城市相互之间的距离。试找出从第1个城市出发到达第 j 个城市距离最短的路径。

- 例1-8 杨辉三角形

- 输出杨辉三角形($n \in [1, 12]$)。
- 如 $n=5$ 的输出结果：

1					
1	1				
1	2	1			
1	3	3	1		
1	4	6	4	1	
1	5	10	10	5	1



应用需求

第1种解答方法

```
Yanghui1(int n)
{ int a[N][N+1];
  printf("\n1\n\n1%6d\n\n", 1);
  for(i=1; i<n; ++i) {
    a[i-1][0]=a[i-1][i]=1;
    printf("1");
    for(j=1; j<=i; ++j) {
      a[i][j]=a[i-1][j-1]+a[i-1][j]; printf(a[i][j]);}
    printf("1\n");
  }
}
```

//时间复杂度 $O(n^2)$, 空间复杂度 $S(N^2)$

//第2种解答方法

```
Yanghui2(int n)
{ int a[2][N+1];
  printf("\n1\n\n1%6d\n\n", 1);
  for(i=1; i<n; ++i) {
    i0=(i-1)%2, i1=i%2;
    a[i0][0]=a[i0][i]=1;
    printf("1");
    for(j=1; j<=i; ++j)
    { a[i1][j]=a[i0][j-1]+a[i0][j];
      printf(a[i1][j]);}
    printf("1\n");
  }
}
```

// 时间复杂度 $O(n^2)$, 空间复杂度 $S(N)$

应用需求

//第3种解答方法

Yanghui3(int n)

```
{ printf("\n1\n\n1%6d\n\n", 1);
```

```
  for(i=2; i<=n; ++i) {
```

```
    c1=c2=1; printf("1");
```

```
    for(j=1; j<i; ++j)
```

```
    { c1*=(i-j+1); c2*=j;
```

```
    printf(c1/c2); }
```

```
    printf("1\n");
```

```
  }
```

```
} // 时间复杂度为 $O(n^2)$ , 空间复杂度为 $S(1)$ 
```

目录

	1	课程简介
	2	入门须知
	3	课程要求
	4	授课内容
	5	总结

阁下在此

迷茫



格物致知

幼儿时期

- 听爸爸妈妈的话

小学时期

- 爸爸妈妈说的不对
- 听老师的话

初中时期

- 爸爸妈妈和老师说的不对
- 模仿同学朋友

高中时期

- 爸爸妈妈老师和朋友说的不对
- 我是我，不一样的烟火

学习目标

- 早定目标

- 从事有意义的工作，个人价值、社会贡献
- 下一步继续深造攻读研究生？在IT企业求职？

- 规划路径

- 了解达到目标必须条件，提早努力
- 条件：GPA、学术竞赛获奖、研发项目、专业实习

- 随时调整

- 个人特质，周遭形势

各科的重要性

- 语文（表达）很重要
- 数学（算法）很重要
- 英语（语法）很重要
- 专业课很重要
- 其他课程也很重要（GPA）

程序设计与法律

- 民事

- 著作权纠纷；开发合同纠纷；泄密或竞业限制

- 刑事

- 第二百八十五条 非法侵入计算机信息系统罪
 - 第二百八十六条 破坏计算机信息系统罪
 - 第二百八十六条之一 拒不履行信息网络安全管理义务罪
 - 第二百八十七条 利用计算机实施犯罪的提示性规定
 - 第二百八十七条之一 非法利用信息网络罪
 - 第二百八十七条之二 帮助信息网络犯罪活动罪
 - 其它：共同犯罪：开设赌场罪

学习方法

- 如何学好数据结构与算法

- (1)掌握数据的抽象、组织和操作

- ✓ 表结构 ✓ 树结构 ✓ 图结构

- ✓ 查找 ✓ 排序

- (2)掌握数据在计算机中的表示、存储和算法实现。

- (3)通过典型应用，学会算法选择和性能优化，提高算法质量。

- (4)通过实验训练，提高计算思维和构造性思维能力，掌握问题的解决方法。

学习方法

- 数据结构与算法的学习过程，是进行程序设计的训练过程。学习数据结构，必须经过大量的实例训练，在实践中体会计算思维和构造性思维方法，掌握数据组织和程序设计技术。
 - 计算思维：是运用计算机科学的基础概念进行问题求解、系统设计、以及人类行为理解等涵盖计算机科学之广度的一系列思维活动。由美国CMU大学周以真教授于2006年3月首次提出。
 - 构造性思维：利用具体问题的典型特征，为待解问题设计一个合理的框架，从而使问题转化并得到解决。

如何学好这门课？

- 动手画图：遇到不懂的结构（比如二叉树遍历），别空想，拿出纸笔，画出来！
- 手写代码：别只看不练。核心算法（如链表增删、快排），合上书，自己手写一遍。
- 理解思想：别死记硬背代码。搞懂“为什么”要这么设计（比如为什么要有循环链表？），比记住代码更重要。
- 善用调试：程序跑不通？学会用调试器（Debugger）一步步看变量的变化，这是程序员的必备技能。

目录

	1	课程简介
	2	入门须知
	3	课程要求
	4	授课内容
	5	总结

课程要求

• 理论课

- 预习：提前阅读课件，调试程序，提出问题
- 课前：适当提前到课，着装得体，不迟到缺勤
- 课中：电子设备静音，不打闹，不做与课堂无关的事
- 课中：带上**笔记簿和笔**，做好笔记，下课凭笔记照片积分
- 课后：及时复习，按上课笔记重复演练，完成作业
- 课后：学有余力的同学保持刷题，规划算法分析学习

课程要求

- 实验课

- 纪律：禁止在机房用餐，不做与课堂无关的事
- 纪律：下课后电脑关机，桌椅和设备恢复原状
- 课前报名闭卷编程数据结构操作，优先坐中排，可以加分。
- 课中可以提问答疑
- 如果有NOIP基础可以提出，协助答疑

课程要求

- 出勤要求

- 旷课按次扣分

- 事前请假可通过QQ私信，事后请假应办理请假手续

- 按校规，理论课或实验课缺勤（含请假）达1/3者，记0分

- 课后要求

- 鼓励上机练习编程，不在练习出错，必在工作出错。

- 不要害怕软件错误，只要不是物理损坏，都可以恢复。

- 多动笔，多动脑，多思考，多提问。

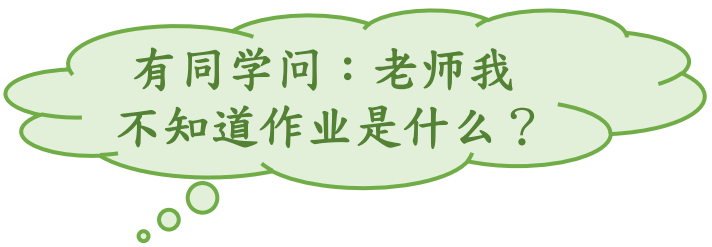
- 阅读很重要，只写不读，必然导致土话连篇。

- 国内外很多很好的代码供大家阅读学习。

作业要求

- 理论作业

- 课程提供习题集，每节课布置作业
- 自授课当日为第一日，第七日结束前提交



有同学问：老师我不知道作业是什么？

- 实验作业

- 在拼题 A 网（<https://pintia.cn/>），先注册，有八次实验
- 截止时间由实验老师确定

- 按时完成，认真作答，鼓励展开讨论

- 严禁作弊，不得抄袭或协助他人抄袭作业
- 合理使用AIGC工具，不要代劳主要工作（期末有上机考）

作业要求

- 作业的批改采取登记制
 - 正常提交，不乱写，少量错误，100分。
 - 认真写且补交：60分。
 - AI代写、乱写、缺交，一经发现，0分。
- 学有余力的同学：外网在线测评系统做题
 - 按NOIP的要求学习算法
 - 上OJ系统（洛谷、力扣等）寻找题目，先易后难
 - 参加ICPC等程序设计竞赛

期末成绩构成

- 期末笔试（ 40% ）
 - 统一闭卷笔试
- 平时成绩（ 60% ）
 - 期中考试（ 10% ）
 - 期末上机考试（ 20% ）
 - 理论作业（ 16% ）
 - 实验作业（ 24% ）
 - 考勤：倒扣分
 - 迟到5分钟以上扣0.5分，迟到23分钟以上扣1分

目录

	1	课程简介
	2	入门须知
	3	课程要求
	4	授课内容
	5	总结

研究数据表示

- 程序 = 数据结构 + 算法

例：位图，一般采用二维数组存储其灰度（亮度）信息。为什么？

- 数据结构：程序以何种形式来存储信息

- 示例：

- 题：某影评机构请你做一个程序，输入客户看过的电影列表，记录片名、（中间省略若干字）、您的评价等。

- 拟采用数据结构

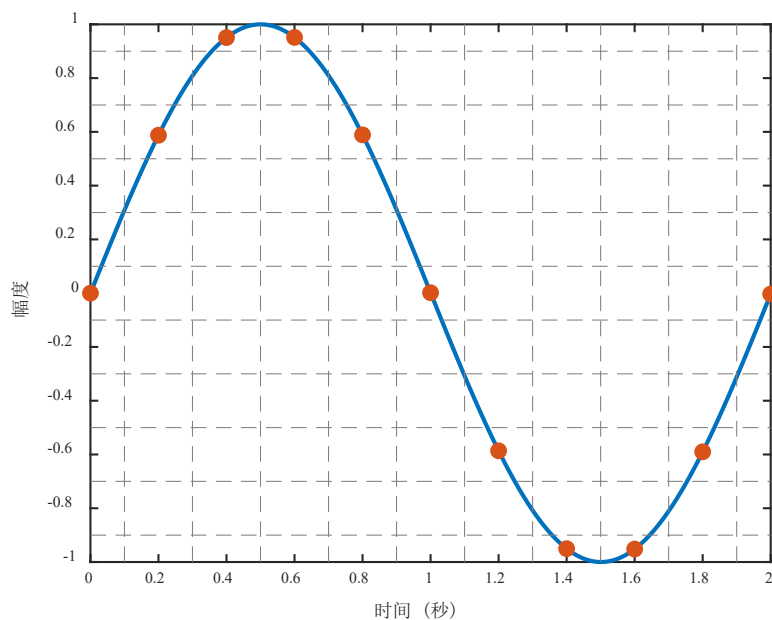
- 结构体：意义不同，数据类型不同，范畴相近

- 数组：意义相同，数据类型相同，仅有次序区别

基本概念

- 数据 (Data) : 万物皆可输入

- 将自然界的模拟信号转换成数字形式的过程称为数字化。
- 脉冲编码调制采样间隔 $125\mu\text{s}$ ，每个样本用0~255的整数表示



$(0,0), (0.2,0.6), (0.4,1), (0.6,1), (0.8,0.6), (1,0), (1.2,-0.6), (1.4,-1), (1.6,-1), (1.8,-0.6), (2,0)$

```
int A = [0, 3, 5, 5, 3, 0, -3, -5, -5, -3, 0];
```

基本概念

- **数据（Data）：万物皆可输入**
 - 数据是对客观事物的符号表示，是所有能够输入计算机并被计算机程序处理的符号的总称。
- **数据元素（Data Element）：打包处理的整体**
 - 数据元素是数据的基本单位。
 - 在计算机程序中通常作为一个整体进行考虑和处理。
 - 提到《走向共和》的时候，已经包括片名、导演、评分等影评整体。
- **数据对象（Data Object）：强调“同类元素的集合”**
 - 数据对象是性质相同的数据元素集合，是数据的一个子集。
 - 把张三、李四和王五等提交的影评记录打包成整体，称数据对象。

基本概念

- 具有相同性质的数据元素的集合。如
 - (1)英文字母数据对象
 - $C=\{ 'A', 'B', \dots, 'Z', 'a', 'b', \dots, 'z' \}$ 。
 - (2)自然数数据对象 $N=\{1, 2, \dots\}$ 。
 - (3)书目信息数据对象：包含书号, 书名, 作者, 出版社, 出版日期等数据项的数据元素集。

基本概念

- 利用数组解决问题的局限性：不可扩充
 - 虽然电影名总长度可调研，预先设定数组长度
 - 电影数量不可预设，用户也没法算一辈子会看几部电影
- 改用结构体链表解决问题，但仍存在局限性
 - 编码细节（链表增删）与概念模型（影评业务）混合了
 - 如果你想做一个电话簿管理软件，仅做小修改是不够的
 - 如果某天我发现了影评编码的错误，很难平移到电话簿

基本概念

- C语言中没有合适影评和电话簿的数据类型
 - 我们设计了一个结构体，再变成链表，来表示影评。
 - 我们设计了一个新结构体，再变成链表，来表示电话簿。
- 系统性的做法：定义类型（Type）
 - 数据类型（Data type）是一个值的集合和定义在这个值集上的一组操作的总称。
 - 操作是核心部分，没有操作的数据类型是没有什么用的
 - 设想：一个不能加减乘除的int类型，是没有用的
 - 数学提供了整数的抽象概念，C提供了概念的实现
 - 但是int类型并没有很好地实现整数，整数是无穷的

基本概念

- 定义一个新的类型的成功方法
 - 为类型的属性和可对类型执行的操作提供一个抽象的描述
 - 开发一个实现该ADT的编程接口，编写代码来实现接口
- 抽象数据类型（ Abstract Data Type ， ADT ）
 - ADT是指一个数学模型以及定义在该模型上的一组操作。
 - ADT定义取决于数据类型的数学（逻辑）特性，与在计算机内部表示和实现无关。
 - 抽象数据类型和数据类型实质上相同。

类型

如：int类型

属性集

整数值、正负

加减乘除模、比较.....

操作集

核心部分

基本概念

- 数据结构（ Data Structure ）
 - 数据结构研究数据元素之间的相互关系，以及定义在其上的一组基本操作。
- 数据结构主要包括三部分的内容
 - 逻辑结构：描述数据元素之间的逻辑关系
 - 如：线性结构？树形结构？圆形结构？
 - 存储结构：逻辑结构在计算机中的物理实现
 - 如：顺序存储（数组）？链式存储（链表）？索引？散列？
 - 运算集合：实现对数据元素及其逻辑关系的基本操作
 - 如：插入、删除、查找、输出等。

例题

例 1-2 “成绩表”的存储结构描述：

```
typedef struct
{   char No[14];    // 学号
    char Name[20];  // 姓名
    int Attend[2];   // 出勤
    int Homework[3]; // 平时成绩
    .....
} Elem; //数据类型描述
```

用顺序表描述逻辑关系 Elem Info[N];
还可以用链式存储方式描述逻辑关系。

例题

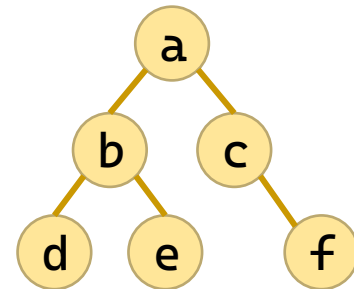
• 数据结构的形式定义： $\text{Data_Structure} = (D, R)$

– D : 数据对象， R : 定义在 D 上的关系集。

例1-3 数据结构 $DS=(D, R)$ ，其中， $D=\{a_1, a_2, a_3, a_4, a_5, a_6\}$ ；
 $R=\{<a_1, a_2>, <a_2, a_3>, <a_4, a_5>, <a_5, a_6>, <a_1, a_4>, <a_2, a_5>, <a_3, a_6>\}$ 。

a_1	a_2	a_3
a_4	a_5	a_6

例1-4 数据结构 $DS=(D, R)$ ，其中，
 $D=\{a, b, c, d, e, f\}$
 $R=\{<a, b>, <a, c>, <b, d>, <b, e>, <c, f>\}$



基本概念

- ADT 的两个重要特征：
 - 数据抽象性：用ADT描述程序的实体时，强调的是其本质特征、所能完成的功能，以及它和外部用户的接口。
 - 数据封装性：将实体外部特性和内部实现细节分离，并对外部用户隐藏内部实现细节。
- 抽象数据类型的形式定义是一个三元组
 - $ADT = (D, R, P)$
 - 其中，D是数据对象，R是D上的关系集，P是对D和R的基本操作集。

基本概念

- 抽象数据类型的定义格式：

ADT 抽象数据类型名 {

 数据对象：<数据对象的定义>

 数据关系：<数据关系的定义>

 基本操作：<基本操作的定义>

} ADT 抽象数据类型名

- 其中，数据对象和数据关系的定义一般采用伪码描述。

- 基本操作的定义格式如下：

- 基本操作名：(参数表)

- 初始条件：<描述操作执行之前，数据结构和参数应满足的条件>

- 操作结果：<说明操作正常结束之后，数据结构的变化情况和应返回的结果>

示例

例1-5 抽象数据类型复数的定义。

ADT Complex {

数据对象: $D = \{e_1, e_2 \mid e_1, e_2 \in \text{RealSet}\}$

数据关系: $R = \{r_1\}$

$r_1 = \{ \langle e_1, e_2 \rangle \mid e_1 \text{表示复数的实数部分, } e_2 \text{是复数的虚数部分} \}$

基本操作:

InitComplex (&Z, v1, v2)

操作结果: 构造复数Z, 实部和虚部分别被赋以参数v1和v2的值。

DestroyComplex (&Z)

操作结果: 销毁复数Z。

GetReal (Z, & zr)

初始条件: 复数已存在。

操作结果: 用zr返回复数Z的实部值。

GetImag (Z, & zi)

初始条件: 复数已存在。

操作结果: 用zi返回复数Z的虚部值。

Add (Z1, Z2, & sum)

初始条件: Z1, Z2是复数。

操作结果: 用sum返回Z1与Z2的和值。

} ADT Complex

基本概念

- 算法 (Algorithm)
 - 算法是求解特定问题的过程描述、操作步骤或指令序列。
- 数据结构和算法的关系
 - 好的数据结构可以提高算法性能；
 - 通过算法研究可以加深理解数据结构。
- 算法的5个重要特性
 - 有穷性 (Finiteness)：对于任意合法输入，一个算法必须总在执行有穷步之后结束，且每一步可在有穷时间内完成。
 - 算法必须是有穷的，而程序可以是无穷的

基本概念

- 算法的5个重要特性

- 确定性（Definiteness）：算法中每条指令必须有确切的含义，相同输入经过相同的执行路径，得到相同输出。
- 可行性（Effectiveness）：算法中描述的操作都可以通过已经实现的基本运算执行有限次来实现。
- 输入项（Input）：一个算法有零个或多个输入，这些输入取自于某个特定的对象的集合。
- 输出项（Output）：一个算法有一个或多个输出，这些输出是与输入有着某种特定关系的量。

基本概念

- 算法设计的要求

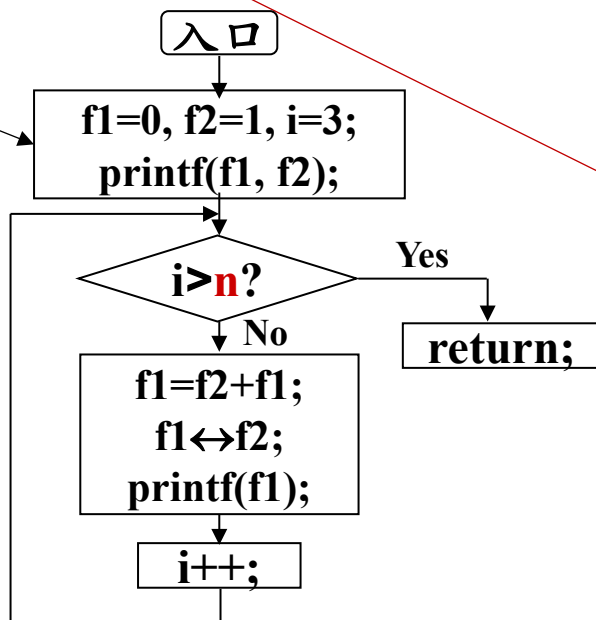
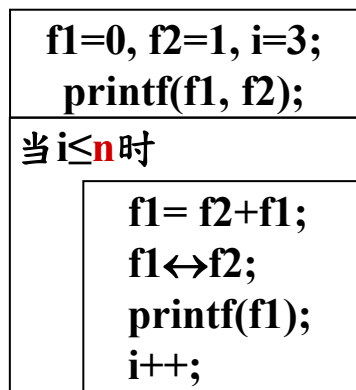
- 正确性：算法应能够正确地解决求解问题。
- 可读性：算法应具有良好的可读性，以帮助人们理解。
- 健壮性：输入非法数据时，算法能适当地做出反应或进行处理，而不会产生莫名其妙的输出结果。
- 高效率：效率是指算法的执行时间。力求设计一个高效率的解决特定问题的算法。
- 低存储：存储量是指算法执行过程中所需的最大存储空间。算法设计应考虑尽可能低的空间开销问题。

基本概念

• 算法描述方法

- 类C语言描述
- 数学方法描述
- 流程图
- N-S图描述
- 自然语言描述

```
Yanghui(int n) {  
    for (i = 2; i ≤ n; ++i) {  
        m1 = m2 = 1;  
        for (j = 1; j < i; ++j) {  
            m1 = m1 * (i - j + 1);  
            m2 = m2 * j;  
            printf(m1 / m2);  
        }  
    }  
}
```



- ① f1=0, f2=1, i=3 ;
- ② 输出 f1, f2 ;
- ③ if (i>n) return ;
- ④ f1=f2+f1 ;
- ⑤ 交换f1和f2的值 ;
- ⑥ 输出 f1 ;
- ⑦ i++ ;
- ⑧ go ③ ;

算法复杂度

- 算法复杂度是算法占用机器资源的量
 - 通常表示为，与问题规模 n 之间的关系
- 时间复杂度(Time Complexity) $T(n)=O(f(n))$
 - 算法运行所需要的执行时间度量。问题规模 n 之间的关系。
 - 最坏时间复杂度：最坏情况下
 - 平均时间复杂度：所有可能输入等概率情况下
 - 最好时间复杂度：最好情况下
- 空间复杂度(Space Complexity) $S(n)=O(f(n))$
 - 算法运行所需要的存储空间度量。

基本概念

- 时间复杂度的计算

- 语句频度

- 第2、7行1次
 - 第3行 $n+1$ 次
 - 第4、5行 n 次

```
1. void loveDSA(int n) {  
2.     int i = 1;  
3.     while (i <= n) {  
4.         i++;  
5.         printf("I love DSA %d\n", i);  
6.     }  
7.     printf("I love DSA more than %d\n", n);  
8. }
```

- 时间开销与问题规模 n 的关系： $T(n)=3n+3$

- 复杂的程序精确计算 $T(n)$ 是困难的

- 关注于数量级，可以简化问题 $T(n)=3n+3=O(n)$

- 用 $O(\cdot)$ 表示同阶，即：当 $n \rightarrow \infty$ 时，二者之比为常数

基本概念

- 规则

- 多项相加，只保留最高阶的项，且系数变为1

$$O(f(n)) + O(g(n)) = O(\max(f(n), g(n)))$$

- 多项相乘，都保留

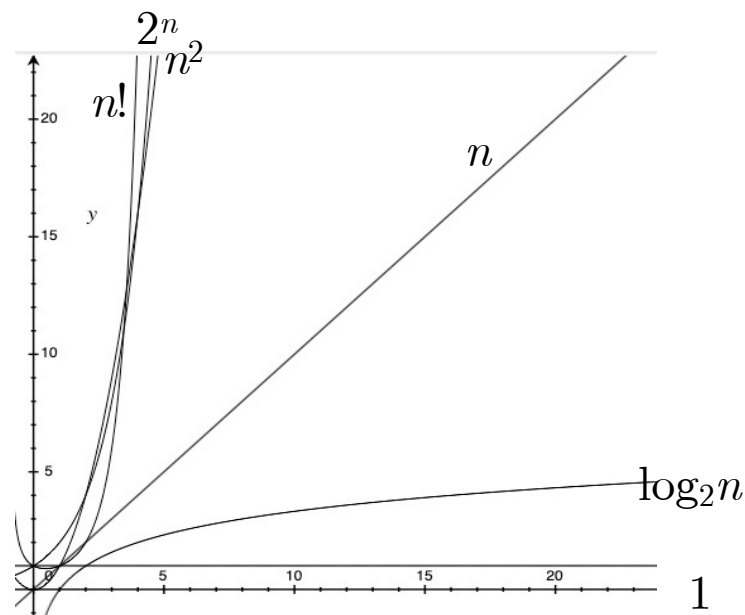
$$O(f(n)) \cdot O(g(n)) = O(f(n) \cdot g(n))$$

$$O(1) < O(\log_2 n) < O(n)$$

$$< O(n \log_2 n) < O(n^2) < O(n^3)$$

$$< O(2^n) < O(n!) < O(n^n)$$

算法的性能问题只有在 n 很大时才会暴露出来。



基本概念

- 空间复杂度的计算

- 无论问题规模怎么变，算法运行所需的内存空间都是固定的常量，算法空间复杂度为 $S(n) = O(1)$
- 多维数组带来的复杂度开销
- 递归带来的复杂度开销：与递归调用的深度相关

```
1. int Search(int A[], int n, int k) {  
2.     int i = n - 1;  
3.     while (i >= 0 && A[i] != k)  
4.         i--;  
5.     return i;  
6. }
```

基本概念

- 算法时间复杂度的估算方法

- 原操作：对于所研究问题的基本操作

- 它是算法中重复执行次数最多、且与问题规模 n 直接相关的那个操作。通常是循环体最深处的语句。

- 从算法中选取一种原操作，将该操作重复执行的次数作为算法时间复杂度的衡量准则。

- 时间复杂度与原操作的执行次数之和成正比。

- 例 `for (i=1; i<=n; ++i) s+=i;` $\Rightarrow T(n)=O(n)$

基本概念

例1-5 分析下列算法的时间复杂度。

```
F(int a[ ],int n)
{ for(i=0;i<n-1;++i)
  { j=i;
    for(k=i+1;k<n;++k )
      if(a[k]<a[j]) j=k; //原操作
      a[j]↔a[i] //交换a[j]和
a[i]
  }
}
```

算法的时间复杂度为 $O(n^2)$

```
F(int A[ ],int l,int r)
{ while(l<r)
  { while(A[r]≥0) --r;
    A[l]=A[r];
    while(A[l]≤0) ++l;
    A[r]=A[l];
  }
}
```

算法的时间复杂度为 $O(n)$

课程知识安排

2

线性表

3

栈和队列

4

串

5

数组和广义表

6

树和二叉树

7

图

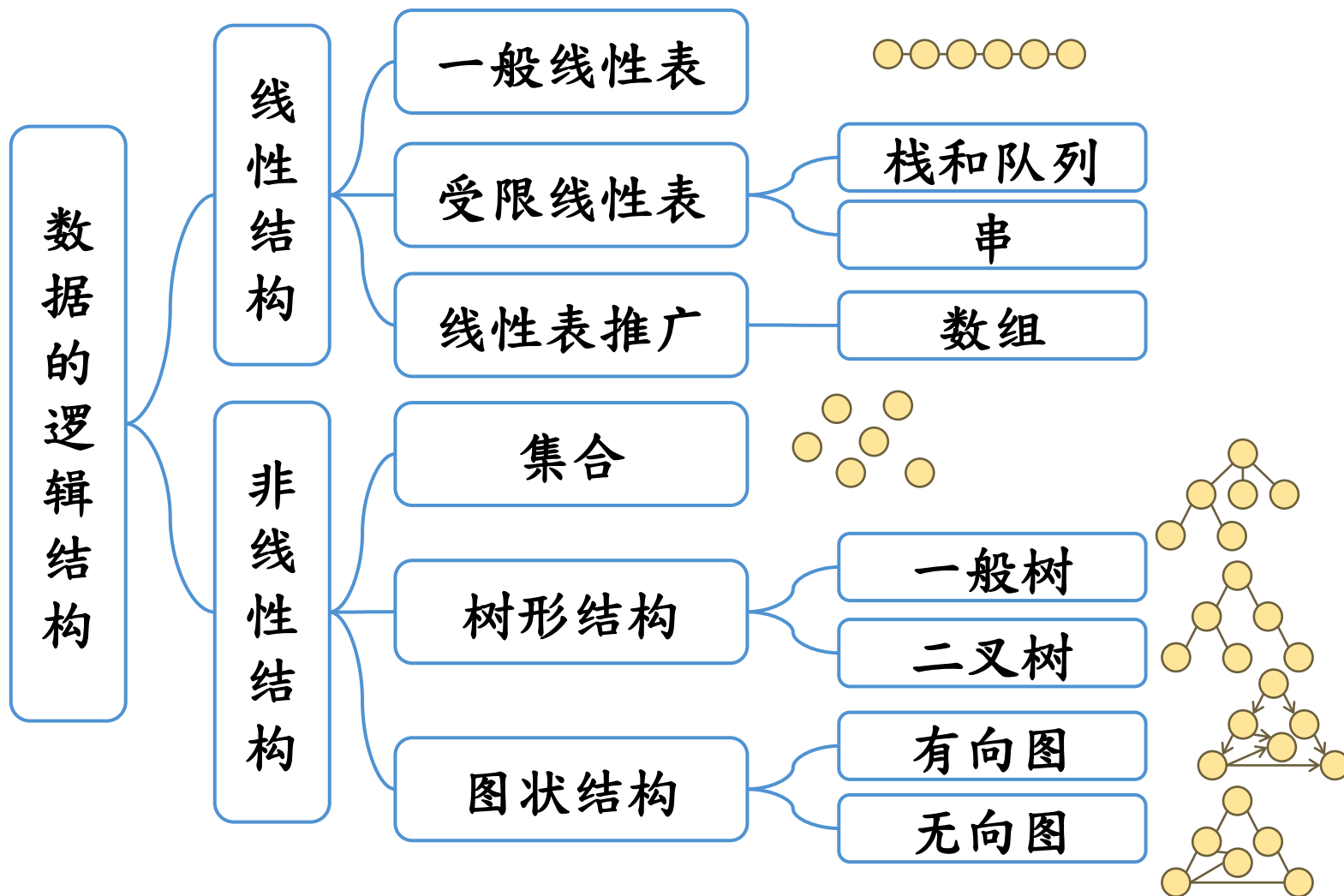
9

查找

10

内部排序

主要内容



主要内容

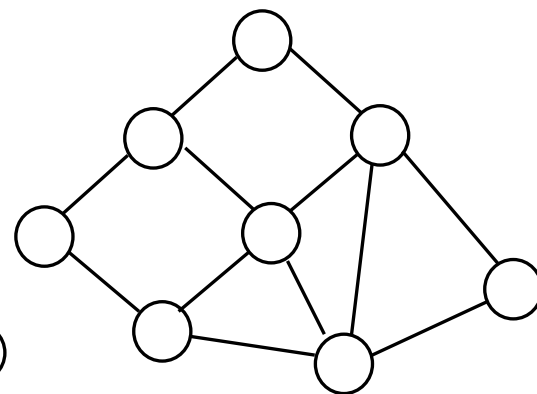
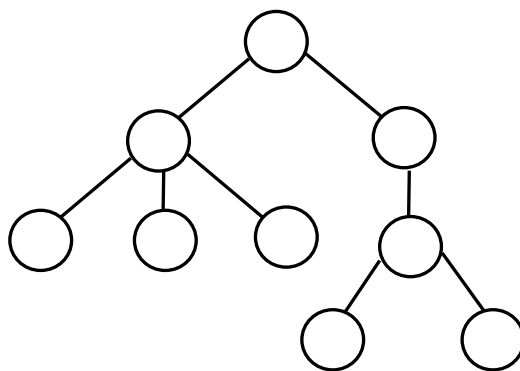
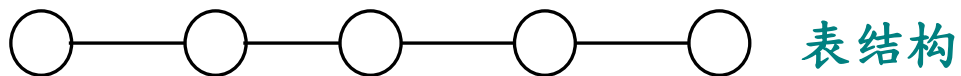
- 数据结构与算法学什么？

- 表结构

- 树结构

- 图结构

- 查找、排序



主要内容

- 线性表
 - 顺序表：线性表；三元组顺序表
 - 链表：循环链表；双向链表
- 栈；队列；循环队列
 - 迷宫问题；括号匹配检验；算术表达式求值；递归算法
- 树的基本概念；二叉树的性质和存储结构
 - 二叉树的遍历方法(先序、中序、后序)
 - 线索二叉树；哈夫曼树；哈夫曼编码
 - 树的存储结构；子集树；排列树

主要内容

- 图的基本概念、存储结构与遍历
 - 最小生成树；Prim算法；Kruskal算法
 - 最短路径；Dijkstra算法；Floyd算法
 - 有向无环图；拓扑排序；关键路径
- 查找
 - 静态查找表
 - 动态查找表：二叉排序树；平衡二叉树
 - 散列表基本概念；哈希函数；处理冲突的方法

主要内容

- 排序

- 简单排序：插入排序；冒泡排序；shell排序
- 归并排序；快速排序；堆排序；计数排序
- 树形选择排序；基数排序；桶排序

目录

	1	课程简介
	2	入门须知
	3	课程要求
	4	授课内容
	5	总结

内容要点

- 课程简介
- 入门须知
- 课程要求
- 授课内容

数据结构与算法

Data Structures

1

谢谢观看

理论课程



廈門大學
XIAMEN UNIVERSITY



信息学院 黃 煒
(特色化示范性软件学院) 博士, 副教授
School of Informatics Wei Huang

2

线性表

理论课程



廈門大學
XIAMEN UNIVERSITY



信息学院 黃 煒
(特色化示范性软件学院) 博士, 副教授
School of Informatics Wei Huang

内容要点

- 线性表的基本概念
- 顺序表
 - 初始化、创建、插入、删除
 - 遍历查找、遍历显示、获取数据
 - 合并
- 单链表
 - 初始化；创建：头插法、尾插法；插入、删除
 - 遍历查找、遍历显示、获取数据销毁、求差集

内容要点

- 循环链表
 - 约瑟夫环
- 双向链表
 - 插入、删除
- 链表的应用
 - 逆置
 - 倒数第 k 个元素

目录

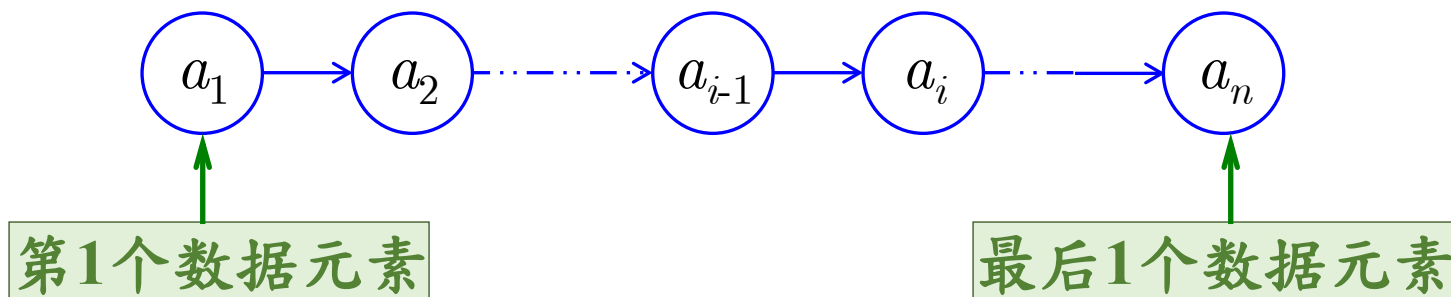
	1	基本概念
	2	顺序表
	3	单链表
	4	循环链表
	5	双向链表

线性结构

- 特征

- 有而且只有一个“第一元素”；
- 有而且只有一个“最后元素”；
- 除第一元素之外，其他元素都有唯一的直接前驱；
- 除最后元素之外，其他元素都有唯一的直接后继。

- 线性表是一种常用的简单的线性结构。



线性表

- 线性表 (Linear List)
- 定义：具有相同类型数据元素的有限序列

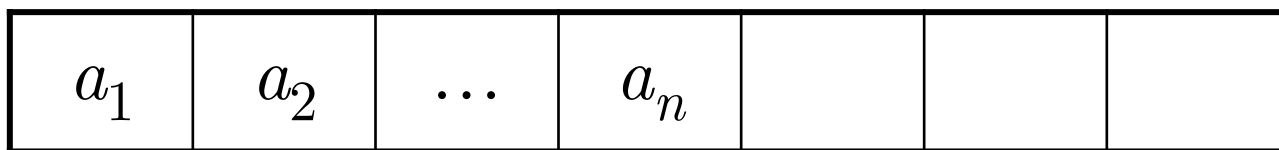
$$\mathbf{D} = \{ a_i \mid a_i \in \text{ElemSet}, \\ i=1, 2, \dots, n, n \geq 0 \}$$

$$(a_1, a_2, \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_n)$$

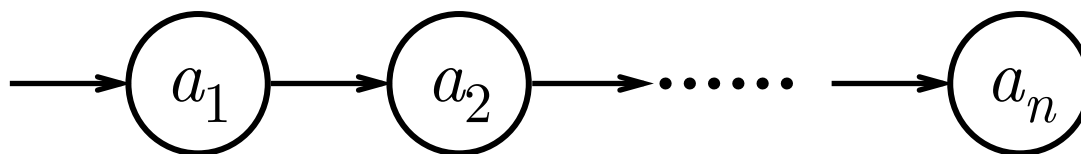
- 其中， i 是数据元素 a_i 在线性表 $\mathbf{D}$ 中的位序；
- n 是线性表的长度，即元素的个数；当 $n=0$ 时，称为空表。

线性表

- 顺序存储结构——由位置描述逻辑关系



- 链式存储结构——由指针描述逻辑关系



目录

	1	基本概念
	2	顺序表
	3	单链表
	4	循环链表
	5	双向链表

顺序表：存储结构

- 用一组地址连续的存储单元存储线性表的数据元素

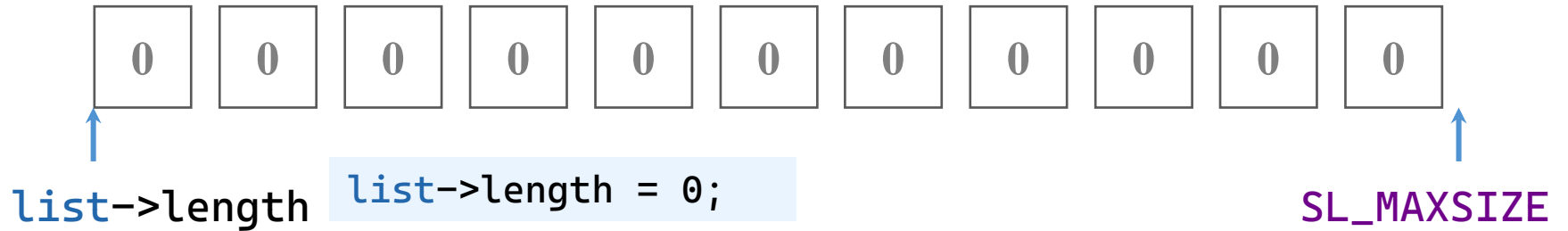
下标	0	1	...	$n-2$	$n-1$		
存储内容	a_1	a_2	...	a_{n-1}	a_n		
存储地址	k_0	k_1	...	k_{n-2}	k_{n-1}		

- 顺序表存储结构

```
#define SL_MAXSIZE 100
typedef struct SeqList {
    int data[SL_MAXSIZE];
    int length;
} SeqList;
```

顺序表：初始化

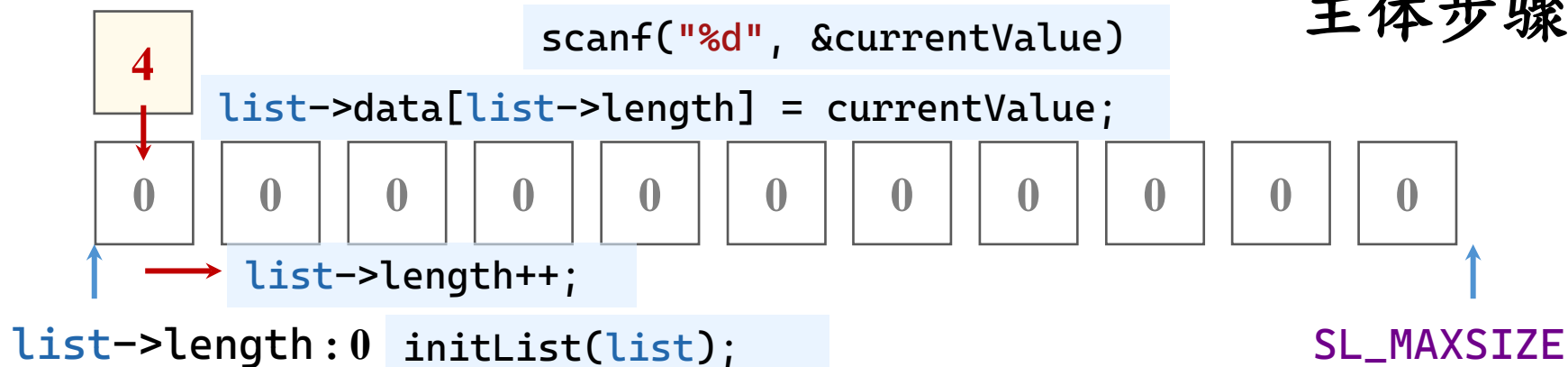
- 初始化：长度清零



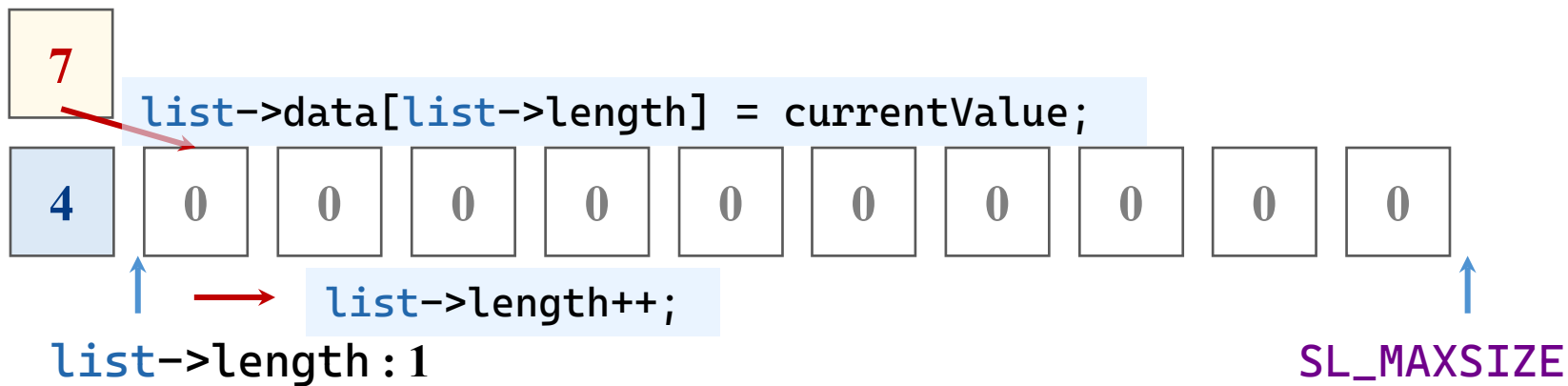
```
void initList(SeqList* list) {  
    if (list != NULL) {  
        list->length = 0;  
    }  
}
```

顺序表：创建

currentValue

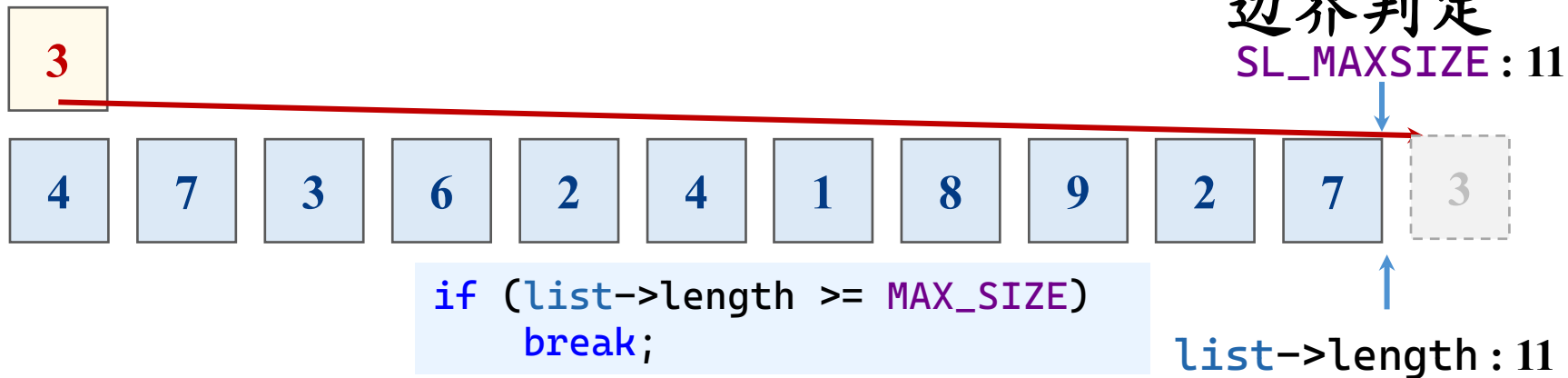


currentValue



顺序表：创建

currentValue



```
void createList(SeqList* list) {
    int currentValue = 0;
    initList(list);
    while (scanf("%d", &currentValue) == 1) {
        list->data[list->length] = currentValue;
        list->length++;
    }
}
```

顺序表：插入

• 顺序表的插入

– 将插入位置后的元素后移，再把新值赋值给插入位置

▪ 必须从尾部逆序，否则旧值会被覆盖，永久丢失

– 顺序表长度自增1。

```
for (i = list->length  
- 1; i >= index; i--)
```

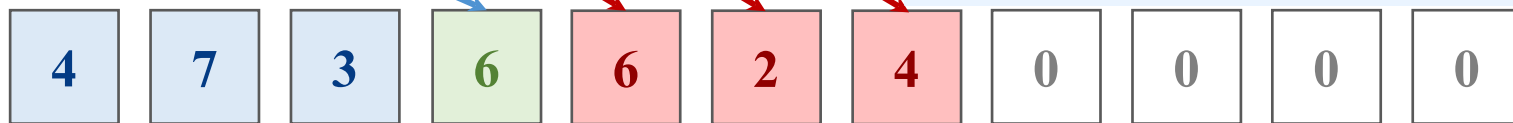
`list->length: 6`

`list->length++;`

主体步骤



`list->data[i+1] = list->data[i];`



data



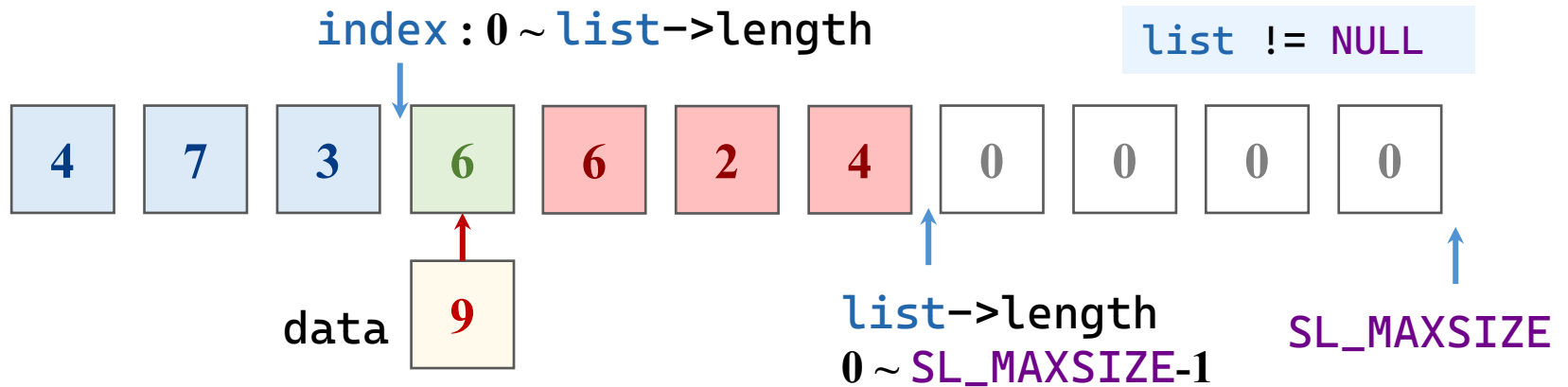
`list->data[index] = data;`

SL_MAXSIZE

顺序表：插入

- 顺序表的插入

边界判定



```
int insertList(const SeqList* list, const int index,
const int data) {
    int i = 0;
    if (list == NULL)
        return NULL_LIST;
    if (list->length >= MAX_SIZE)
        return ERROR;
    if (index < 0 || index > list->length)
        return NOT_FOUND;
    for (i = list->length - 1; i >= index; i--) {
        list->data[i + 1] = list->data[i];
    }
    list->data[index] = data;
    list->length++;
    return OK;
}
```

顺序表：删除

• 顺序表的删除

- 将删除位置的值赋给变量，删除位置后的元素前移
 - 必须从插入处顺序，否则旧值会被覆盖，永久丢失
- 顺序表长度自减1。

主体步骤

data

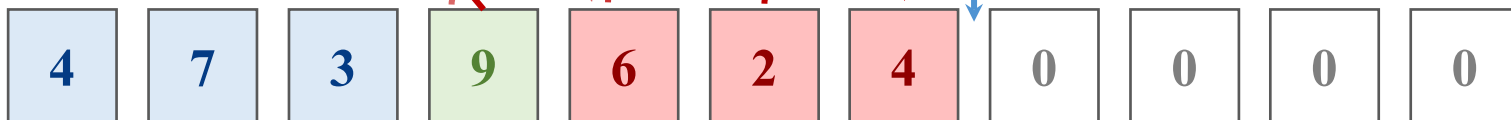
9

```
*data = list->data[index];
```

```
for (i = index; i < list->length - 1; i++)
```

list->length: 6

```
list->length--;
```



index: 3

```
list->data[i] = list->data[i+1];
```

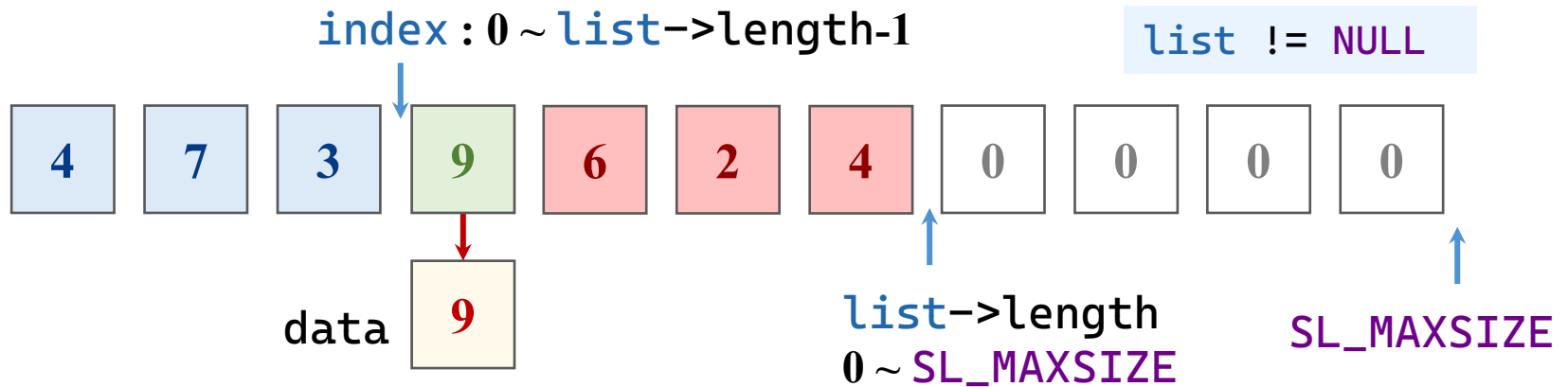


SL_MAXSIZE

顺序表：删除

- 顺序表的删除

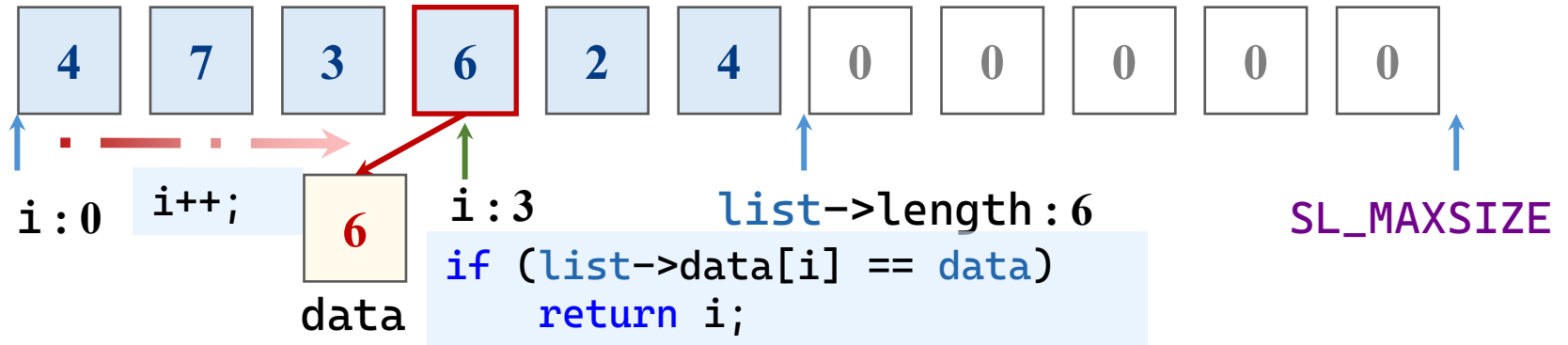
边界判定



```
int deleteList(SeqList* list, int index, int* data) {  
    int i = 0;  
    if (list == NULL || data == NULL)  
        return NULL_LIST;  
    if (list->length <= 0)  
        return ERROR;  
    if (index < 0 || index >= list->length)  
        return NOT_FOUND;  
  
    *data = list->data[index];  
    for (i = index; i < list->length - 1; i++)  
        list->data[i] = list->data[i + 1];  
    list->length--;  
    return OK;  
}
```

顺序表：遍历查找

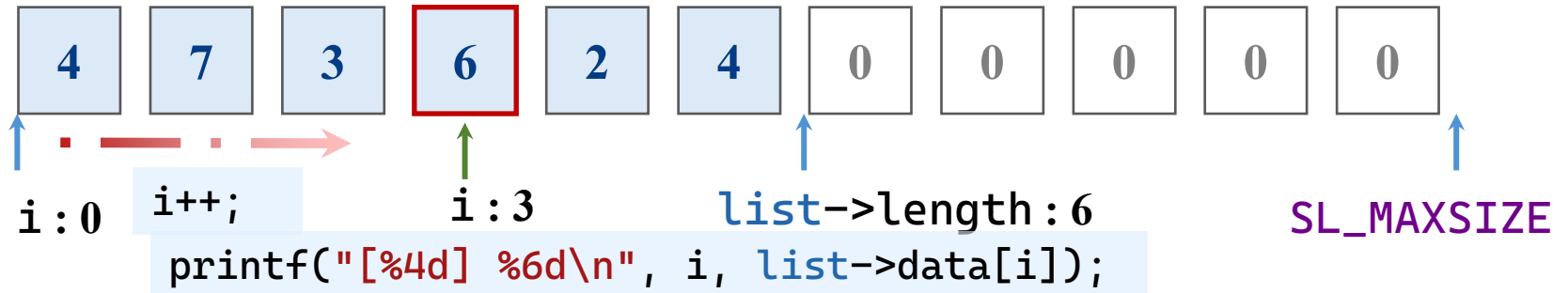
• 顺序表：遍历查找



```
int locate(const SeqList* list, int data) {
    int i = 0;
    if (list == NULL)
        return NOT_FOUND;
    for (i = 0; i < list->length; i++)
        if (list->data[i] == data)
            return i;
    return NOT_FOUND;
}
```


顺序表：遍历显示

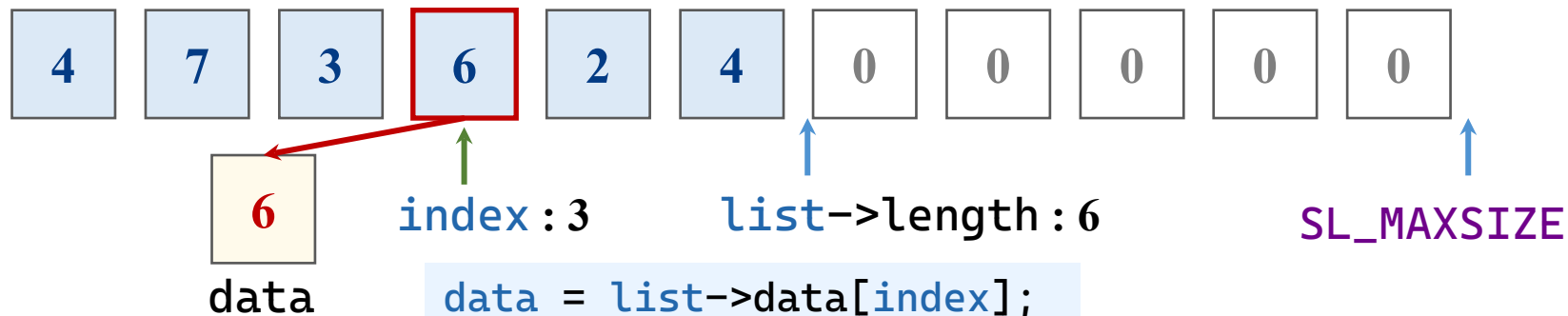
• 顺序表：遍历显示



```
int displayList(const SeqList* list) {  
    int i = 0;  
    if (list == NULL || list->length <= 0)  
        printf("这是个空表! \n");  
    else  
        for (i = 0; i < list->length; i++)  
            printf("[%4d] %6d\n", i, list->data[i]);  
    return OK;  
}
```

顺序表：获取数据

- 顺序表：根据索引获取数据



```
int getData(const SeqList* list, int index, int* data) {  
    if (list == NULL || data == NULL)  
        return NULL_LIST;  
    if (index < 0 || index >= list->length)  
        return NOT_FOUND;  
    *data = list->data[index];  
    return OK;  
}
```

顺序表：合并数据

- ## • 合并两个升序表

```
if (listA->data[i] <= listB->data[j]) {
}
```



顺序表：合并数据

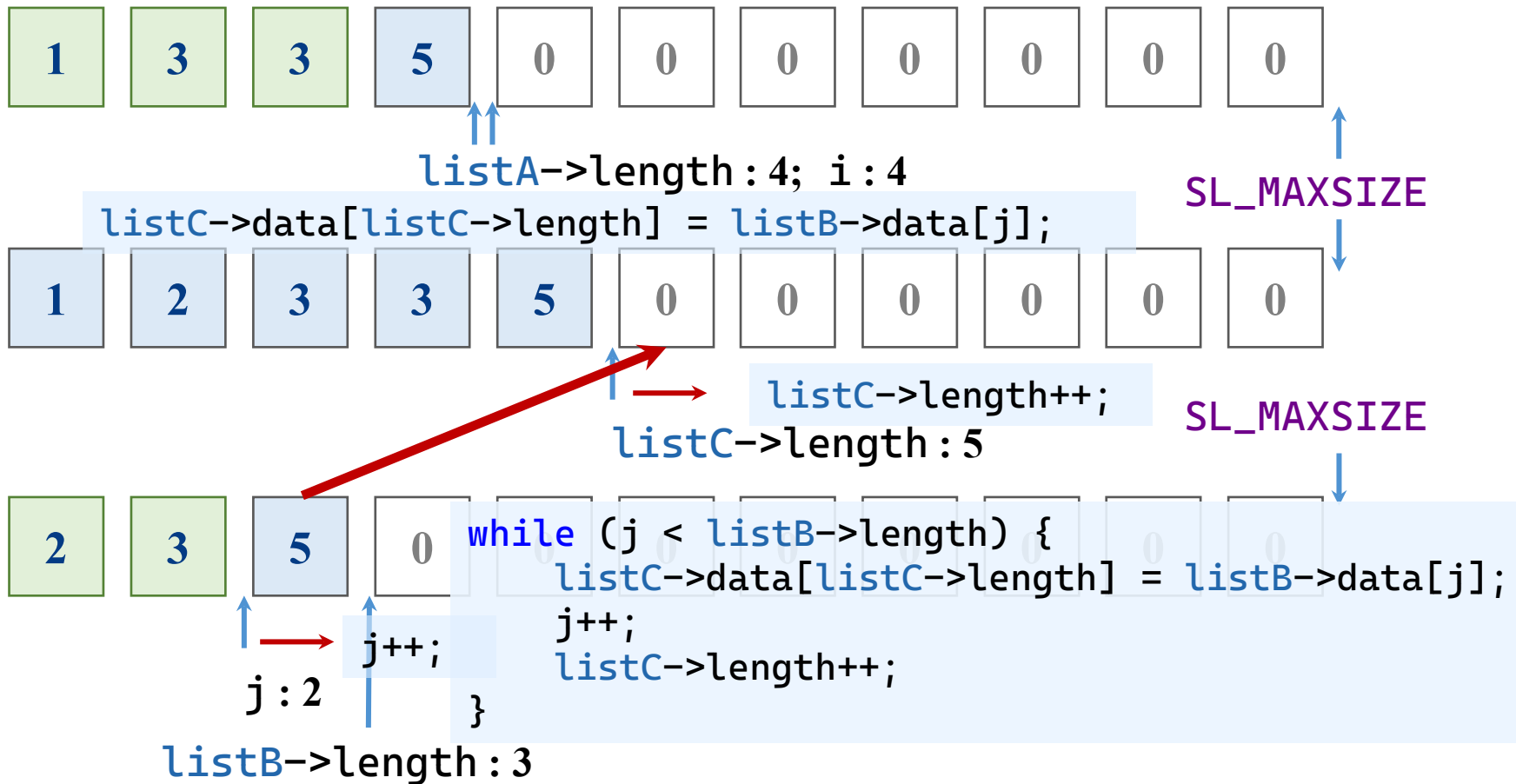
- 谁小就取谁

```
if (listA->data[i] <= listB->data[j])  
else {  
}
```



顺序表：合并数据

- 一方取完，没取完的一方继续取完



顺序表：合并数据

- 合并两个升序表
- 基本操作
 - 从头到尾遍历两个基础表，谁小就取谁
 - 取中的一方往前进，合并表也往前进
 - 没有取完的一方复制到合并表末尾
- 前后操作
 - 判断两个合并表长度之和是否越界
 - 升序排列两个基础表
 - 初始化合并表

```
int bubbleSort(SeqList* list) {  
    int i = 0;  
    int size = 0;  
    int temp = 0;  
    if (list == NULL)  
        return NULL_LIST;  
    for (size = list->length; size > 1; size--) {  
        for (i = 1; i < size; i++) {  
            if (list->data[i - 1] > list->data[i]) {  
                temp = list->data[i - 1];  
                list->data[i - 1] = list->data[i];  
                list->data[i] = temp;  
            }  
        }  
    }  
    return OK;  
}
```

```

int mergeList(SeqList* listA, SeqList* listB, SeqList* listC) {
    int i = 0;
    int j = 0;
    if (listA == NULL || listB == NULL || listC == NULL)
        return NULL_LIST;
    if (listA->length + listB->length > MAX_SIZE)
        return ERROR;
    bubbleSort(listA);
    bubbleSort(listB);
    initList(listC);
    while (i < listA->length && j < listB->length) {
        if (listA->data[i] <= listB->data[j]) {
            listC->data[listC->length] = listA->data[i];
            i++;
        } else {
            listC->data[listC->length] = listB->data[j];
            j++;
        }
        listC->length++;
    }
}

```



```
while (i < listA->length) {  
    listC->data[listC->length] = listA->data[i];  
    i++;  
    listC->length++;  
}  
  
while (j < listB->length) {  
    listC->data[listC->length] = listB->data[j];  
    j++;  
    listC->length++;  
}  
  
return OK;  
}
```

时间复杂度分析

- 冒泡排序法的时间复杂度 $O(n^2)$ 。
 - 两个线性表的时间复杂度 $O(m^2)$, $O(n^2)$
 - 用快速排序或归并排序，排序复杂度为 $O(m\log_2 m + n\log_2 n)$
- 顺序表合并
 - 这3个循环整体 i 和 j 是单调递增的，最多遍历 $m+n$ 次。
 - 时间复杂度 $O(m+n)$
- 总体时间复杂度
 - $T(m, n) = O(m^2) + O(n^2) + O(m+n) = O(m^2) + O(n^2)$

为数据结构书写测试程序

- 学完一个数据结构时，应书写测试程序，整理知识
- 书写一个菜单项程序
- 完善数据结构的各种操作
 - 先画图理解操作的过程
 - 再书写程序将图翻译为代码
 - 调试代码
- 优化输入输出，便于使用

```

#include <stdio.h>
#define MAX_SIZE 100
#define SL_MAXSIZE 100
#define OK 0
#define ERROR -1
#define NOT_FOUND -2
#define NULL_LIST -3
void clearInputLine(void) {
    int ch = 0;
    while ((ch = getchar()) != '\n' && ch != EOF) {
    }
}
int showMenu(void) {
    int choice = 0;
    printf("*****\n");
    printf("*          2.2 线性表的顺序存储          *\n");
    printf("*****\n");
    printf("    1.新建数据表\n");
    printf("    2.查找\n");
    printf("        21.按序号查找\n");
    printf("        22.按内容查找 [算法2.1]\n");
    printf("    3.插入 [算法2.2]\n");
    printf("    4.删除 [算法2.3]\n");
    printf("    5.合并 [算法2.4]\n");
    printf("    6.显示内容\n");
    printf("    7.退出\n");
}

```

```

    printf("=====\\n");
    printf("请选择: ");
    scanf("%d", &choice);
    return choice;
}
int main(void) {
    int ch = 0;
    int i = 0;
    int e = 0;
    SeqList listA, listB, listC;
    initList(&listA);
    initList(&listB);
    initList(&listC);
    while (ch != 7) {
        ch = showMenu();
        switch (ch) {
            case 1:
                createList(&listA);
                break;
            case 21:
                printf("请输入要查找的序号: ");
                scanf("%d", &i);
                if (getData(&listA, i, &e) == OK)
                    printf("第%d个元素为%d。\\n", i, e);
                else
                    printf("第%d个元素不存在。\\n", i);
        }
    }
}

```

```

        break;
    case 22:
        printf("输入要查找的元素值: ");
        scanf("%d", &e);
        i = locate(&listA, e);
        if (i != NOT_FOUND)
            printf("值为%d的元素第1次出现位置为%d。\\n", e, i);
        else
            printf("值为%d的元素并未在顺序表中出现\\n", e);
        break;
    case 3:
        printf("输入要插入的元素的值: ");
        scanf("%d", &e);
        printf("输入要插入的元素的位置: ");
        scanf("%d", &i);
        if (insertList(&listA, i, e) == OK)
            printf("插入成功! \\n");
        else
            printf("插入失败! 表已满或者插入位置错误。\\n");
        break;
    case 4:
        printf("输入要删除的元素的位置: ");
        scanf("%d", &i);
        if (deleteList(&listA, i, &e) == OK)
            printf("删除成功! 删去的值为%d。\\n", e);
        else

```

```

        printf("删除失败！表为空或者删除位置错误。\\n");
    break;
case 5:
    printf("主表为原表，现在输入副表：\\n");
    createList(&listB);
    initList(&listC);
    if (mergeList(&listA, &listB, &listC) == OK) {
        printf("主表：\\n");
        displayList(&listA);
        printf("副表：\\n");
        displayList(&listB);
        printf("合并后的表：\\n");
        displayList(&listC);
    } else {
        printf("合并失败！表长超出上限或者输入有误。\\n");
    }
    break;
case 6:
    printf("主表：\\n");
    displayList(&listA);
    break;
case 7:
    printf("结束程序。\\n");
    break;
default:
    printf("输入错误！请重新输入！\\n\\n");

```

```
        break;
    }

    if (ch != 7) {
        printf("单击空格键继续...\n");
        clearInputLine();
    }
}

clearInputLine();
return 0;
}
```


顺序表的主要特点

- 存储空间利用率高：只存储元素值。
- 随机存取
 - 可以通过计算来确定顺序表中存储地址
 - 第 i 个数据元素的地址是： $L_i = L_0 + (i - 1) \cdot m$
 - 其中， L_0 为第一个数据元素的存储地址， m 为每个数据元素所占用的存储单元数。
- 插入和删除数据元素会引起大量结点移动。

目录

	2	顺序表
	3	单链表
	4	循环链表
	5	双向链表
	6	链表的应用

链表

- 链表是一种能够动态进行存储分配的数据结构。当需要插入数据元素时，申请一个结点的地址单元，并将该单元链入线性表中。
- 定义
 - 链表：以结点序列表示的线性表。
 - 结点：一个数据元素在内存中的映象（由连续的若干字节组成，该区域正好可以存储一个数据元素），包括数据域和指针域。
 - 数据域：存储数据元素的内容。
 - 指针域：存储后继元素的首地址。

链表

- 定义

- 线性链表（单链表）：链表的每个结点只包含一个指针域。
- 头结点：附加在第一个数据元素之前的结点，该结点的数据域一般为“空”、指针域存放第一个数据元素的地址。
- 头指针：线性链表中第一个结点或头结点的存储地址，它是访问链表的起始点。
- 动态链表：采用动态地址分配方法建立的线性链表（指针型描述）。

链表

- 线性链表操作要点：

- 访问链表，只能从头指针开始，沿着指针*next的指向进行访问(不能逆向)。
- 头指针不能丢——丢了，链表就没了。
- 在断开链表之前，注意保存后段链表的“头指针”，否则后段链表将丢失。

链表：数据结构

- 结点的存储结构

- 数据（data是数据域的示例）

- 下一结点指针 `p->next = NULL;`

```
typedef struct Node {  
    int data;  
    struct Node* next;  
} Node;  
typedef Node* LinkList;
```

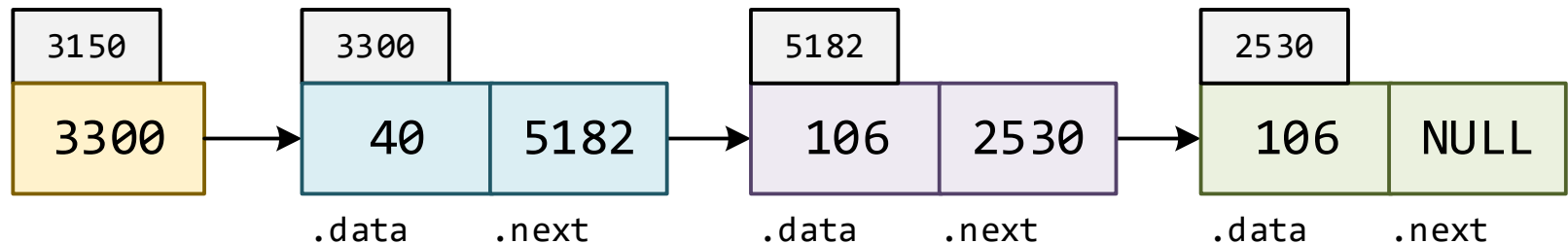
- 链表是节点的指针

- 链表头：指向第一个结点的指针

```
LinkList listA;
```

- 链表中：next成员指向下一个结点

- 链表尾：next成员置为空



链表：创建结点并链接到另一结点

- 新建结点

```
node *p = (node*)malloc(sizeof(node));
```

- 赋初始值

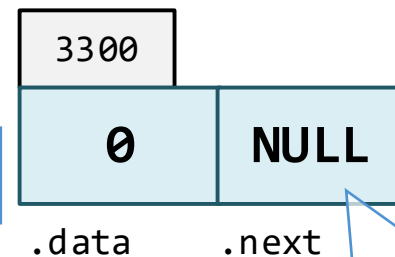
- next初始值一般赋值为NULL

- 成员初始值一般按惯例赋值

- 形成前后顺序

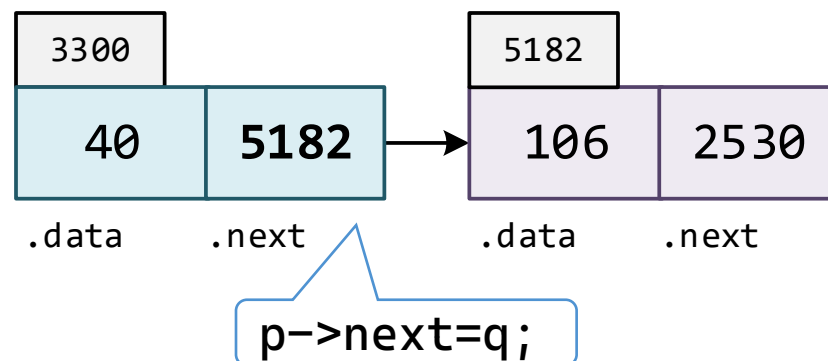
- next赋值为另一结点的地址

```
p->data = 40;  
p->next = q;
```



```
p->next=NULL;
```

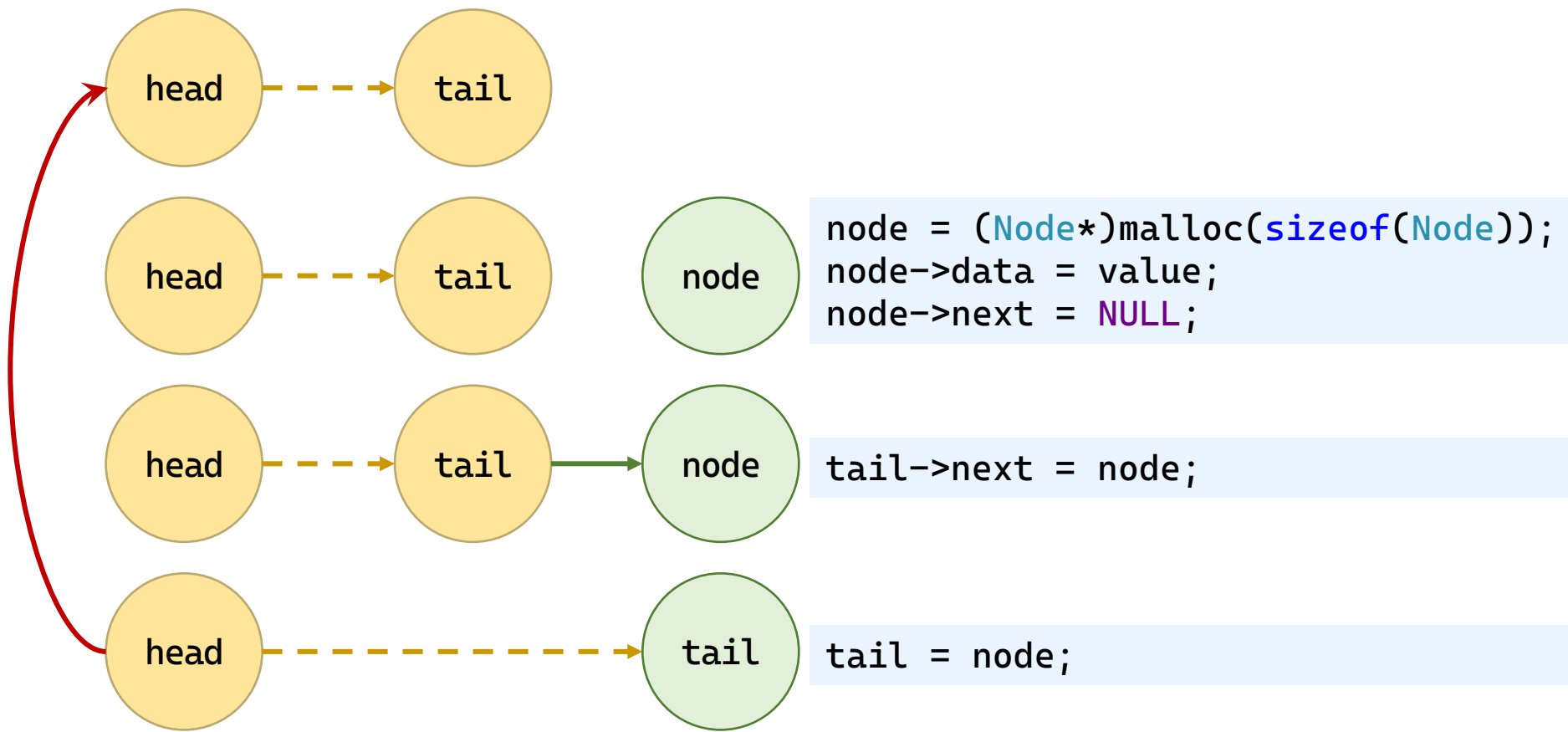
```
p->data = 0;  
p->next = NULL;
```



创建列表：尾插法

• 图示

主体步骤



链表：创建列表（尾插法）

- 初步代码

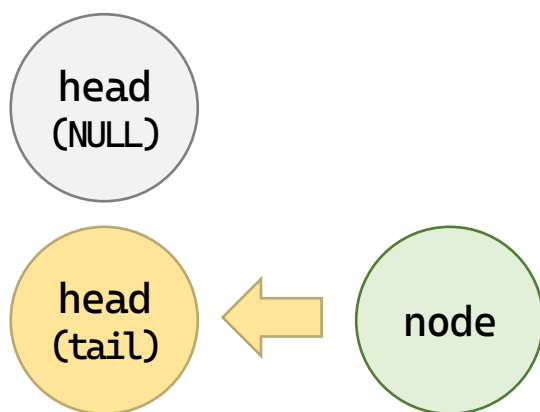
```
LinkedList create_from_tail(void) {  
    LinkedList head = NULL;  
    Node* tail = NULL, * node;  
    int value;  
    printf("请输入元素，输入非数字结束：\n");  
    while (scanf("%d", &value) == 1) {  
        node = (Node*)malloc(sizeof(Node));  
        node->data = value;  
        node->next = NULL;  
        tail->next = node;  
        tail = node;  
    }  
    return head;  
}
```

链表：创建列表（尾插法）

• 图示：链表还没建立的时候

特殊处理

1. head 为 NULL 时



2. node 生成失败时，报错、销毁

```
if (node == NULL) {  
    printf("内存分配失败。\\n");  
    free_list(&head);  
    clear_input_line();  
    return NULL;  
}
```

```
if (head == NULL) {  
    head = tail = node;  
} else {  
    tail->next = node;  
    tail = node;  
}
```

```

LinkedList create_from_tail(void) {
    LinkedList head = NULL;
    Node* tail = NULL, * node;
    int value;
    printf("请输入元素，输入非数字结束：\n");
    while (scanf("%d", &value) == 1) {
        node = (Node*)malloc(sizeof(Node));
        if (node == NULL) {
            printf("内存分配失败。\\n");
            free_list(&head);
            clear_input_line();
            return NULL;
        }
        node->data = value;
        node->next = NULL;
        if (head == NULL) {
            head = tail = node;
        } else {
            tail->next = node;
            tail = node;
        }
    }
    clear_input_line();
    return head;
}

```

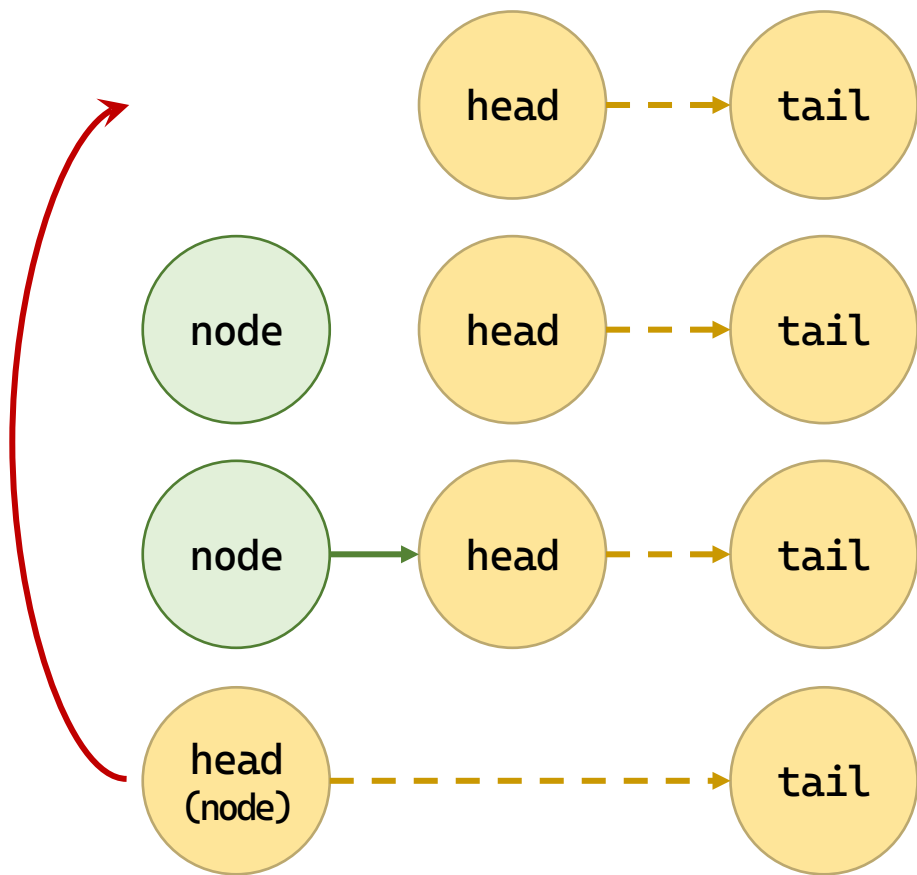
链表：创建列表（头插法）

• 图示

1. head为NULL无例外

2. node 生成失败时，报错、销毁

主体步骤



```
node = (Node*)malloc(sizeof(Node));  
node->data = value;
```

```
node->next = head;
```

```
head = node;
```

```
LinkedList create_from_head(void) {  
    LinkedList head = NULL;  
    Node* node;  
    int value;  
    printf("请输入元素, 输入非数字结束: \n");  
    while (scanf("%d", &value) == 1) {  
        node = (Node*)malloc(sizeof(Node));  
        if (node == NULL) {  
            printf("内存分配失败. \n");  
            free_list(&head);  
            clear_input_line();  
            return NULL;  
        }  
        node->data = value;  
        node->next = head;  
        head = node;  
    }  
    clear_input_line();  
    return head;  
}
```

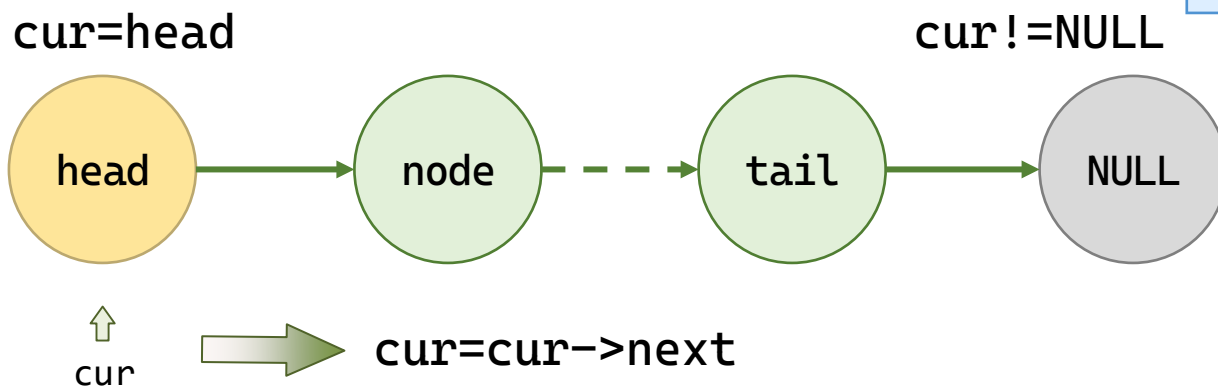
链表：遍历链表

- 按顺序遍历数组和链表比较

功能	数组	链表
定位到第一个元素	<code>i=0</code>	<code>cur=head</code>
判断最后一个元素	<code>i<N</code>	<code>cur!=NULL</code>
移动到下一个元素	<code>i++</code>	<code>cur=cur->next</code>

```
i = 0;
while (i < N) {
    ...;
    i++;
}
```

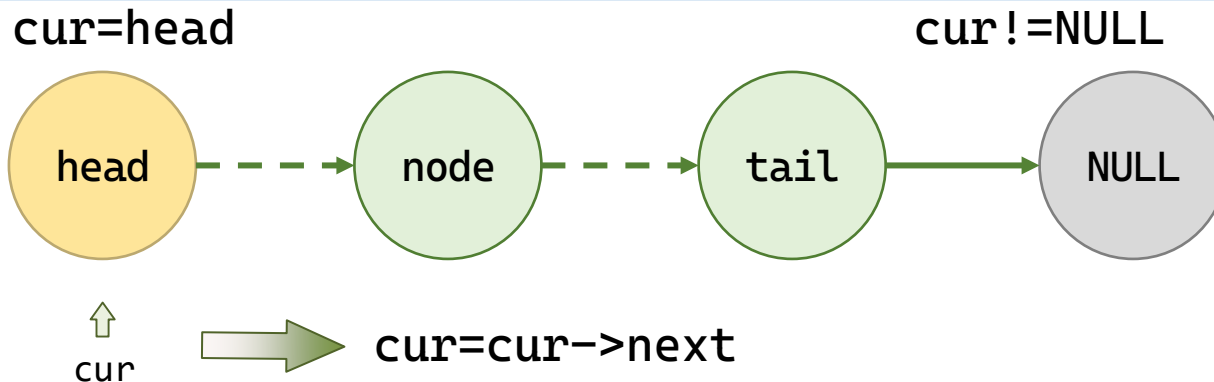
```
cur = head;
while (cur != NULL) {
    ...;
    cur = cur->next;
}
```



链表：计算元素个数

• 示例代码

```
int list_length(LinkList head) {  
    int length = 0;  
    Node* current = head;  
    while (current != NULL) {  
        length++;  
        current = current->next;  
    }  
    return length;  
}
```

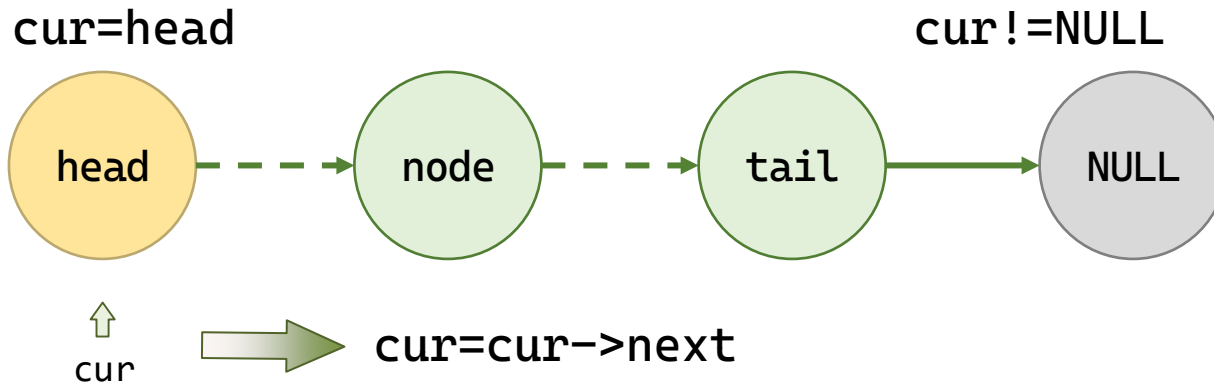


链表：定位元素

• 示例代码

```
Node* get_node(LinkList head, int index) {  
    int i;  
    Node* current = head;  
    for (i = 0; current != NULL && i < index; i++)  
        current = current->next;  
    return current;  
}
```

点数，数到了就停止



链表：查找元素

• 示例代码

```
Node* locate_node(LinkList head, int value, int* index) {  
    int i;  
    Node* current = head;  
    i = 0;  
    while (current != NULL && current->data != value) {  
        current = current->next;  
        i++;  
    }  
    if (index != NULL) {  
        *index = i;  
    }  
    return current;  
}
```

先有cur不为空才有cur->data

停止条件不是点数，而是找到了

如果未找到不应有副作用，
此时cur是NULL，不需要特殊处理

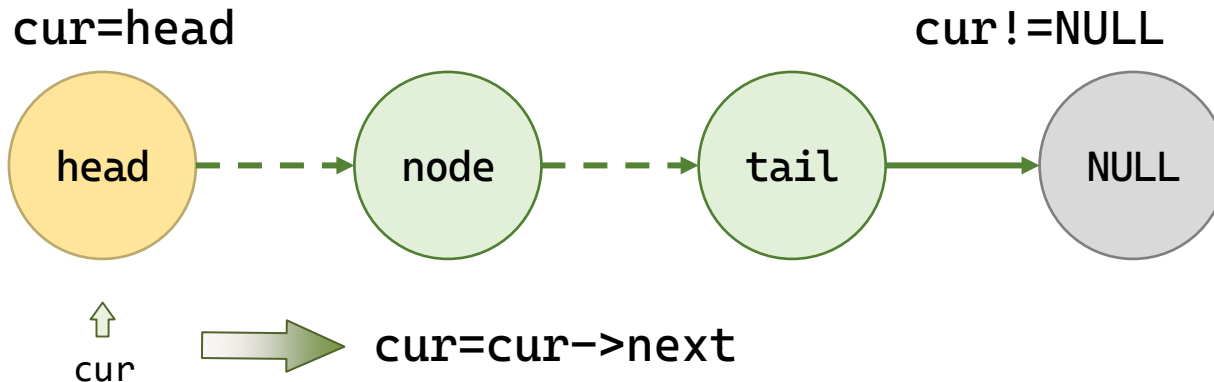
链表：显示链表

- 示例代码

- 遍历列表
- 输出内容

```
Node* current = head;  
while (current != NULL) {  
    current = current->next;  
}
```

```
printf("%4zu\t%8d\t%016p\t%016p\n", ++i, current->data,  
      (void*)current->next, (void*)current);
```



```

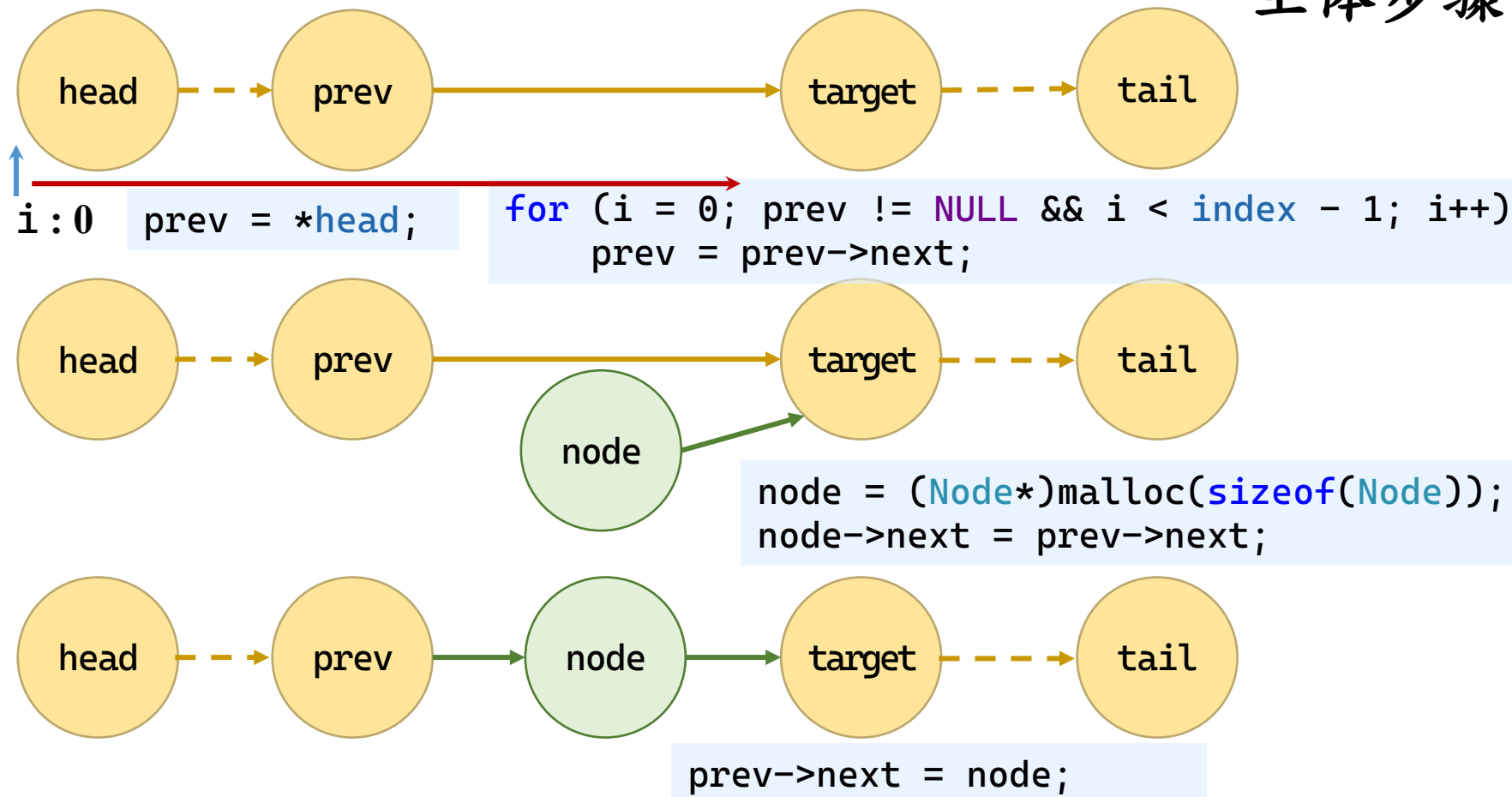
int display_list(LinkList head) {
    Node* current = head;
    size_t i = 0;
    if (current == NULL) {
        fputs("这是个空表.\n", stderr);
        return OK;
    }
    printf("%-6s\t%-8s\t%-16s\t%-16s\n", "ID", "数据", "后续", "
    地址");
    printf("=====\t=====\t=====\t=====\n");
    while (current != NULL) {
        printf("%4zu\t%8d\t%016p\t%016p\n", ++i, current->data,
            (void*)current->next, (void*)current);
        current = current->next;
    }
    return OK;
}

```

链表：插入元素

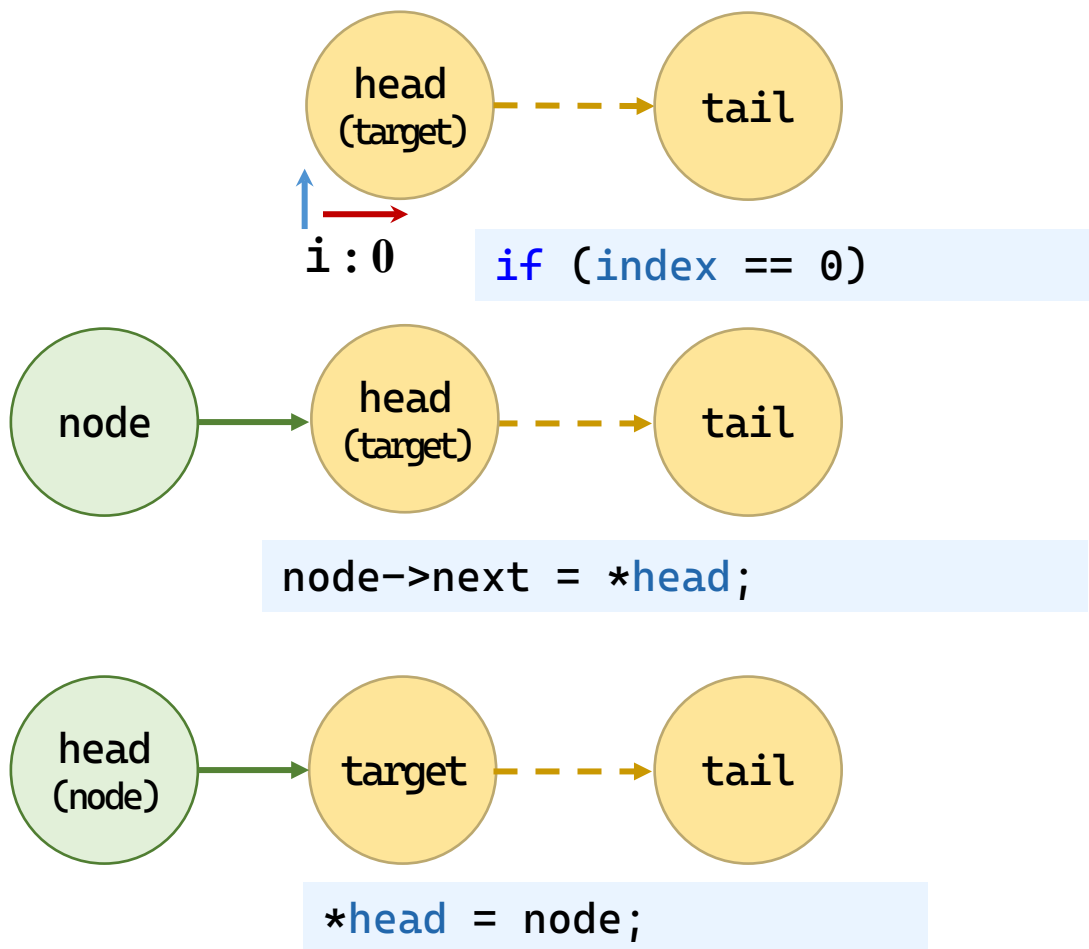
• 插入元素：有前一节点

主体步骤



链表：插入元素

• 插入元素：在头节点插入



特殊情况

1. `*head`为NULL无例外
2. `head`为NULL判定
3. 错误参数`index`判定
4. `node`生成失败判定

```
int insert_list(LinkList* head, int index, int value) {  
    int i;  
    Node* prev;  
    Node* node;  
    if (head == NULL)  
        return NULL_LIST;  
    if (index < 0)  
        return NOT_FOUND;  
    if (index == 0) {  
        node = (Node*)malloc(sizeof(Node));  
        if (node == NULL)  
            return ERROR;  
        node->data = value;  
        node->next = *head;  
        *head = node;  
        return OK;  
    }  
}
```

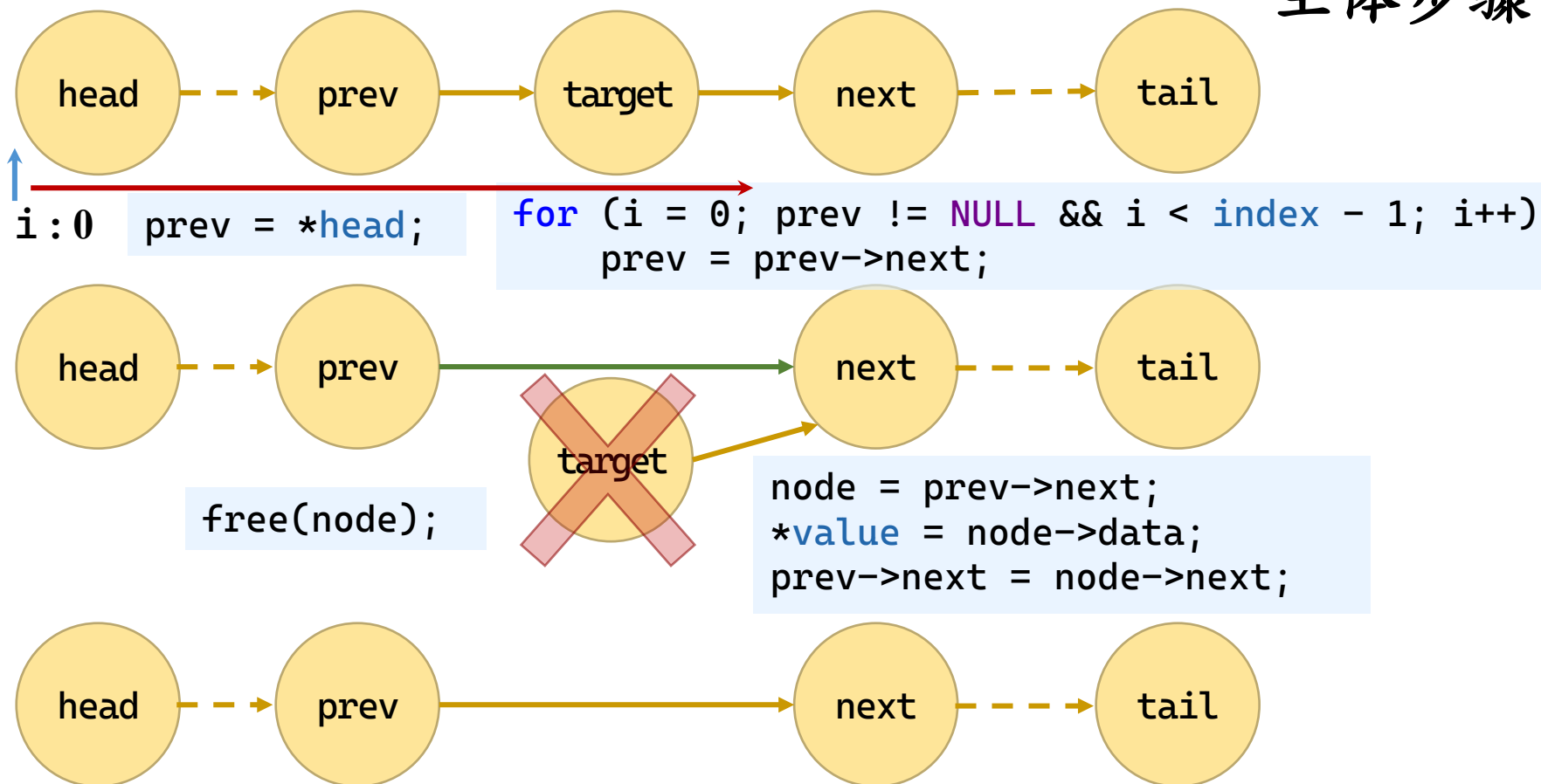
```
prev = *head;
for (i = 0; prev != NULL && i < index - 1; i++) {
    prev = prev->next;
}
if (prev == NULL) {
    return NOT_FOUND;
}

node = (Node*)malloc(sizeof(Node));
if (node == NULL) {
    return ERROR;
}
node->data = value;
node->next = prev->next;
prev->next = node;
return OK;
}
```

链表：删除元素

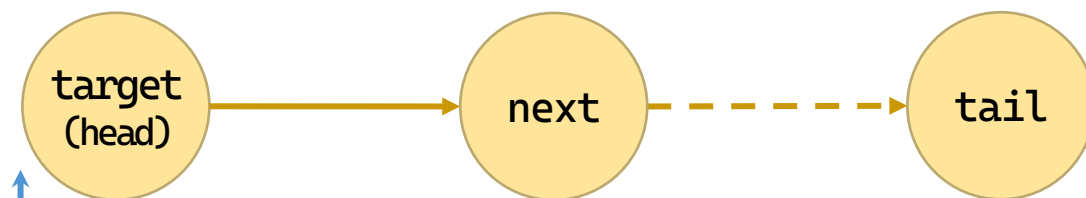
• 删除元素：有前一节点

主体步骤



链表：删除元素

• 删除元素：在头节点删除



↑
i : 0 `if (index == 0)`



`free(node);`

```
node = *head;  
*value = node->data;  
*head = node->next;
```



特殊情况

1. *head为NULL判定
2. head为NULL判定
3. 错误参数index判定

```
int delete_list(LinkList* head, int index, int* value) {
    int i;
    Node* prev;
    Node* node;

    if (head == NULL || value == NULL || *head == NULL
        || index < 0) {
        return ERROR;
    }

    if (index == 0) {
        node = *head;
        *value = node->data;
        *head = node->next;
        free(node);
        return OK;
    }
}
```

在头删除的情况：
调整结点链接，当前链表头后移；
释放原链表头。

```
prev = *head;
for (i = 0; prev != NULL && i < index - 1; i++) {
    prev = prev->next;
}
if (prev == NULL || prev->next == NULL) {
    return NOT_FOUND;
}
```

寻找插入位置的前一结点
如果不存在该位置报错

```
node = prev->next;
*value = node->data;
prev->next = node->next;
free(node);
return OK;
```

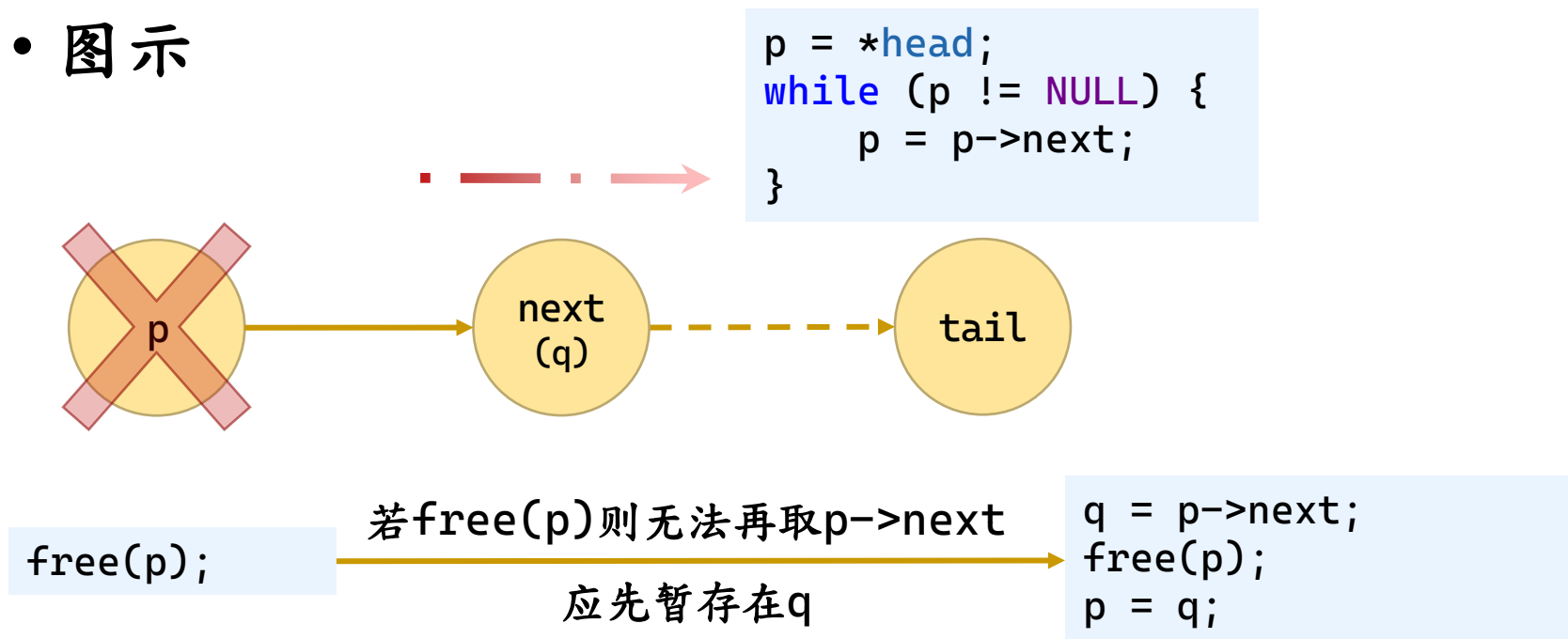
一般删除的情况：
调整结点链接，前一结点跳过待删结点，后续为待删结点的后续；
释放待删结点。

```
}
```

链表：销毁

- 由malloc()开辟的空间在程序终止时释放
 - 但程序员不能放任这种情况，应调用free()释放

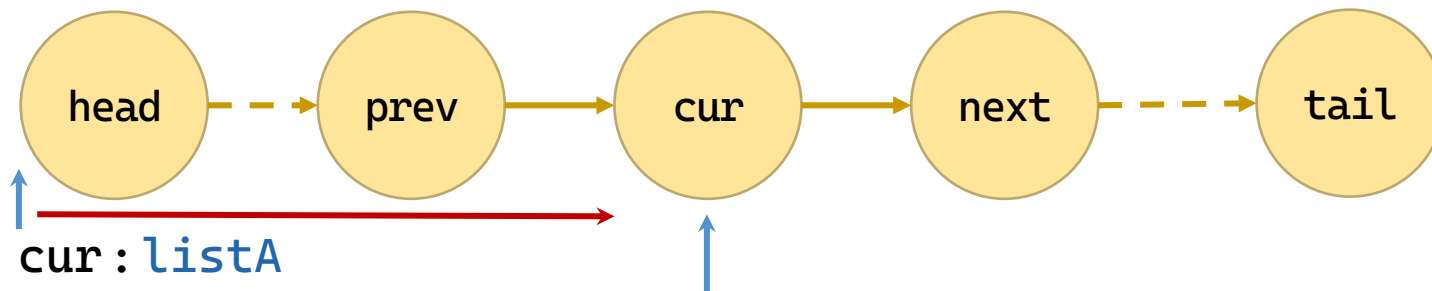
- 图示



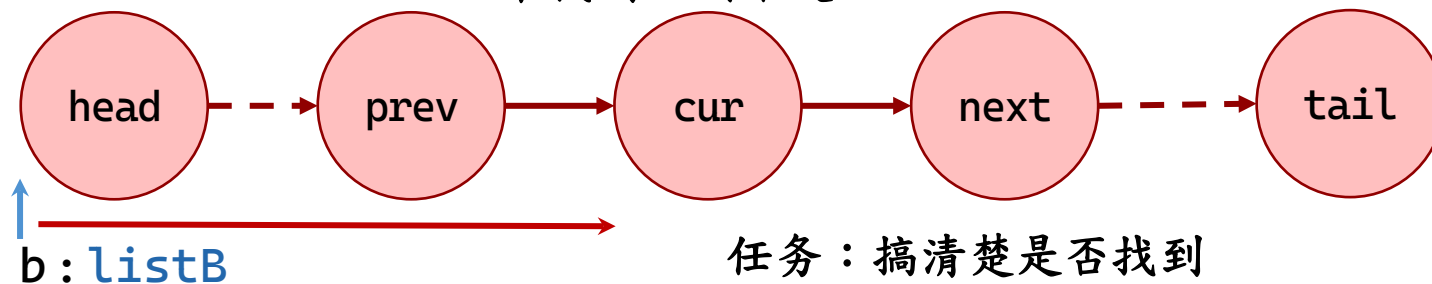
```
void free_list(LinkList* head) {  
    Node* p, * q;  
    if (head == NULL)  
        return;  
    p = *head;  
    while (p != NULL) {  
        q = p->next;  
        free(p);  
        p = q;  
    }  
    *head = NULL;  
}
```

链表：求差集

- 求两个链表A、B的差集A-B



是否找到？找到，则删除；
未找到，则不处理。



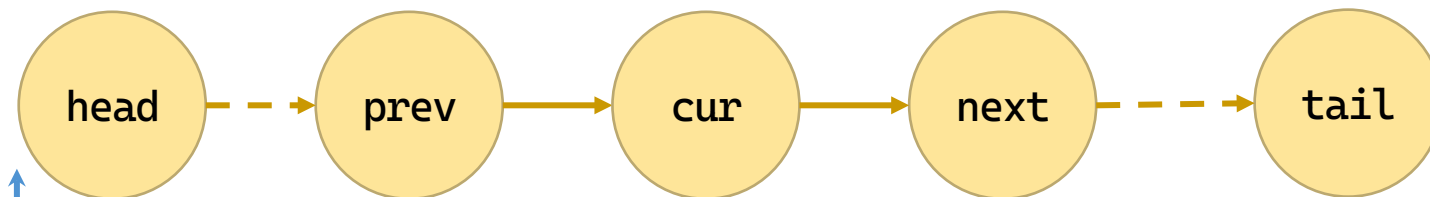
任务：搞清楚是否找到

```
while (b != NULL) {  
    // ...  
    b = b->next;  
}
```

```
if (current->data == b->data) {  
    found = 1;  
    break;  
}
```

链表：求差集

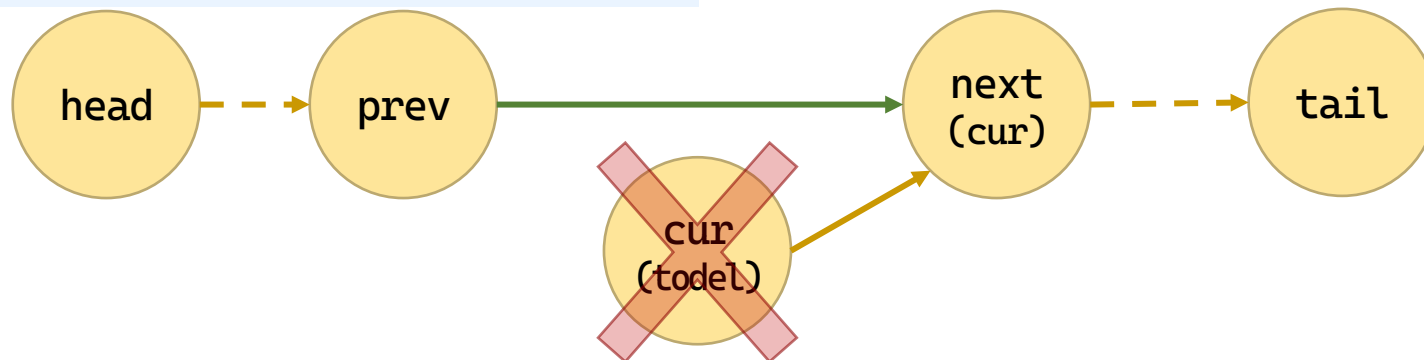
- 如果找到了，则删除，前导不动；否则，都动



cur: listA

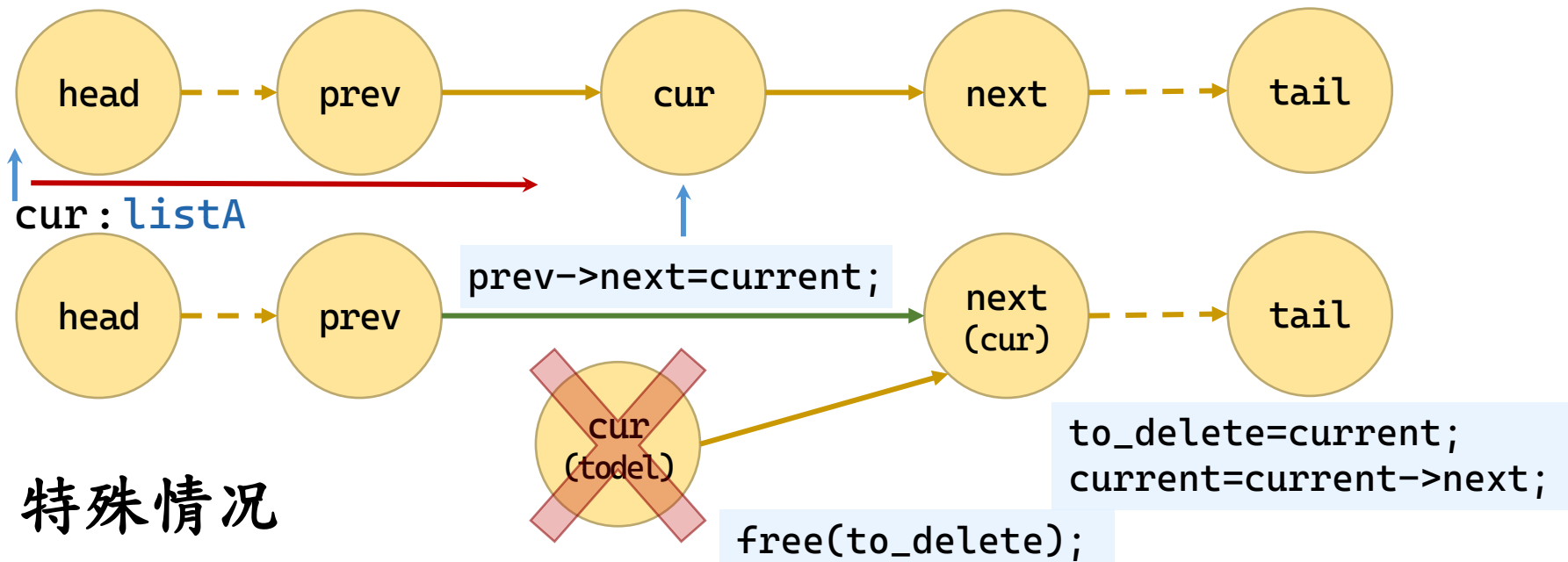
```
while (current != NULL) {  
    // found cur in List B?  
    prev = current;  
    current = current->next;  
}
```

```
if (found) {  
    to_delete = current;  
    current = current->next;  
    prev->next = current;  
    free(to_delete);  
}
```



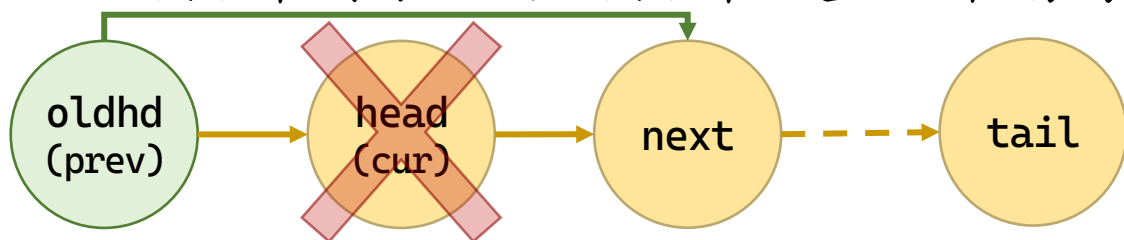
链表：求差集

- 如果找到了，则删除，前导不动



- 特殊情况

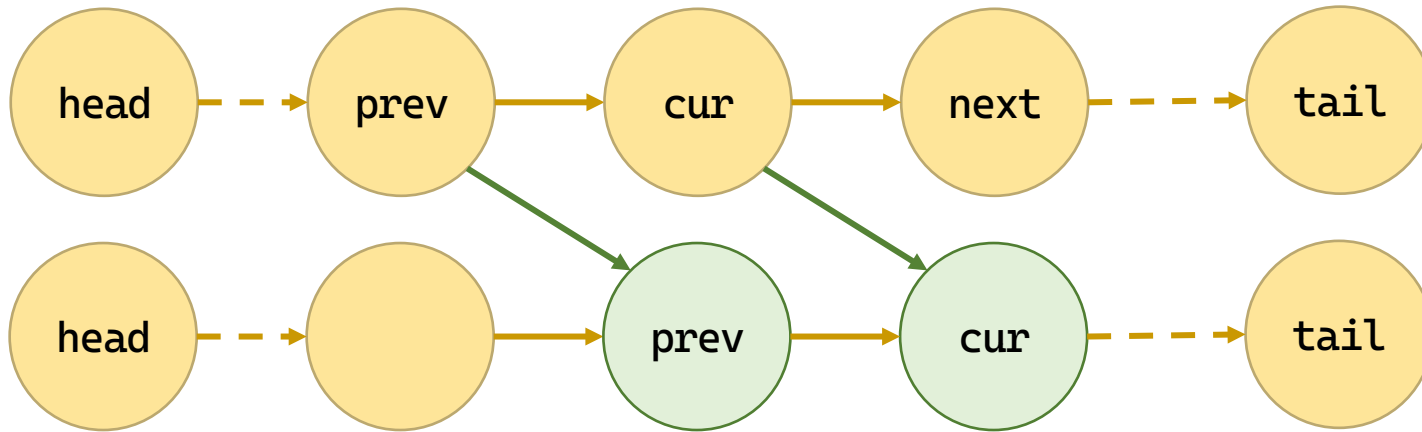
– 头指针删除：为头指针设置一个前导



```
old_head.next = *listA;  
prev = &old_head;  
current = old_head.next;  
...  
*listA = old_head.next;
```


链表：求差集

- 如果找不到，则不删除，前导前进



```
while (current != NULL) {  
    // found cur in List B?  
    prev = current;  
    current = current->next;  
}
```

```
if (found) {  
    ...  
} else {  
    prev = current;  
    current = current->next;  
}
```

```

void difference_list(LinkList* listA, LinkList* listB) {
    Node old_head;
    Node* prev, * current, * b, * to_delete;
    int found;
    if (listA == NULL || *listA == NULL)
        return;
    old_head.next = *listA;
    prev = &old_head;
    current = old_head.next;
    while (current != NULL) {
        found = 0;
        b = (listB != NULL) ? *listB : NULL;
        while (b != NULL) {
            if (current->data == b->data) {
                found = 1;
                break;
            }
            b = b->next;
        }
    }
}

```

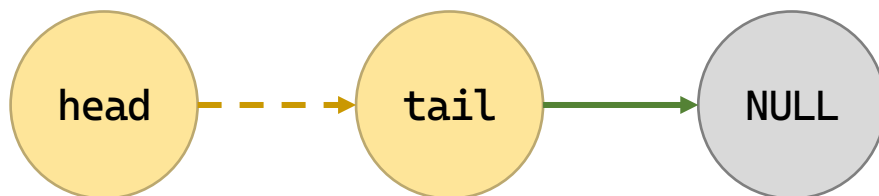
```
    }  
    if (found) {  
        to_delete = current;  
        current = current->next;  
        prev->next = current;  
        free(to_delete);  
    } else {  
        prev = current;  
        current = current->next;  
    }  
}  
*listA = old_head.next;  
}
```

目录

	2	顺序表
	3	单链表
	4	循环链表
	5	双向链表
	6	链表的应用

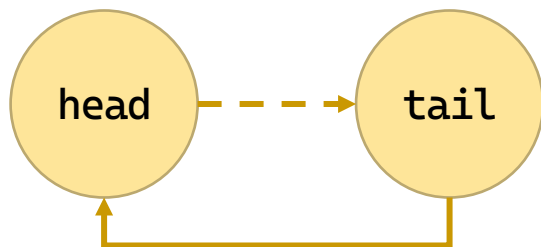
循环链表

- 普通链表尾部的下一节点是空指针



```
tail->next = NULL;
```

- 循环链表尾部的下一节点是链表头

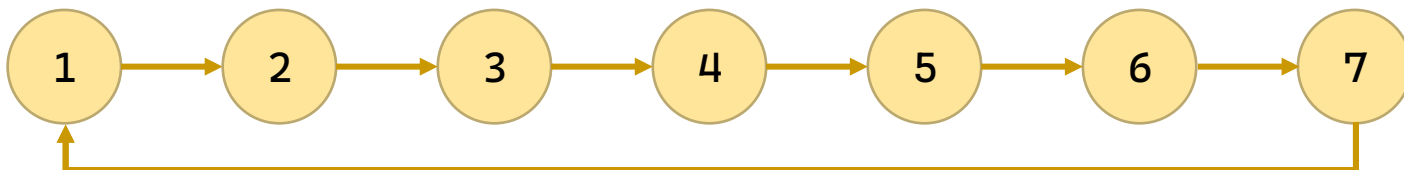


```
tail->next = head;
```

循环链表：应用

• 例2-5 约瑟夫环

- 设有 n 个数构成一个环链，要求从第 k 个数开始数数，数到 m 的那个数被弹出，然后从该数的下一个数重新数数，数到 m 的那个数又被弹出，如此重复，直到所有的数均被弹出为止。输出这些数弹出的序列。
- 例如： $n=7$ ， $k=2$ ， $m=3$
- 输出：4 7 3 1 6 2 5



循环链表：应用

- 主体步骤

- 找到第 k 个节点

```
current = head;  
for (count = 1; count < k; count++) {  
    current = current->next;  
}
```

- 数到第 m 个节点

```
for (count = 1; count < m; count++) {  
    current = current->next;  
}
```

- 打印并删除数到的节点

```
to_delete = current;  
current = current->next;  
prev->next = current;  
printf("%d -> ", to_delete->data);  
free(to_delete);
```

需要用到pre，
在上面步骤要维护pre



```
prev = current;  
current = current->next;
```

- 重复至链表已空

- 应判断只剩一个节点

```
while (current->next != current) {  
    ...  
}
```

循环链表：初步代码

```
void josephus_ring(LinkList head, int k, int m) {
    int count;
    Node *prev, *current, *to_delete;
    current = head;
    for (count = 1; count < k; count++) {
        prev = current;
        current = current->next;
    }
    while (current->next != current) {
        for (count = 1; count < m; count++) {
            prev = current;
            current = current->next;
        }
        to_delete = current;
        current = current->next;
        prev->next = current;
        printf("%d -> ", to_delete->data);
        free(to_delete);
    }
}
```



```
#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
    int data;
    struct Node* next;
} Node;
typedef Node* LinkList;
LinkList create_list(int n) {
    if (n <= 0)
        return NULL;
    Node* head = (Node*)malloc(sizeof(Node));
    head->data = 1;
    Node* current = head;
    for (int i = 2; i <= n; i++) {
        current->next = (Node*)malloc(sizeof(Node));
        current = current->next;
        current->data = i;
    }
    current->next = head; // 尾指向首, 连成环
    return head;
}
```

```
int main(void) {
    /* 定义测试参数表 */
    int node_counts[] = {0, 1, 5, 5, 0, 5, 5, 5, 5};
    int k_vals[]      = {1, 1, 1, -2, 0, 45, 1, 1, 1};
    int m_vals[]      = {3, 3, 3, 3, 3, 3, -4, 0, 80};
    int total_tests = sizeof(node_counts) /
        sizeof(node_counts[0]);

    /* 使用一个极其简单的 for 循环驱动全部测试例 */
    for (int i = 0; i < total_tests; i++) {
        printf("\n节点数: %d | k = %d | m = %d\n", i + 1,
            node_counts[i], k_vals[i], m_vals[i]);
        josephus_ring(create_list(node_counts[i]), k_vals[i],
            m_vals[i]);
    }

    return 0;
}
```

循环链表：通过测试完善代码

- 先确定定义域，再定义测试域

参数名	取值范围	测试用例
head	任意链表	空表、一个节点表、五节点表
k	任意整数	负数、0、小正数、大正数
m	任意整数	负数、0、小正数、大正数

- 测试发现错误情况

- error C4703: 使用了可能未初始化的本地指针变量 “prev”
- 空表时，在while (current->next != current)出空指针

- 应对：

```
if (head == NULL) return;
```

- $k \leq 0$ 或 $m \leq 0$ ，出现
prev为空错误

```
if (head == NULL || k < 1 || m < 1)  
    return;
```

循环链表：通过测试完善代码

- 测试发现错误情况

- 错误：未输出最后一个节点

【测试用例】节点数: 5 | 起始位置 k: 1 | 每次报数 m: 3
3 -> 1 -> 5 -> 2 ->

- 应对

```
printf("%d\n", current->data);  
free(current);
```

- 错误： $k=1$ 或 $m=1$ ，出现prev为空错误

```
int len = 1;  
Node* tail = head;  
while (tail->next != head) {  
    len++;  
    tail = tail->next;  
}  
prev = tail;  
k = (k - 1) % len + 1;  
m = (m - 1) % len + 1;
```

```

void josephus_ring(LinkList head, int k, int m) {
    if (head == NULL) {
        printf("链表为空, 无法执行约瑟夫环模拟。\\n");
        return;
    }
    int len = 1;
    Node* tail = head;
    while (tail->next != head) {
        len++;
        tail = tail->next;
    }
    Node* prev = tail;
    if (k < 1)
        k = 1;
    else
        k = (k - 1) % len + 1;
    if (m < 1)
        m = 1;
}

```

```

Node* current = head;
for (int count = 1; count < k; count++) {
    prev = current;
    current = current->next;
}
printf("弹出序列: ");
while (current->next != current) {
    for (int count = 1; count < m; count++) {
        prev = current;
        current = current->next;
    }
    Node* to_delete = current;
    current = current->next;
    prev->next = current;
    printf("%d -> ", to_delete->data);
    free(to_delete);
}
printf("%d\n", current->data);
free(current);
}

```

时间复杂度分析

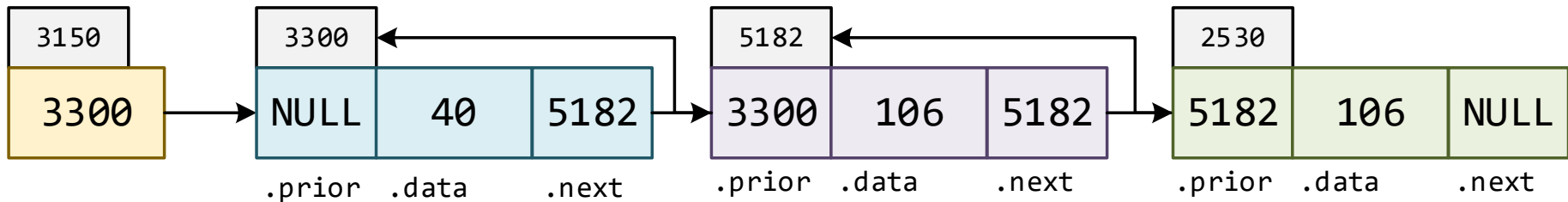
- 定位计数起始位置的时间
 - 通过 $k=(k-1)\%len+1$; 已经从 $O(k)$ 优化为 $O(n)$
- 对所有数据元素，依次完成计数到 m 及弹出数据元素
 - 时间复杂度： $O(m*n)$ 。
- 算法的时间复杂度为：
 - 时间复杂度： $T(m,n)=O(n)+O(m*n)=O(m*n)$

目录

	1	基本概念
	2	顺序表
	3	单链表
	4	循环链表
	5	双向链表

双向链表

- 双向链表增加了前节点域



```
typedef struct DNode {  
    int data;  
    struct DNode* prior;  
    struct DNode* next;  
} DNode, *DLinkedList;
```

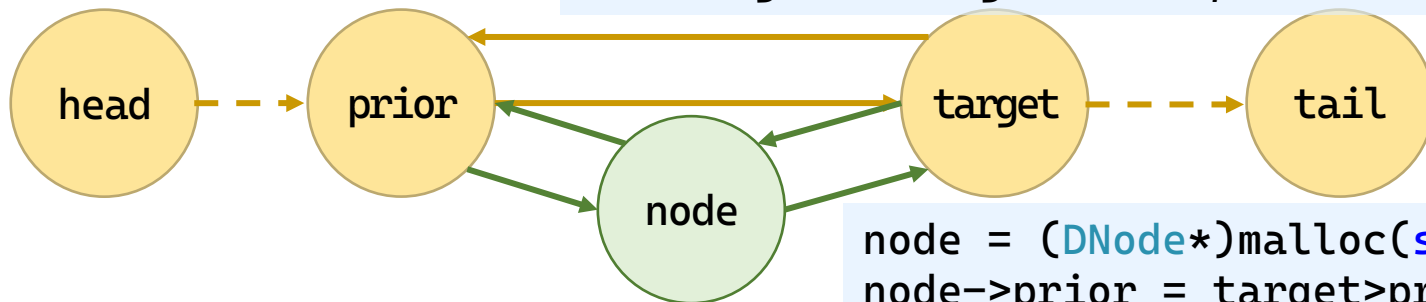
双向链表：插入元素

• 插入元素：有前一节点

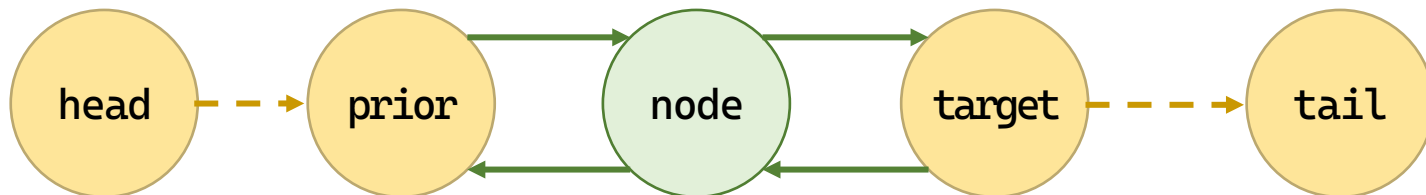
主体步骤



`i:0` `target = *head;` `for (i = 0; target != NULL && i < index; i++)`
`target = target->next;`



```
node = (DNode*)malloc(sizeof(DNode));  
node->prior = target->prior;  
target->prior->next = node;  
target->prior = node;
```

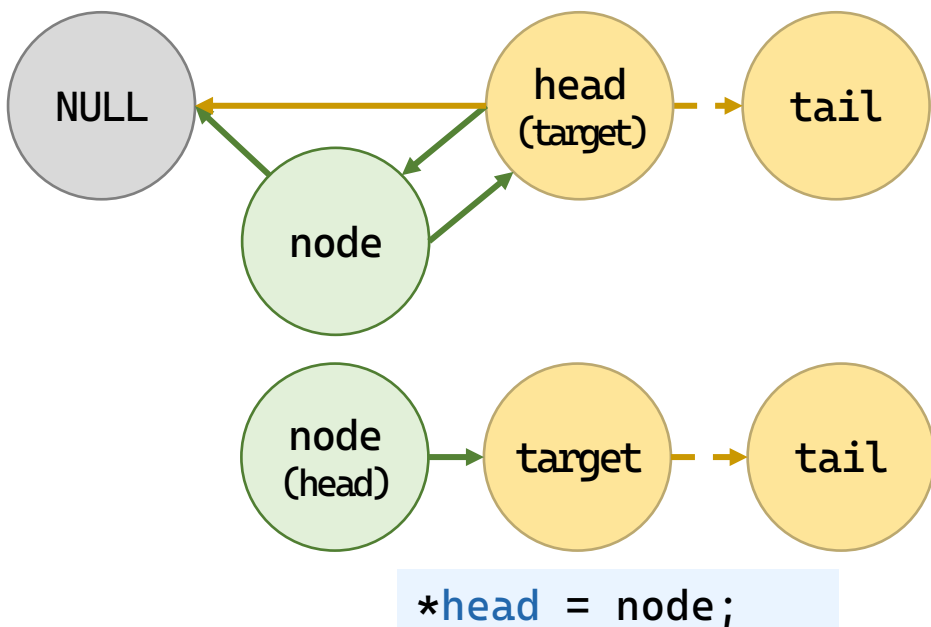
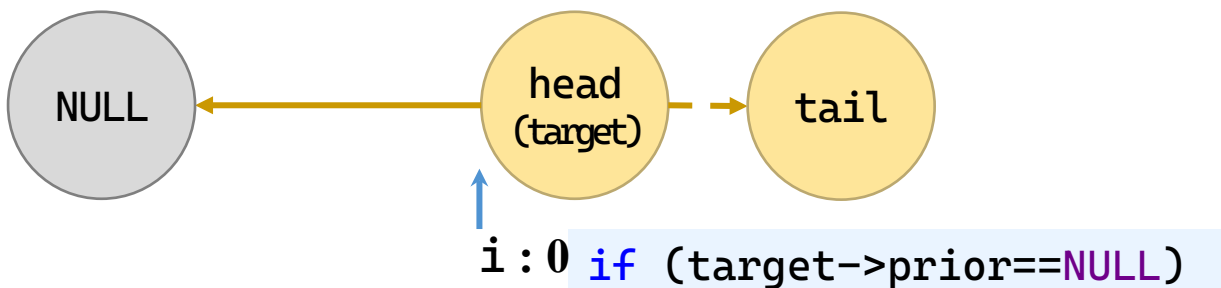


双向链表：插入元素

• 插入元素：在头节点插入

特殊情况

1. *head为NULL例外
2. head为NULL判定
3. 错误参数index判定
4. node 生成失败判定

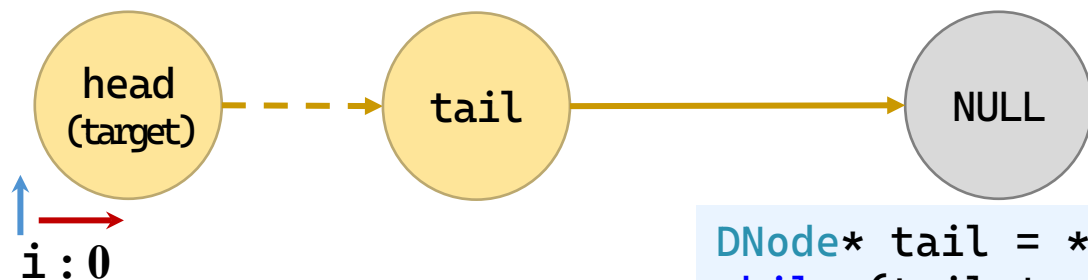


```
node = (DNode*)malloc(sizeof(DNode));
if (node == NULL)
    return ERROR;
node->data = value;
node->next = target;
node->prior = target->prior;
if (target->prior != NULL)
    target->prior->next = node;
else
    *head = node;
target->prior = node;
```

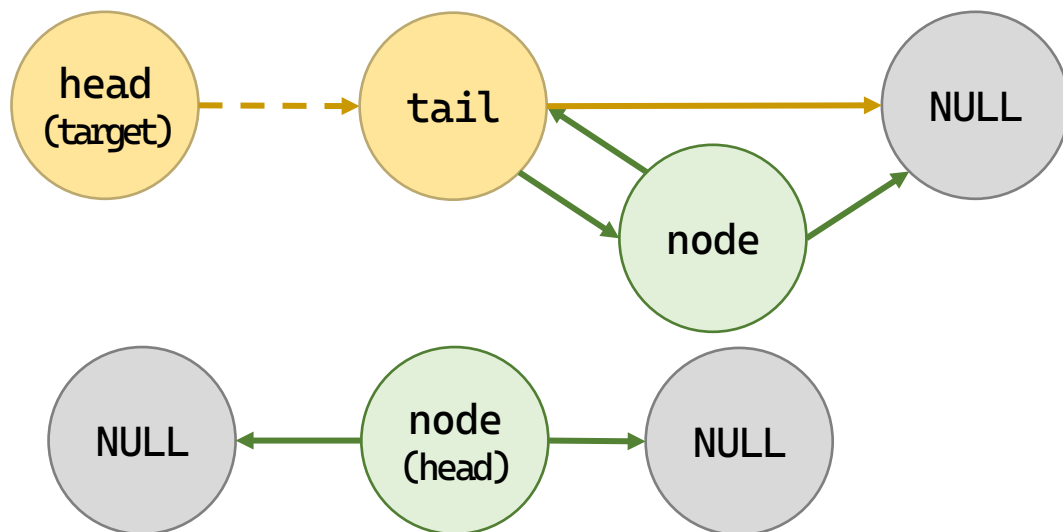
双向链表：插入元素

- 插入元素：找不到的时候尾插

特殊情况



```
DNode* tail = *head;  
while (tail != NULL && tail->next != NULL)  
    tail = tail->next;
```



```
node->next = NULL;  
node->prior = tail;
```

```
if (tail != NULL)  
    tail->next = node;  
else  
    *head = node;
```

```
int insert_list(DLinkedList* head, int index, int value) {
    DNode* target;
    DNode* node;
    int i;
    if (head == NULL)
        return NULL_LIST;
    if (index < 0)
        return NOT_FOUND;
    target = *head;
    for (i = 0; target != NULL && i < index; i++)
        target = target->next;
    if (i < index)
        return NOT_FOUND;
    node = (DNode*)malloc(sizeof(DNode));
    if (node == NULL)
        return ERROR;
    node->data = value;
    if (target != NULL) {
        node->next = target;
    }
}
```

```

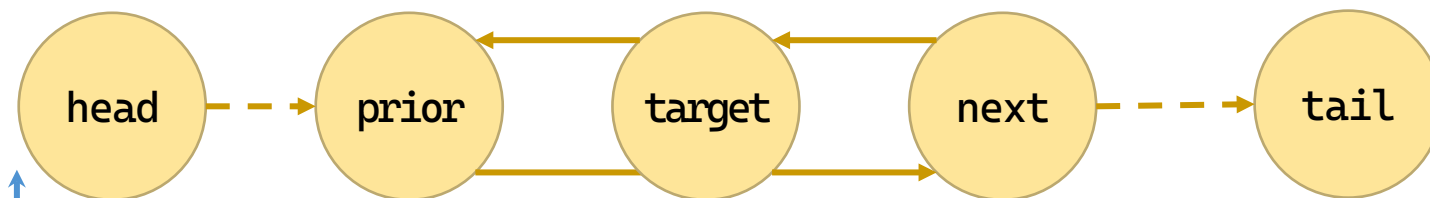
        node->prior = target->prior;
        if (target->prior != NULL)
            target->prior->next = node;
        else
            *head = node;
    }
    target->prior = node;
    return OK;
}
DNode* tail = *head;
while (tail != NULL && tail->next != NULL)
    tail = tail->next;
node->next = NULL;
node->prior = tail;
if (tail != NULL)
    tail->next = node;
else
    *head = node;
return OK;
}

```

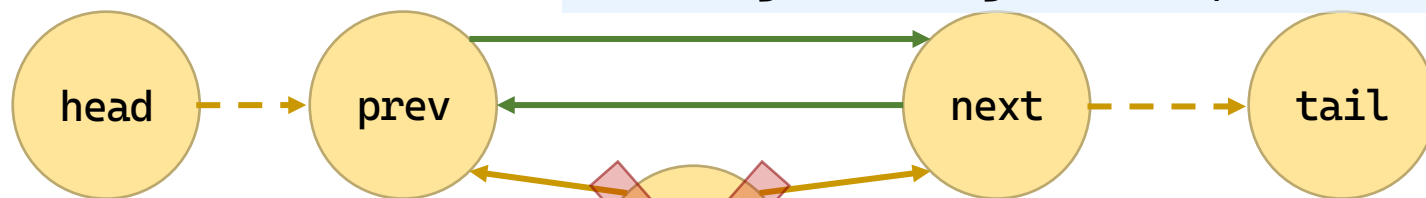
双向链表：删除元素

• 删除元素：有前一节点

主体步骤



`i:0` `target = *head;` `for (i = 0; target != NULL && i < index; i++)`
`target = target->next;`



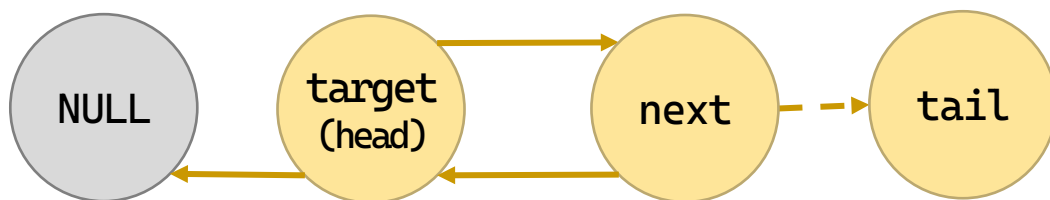
`free(node);`

`target->prior->next = target->next;`
`if (target->next != NULL)`
`target->next->prior = target->prior;`

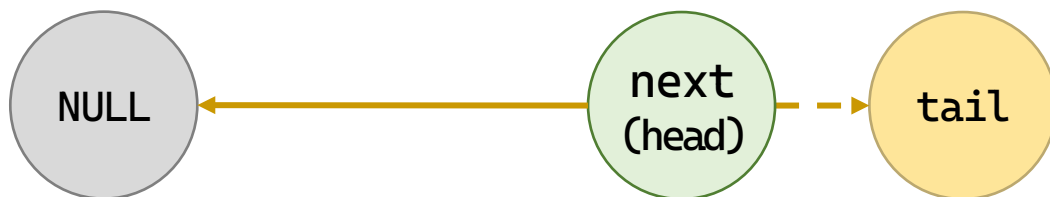
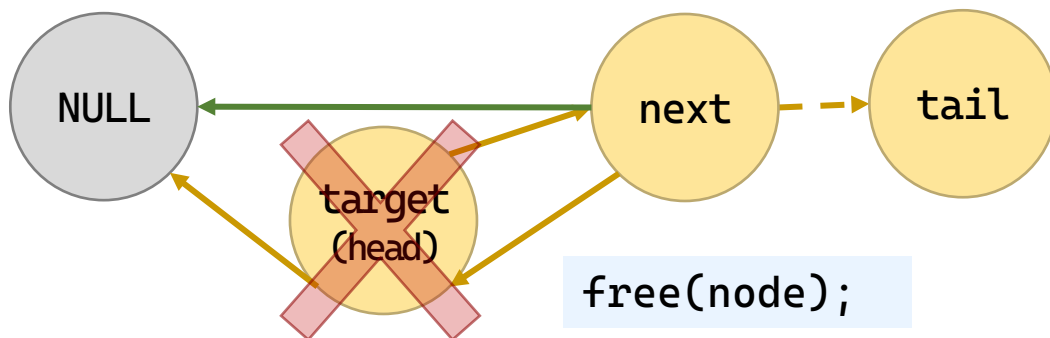


链表：删除元素

• 删除元素：在头节点删除



```
if (target == *head)
```



```
*head = target->next;
```

特殊情况

1. *head为NULL判定
2. head为NULL判定
3. 错误参数index判定

```
if (*head != NULL)  
    (*head)->prior = NULL;
```



```

int delete_list(DLinkedList* head, int index, int* value) {
    DNode* target;
    int i;
    if (head == NULL || value == NULL || *head == NULL
        || index < 0)
        return ERROR;
    target = *head;
    for (i = 0; target != NULL && i < index; i++)
        target = target->next;
    if (target == NULL)
        return NOT_FOUND;
    *value = target->data;
    if (target == *head) {
        *head = target->next;
        if (*head != NULL)
            (*head)->prior = NULL;
    }
    else {

```

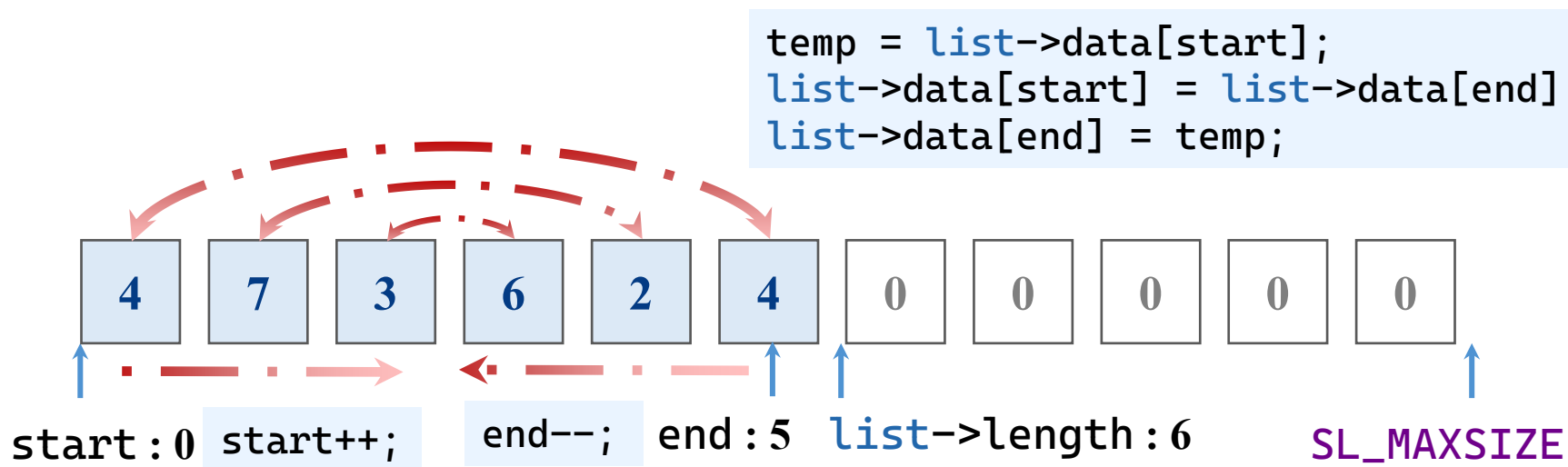
```
        target->prior->next = target->next;
        if (target->next != NULL)
            target->next->prior = target->prior;
    }
    free(target);
    return OK;
}
```

目录

	2	顺序表
	3	单链表
	4	循环链表
	5	双向链表
	6	链表的应用

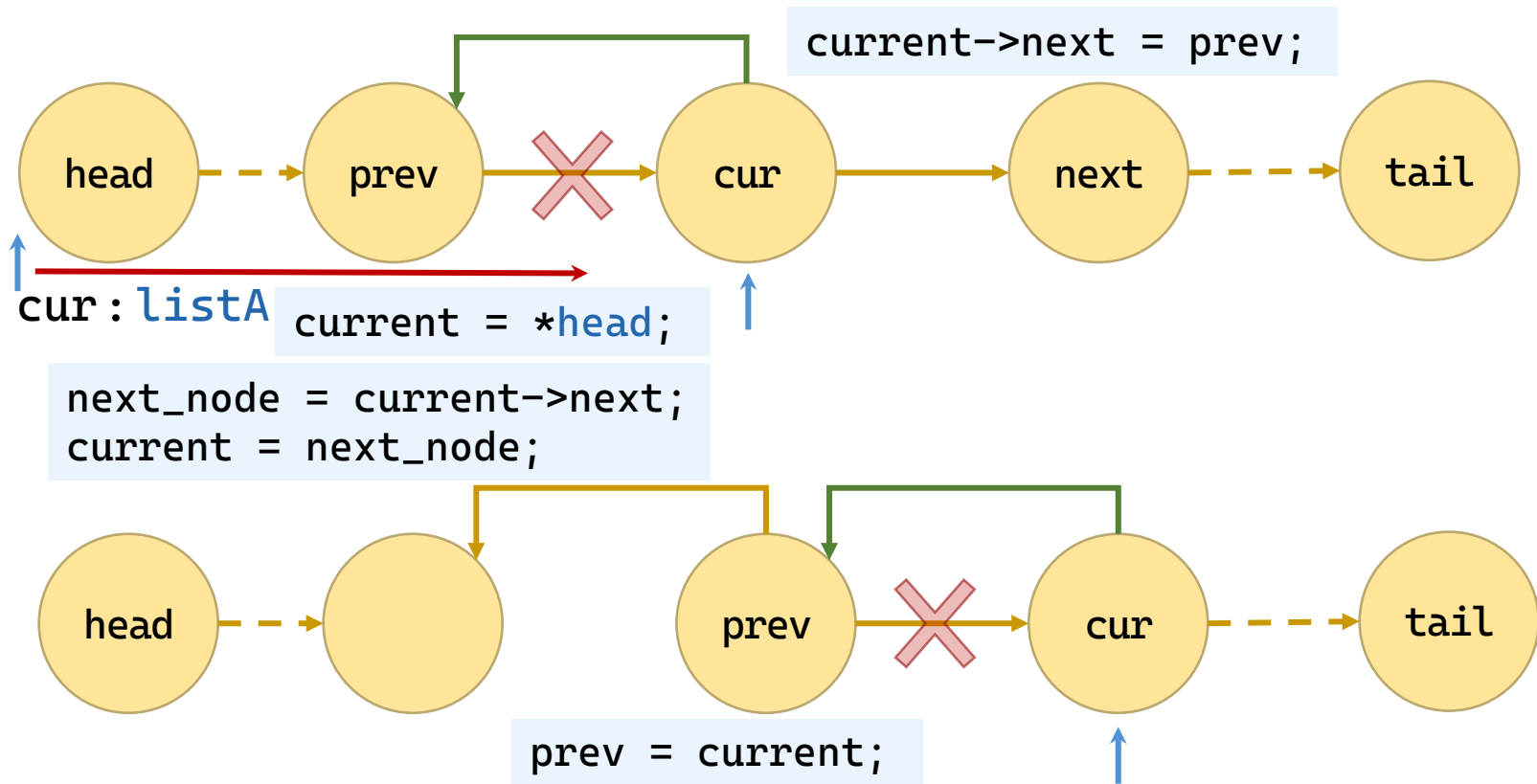
线性表逆置

- 例题：设计算法，将线性表L的数据元素逆置。
 - (1) 顺序表：利用随机存取特点，将头尾对称的位置交换。



线性表逆置

- 例题：设计算法，将线性表L的数据元素逆置。
 - (2) 链表：在访问过程中，将指针“逆置”。



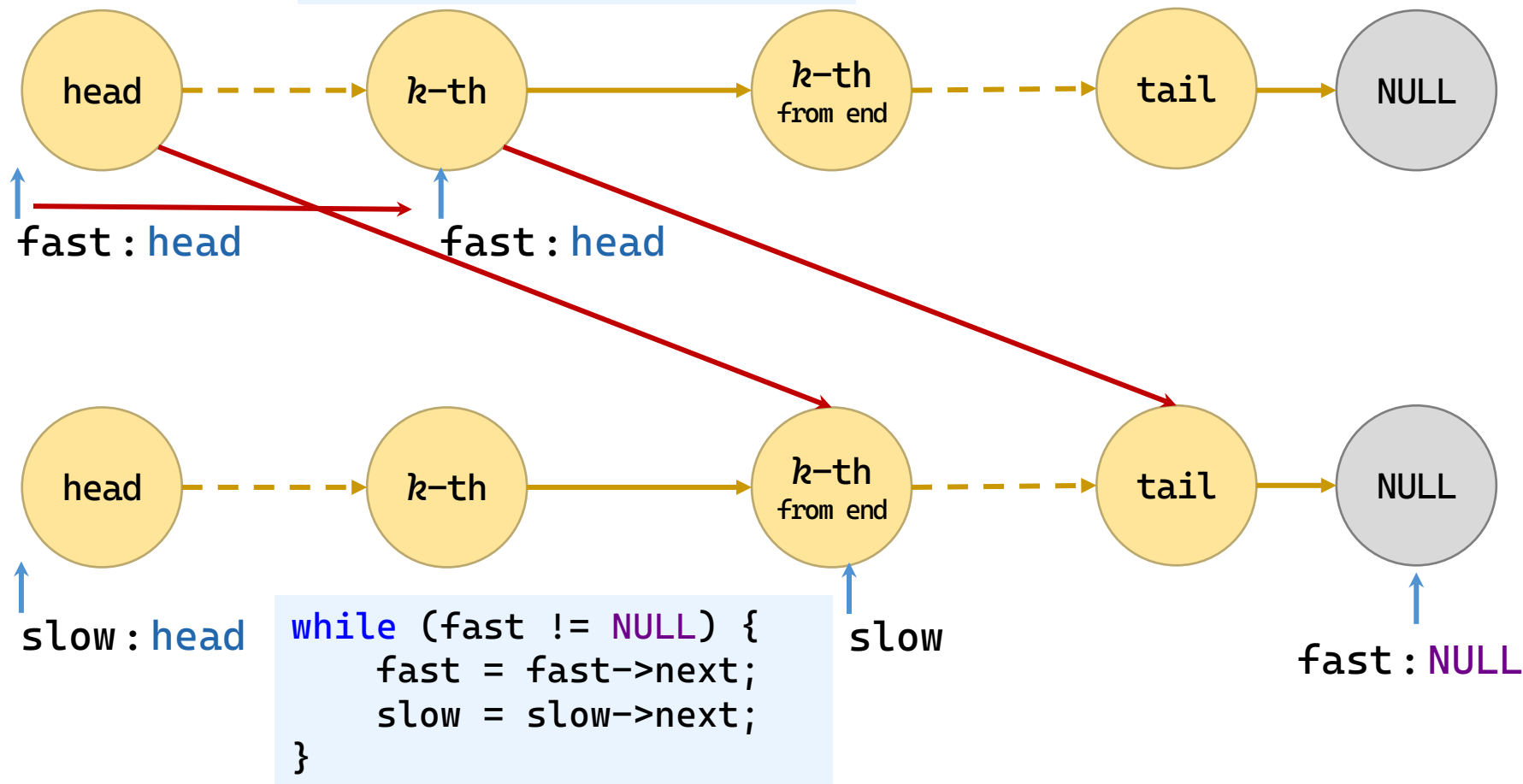
```
void reverse_list(LinkList* head) {  
    Node* prev = NULL;  
    Node* current;  
    Node* next_node = NULL;  
    if (head == NULL || *head == NULL)  
        return;  
    current = *head;  
    while (current != NULL) {  
        next_node = current->next;  
        current->next = prev;  
        prev = current;  
        current = next_node;  
    }  
    *head = prev;  
}
```

查找倒数第 k 个位置上的结点

- 例：给定链表的头指针 L 和一个正整数 k 。试设计一个尽可能高效的算法，用于查找链表 L 中倒数第 k 个位置上的结点。
- 算法思路1：
 - 先计算单链表的长度 n ，再查找第 $n-k+1$ 个结点。
 - 算法的时间复杂度为 $O(2n-k)$ 。
- 算法思路2：（更优）
 - 快指针先走 k 步，然后快指针和慢指针同时走 $n-k$ 步。
 - 算法的时间复杂度为 $O(n)$ 。

查找倒数第 k 个位置上的结点

```
for (i = 0; i < k; i++)  
    fast = fast->next;
```




```
Node* find_kth_from_end(LinkList head, int k) {
    Node* fast = head;
    Node* slow = head;
    int i;
    if (head == NULL || k <= 0)
        return NULL;
    for (i = 0; i < k; i++) {
        if (fast == NULL)
            return NULL;
        fast = fast->next;
    }
    while (fast != NULL) {
        fast = fast->next;
        slow = slow->next;
    }
    return slow;
}
```

2

谢谢观看

理论课程



廈門大學
XIAMEN UNIVERSITY



信息学院 黄 焯
(特色化示范性软件学院) 博士, 副教授
School of Informatics Wei Huang

3

栈和队列

理论课程



廈門大學
XIAMEN UNIVERSITY



信息学院 黃 煒
(特色化示范性软件学院) 博士, 副教授
School of Informatics Wei Huang

内容要点

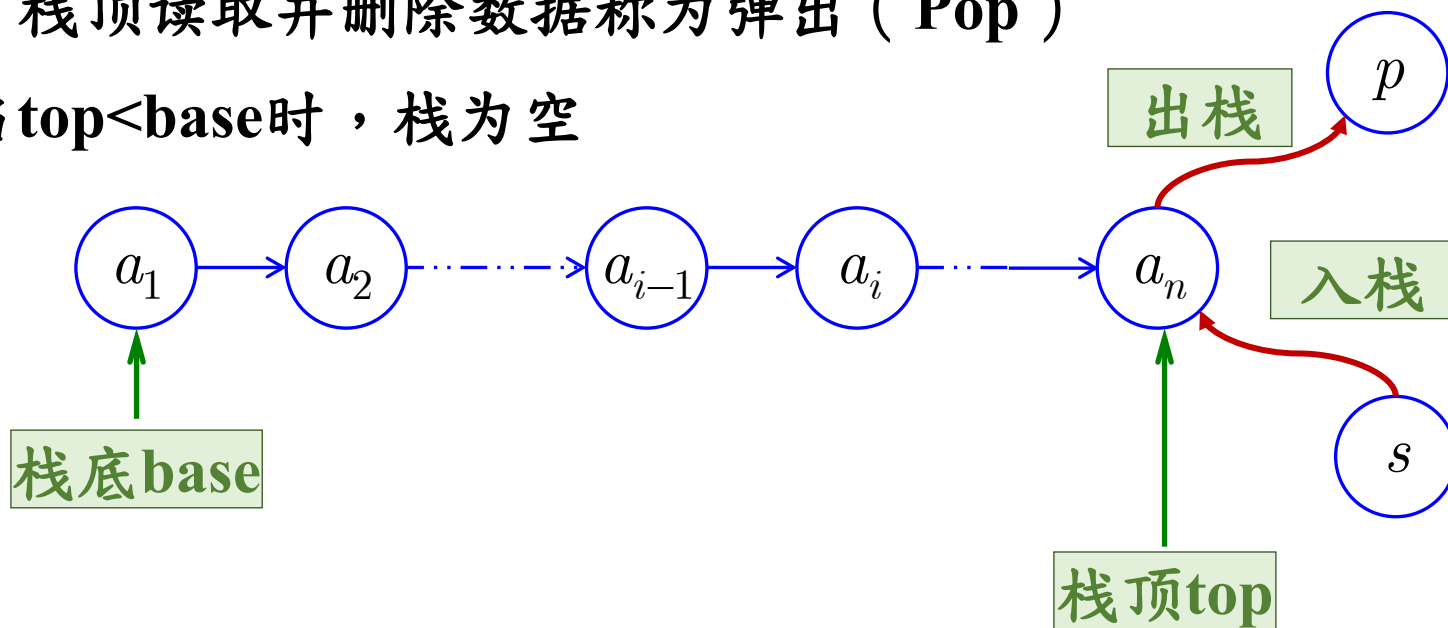
- 栈
 - 顺序栈
 - 链栈
 - 栈的应用
- 队列
 - 顺序队列
 - 链队列
 - 队列的应用

目录

	1	栈
	1.1	基本概念
	1.2	顺序栈
	2	队列
	3	小结

基本概念

- 栈和队列都是限制在端点进行插入和删除的线性表。
- 栈是一种后进先出(LIFO)的线性表。
 - 插入数据至栈顶称为压入 (Push)
 - 自栈顶读取并删除数据称为弹出 (Pop)
 - 当 $\text{top} < \text{base}$ 时，栈为空



例2-6 一个栈的输入序列为 $a_1 a_2 a_3 a_4 a_5$ ，
则栈的输出序列不可能是()。

(A) $a_2 a_3 a_4 a_1 a_5$

(B) $a_2 a_3 a_1 a_4 a_5$

(C) $a_5 a_4 a_1 a_3 a_2$

(D) $a_1 a_5 a_4 a_3 a_2$

栈的抽象数据类型定义

- 定义

ADT Stack{

数据对象： $\mathbf{D} = \{ a_i \mid a_i \in \text{ElemSet}, i=1,2,\dots,n, n \geq 0 \}$

数据关系：

$\mathbf{R} = \{ \langle a_{i-1}, a_i \rangle \mid a_{i-1}, a_i \in \mathbf{D}, i=2,3,\dots,n \}$

基本操作：

初始化、进栈、出栈、取栈顶元素、

判断栈空等

} ADT Stack

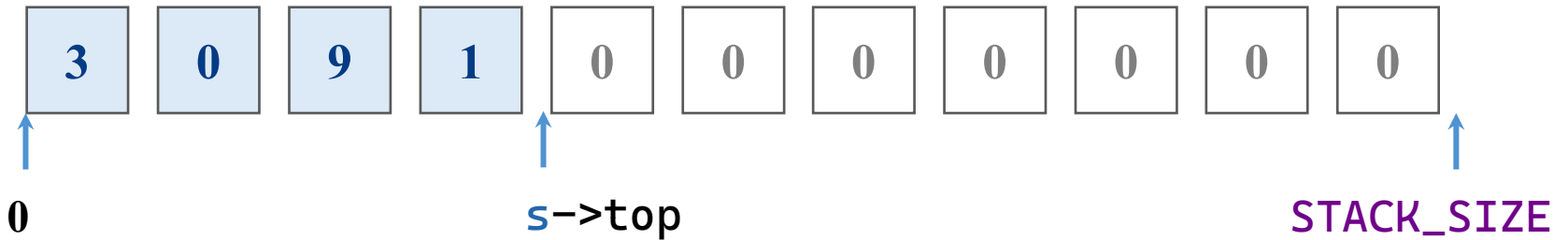
目录

	1	栈
	1.1	基本概念
	1.2	顺序栈
	1.3	链栈
	2	队列

顺序栈的存储结构

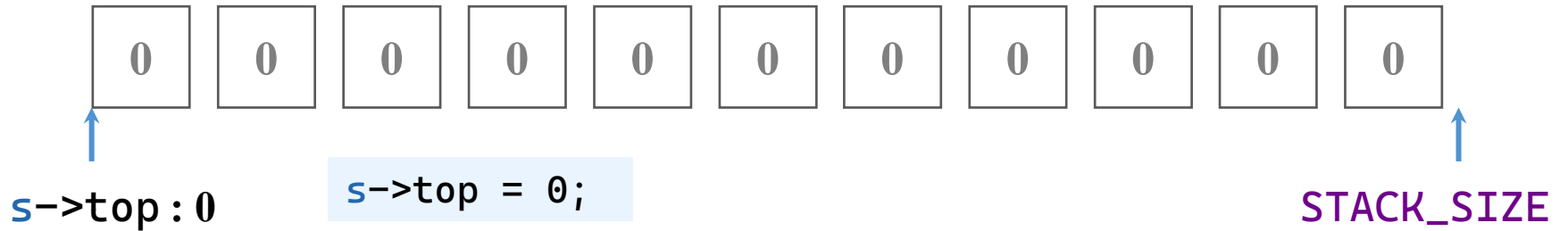
- 顺序栈的存储结构

```
typedef struct {  
    int data[STACK_SIZE];  
    int top;  
} SeqStack;
```



顺序栈：初始化

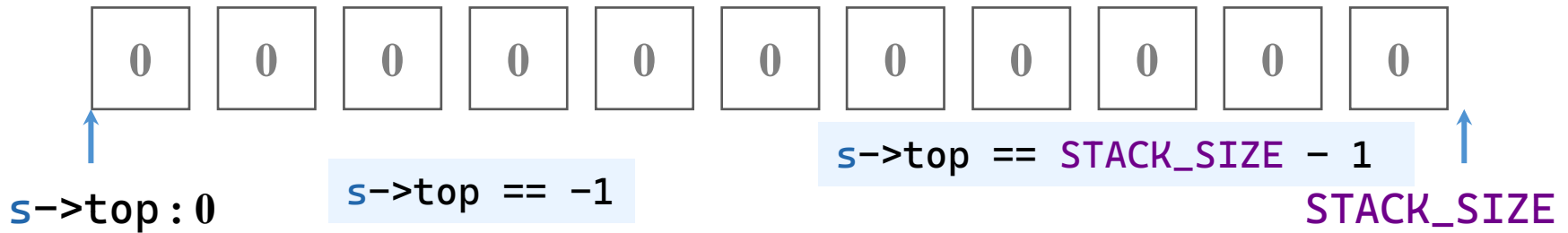
- 初始化：长度清零（栈顶为-1）



```
int init_stack(SeqStack* s) {  
    if (s == NULL)  
        return ERROR;  
    s->top = -1;  
    return OK;  
}
```

顺序栈：判断空和满

- 判断空：栈顶是否小于0
- 判断满：栈顶是否达到栈长度最大值减一



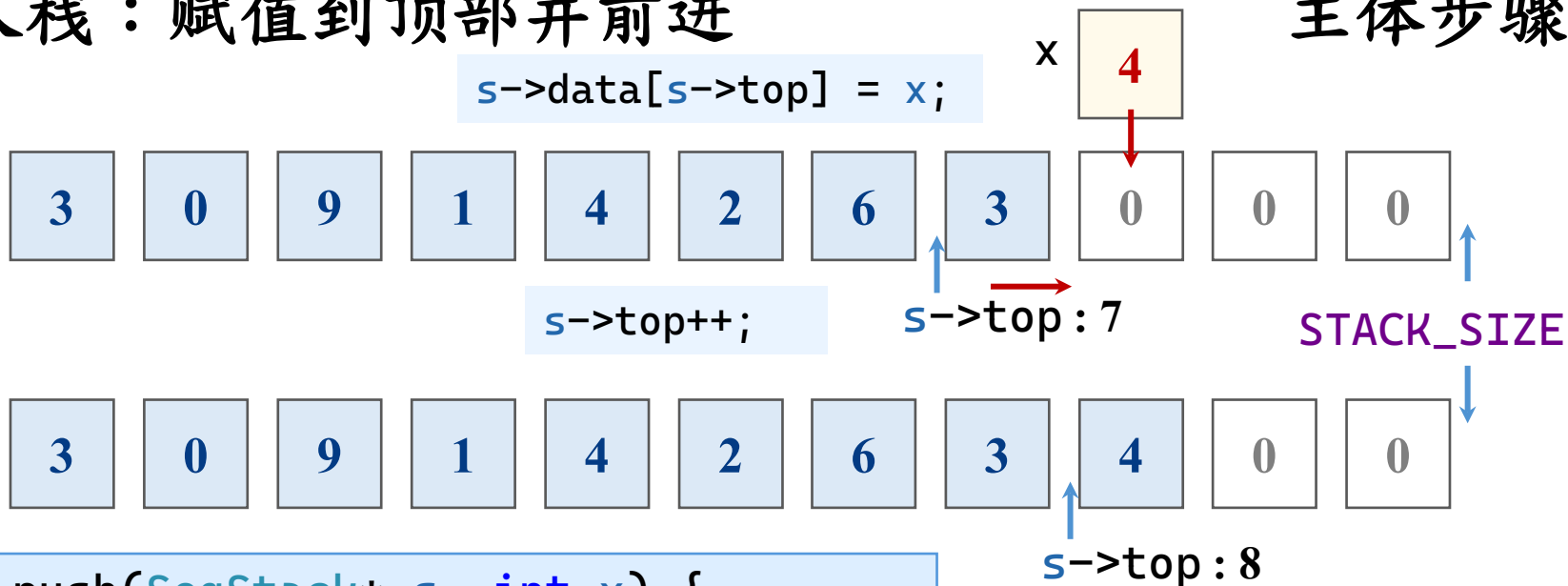
```
int is_empty(SeqStack* s) {  
    if (s == NULL)  
        return ERROR;  
    return (s->top == -1);  
}
```

```
int is_full(SeqStack* s) {  
    if (s == NULL)  
        return ERROR;  
    return (s->top == STACK_SIZE - 1);  
}
```

顺序栈：压入栈

- 入栈：赋值到顶部并前进

主体步骤



```
int push(SeqStack* s, int x) {  
    if (s == NULL || is_full(s))  
        return ERROR;  
    s->top++;  
    s->data[s->top] = x;  
    return OK;  
}
```

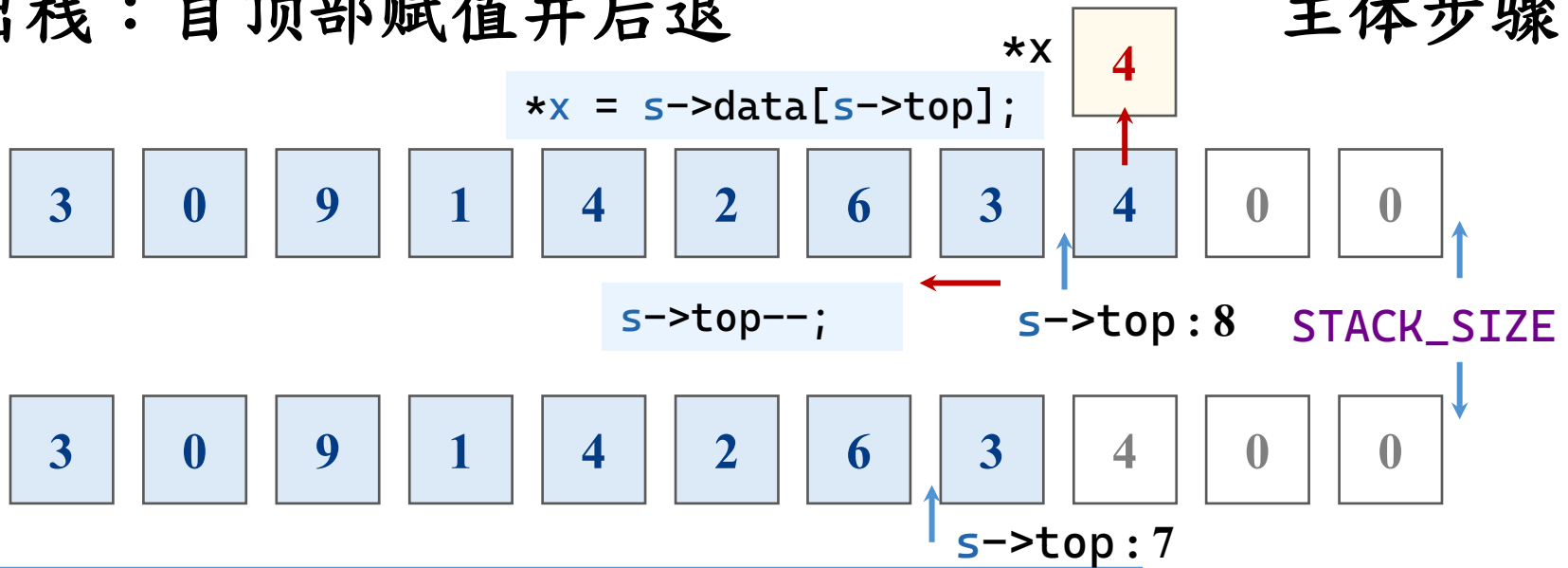
边界判定

1. 栈指针空
2. 栈满

顺序栈：弹出栈

- 出栈：自顶部赋值并后退

主体步骤



```
int pop(SeqStack* s, int* x) {  
    if (s == NULL || is_empty(s) || x == NULL)  
        return ERROR;  
    *x = s->data[s->top];  
    s->top--;  
    return OK;  
}
```

边界判定

1. 栈指针空
2. 栈空
3. 出指针空

顺序栈：取栈顶

- 出栈：自顶部赋值并不动

主体步骤



```
int get_top(SeqStack* s, int* x) {  
    if (s == NULL || is_empty(s) || x == NULL)  
        return ERROR;  
    *x = s->data[s->top];  
    return OK;  
}
```

边界判定

1. 栈指针空
2. 栈空
3. 出指针空

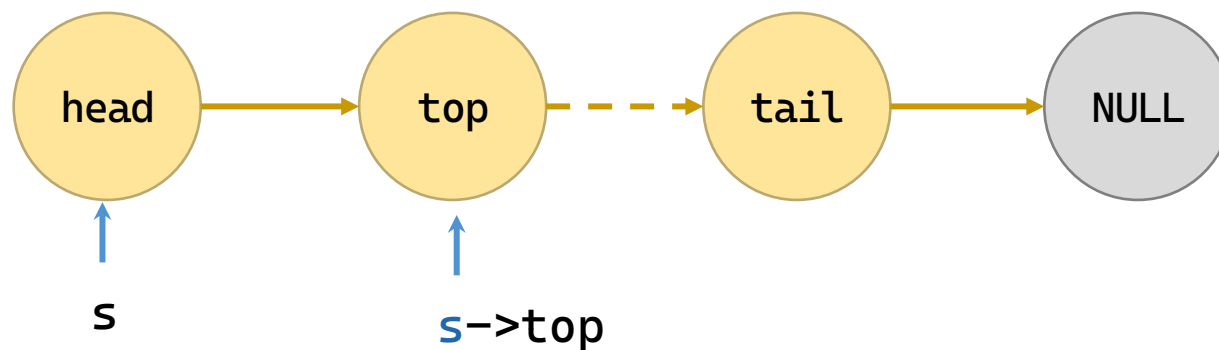
目录

	1	栈
	1.2	顺序栈
	1.3	链栈
	1.4	栈的应用
	2	队列

链栈的存储结构

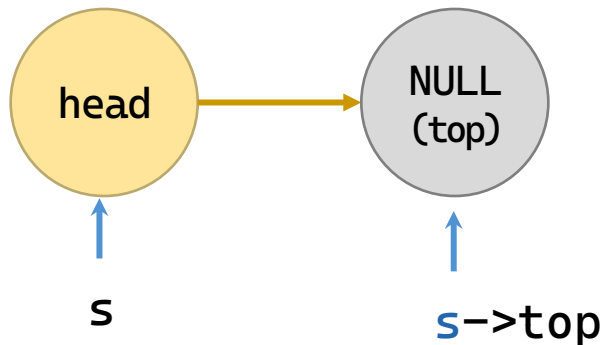
- 顺序栈的存储结构

```
typedef struct LinkNode {  
    int data;  
    struct LinkNode* next;  
} LinkNode;  
typedef LinkNode* LinkStack;
```



链栈：初始化

- 初始化：长度清零（栈顶为-1）



```
LinkStack init_stack(void) {  
    LinkNode* head = (LinkNode*)malloc(sizeof(LinkNode));  
    if (head != NULL) {  
        head->data = 0;           // 头结点数据域未使用  
        head->next = NULL;  
    }  
    return head;                  // 返回头结点指针，失败则返回 NULL  
}
```

链栈：判断空

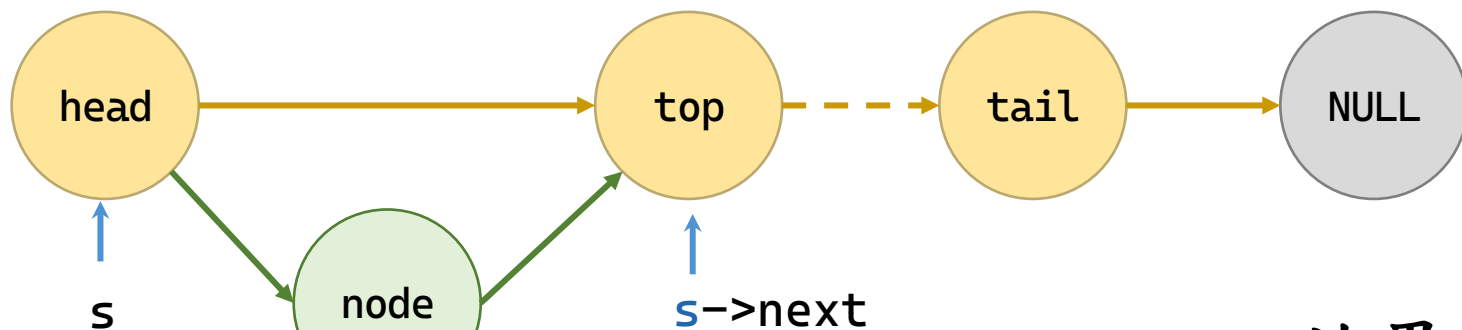
- 判断空：栈顶是否小于0
- 判断满：内部没有办法申请新的空间即是满

```
int is_empty(LinkStack stack) {  
    if (stack == NULL)  
        return ERROR;           // 栈未初始化  
    return (stack->next == NULL);  
}
```

链表：压入栈

• 入栈：赋值到顶部

主体步骤



`stack->next = node;`

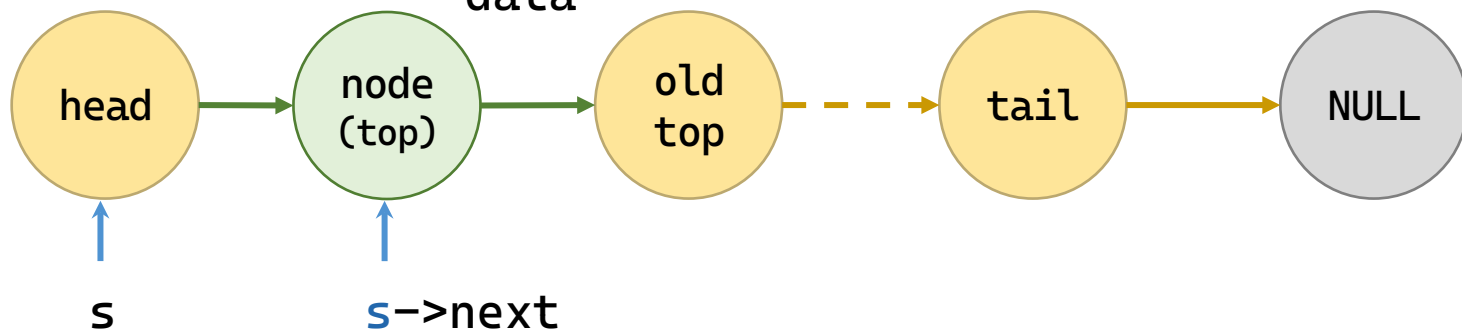
`node->data = data;`
`node->next = stack->next;`

4

data

边界判定

1. 栈指针空
2. 栈满

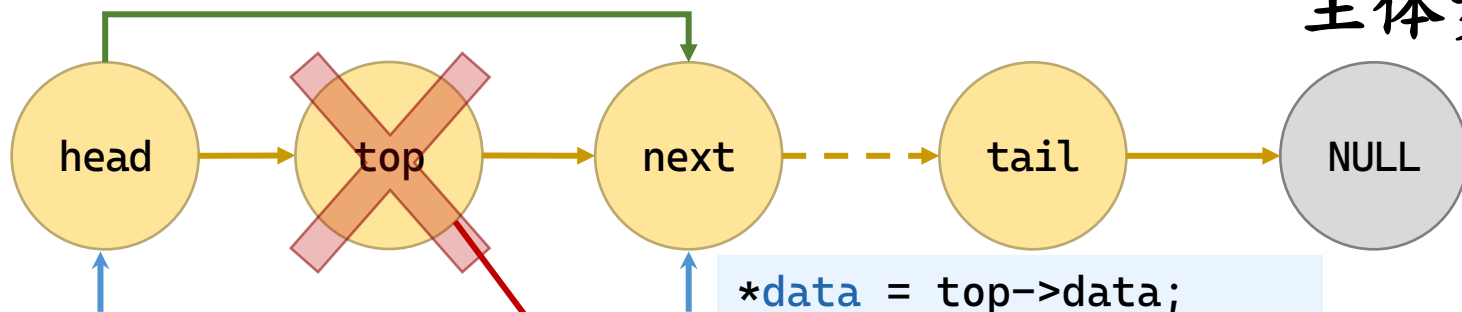


```
int push(LinkStack stack, int data) {  
    if (stack == NULL)  
        return ERROR;  
    LinkNode* node = (LinkNode*)malloc(sizeof(LinkNode));  
    if (node == NULL)  
        return ERROR;  
    node->data = data;  
    node->next = stack->next;  
    stack->next = node;  
    return OK;  
}
```

链表：弹出栈和取栈顶

- 出栈：自顶部赋值并前进；取栈顶：取值后不动

主体步骤



```
top = stack->next;  
stack->next = top->next;
```

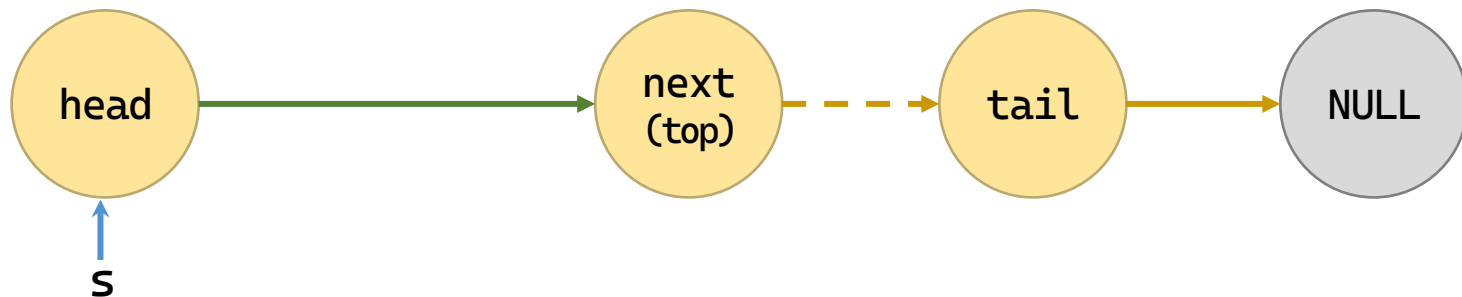
top->next

4

data

边界判定

1. 栈指针空
2. 栈空
3. 出指针空



```
int pop(LinkStack stack, int* data) {  
    if (stack == NULL)  
        return ERROR;  
    LinkNode* top = stack->next;  
    if (top == NULL)  
        return ERROR;  
    *data = top->data;  
    stack->next = top->next;  
    free(top);  
    return OK;  
}
```

```
int get_top(LinkStack stack, int* data) {  
    if (stack == NULL)  
        return ERROR;  
    LinkNode* top = stack->next;  
    if (top == NULL)  
        return ERROR;  
    *data = top->data;  
    return OK;  
}
```


目录

	1	栈
	1.2	顺序栈
	1.3	链栈
	1.4	栈的应用
	2	队列

栈应用：进制转换

- 例2-8 输入10进制数n，输出d进制数。

– 原理： $n = (n \text{ div } d) \times d + n \text{ mod } d$

- 试算： $(1348)_{10} = (2504)_8$ ，运算过程：

$$1348 \div 8 = 168 \dots\dots 4$$

$$168 \div 8 = 21 \dots\dots 0$$

$$21 \div 8 = 2 \dots\dots 5$$

$$2 \div 8 = 0 \dots\dots 2$$

- 逆序输出

```
int temp = n;
do {
    push(stack, temp % d);
    temp /= d;
} while (temp != 0);
```

```
while (!is_empty(stack)) {
    pop(stack, &digit);
    if (digit < 10)
        printf("%d", digit);
    else
        printf("%c", 'A'+digit-10);
}
```

```
void convert_base(void) {
    int n, d, digit;
    LinkStack stack = init_stack();    // 创建临时栈
    if (stack == NULL) {
        printf("栈初始化失败! \n");
        return;
    }

    printf("请输入十进制整数 n: ");
    scanf("%d", &n);
    printf("请输入目标进制 d (2~16) : ");
    scanf("%d", &d);

    if (d < 2 || d > 16) {
        printf("进制输入无效, 请输入2~16之间的整数. \n");
        free(stack);
        return;
    }
}
```

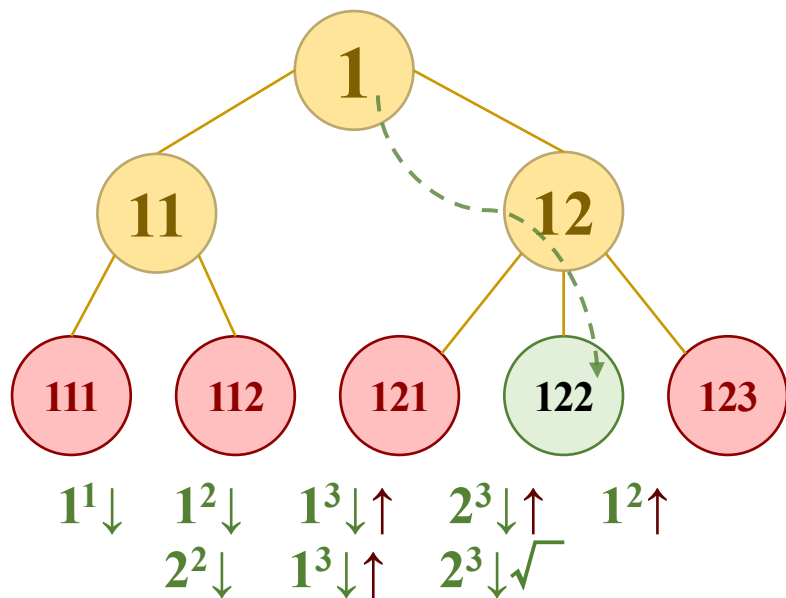
```
int temp = n;
do {
    push(stack, temp % d);
    temp /= d;
} while (temp != 0);

printf("十进制 %d 转换为 %d 进制为: ", n, d);
while (!is_empty(stack)) {
    pop(stack, &digit);
    if (digit < 10)
        printf("%d", digit);
    else
        printf("%c", 'A' + digit - 10);
}
printf("\n");

free(stack);
}
```

栈应用：迷宫

- 例2-9 假设迷宫由m行n列构成，有一个入口(1, 1)和一个出口(m, n)。试找出一条从入口通往出口的可行路径（如：输出可通行路径方格的坐标序列）。
- 思路：深度优先搜索



	1	2	3	4	5	6	7	8	
1	→	→				*			1
2		↓	→	→		*			2
3	*	*	←	↓	*				3
4		←	↓	*	↑	→	→		4
5		↓	*		↑	*	↓	*	5
6		↓	*	↑	→	*	↓	→	6
7		↓	→	→	*	*		↓	7
	1	2	3	4	5	6	7	8	

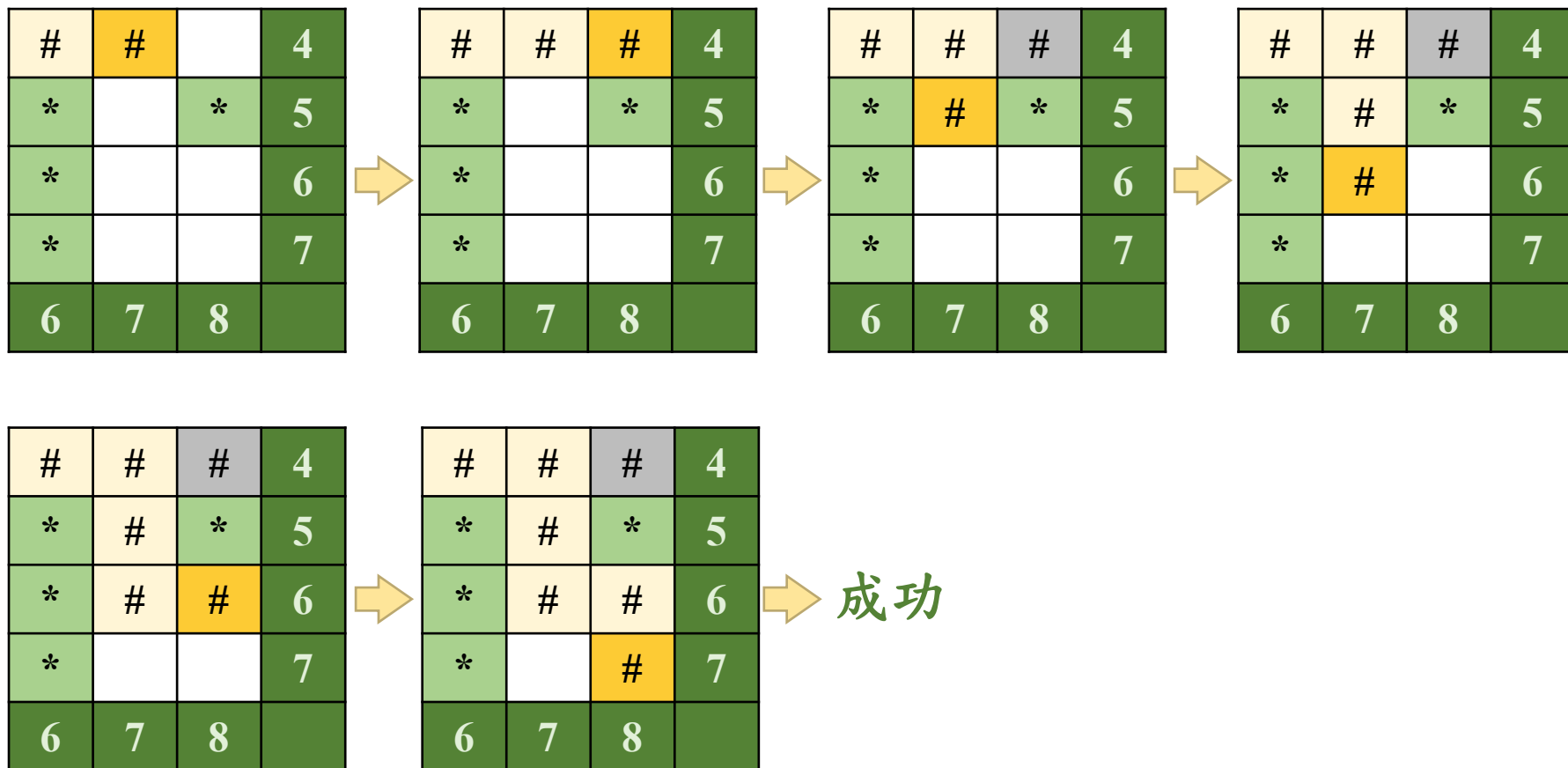
栈应用：迷宫

- 思路：

- 从入口开始。
- 如果当前位置有通行路径，纳入可行路径（进栈），按照右→下→左→上的顺序依次选择1个方向前进；
- 如果当前位置的三个前进方向均不可通行，后退，并将当前位置从可行路径中删去（出栈）；
- 如果当前位置为出口位置，搜索成功；
- 如果当前位置为入口位置，且没有可通行路径，搜索失败。

栈应用：迷宫

- 思路：数据结构：栈；算法：DFS



栈应用：迷宫

- 主体部分：按四个方向走一步，走即入栈
 - 下一步：判断已走过和走不通

```
int dirs[4][2] = { {0,1}, {1,0}, {0,-1}, {-1,0} };
push(stack, 0);
while (!is_empty(stack)) {
    int pos;
    get_top(stack, &pos);
    int x = pos % cols, y = pos / cols; // 列, 行
    for (int d = 0; d < 4; d++) {
        int nx = x + dirs[d][0], ny = y + dirs[d][1];
        if (nx >= 0 && nx < cols && ny >= 0 && ny < rows) {
            int npos = ny * cols + nx;
            push(stack, npos);
        }
    }
}
```


栈应用：迷宫

- 补充：已走过则标记上，走不通就是没动过

```
int moved = 0;
for (int d = 0; d < 4; d++) {
    int nx = x + dirs[d][0], ny = y + dirs[d][1];
    if (nx >= 0 && nx < cols && ny >= 0 && ny < rows) {
        int npos = ny * cols + nx;
        if (maze[npos] == 0 && !visited[npos]) {
            push(stack, npos);
            visited[npos] = 1;
            moved = 1;
            break;
        }
    }
}
if (!moved) { // 无路可走，回退
    int tmp;
    pop(stack, &tmp);
}
```

栈应用：迷宫

- 记录找到了，找到则输出（应该逆序输出）

```
if (x == cols - 1 && y == rows - 1) { // 到达出口
    found = 1;
    break;
}
```

```
if (found) {
    printf("找到路径: \n");
    while (!is_empty(stack)) {
        int p;
        pop(stack, &p);
        printf("(%d, %d) ", p % cols, p / cols);
    }
    printf("\n");
    free(rev);
} else
    printf("无法找到路径! \n");
```

```

void maze_path(void) {
    int rows, cols;
    printf("请输入迷宫行数和列数: ");
    scanf("%d %d", &rows, &cols);
    // 一维数组存储迷宫, 下标 = y*cols + x, calloc初始化为0
    int* maze = (int*)malloc(rows * cols * sizeof(int));
    int* visited = (int*)calloc(rows * cols, sizeof(int));
    if (maze == NULL || visited == NULL) {
        printf("内存分配失败! \n");
        free(maze); free(visited);
        return;
    }
    printf("请输入迷宫矩阵 (0通路, 1墙) : \n");
    for (int i = 0; i < rows * cols; i++)
        scanf("%d", &maze[i]);    // 直接按一维顺序读入
    // 方向: 右、下、左、上 (列增量, 行增量)
    int dirs[4][2] = { {0,1}, {1,0}, {0,-1}, {-1,0} };
    LinkStack stack = init_stack();
    if (stack == NULL) {
        printf("栈初始化失败! \n");
        free(maze); free(visited);
        return;
    }
}

```

```

push(stack, 0); visited[0] = 1; // 起点 (0,0), 0*cols+0=0
int found = 0;
while (!is_empty(stack)) {
    int pos; get_top(stack, &pos);
    int x = pos % cols, y = pos / cols; // 列, 行
    if (x == cols - 1 && y == rows - 1) { // 到达出口
        found = 1; break;
    }
    int moved = 0;
    for (int d = 0; d < 4; d++) {
        int nx = x + dirs[d][0], ny = y + dirs[d][1];
        if (nx >= 0 && nx < cols && ny >= 0 && ny < rows) {
            int npos = ny * cols + nx;
            if (maze[npos] == 0 && !visited[npos]) {
                push(stack, npos);
                visited[npos] = 1;
                moved = 1;
                break;
            }
        }
    }
}
if (!moved) { // 无路可走, 回退
    int tmp; pop(stack, &tmp);
}
}

```

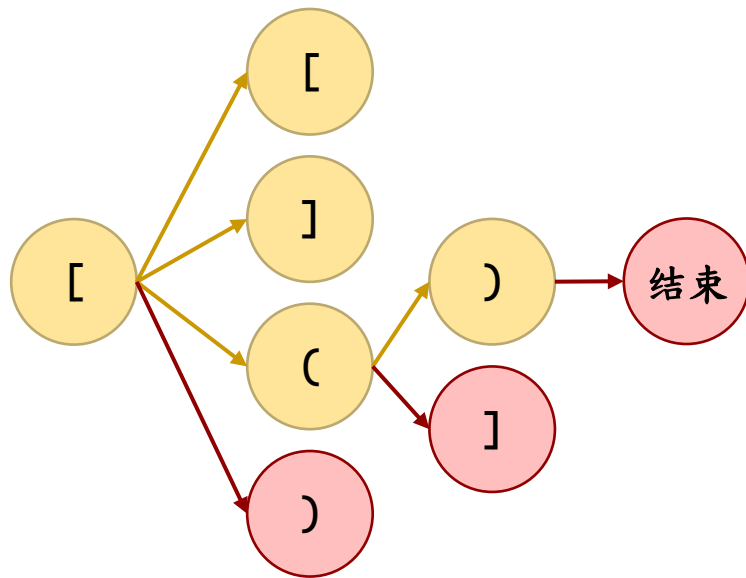
```

if (found) {
    printf("找到路径: \n");
    // 逆序输出 (用辅助栈)
    LinkStack rev = init_stack();
    while (!is_empty(stack)) {
        int p;
        pop(stack, &p);
        push(rev, p);
    }
    while (!is_empty(rev)) {
        int p; pop(rev, &p);
        printf("(%d, %d) ", p % cols, p / cols);
    }
    printf("\n");
    while (!is_empty(rev)) {
        int tmp; pop(rev, &tmp);
    }
    free(rev);
} else
    printf("无法找到路径! \n");
free(stack);
free(maze);
free(visited);
}

```

应用：括号匹配检验

- 例2-10 (括号匹配检验) 设在表达式中允许包含[]和()两种括号，约定
 - ([]()) 或 [([] [])] 这类括号匹配格式是正确的
 - [(]) 或 ([()) 或 (()]) 这类括号匹配格式是不正确的。
 - 思路
 - 左 括 号：压栈
 - 右 括 号：匹配则出栈
不匹配则失败
 - 输入结束：非空栈则失败
-
- ```
graph LR; L1["["] --> L2["["]; L1 --> L3["]"]; L1 --> L4["("]; L4 --> L5[")"]; L5 --> End((结)); L4 --> L6["]"];
```



## • 核心代码

```
for (i = 0; expr[i] != '\0'; i++) {
 char ch = expr[i];
 if (ch == '(' || ch == '[' || ch == '{') {
 push(stack, ch);
 } else if (ch == ')' || ch == ']' || ch == '}') {
 if (is_empty(stack))
 break;
 int left; get_top(stack, &left);
 if ((ch == ')' && left == '(') || (ch == ']' && left
== '[') || (ch == '}' && left == '{')) {
 int tmp; pop(stack, &tmp);
 } else
 break;
 }
}
if (expr[i] == '\0' && is_empty(stack))
 printf("括号匹配正确! \n");
else
 printf("括号匹配错误! \n");
```

```

void bracket_match(void) {
 char expr[256];
 printf("请输入包含括号的表达式: ");
 scanf("%s", expr);
 LinkStack stack = init_stack();
 if (stack == NULL) {
 printf("栈初始化失败! \n");
 return;
 }
 int i = 0;
 for (i = 0; expr[i] != '\0'; i++) {
 char ch = expr[i];
 if (ch == '(' || ch == '[' || ch == '{') {
 push(stack, ch); // 将字符 ASCII 码压栈
 } else if (ch == ')' || ch == ']' || ch == '}') {
 if (is_empty(stack))
 break;
 int topChar;
 get_top(stack, &topChar);
 char left = (char)topChar;

```



```

 if ((ch == ')' && left == '(') ||
 (ch == ']' && left == '[') ||
 (ch == '}' && left == '{')) {
 int tmp;
 pop(stack, &tmp);
 } else
 break;
 } // 忽略其他字符
}
if (expr[i] == '\0' && is_empty(stack)) {
 printf("括号匹配正确! \n");
} else {
 printf("括号匹配错误! \n");
}
while (!is_empty(stack)) { // 释放栈
 int tmp;
 pop(stack, &tmp);
}
free(stack);
}

```

# 应用：算术表达式求值

- 例2-11 （ 算术表达式求值 ） 设有
  - 符号集： $\{ 0, 1, \dots, 9, ., +, -, *, /, (, ), \# \}$
  - 运算规则（ $\#$ 为表达式的结束标识）：
    - (1)先计算最内层括号内的表达式；
    - (2)先 $*$ 、 $/$ ，后 $+$ 、 $-$ ；
    - (3)从左至右计算 $*$ 和 $/$ 或者 $+$ 和 $-$ 。
- 思路
  - 先把中缀表达式转换为后缀表达式
  - 对后缀表达式求值

# 应用：算术表达式求值

- 步骤1：先把中缀表达式转换为后缀表达式

式： $10+(18+9*3)/15-6 \rightarrow 10,18,9,3,*,+,15,/,6,-$

| 读 | 处理 | 栈    | 后缀表达式      |
|---|----|------|------------|
| 1 | 存  |      | 1          |
| 0 | 存  |      | 10,        |
| + | 入  | +    | 10,        |
| ( | 入  | +(   | 10,        |
| 1 | 存  | +(   | 10,1       |
| 8 | 存  | +(   | 10,18,     |
| + | 存  | +(+  | 10,18,     |
| 9 | 存  | +(+  | 10,18,9,   |
| * | 入  | +(+* | 10,18,9,   |
| 3 | 存  | +(+* | 10,18,9,3, |

| 读 | 处理 | 栈  | 后缀表达式                   |
|---|----|----|-------------------------|
| b | 出  | s  | 10,18,9,3,*,            |
|   | 出  | +( | 10,18,9,3,*,+,          |
|   | 出  | +  | 10,18,9,3,*,+,          |
| / | 入  | +/ | 10,18,9,3,*,+,          |
| 1 | 存  | +/ | 10,18,9,3,*,+,1         |
| 5 | 存  | +/ | 10,18,9,3,*,+,15,       |
| - | 出  | +  | 10,18,9,3,*,+,15,/,     |
|   | 出  |    | 10,18,9,3,*,+,15,/,+,   |
|   | 入  | -  | 10,18,9,3,*,+,15,/,+,   |
| 6 | 存  | -  | 10,18,9,3,*,+,15,/,+,6, |

# 应用：算术表达式求值

- 步骤1：先把中缀表达式转换为后缀表达式

式： $10+(18+9*3)/15-6 \rightarrow 10, 18, 9, 3, *, +, 15, /, 6, -$

— 规矩：数字入串，优先级低则入栈，高则出栈入串

- 遇到数字，则存入后缀串
- 遇到左括号，则入栈
- 遇到运算符且栈顶优先级更低，则入栈
- 遇到右括号，则出栈，存入后缀串
- 遇到运算符且栈顶优先级更高，则出栈，存入后缀串
- 遇到输入结束，则出栈，存入后缀串

```

static int priority(char op) {
 switch (op) {
 case '+': case '-': return 1;
 case '*': case '/': return 2;
 default: return 0;
 }
}

void infix_to_postfix(const char* infix, char* postfix) {
 LinkStack ops = init_stack();
 if (!ops) return;
 int i = 0, j = 0;
 while (infix[i] != '#' && infix[i] != '\0') {
 char ch = infix[i];
 if (ch >= '0' && ch <= '9') {
 while (ch >= '0' && ch <= '9') {
 postfix[j++] = ch;
 i++;
 ch = infix[i];
 }
 postfix[j++] = ' ';
 continue;
 }
 }
}

```

```

} else if (ch == '(') {
 push(ops, ch);
} else if (ch == ')') {
 int top;
 get_top(ops, &top);
 while ((char)top != '(') {
 postfix[j++] = (char)top;
 postfix[j++] = ' ';
 pop(ops, &top);
 get_top(ops, &top);
 }
 pop(ops, &top); // 弹出 '('
} else if (ch == '+' || ch == '-' || ch == '*' || ch == '/') {
 int top;
 get_top(ops, &top);
 while (!is_empty(ops) && (char)top != '('
 && priority((char)top) >= priority(ch)) {
 postfix[j++] = (char)top;
 postfix[j++] = ' ';
 pop(ops, &top);
 get_top(ops, &top);
 }
}

```

```

 }
 push(ops, ch);
 }
 i++;
}
while (!is_empty(ops)) {
 int top;
 pop(ops, &top);
 postfix[j++] = (char)top;
 postfix[j++] = ' ';
}
postfix[j] = '\0';
while (!is_empty(ops)) {
 int tmp; pop(ops, &tmp);
}
free(ops);
}

```

# 应用：算术表达式求值

## • 步骤2：后缀表达式求值

式：10, 18, 9, 3, \*, +, 15, /, +, 6, -

式：10, 18, 27, +, 15, /, +, 6, -

式：10, 45, 15, /, +, 6, -

式：10, 3, +, 6, -

式：13, 6, -

式：7

— 规矩：

- 遇到数字，则入栈
- 遇到运算符，则两次出栈，得到的两个运算数与运算符进行运算



```

int eval_postfix(const char* postfix) {
 LinkStack nums = init_stack();
 if (!nums) return ERROR;
 int i = 0, error = 0;
 while (postfix[i] != '\0' && !error) {
 if (postfix[i] >= '0' && postfix[i] <= '9') {
 int num = 0;
 while (postfix[i] >= '0' && postfix[i] <= '9') {
 num = num * 10 + (postfix[i] - '0');
 i++;
 }
 push(nums, num);
 } else if (postfix[i] == '+' || postfix[i] == '-'
 || postfix[i] == '*' || postfix[i] == '/') {
 int b, a, res;
 pop(nums, &b);
 pop(nums, &a);
 switch (postfix[i]) {
 case '+': res = a + b; break;
 case '-': res = a - b; break;
 case '*': res = a * b; break;
 case '/':
 if (b == 0) {
 printf("错误: 除零! \n"); error = 1; res = 0;
 }
 }
 }
 i++;
 }
 return res;
}

```

```

 } else
 res = a / b;
 break;
 }
 if (!error) push(nums, res);
}
i++;
}
int result;
if (!error && get_top(nums, &result) == OK) {
 while (!is_empty(nums)) {
 int tmp; pop(nums, &tmp);
 }
 free(nums);
 return result;
} else {
 while (!is_empty(nums)) {
 int tmp; pop(nums, &tmp);
 }
 free(nums);
 return ERROR;
}
}

```

# 应用：递归

- 递归函数：一个利用自身定义的函数。
- 递归算法
  - 将待求解问题 $F(n)$ 进行分解
  - 转化为求解相对简单的子问题，如 $F(n-1)$
  - 其中一些子问题比较容易得到解，如 $F(1)$ ，称为递归出口。
  - 而其它子问题可以用同样方法进行求解，但问题规模较小。
- 一个递归算法应具有下列特点
  - (1)递归出口至少一个。否则，算法将永远不会有结果
  - (2)在经过有限次的递归调用后，能够导致递归出口的出现。

# 应用：递归（Ackerman函数）

- **例1-12** Ackerman函数。它是一种双递归函数。

$$\text{Ack}(m, n) = \begin{cases} n+1, & m=0 \\ \text{Ack}(m-1, 1), & m>0 \text{ 且 } n=0 \\ \text{Ack}(m-1, \text{Ack}(m, n-1)), & m>0 \text{ 且 } n>0 \end{cases}$$

```
int ack(int m, int n) {
 if (m == 0) {
 return n + 1;
 } else if (n == 0) {
 return ack(m - 1, 1);
 } else {
 return ack(m - 1, ack(m, n - 1));
 }
}
```

# 应用：递归（Ackerman函数）

## • 代码分析

- 栈溢出风险：Ackerman函数的递归深度极深。
  - 仅仅计算  $A(4, 1)$  就会产生极其庞大的递归调用树，这会导致程序直接因为内存耗尽（栈溢出）而崩溃。建议  $m$  的值不要超过 3。
- 时间复杂度：计算时间随着  $m$  和  $n$  的增加呈爆炸式增长。
  - $A(3, 2)$  的计算已经需要几十次递归调用，而  $A(3, 3)$  的结果是 61，但计算过程却极其漫长。
- 记忆化优化：计算过程中会产生大量重复的子问题，可以通过引入二维数组作为缓存表来存储已经计算过的结果，从而大幅减少重复计算的时间。

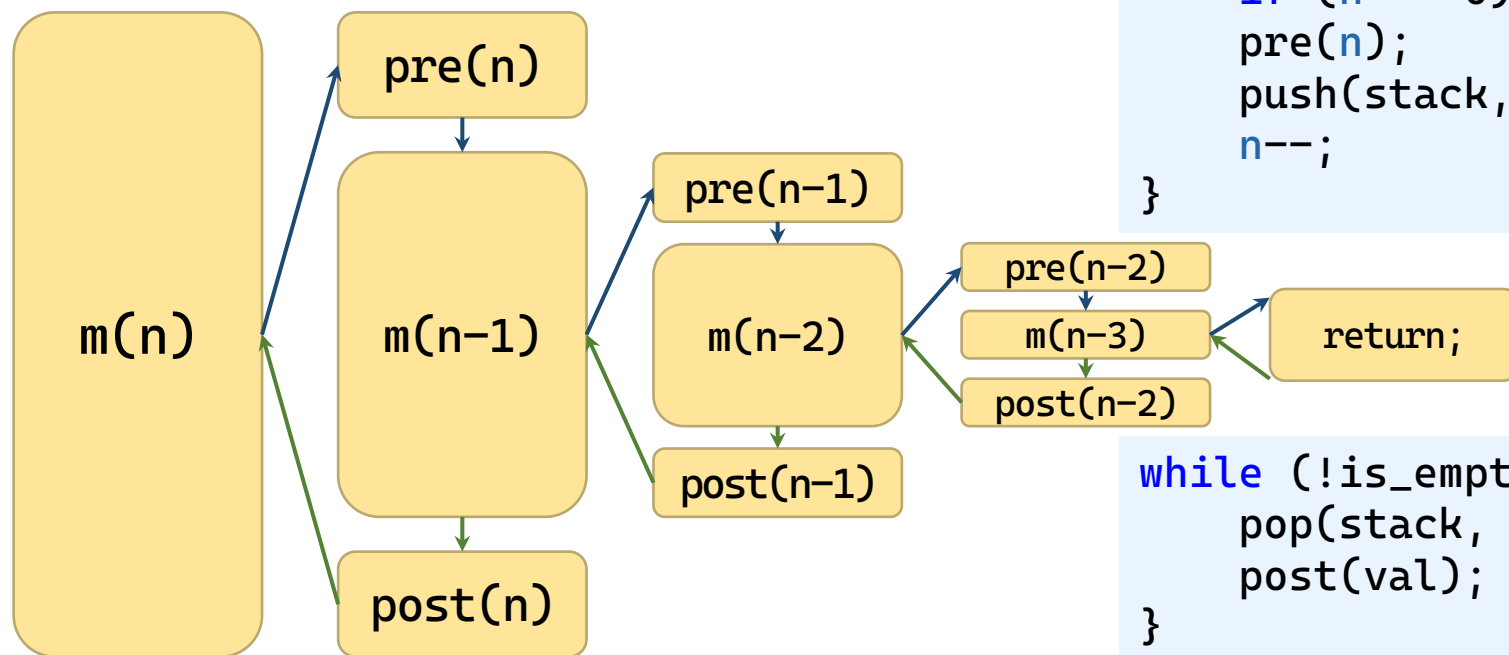
# 递归的栈实现

- 进入递归时，系统的操作
  - 保存当前状态入栈：参数值、局部变量、当前执行位置
  - 进入新一轮函数调用
- 退出递归时，系统的操作
  - 获取返回值作为被调用函数的值
  - 恢复栈顶的状态，继续执行主调用函数
- 问题：系统维护的栈通常只有1MB大小
- 如果需要栈超过系统限制，需要自己改用栈实现

# 递归的栈实现

## • 原递归

```
void m(int n) {
 if (n == 0) return; // 出口
 pre(n);
 m(n - 1);
 post(n);
}
```



```
while (1) {
 if (n == 0) break; // 出口
 pre(n);
 push(stack, n);
 n--;
}
```

```
while (!is_empty(stack)) {
 pop(stack, &val);
 post(val);
}
```

```

void iterative_m(int n) {
 LinkStack stack = init_stack();
 if (!stack) return;
 // 阶段1: 模拟递归进入
 while (1) {
 if (n == 0) break; // 递归出口, 退出进入阶段
 pre(n);
 push(stack, n);
 n--;
 }
 // 阶段2: 模拟递归返回
 int val;
 while (!is_empty(stack)) {
 pop(stack, &val);
 post(val);
 }
 free(stack);
}

```



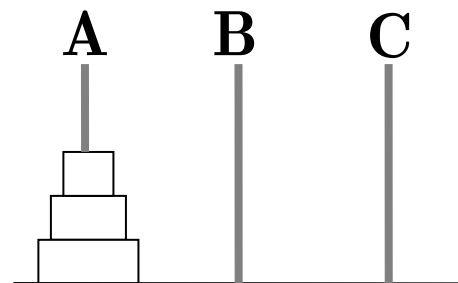
```

int ackermann_with_stack(int m, int n) {
 LinkStack stack = init_stack();
 if (stack == NULL) return ERROR; // 栈初始化失败
 while (m > 0) {
 if (push(stack, m) == ERROR) return ERROR;
 m--;
 }
 int result = n + 1; // A(0, n) = n + 1
 int cur_m;
 while (!is_empty(stack)) { // 不断处理栈中的 m
 if (pop(stack, &cur_m) == ERROR) return ERROR;
 if (cur_m == 1) {
 result = result + 1;
 }
 else {
 if (push(stack, cur_m-1) == ERROR) return ERROR;
 result = 1;
 }
 }
 return result;
}

```

# 应用：Hanoi塔

- 例1-13 Hanoi塔问题：已知有3个塔座A, B, C和 $n$ 个圆盘（直径 $d_i$ 满足 $d_1 < d_2 < \dots < d_n$ ）。假设已经从小到大依次将这些圆盘叠放在塔座x上。要求按照下列规则，将塔座x上的 $n$ 个圆盘都移动到塔座z上：
  - (1) 一次只能移动一个圆盘；
  - (2) 圆盘可以临时叠放在x, y或z塔座上；
  - (3) 任一次移动后，任一塔座上都不能出现大圆盘在小圆盘之上的现象。



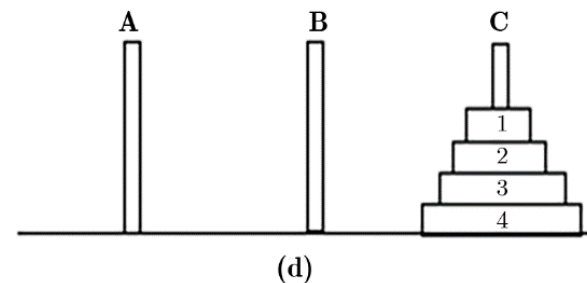
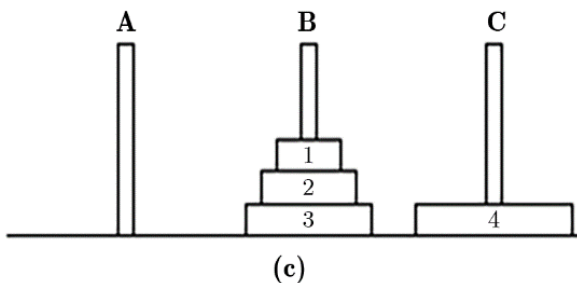
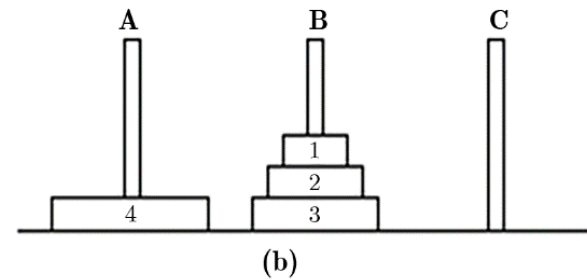
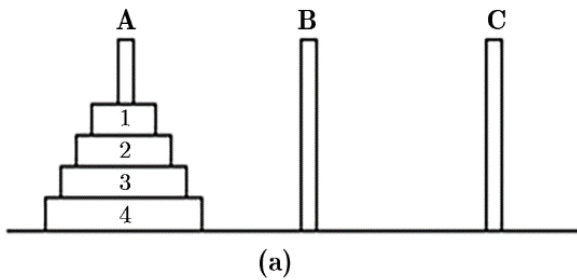
# 应用：Hanoi塔

## • 汉诺塔

–  $1 \sim n-1$  :  $A \rightarrow B$

–  $n$  :  $A \rightarrow C$

–  $1 \sim n-1$  :  $B \rightarrow C$



```
void hanoi(int n, char from, char to, char aux) {
 if (n == 0) return;
 hanoi(n - 1, from, aux, to);
 move_count++;
 printf("%d: %c --> %c\n", n, from, to);
 hanoi(n - 1, aux, to, from);
}
```

```

void hanoi_stack(int n, char from, char to, char aux) {
 LinkStack stack = init_stack();
 if (!stack) {
 printf("栈初始化失败! \n");
 return;
 }
 push(stack, (HanoiParam) { n, from, to, aux, 0 }); // 初始
 while (!is_empty(stack)) {
 HanoiParam f;
 pop(stack, &f);
 if (f.n == 0) continue; // 出口
 if (f.stage == 0) {
 f.stage = 1; // 当前帧 stage 设为 1
 push(stack, f); // 重新压栈一遍激发第二个递归
 push(stack, (HanoiParam) { f.n - 1, f.from, f.aux,
 f.to, 0 }); // 第1递归 hanoi(n-1, from, aux, to)
 } else { // stage == 1
 printf("%d: %c --> %c\n", f.n, f.from, f.to);
 push(stack, (HanoiParam) { f.n - 1, f.aux, f.to,
 f.from, 0 }); // 第2个递归 hanoi(n-1, aux, to, from)
 }
 }
 free(stack); // 释放头结点
}

```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
/* 返回值状态码 */
#define ERROR -1
#define OK 1
typedef struct {
 int n; // 盘子数
 char from; // 源柱
 char to; // 目标柱
 char aux; // 辅助柱
 int stage; // 0: 第一个递归前; 1: 第二个递归前
} HanoiParam;
typedef struct LinkNode {
 HanoiParam data;
 struct LinkNode* next;
} LinkNode; // 链式栈结点
typedef LinkNode* LinkStack; // 栈类型: 指向头结点的指针
int is_empty(LinkStack stack) {
 if (stack == NULL) return ERROR; // 栈未初始化
 return (stack->next == NULL);
}

```

```

LinkStack init_stack(void) {
 LinkNode* head = (LinkNode*)malloc(sizeof(LinkNode));
 if (head != NULL)
 memset(head, 0, sizeof(*head));
 return head; // 返回头结点指针, 失败则返回 NULL
}

int push(LinkStack stack, HanoiParam data) {
 if (stack == NULL) return ERROR; // 栈未初始化
 LinkNode* newNode = (LinkNode*)malloc(sizeof(LinkNode));
 if (newNode == NULL) return ERROR; // 内存分配失败
 newNode->data = data;
 newNode->next = stack->next; // 新结点指向原栈顶
 stack->next = newNode; // 头结点指向新栈顶
 return OK;
}

int pop(LinkStack stack, HanoiParam* data) {
 if (stack == NULL) return ERROR;
 LinkNode* topNode = stack->next; // 获取栈顶结点
 if (topNode == NULL) return ERROR; // 栈空
 *data = topNode->data; // 返回数据
 stack->next = topNode->next; // 头结点跳过栈顶
 free(topNode); // 释放原栈顶
}

```

```

 return OK;
}
int get_top(LinkStack stack, HanoiParam* data) {
 if (stack == NULL) return ERROR;
 LinkNode* topNode = stack->next;
 if (topNode == NULL) return ERROR;
 *data = topNode->data;
 return OK;
}
int main() {
 int n;
 printf("请输入汉诺塔的盘子数量: ");
 if (scanf("%d", &n) != 1 || n <= 0) {
 printf("输入无效, 请输入正整数! \n");
 return 1;
 }
 printf("汉诺塔 %d 层移动步骤: \n", n);
 hanoi_stack(n, 'A', 'C', 'B');
 return 0;
}

```

# 应用：背包问题

- 例1-14（背包问题） 设有  $n$  个物品，每个物品的重量为  $w_i$ ,  $i=1, 2, \dots, n$ 。试从这  $n$  个物品中选取若干个，使其重量之和 = 背包的容量  $T$ 。
- 例：  $n=5$ ，  $W=\{17, 51, 28, 32, 63\}$ ，  $T=100$ 。
- 选择  $w_1 + w_2 + w_4 = 17 + 51 + 32 = 100$
- 思路：DFS
  - 如果  $S \subseteq W = \{w_i\}$  是  $(W, T)$  的可行解
  - 对于任何  $s_i \in S$ ，  $S \setminus \{s_i\}$  也是  $(W \setminus \{s_i\}, T - s_i)$  可行解
  - 对于任何  $s_i \notin S$ ，  $S \cup \{s_i\}$  也是  $(W \cup \{s_i\}, T + s_i)$  可行解



```

#include <stdio.h>
#include <stdlib.h>
#define MAX_N 100
int w[MAX_N]; // 物品重量数组
int selected[MAX_N]; // 标记选中物品 (1选中, 0未选)
int n, T; // 物品个数, 背包容量

int knap_rec(int i, int sum) {
 if (sum == T) return 1; // 出口, 找到一组可行解
 if (i == n || sum > T) return 0; // 无解
 selected[i] = 1; // 尝试选第 i 个物品
 if (knap_rec(i + 1, sum + w[i]))
 return 1; // 有选可行, 则传到至上层
 selected[i] = 0; // 尝试“不选”第 i 个物品
 if (knap_rec(i + 1, sum))
 return 1; // 不选可行, 则传到至上层
 return 0; // 都不可行
}

```

```
int main() {
 printf("请输入物品个数 n: "); scanf("%d", &n);
 printf("请输入背包容量 T: "); scanf("%d", &T);
 printf("请输入 %d 个物品的重量: ", n);
 for (int i = 0; i < n; i++)
 scanf("%d", &w[i]);
 if (knap_rec(0, 0)) {
 printf("找到一组解: ");
 for (int i = 0; i < n; i++) {
 if (selected[i])
 printf("%d ", w[i]);
 }
 printf("= %d\n", T);
 } else {
 printf("无解! \n");
 }
 return 0;
}
```

```

typedef struct {
 int i; // 当前物品下标
 int sum; // 当前已选重量
 int stage; // 0: 刚进入, 准备尝试选; 1: 选已尝试, 准备尝试不选
} Param;

typedef struct LinkNode {
 Param data;
 struct LinkNode* next;
} LinkNode;

typedef LinkNode* LinkStack;

#define MAX_N 100
int w[MAX_N]; // 物品重量数组
int selected[MAX_N]; // 标记选中物品 (1选中, 0未选)
int n, T; // 物品个数, 背包容量

int knap_stack(void) {
 LinkStack stack = init_stack();
 if (stack == NULL) {
 printf("栈初始化失败! \n");
 return 0;
 }
 push(stack, (Param) { 0, 0, 0 });
}

```

```

while (!is_empty(stack)) {
 Param f; pop(stack, &f);
 if (f.sum == T) { // 递归出口: 找到解
 while (!is_empty(stack)) {
 Param tmp; pop(stack, &tmp);
 }
 free(stack);
 return 1;
 }
 if (f.i == n || f.sum > T)
 continue; // 无解: 物品用完或超重
 if (f.stage == 0) { // 已在待选阶段
 // 压栈, 以备稍后处理“不选”
 push(stack, (Param) { f.i, f.sum, 1 });
 selected[f.i] = 1; // 标记选中
 push(stack, (Param) { f.i + 1, f.sum + w[f.i], 0 });
 } else { // 尝试“不选”当前物品
 selected[f.i] = 0; // 标记不选
 push(stack, (Param) { f.i + 1, f.sum, 0 });
 }
}
free(stack);
return 0; // 无解
}

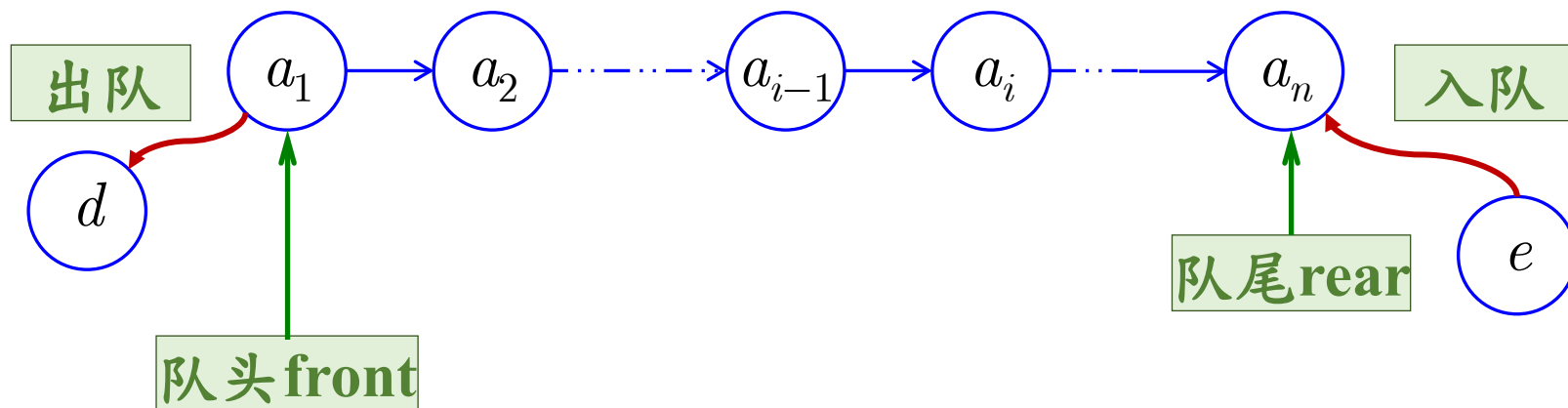
```

# 目录

|  |     |      |
|--|-----|------|
|  | 1   | 栈    |
|  | 2   | 队列   |
|  | 2.1 | 基本概念 |
|  | 2.2 | 顺序队列 |
|  | 2.3 | 链队列  |

# 基本概念

- 栈和队列都是限制在端点进行插入和删除的线性表。
- 队列是一种先进先出(FIFO)的线性表。
  - 插入数据至队列尾部顶称为入队 ( Enqueue )
  - 自队列头部读取并删除数据称为出队 ( Dequeue )
  - 当 $\text{front}=\text{rear}$ 时，队为空



# 队列的抽象数据类型定义

- 定义

ADT Queue {

数据对象： $D = \{ a_i \mid a_i \in \text{ElemSet}, i=1,2,\dots,n, n \geq 0 \}$

数据关系：

$R = \{ \langle a_{i-1}, a_i \rangle \mid a_{i-1}, a_i \in D, i=2,3,\dots,n \}$

约定  $a_1$  端为队头， $a_n$  端为队尾。

基本操作：

初始化、进队、出队、判断队空等

} ADT Queue

# 目录

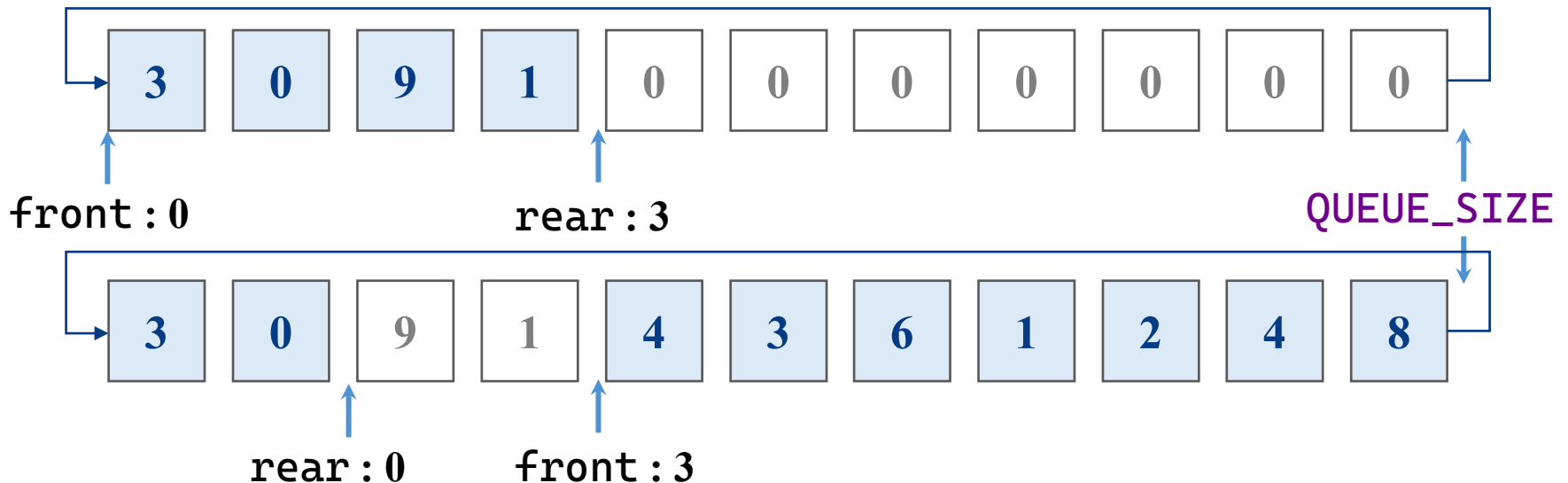
|  |     |      |
|--|-----|------|
|  | 1   | 栈    |
|  | 2   | 队列   |
|  | 2.1 | 基本概念 |
|  | 2.2 | 顺序队列 |
|  | 2.3 | 链队列  |



# 顺序队列的存储结构

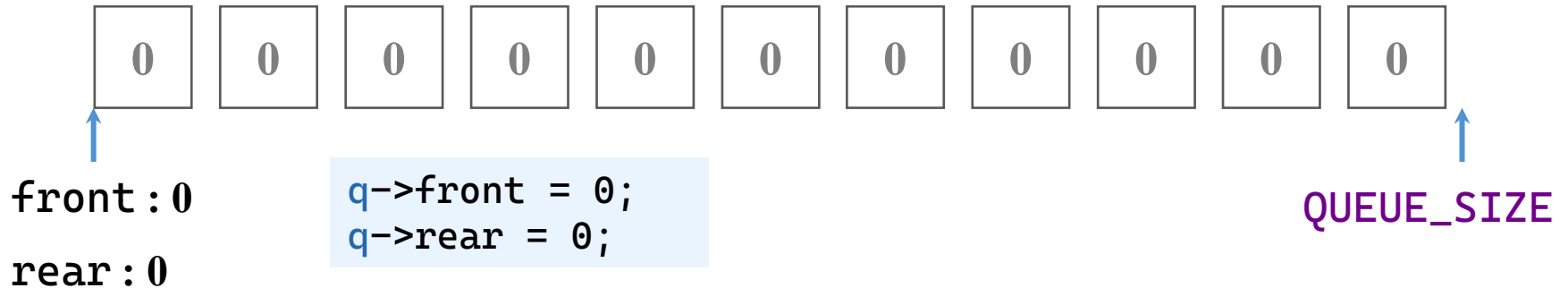
- 顺序队列（本节为循环版本）的存储结构

```
typedef struct {
 int data[QUEUE_SIZE];
 int front;
 int rear;
} SeqQueue;
```



# 顺序队列：初始化

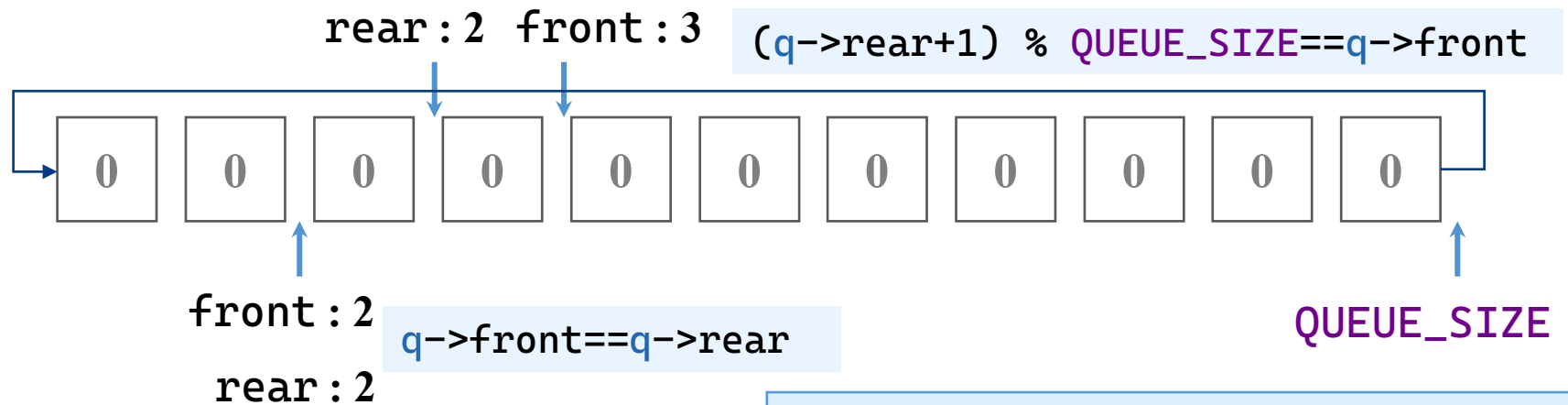
- 初始化：长度清零（栈顶为-1）



```
int init_queue(SeqQueue* q) {
 if (q == NULL)
 return ERROR;
 q->front = 0;
 q->rear = 0;
 return OK;
}
```

# 顺序队列：判断空和满

- 判断空：队列头尾是否相等
- 判断满：队列头是否达到队列尾加一



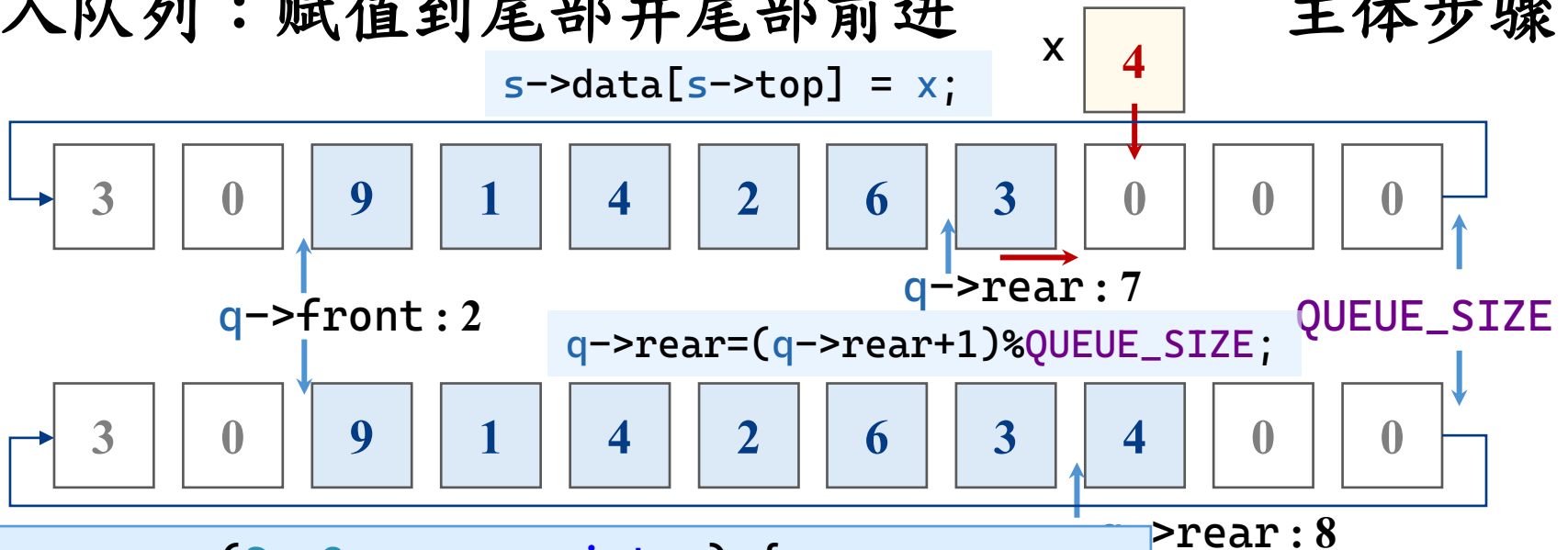
```
int is_empty(SeqQueue* q) {
 if (q == NULL)
 return 1;
 return (q->front == q->rear);
}
```

```
int is_full(const SeqQueue* q) {
 if (q == NULL)
 return 1; // 防止溢出
 return ((q->rear + 1) %
 QUEUE_SIZE == q->front);
}
```

# 顺序队列：进入队列

- 入队列：赋值到尾部并尾部前进

主体步骤



```
int enqueue(SeqQueue* q, int x) {
 if (q == NULL || is_full(q))
 return ERROR;
 q->data[q->rear] = x;
 q->rear = (q->rear + 1) % QUEUE_SIZE;
 return OK;
}
```

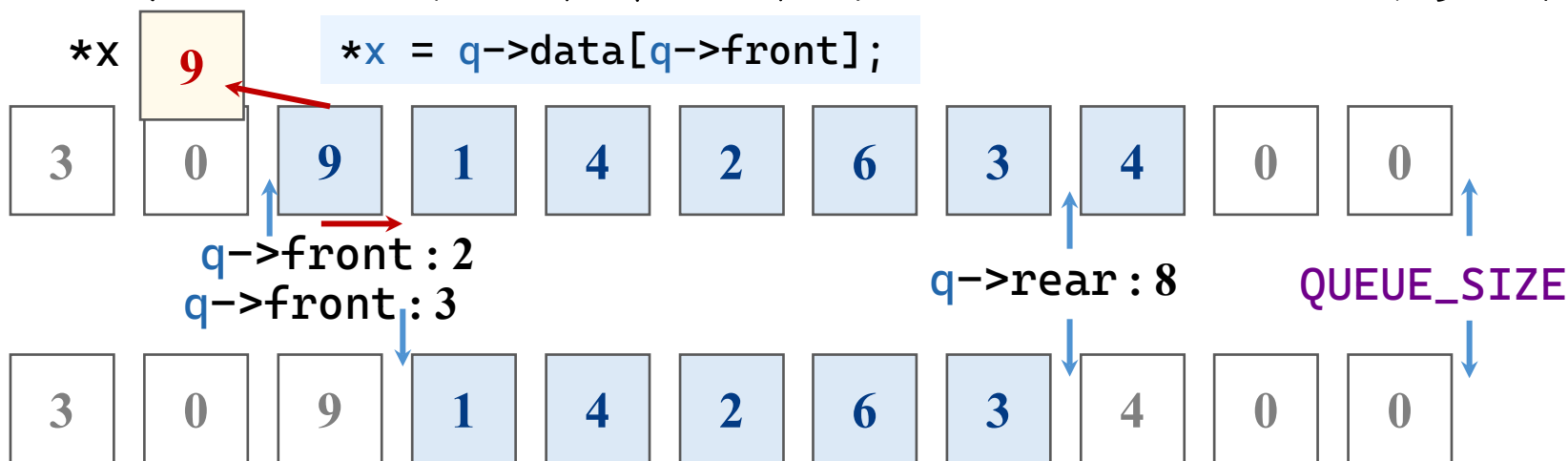
边界判定

1. 队列指针空
2. 队列满

# 顺序队列：弹出队列

- 出队列：赋值到头部并头部前进

主体步骤



```
int dequeue(SeqQueue* q, int* x) {
 if (q == NULL || is_empty(q) || x == NULL)
 return ERROR;
 *x = q->data[q->front];
 q->front = (q->front + 1) % QUEUE_SIZE;
 return OK;
}
```

## 边界判定

1. 队列指针空
2. 队列空
3. 出指针空

# 目录

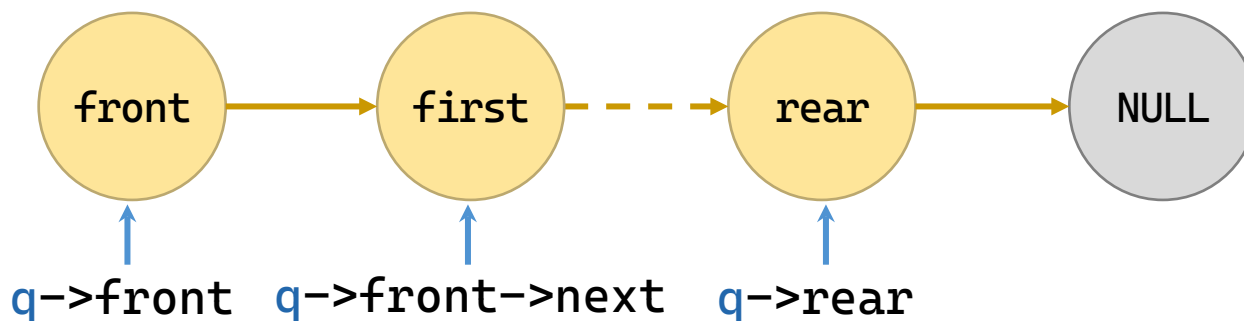
|  |     |       |
|--|-----|-------|
|  | 2   | 队列    |
|  | 2.2 | 顺序队列  |
|  | 2.3 | 链队列   |
|  | 2.4 | 队列的应用 |
|  | 3   | 小结    |

# 链队列的存储结构

- 顺序栈的存储结构

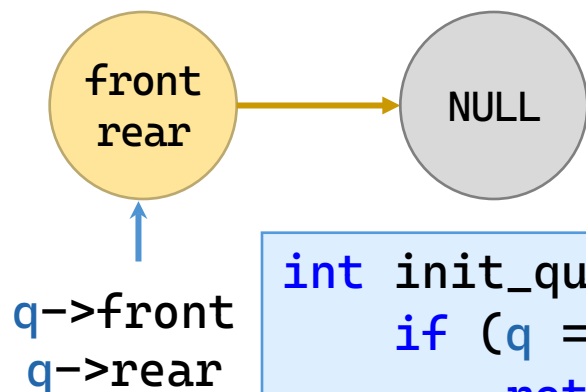
- front称呼头节点的前指针而不是第一节点，是为方便出队

```
typedef struct Node {
 int data;
 struct Node* next;
} LinkNode;
typedef struct {
 LinkNode* front; // 队头的前指针（指向头结点）
 LinkNode* rear; // 队尾指针
} LinkQueue;
```



# 链队列：初始化

- 初始化：长度清零（队列顶为-1）



```
int init_queue(LinkQueue* q) {
 if (q == NULL)
 return ERROR;
 LinkNode* head = (LinkNode*)malloc(sizeof(LinkNode));
 if (head == NULL)
 return ERROR;
 head->next = NULL;
 q->front = head;
 q->rear = head;
 return OK;
}
```



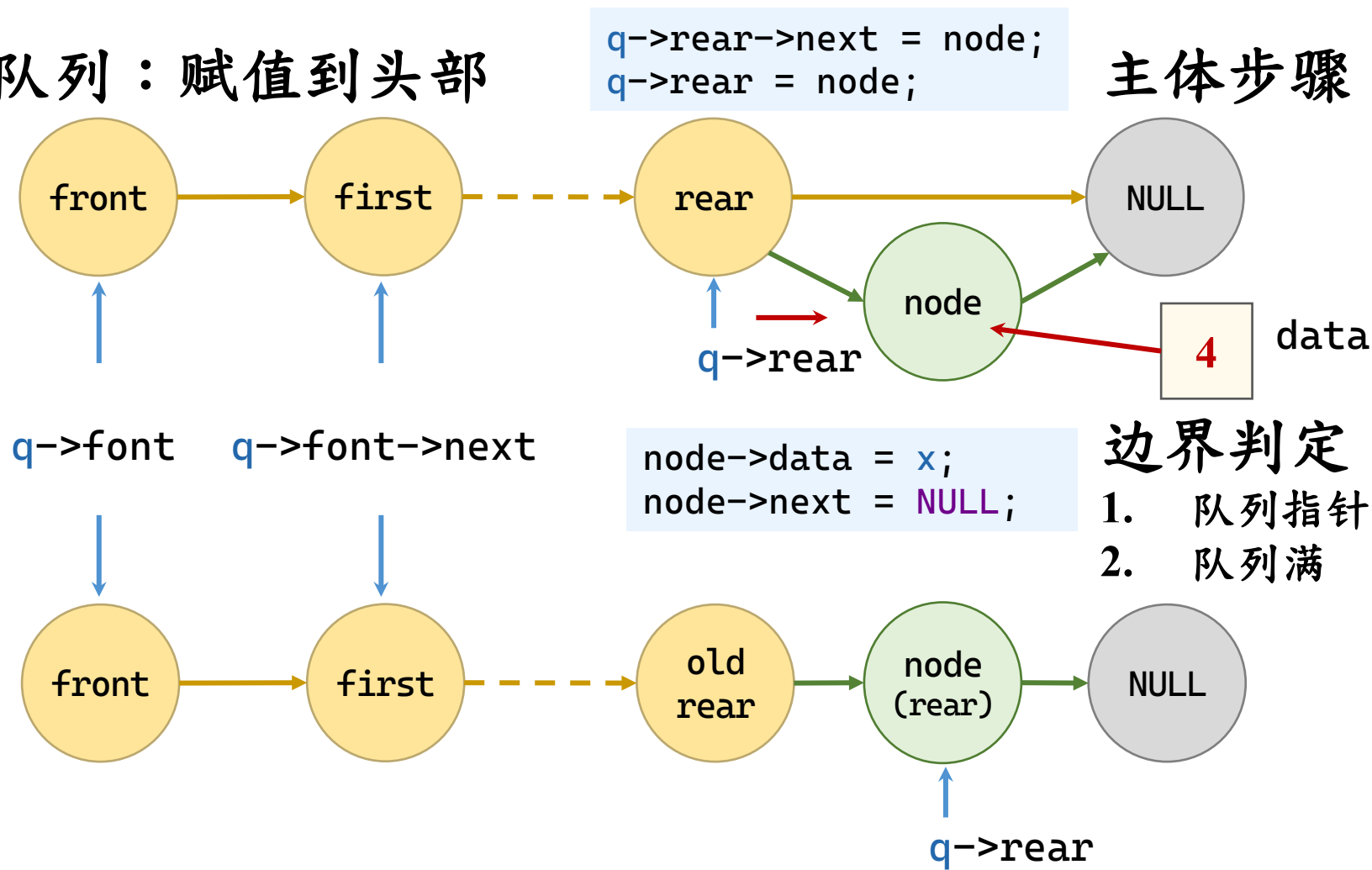
# 链队列：判断空

- 判断空：队列头尾是否相等
- 判断满：内部没有办法申请新的空间即是满

```
int is_empty(const LinkQueue* q) {
 if (q == NULL) {
 return 1;
 }
 return (q->front == q->rear);
}
```

# 链队列：入队

## • 入队列：赋值到头部

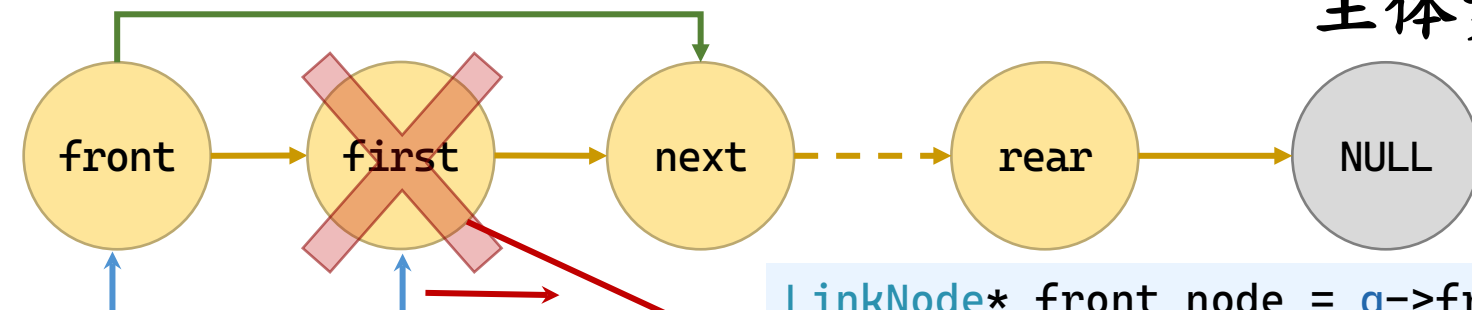


```
int enqueue(LinkQueue* q, int x) {
 if (q == NULL)
 return ERROR;
 LinkNode* node = (LinkNode*)malloc(sizeof(LinkNode));
 if (node == NULL)
 return ERROR;
 node->data = x;
 node->next = NULL;
 q->rear->next = node;
 q->rear = node;
 return OK;
}
```

# 链队列：出队

- 出队列：自顶部赋值并前进

## 主体步骤



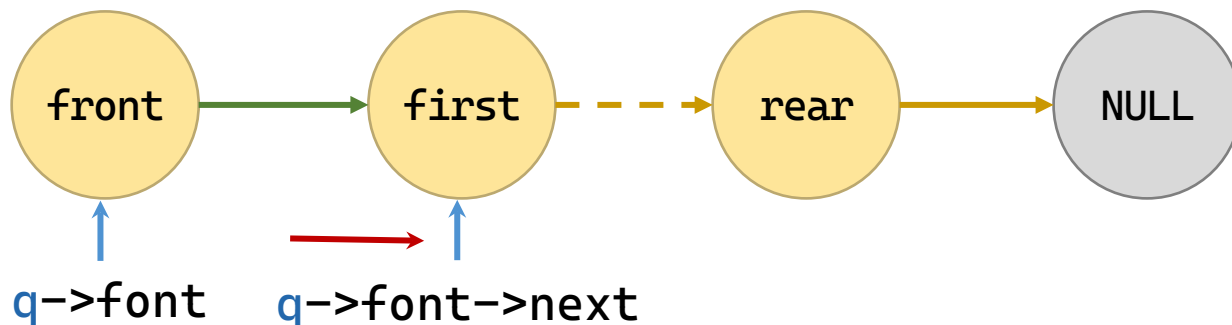
`q->font q->font->next`

```
front_node->next=first->next;
free(first);
```

```
LinkNode* front_node = q->font;
LinkNode* first = front_node->next;
*x = first->data;
```

## 边界判定

1. 队列指针空
2. 队列空
3. 出指针空

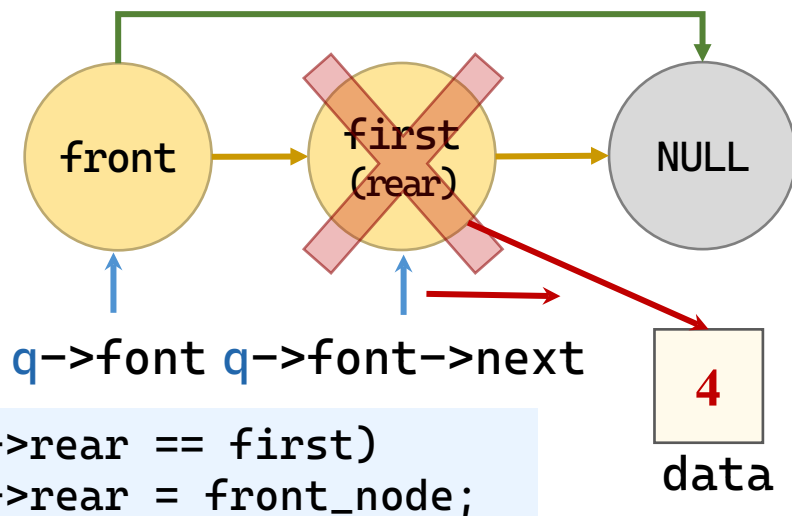


`q->font`

`q->font->next`

# 链队列：出队

- 出队列：自顶部赋值并前进



## 特殊情况

当队列只剩一个节点时，  
释放第一节点相当于释放  
尾部，此时头尾节点一致。

```
int dequeue(LinkQueue* q, int* x) {
 if (q == NULL || x == NULL || is_empty(q)) {
 return ERROR;
 }
 LinkNode* front_node = q->front; // 头结点
 LinkNode* first = front_node->next; // 第一个数据结点
 *x = first->data;
 front_node->next = first->next;
 if (q->rear == first) { // 队列只剩一个数据结点
 q->rear = front_node;
 }
 free(first);
 return OK;
}
```

```
int get_top(LinkStack stack, int* data) {
 if (stack == NULL)
 return ERROR;
 LinkNode* top = stack->next;
 if (top == NULL)
 return ERROR;
 *data = top->data;
 return OK;
}
```

# 目录

|  |     |       |
|--|-----|-------|
|  | 2   | 队列    |
|  | 2.2 | 顺序队列  |
|  | 2.3 | 链队列   |
|  | 2.4 | 队列的应用 |
|  | 3   | 小结    |



# 应用：腐烂的橘子

- LeetCode 994 ( 腐烂的橘子 ) 在给定的  $m \times n$  网格 grid 中，每个单元格可以有以下三个值之一：
  - 值 0 代表空单元格；
  - 值 1 代表新鲜橘子；
  - 值 2 代表腐烂的橘子。
- 每分钟，腐烂的橘子 周围 4 个方向上相邻 的新鲜橘子都会腐烂。
- 返回 直到单元格中没有新鲜橘子为止所必须经过的最小分钟数。如果不可能，返回 -1 。

# 应用：腐烂的橘子

## • 思路

— 将烂橘子入队，记录鲜橘子个数

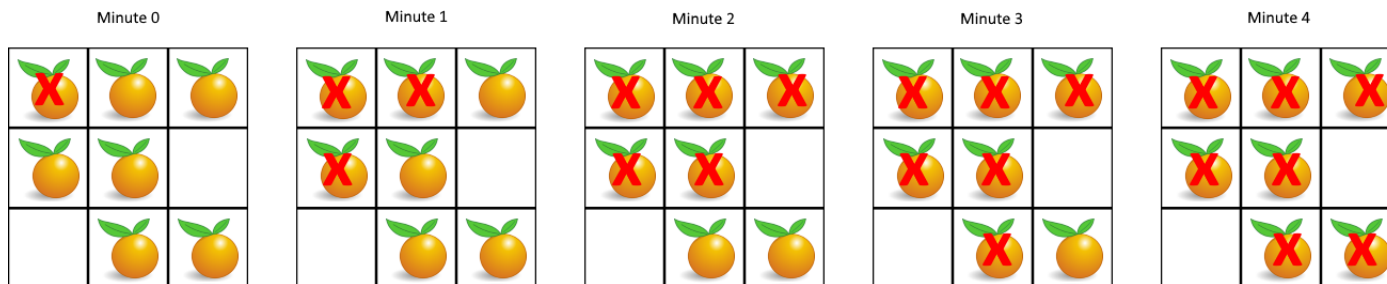
— 依次出队，记录本层长度

- 如果是烂橘子，将周围的橘子入队

- 时间增加一分钟

— 队列为空则表示传染完毕

- 如果已处理了所有鲜橘子，则输出时间减一（有一轮空转）。



```

#include <stdio.h>
typedef struct {
 int row, col;
} Orange;
Orange q[100];
int front = 0, rear = 0;
void enqueue(int r, int c) {
 q[rear].row = r; q[rear++].col = c;
}
Orange dequeue() {
 return q[front++];
}
int isEmpty() {
 return front == rear;
}
int orangesRotting(int** grid, int rows, int cols) {
 front = rear = 0; // 重置队列
 int minutes = 0;
 int freshCnt = 0; // 统计新鲜橘子数量
 for (int i = 0; i < rows; i++) // 1. 传染源入队, 统计新鲜橘子
 for (int j = 0; j < cols; j++) {
 if (grid[i][j] == 2) enqueue(i, j);
 if (grid[i][j] == 1) freshCnt++; // 记录好橘子数量
 }
 if (freshCnt == 0) return 0;
}

```

```

int dir[4][2] = { {-1,0}, {1,0}, {0,-1}, {0,1} };
while (!isEmpty()) { // 2. BFS 水波纹扩散
 int levelSize = rear - front;
 for (int i = 0; i < levelSize; i++) {
 Orange cur = dequeue();
 for (int d = 0; d < 4; d++) {
 int nr = cur.row + dir[d][0];
 int nc = cur.col + dir[d][1];
 if (nr >= 0 && nr < rows && nc >= 0
 && nc < cols && grid[nr][nc] == 1) {
 grid[nr][nc] = 2; // 变成坏橘子
 freshCnt--; // 好橘子被传染, 数量减 1
 enqueue(nr, nc);
 }
 }
 }
 minutes++;
}
return (freshCnt == 0) ? (minutes - 1) : -1;
}

int main() {
 int arr[3][3] = { {2, 1, 1}, {1, 1, 0}, {0, 1, 1} };
 int* grid[3] = { arr[0], arr[1], arr[2] };
 printf("结果: %d\n", orangesRotting(grid, 3, 3));
 return 0;
}

```

# 目录

|  |   |    |
|--|---|----|
|  | 1 | 栈  |
|  | 2 | 队列 |
|  | 3 | 小结 |
|  |   |    |
|  |   |    |

# 小结

- 栈：后进先出。
- 队列：先进先出。
- 栈和队列主要用来“保存”中间结果。
  - 输出结果的次序跟中间结果一致的，采用队列保存；
  - 输出结果的次序跟中间结果可能不一致的，采用栈保存。
- 循环队列：正常不应该写一次性队列。
- 递归算法的栈实现。

3

谢谢观看

理论课程



廈門大學  
XIAMEN UNIVERSITY



信息学院 黃 煒  
(特色化示范性软件学院) 博士, 副教授  
School of Informatics Wei Huang

数据结构与算法

Data Structures

4

串

理论课程



廈門大學  
XIAMEN UNIVERSITY



信息学院 黃 煒  
(特色化示范性软件学院) 博士, 副教授  
School of Informatics Wei Huang



# 内容要点

- 串的基本概念
- 顺序串、堆串、快串
  - 数据定义
  - 赋值、插入、删除、复制
  - 判空、判满、求长、清空、显示
  - 比较、拼接、子串
  - 子串查找：BF算法、KMP算法

# 目录

|  |   |      |
|--|---|------|
|  | 1 | 基本概念 |
|  | 2 | 顺序串  |
|  | 3 | 堆串   |
|  | 4 | 块串   |
|  |   |      |

# 基本概念

- 串（字符串）：是零个或多个字符组成的有限序列。
  - 记作： $S = "a_1 a_2 a_3 \dots"$
  - $S$ 是串名， $a_i (1 \leq i \leq n)$  可以是字母、数字或其它字符。
  - 串值：双引号括起来的字符序列是串值。
  - 串长：串中所包含的字符个数称为该串的长度。
- 空串（空的串）和空格串（空白串）
  - 空串：长度为零的串称为空串，它不包含任何字符。
  - 空格串：构成串的所有字符都是空格的串称为空白串。
  - “ ” 和 “” 分别表示长度为1的空白串和长度为0的空串。

# 基本概念

- 子串(substring)：串中任意个连续字符组成的子序列称为该串的子串，包含子串的串相应地称为主串。
  - 子串的序号：将子串在主串中首次出现时的该子串的首字符对应在主串中的序号，称为子串在主串中的序号（或位置）。
  - 例如，设有串A=“this is a word”，B=“is”，则B是A的子串，A为主串。B在A中出现了两次，其中首次出现所对应的主串位置是3。因此，称B在A中的序号为3。
  - 特别地，空串是任意串的子串，任意串是其自身的子串。

# 基本概念

- 串相等：如果两个串的串值相等（相同），称这两个串相等。换言之，只有当两个串的长度相等，且各个对应位置的字符都相同时才相等。
- 串变量和串常量。
  - 串常量：和整常数、实常数一样，在程序中只能被引用但不能不能改变其值，即只能读不能写。通常串常量是由直接量来表示的，例如语句错误（“溢出”）中“溢出”是直接量。
  - 串变量：和其它类型的变量一样,其值是可以改变。

# 串的抽象数据类型定义

- 定义

ADT String {

    数据对象： $D = \{ a_i \mid a_i \in \text{CharSet}, i=1,2,\dots,n, n \geq 0 \}$

    数据关系：

$R = \{ \langle a_{i-1}, a_i \rangle \mid a_{i-1}, a_i \in D, i=2,3,\dots,n \}$

    基本操作：

        赋值、连接、求长度、求字符串等

} ADT String

# 串的存储表示和实现

- 串在计算机中有3种表示方式：
  - 定长顺序存储表示：将串定义成字符数组，利用串名可以直接访问串值。用这种表示方式，串的存储空间在编译时确定，其大小不能改变。
  - 堆分配存储方式：仍然用一组地址连续的存储单元来依次存储串中的字符序列，但串的存储空间是在程序运行时根据串的实际长度动态分配的。
  - 块链存储方式：是一种链式存储结构表示。

# 目录

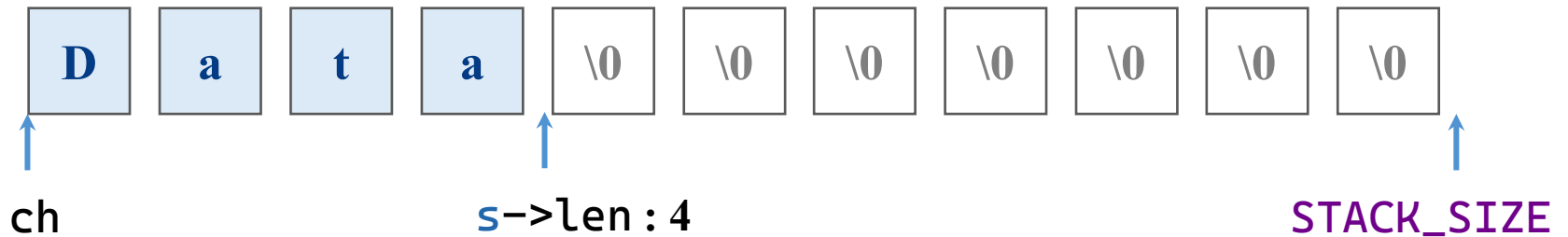
|  |   |      |
|--|---|------|
|  | 1 | 基本概念 |
|  | 2 | 顺序串  |
|  | 3 | 堆串   |
|  | 4 | 块串   |
|  |   |      |



# 顺序串的存储结构

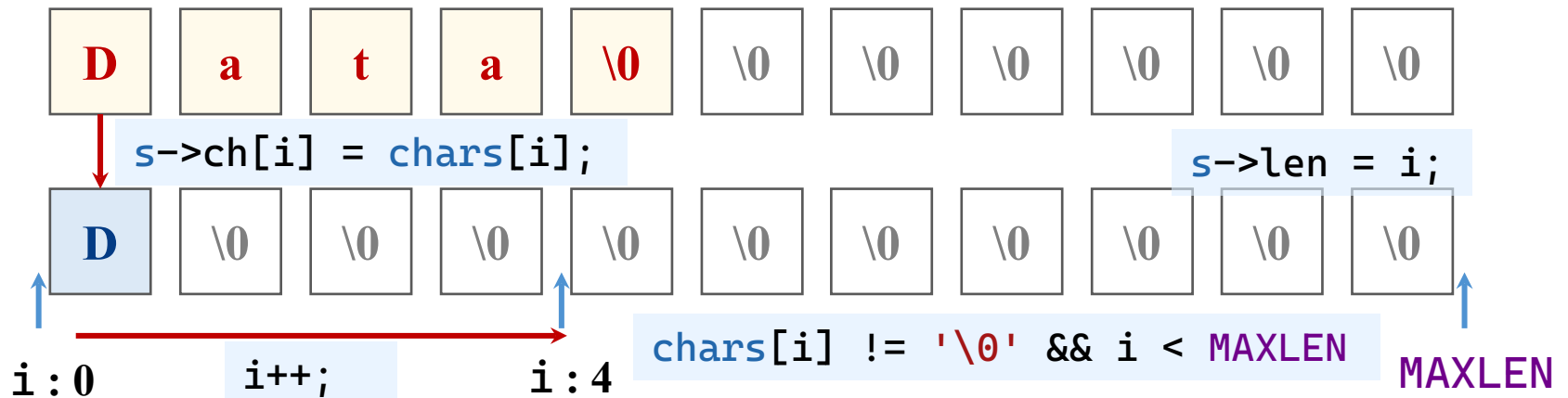
- 顺序串的存储结构

```
typedef struct {
 char ch[MAXLEN];
 int len;
} SeqString;
```



## 顺序串：赋值

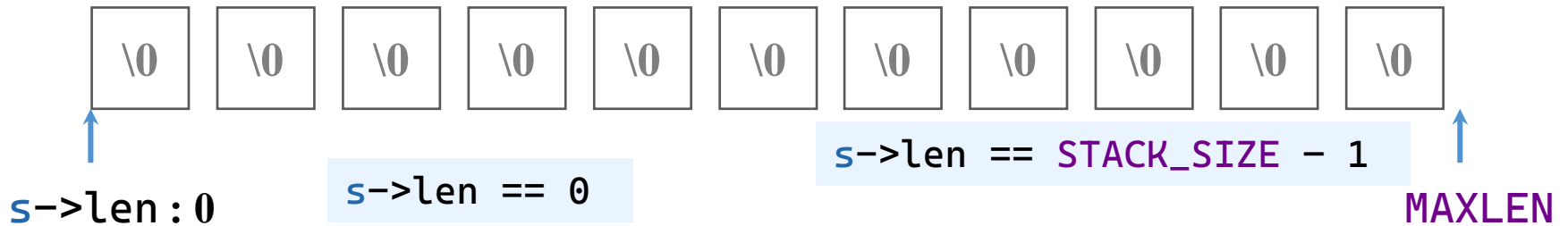
- **赋值：按顺序赋值元素，至\0或过长为止**



```
int str_assign(SeqString* s, const char* chars) {
 if (s == NULL || chars == NULL) return ERROR;
 int i = 0;
 while (chars[i] != '\0' && i < MAXLEN) {
 s->ch[i] = chars[i];
 i++;
 }
 s->len = i;
 if (chars[i] != '\0') return ERROR; // 字符串过长
 return OK;
}
```

# 顺序串：判断空

- 判断空：串长是否为0
- 判断满：串长度是否为最大值减一

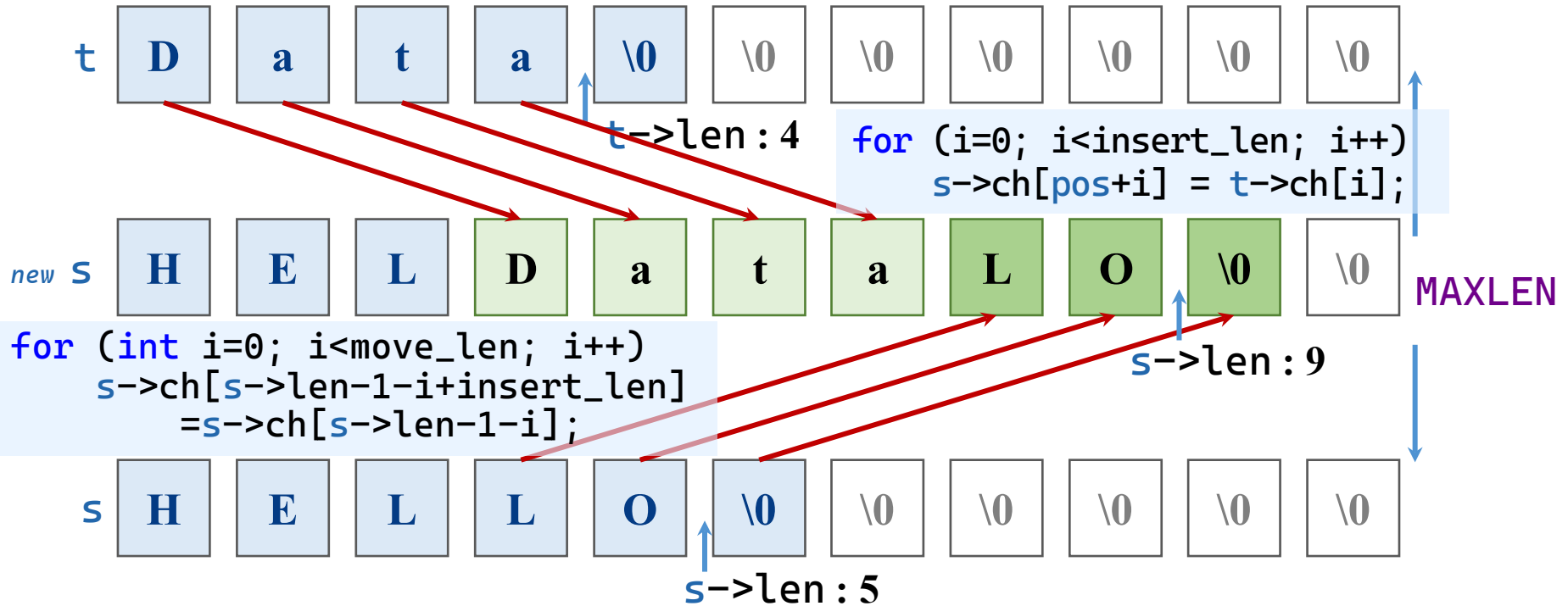


```
int str_empty(const SeqString* s) {
 if (s == NULL) return TRUE; // 视为空
 return (s->len == 0) ? TRUE : FALSE;
}
```

# 顺序串：插入

## • 入串：赋值到顶部并前进

## 主体步骤



```
if (s == NULL || t == NULL) return ERROR;
if (pos < 0 || pos > s->len) {
 printf("位置不合法! \n"); return -2;
}
```

`pos--;`

## 边界判定

1. 串指针空
2. 位置越界

# 顺序串：插入

- 插入：特殊情况计算

- 计算实际能插入的字符数  
(浅绿底色)

```
int insert_len = t->len;
if (s->len + insert_len > MAXLEN) {
 insert_len = MAXLEN - s->len;
 if (insert_len < 0) insert_len = 0;
}
```

- 计算实际需要移动的原串尾部字符数  
(深绿底色)

```
int move_len = s->len - pos;
if (pos + insert_len + move_len > MAXLEN) {
 move_len = MAXLEN - pos - insert_len;
 if (move_len < 0) move_len = 0;
}
```

- 更新长度 `s->len=(s->len+insert_len>MAXLEN)?MAXLEN:s->len+insert_len;`

- 报告是否截断

```
return (insert_len < t->len) ? -1 : OK;
```

```

int str_insert(SeqString* s, const SeqString* t, int pos) {
 if (s == NULL || t == NULL) return ERROR;
 pos--; // 转为 0-based
 if (pos < 0 || pos > s->len) {
 printf("位置不合法! \n");
 return -2;
 }
 // 1. 计算实际能插入的字符数 (受 MAXLEN 限制)
 int insert_len = t->len;
 if (s->len + insert_len > MAXLEN) {
 insert_len = MAXLEN - s->len; // 最多还能容纳的字符数
 if (insert_len < 0) insert_len = 0;
 }
 // 2. 计算实际需要移动的原串尾部字符数 (从 pos 开始)
 int move_len = s->len - pos;
 if (pos + insert_len + move_len > MAXLEN) {
 move_len = MAXLEN - pos - insert_len;
 if (move_len < 0) move_len = 0;
 }
}

```

```

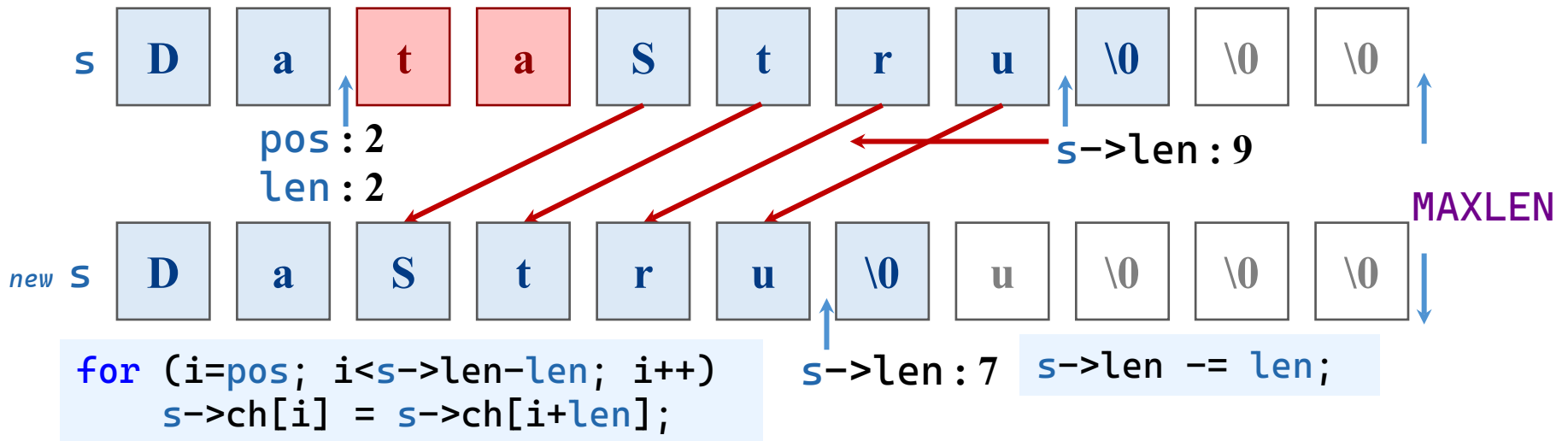
// 3. 统一处理：移动尾部（从后向前）
for (int i = 0; i < move_len; i++) {
 s->ch[s->len - 1 - i + insert_len]
 = s->ch[s->len - 1 - i];
}
// 4. 统一处理：插入新字符（从前向后）
for (int i = 0; i < insert_len; i++) {
 s->ch[pos + i] = t->ch[i];
}
// 5. 更新长度
s->len = (s->len + insert_len > MAXLEN) ?
 MAXLEN : s->len + insert_len;
// 6. 返回结果：若插入长度被截断，返回 -1
return (insert_len < t->len) ? -1 : OK;
}

```

# 顺序串：删除

- 删除：自顶部赋值并后退

主体步骤



- 越界处理

— 删除长度越界

```
for (i=pos; i<s->len-len; i++)
 s->ch[i] = s->ch[i+len];
```

边界判定

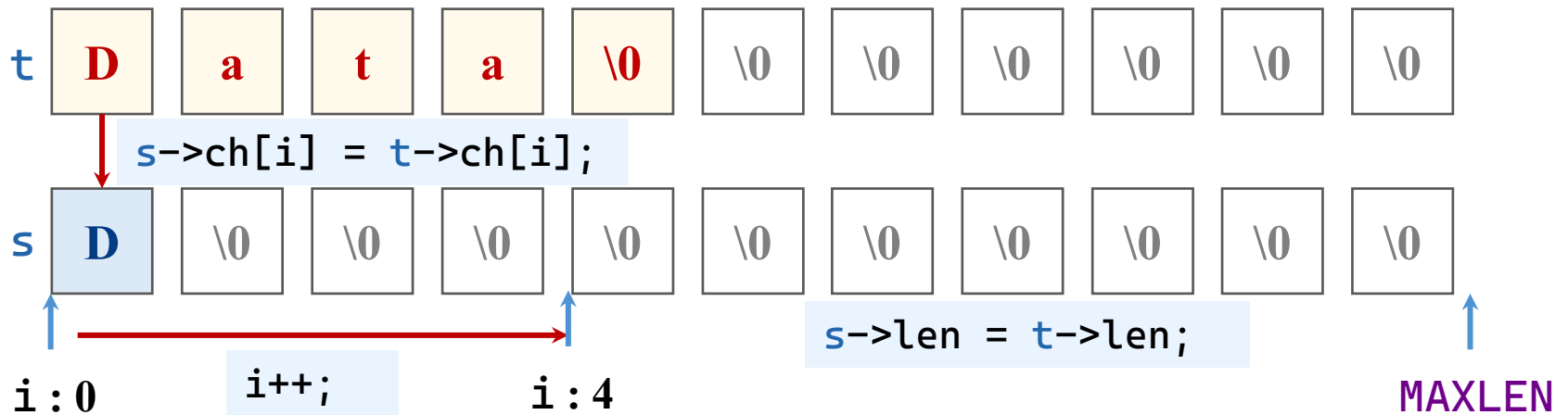
1. 串指针空
2. 位置越界



```
int str_delete(SeqString* s, int pos, int len) {
 if (s == NULL) return ERROR;
 pos--; // 0-based
 if (pos < 0 || pos >= s->len) {
 printf("位置不合法! \n");
 return ERROR;
 }
 if (pos + len > s->len) {
 len = s->len - pos; // 只删除到末尾
 }
 // 将后面的字符前移
 for (int i = pos; i < s->len - len; i++) {
 s->ch[i] = s->ch[i + len];
 }
 s->len -= len;
 return OK;
}
```

# 顺序串：复制

- 复制：按顺序复制元素



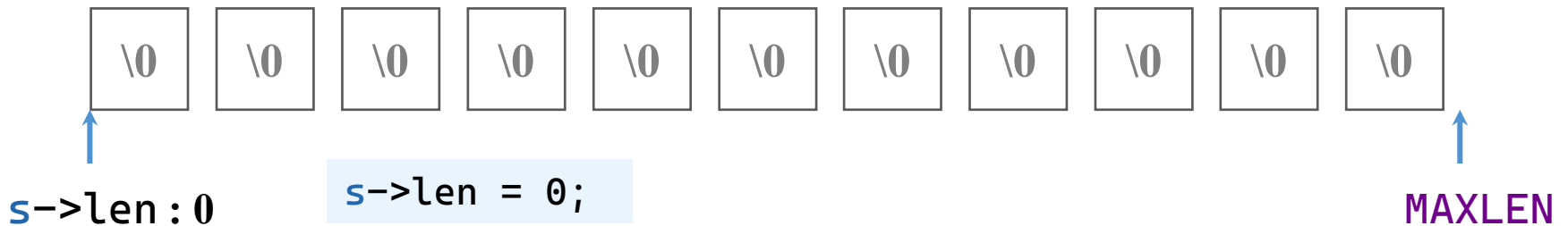
```
void str_copy(SeqString* s, const SeqString* t) {
 if (s == NULL || t == NULL) return;
 for (int i = 0; i < t->len; i++) {
 s->ch[i] = t->ch[i];
 }
 s->len = t->len;
}
```

# 顺序串：求长和清空、显示

- 求长：返回len成员

```
int str_length(const SeqString* s) {
 if (s == NULL) return 0;
 return s->len;
}
```

- 清空：len成员置0

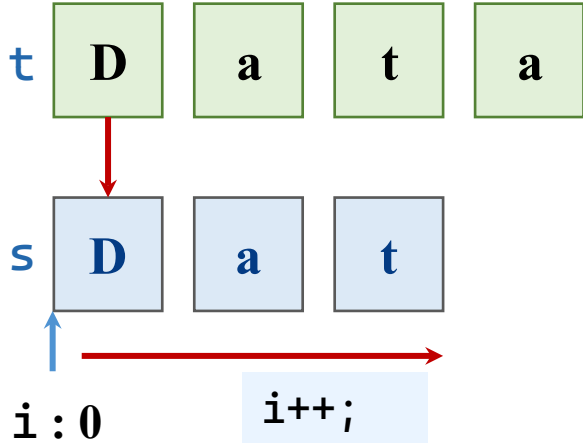


```
void str_clear(SeqString* s) {
 if (s == NULL) return;
 s->len = 0;
}
```

```
void str_display(const SeqString* s) {
 if (s == NULL) return;
 for (int i = 0; i < s->len; i++) {
 putchar(s->ch[i]);
 }
}
```

# 顺序串：比较

- 比较：按顺序比较元素，相等继续，不等输出



```
while (i < s->len && i < t->len) {
 if (s->ch[i] != t->ch[i])
 return (s->ch[i] > t->ch[i]) ? 1 : -1;
 i++;
}
```

```
if (s->len == t->len) return 0;
return (s->len > t->len) ? 1 : -1;
```

边界判定

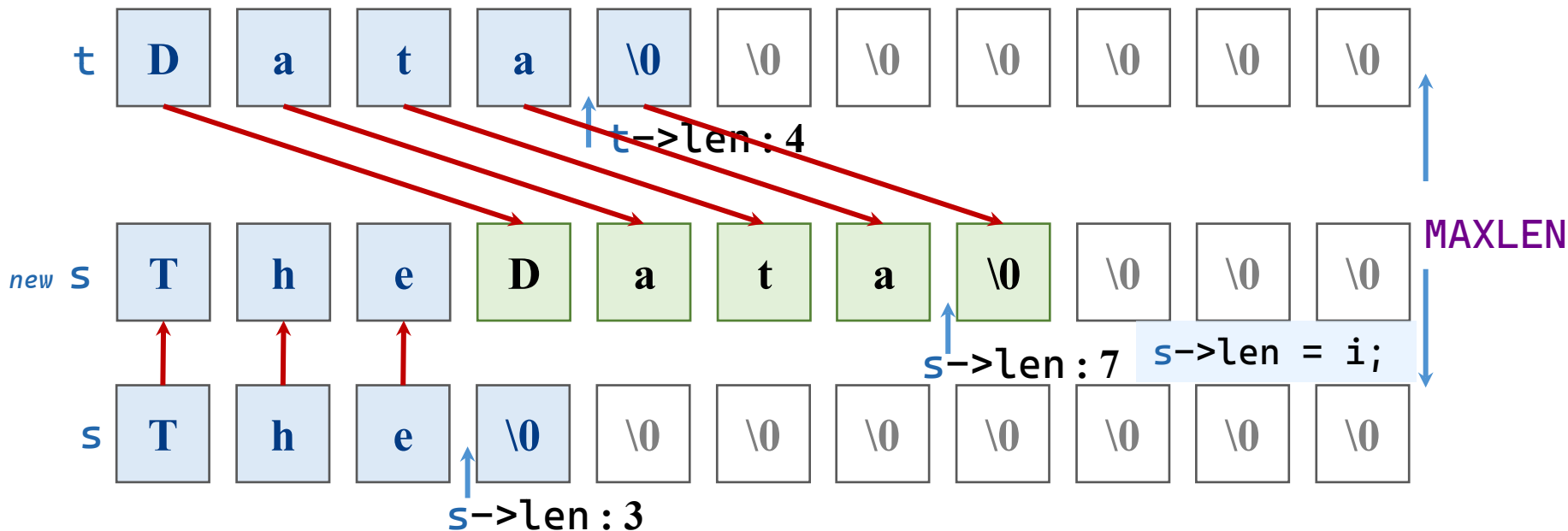
1. 串指针空

```
int str_compare(const SeqString* s, const SeqString* t) {
 if (s == NULL || t == NULL) return 0;
 int i = 0;
 while (i < s->len && i < t->len) {
 if (s->ch[i] != t->ch[i])
 return (s->ch[i] > t->ch[i]) ? 1 : -1;
 i++;
 }
 if (s->len == t->len) return 0;
 return (s->len > t->len) ? 1 : -1;
}
```

# 顺序串：拼接

- 入串：赋值到顶部并前进

主体步骤



```
for (i=s->len; i<MAXLEN && i-s->len<t->len; i++)
 s->ch[i]=t->ch[i-s->len];
```

```
if (s == NULL || t == NULL) return ERROR;
```

边界判定

1. 串指针空
2. 位置越界

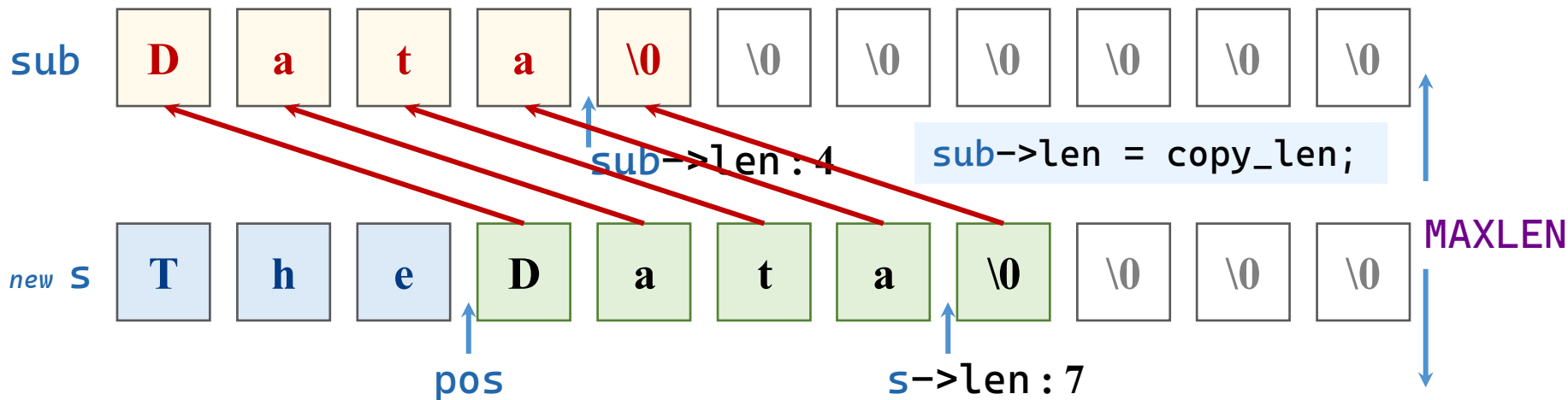
```
int str_cat(SeqString* s, const SeqString* t) {
 if (s == NULL || t == NULL)
 return ERROR;
 int i;
 for (i = s->len; i < MAXLEN && i - s->len < t->len;
 i++) {
 s->ch[i] = t->ch[i - s->len];
 }
 s->len = i;
 if (i == MAXLEN && s->len < s->len + t->len) {
 return ERROR; // 截断
 }
 return OK;
}
```



# 顺序串：子串

- 入串：赋值到顶部并前进

主体步骤



```
if (pos + len > s->len)
 copy_len = s->len - pos;
```

```
for (int i = 0; i < copy_len; i++) {
 sub->ch[i] = s->ch[pos + i];
}
```

边界判定

1. 串指针空
2. 位置越界

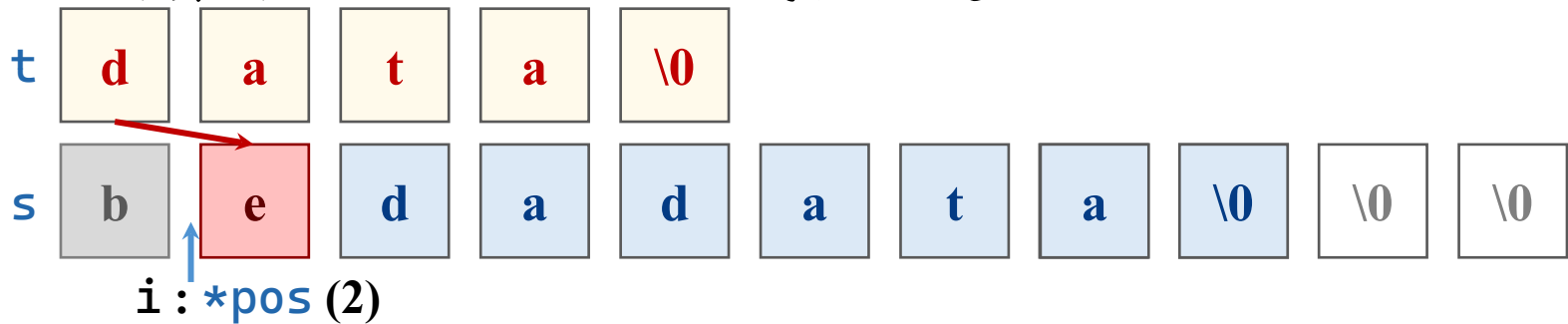
```
if (s == NULL || t == NULL) return ERROR;
```

```
int str_substr(SeqString* sub, const SeqString* s, int
pos, int len) {
 if (sub == NULL || s == NULL) return ERROR;
 pos--; // 0-based
 if (pos < 0 || pos >= s->len) {
 printf("位置不合法! \n");
 return ERROR;
 }
 int copy_len = len;
 if (pos + len > s->len)
 copy_len = s->len - pos;
 for (int i = 0; i < copy_len; i++)
 sub->ch[i] = s->ch[pos + i];
 sub->len = copy_len;
 return OK;
}
```

# 顺序串：子串查找

- Brute-Force模式匹配算法

– 对齐开头，逐位比较；成功就走，失败重头



| i | j | 主串       | 子串   | i | j | 主串       | 子串   |
|---|---|----------|------|---|---|----------|------|
| 0 | 0 | bedadata | data | 3 | 0 | bedadata | data |
| 1 | 0 | bedadata | data | 4 | 0 | bedadata | data |
| 2 | 0 | bedadata | data | 5 | 1 | bedadata | data |
| 3 | 1 | bedadata | data | 6 | 2 | bedadata | data |
| 4 | 2 | bedadata | data | 7 | 3 | bedadata | data |

# 顺序串：子串查找

## • Brute-Force模式匹配算法

— 对齐开头

```
int i = *pos - 1; // 转为 0-based
int j = 0;
```

— 逐位比较

```
while (i < s->len && j < t->len) {
```

— 成功就走

```
 if (s->ch[i] == t->ch[j]) {
 i++;
 j++;
 }
```

— 失败重头

```
 else {
 i = i - j + 1;
 j = 0;
 }
```

```
}
```

## 边界判定

1. 串指针空
2. 位置越界
3. 边界空

```

int str_index(const SeqString* s, const SeqString* t, int* pos) {
 if (s == NULL || t == NULL || pos == NULL) return ERROR;
 if (t->len == 0) return ERROR;
 if (*pos < 1 || *pos > s->len) return ERROR;
 int i = *pos - 1; // 转为 0-based
 int j = 0;
 while (i < s->len && j < t->len) {
 if (s->ch[i] == t->ch[j]) {
 i++;
 j++;
 } else {
 i = i - j + 1;
 j = 0;
 }
 }
 if (j == t->len) {
 *pos = i - j + 1; // 转为 1-based
 return OK;
 }
 return ERROR;
}

```

# 顺序串：子串查找

- 改进算法：KMP算法

- 失败重头太浪费时间
- 先记录下失败重来的位置，从上一步继续，将节省时间
- 当匹配到第  $j$  个字符卡住时，因为前面已经确认匹配好的那一段里，头部和尾部有  $\text{next}[j]$  个字符是重复的，所以  $j$  直接跳到  $\text{next}[j]$ ， $i$  停在原地等着继续比。

```

void get_next(const SeqString* t, int next[]) {
 if (t == NULL || t->len == 0) return;
 int j = 0, k = -1;
 next[0] = -1;
 while (j < t->len - 1)
 if (k == -1 || t->ch[j] == t->ch[k]) {
 j++;
 k++;
 if (t->ch[j] != t->ch[k])
 next[j] = k;
 else
 next[j] = next[k]; // 跳过相同字符, 避免无效比较
 } else
 k = next[k];
}

int kmp_index(const SeqString* s, const SeqString* t, int* pos) {
 if (s == NULL || t == NULL || pos == NULL) return ERROR;
 if (t->len == 0) return ERROR; // 空串直接失败
 if (*pos < 1 || *pos > s->len) return ERROR;
 int next[MAXLEN] = {}; // 存储 next 数组
 get_next(t, next); // 计算 next
}

```

```

int i = *pos - 1; // 主串指针 (0-based)
int j = 0; // 模式串指针

while (i < s->len && j < t->len) {
 if (j == -1 || s->ch[i] == t->ch[j]) {
 i++;
 j++;
 } else {
 j = next[j]; // 主串指针 i 不回溯!
 }
}

if (j == t->len) { // 匹配成功
 *pos = i - j + 1; // 转换为 1-based 位置
 return OK;
}
return ERROR; // 匹配失败
}

```



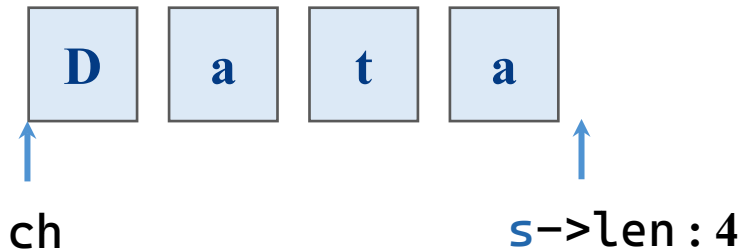
# 目录

|  |   |      |
|--|---|------|
|  | 1 | 基本概念 |
|  | 2 | 顺序串  |
|  | 3 | 堆串   |
|  | 4 | 块串   |
|  |   |      |

# 堆串的存储结构

- 堆串的存储结构

```
typedef struct {
 char* ch;
 int len;
} HeapString;
```



# 堆串：赋值

- 赋值：按顺序赋值元素，至\0或过长为止
- 与顺序串的不同
  - 复制前先释放原空间，再建立新空间
  - 特殊情况：长度为0时，不新建空间
  - 复制可以使用内存复制函数（ `memcpy` ）

```
int str_assign(HeapString* s, const char* chars) {
 if (s == NULL || chars == NULL) return ERROR;
 // 释放原有空间
 if (s->ch != NULL) {
 free(s->ch);
 s->ch = NULL;
 }
 int len = strlen(chars);
 if (len == 0) {
 s->ch = NULL;
 s->len = 0;
 return OK;
 }
 s->ch = (char*)malloc(len + 1); // 多分配一个字节方便, 但不用
 if (s->ch == NULL) return ERROR;
 memcpy(s->ch, chars, len);
 s->len = len;
 return OK;
}
```

# 堆串：判断空

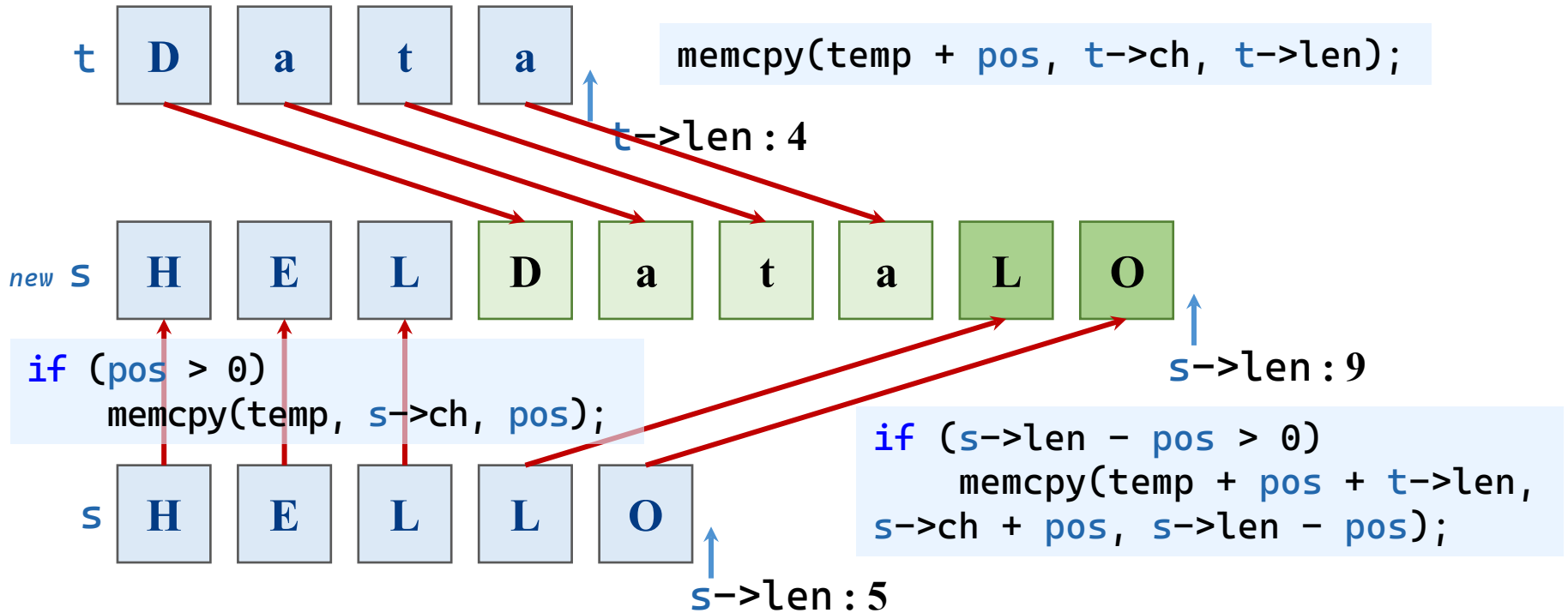
- 判断空：串长是否为0
- 判断满：堆能新建空间即没有满
- 堆串判断空与顺序串一样

```
int str_empty(const HeapString* s) {
 if (s == NULL) return TRUE;
 return (s->len == 0) ? TRUE : FALSE;
}
```

# 堆串：插入

- 入串：拆分并复制到新空间

主体步骤



```
if (s == NULL || t == NULL) return ERROR;
if (pos < 0 || pos > s->len) {
 printf("位置不合法! \n"); return ERROR;
}
```

```
pos--;
```

## 边界判定

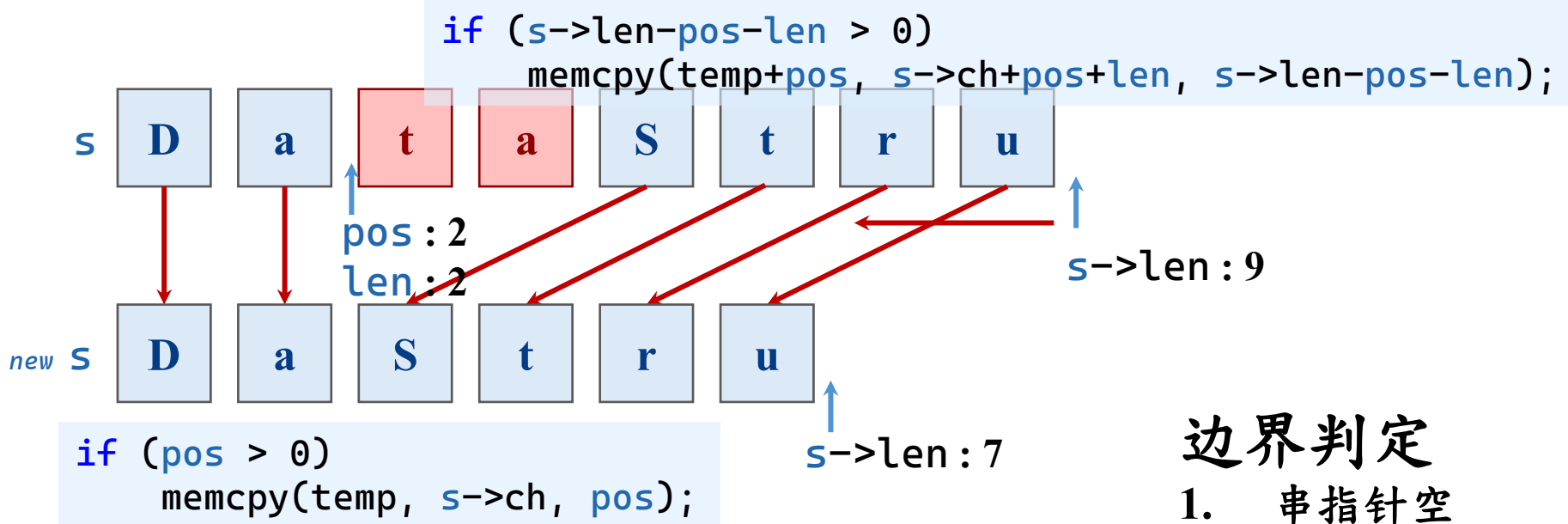
1. 串指针空
2. 位置越界

```
int str_insert(HeapString* s, const HeapString* t, int pos) {
 if (s == NULL || t == NULL) return ERROR;
 pos--; // 转为 0-based
 if (pos < 0 || pos > s->len) return ERROR;
 int new_len = s->len + t->len;
 char* temp = (char*)malloc(new_len);
 if (temp == NULL) return ERROR;
 // 复制 s 的前 pos 个字符
 if (pos > 0) memcpy(temp, s->ch, pos);
 // 复制 t 的所有字符
 memcpy(temp + pos, t->ch, t->len);
 // 复制 s 的剩余字符
 if (s->len - pos > 0)
 memcpy(temp + pos + t->len, s->ch + pos, s->len - pos);
 free(s->ch);
 s->ch = temp;
 s->len = new_len;
 return OK;
}
```

# 堆串：删除

## • 删除：拆分并复制到新空间

## 主体步骤



## 边界判定

1. 串指针空
2. 位置越界

## • 越界处理

### — 删除长度越界

```
if (s == NULL) return ERROR;
if (pos < 0 || pos >= s->len) return ERROR;
if (pos + len > s->len) len = s->len - pos;
if (len <= 0) return OK;
```



```

int str_delete(HeapString* s, int pos, int len) {
 if (s == NULL) return ERROR;
 pos--; // 0-based
 if (pos < 0 || pos >= s->len) return ERROR;
 if (pos + len > s->len) len = s->len - pos; // 删除至末尾
 if (len <= 0) return OK; // 无删除
 int new_len = s->len - len;
 char* temp = (char*)malloc(new_len);
 if (temp == NULL) return ERROR;
 // 复制 pos 前的字符
 if (pos > 0) memcpy(temp, s->ch, pos);
 // 复制 pos+len 之后的字符
 if (s->len - pos - len > 0)
 memcpy(temp + pos, s->ch + pos + len, s->len - pos - len);
 free(s->ch);
 s->ch = temp;
 s->len = new_len;
 return OK;
}

```

# 堆串：复制

- 复制：释放并重建空间，复制元素

– 防止自己复制自己

```
int str_copy(HeapString* s, const HeapString* t) {
 if (s == NULL || t == NULL) return ERROR;
 if (s == t) return OK; // 自己复制自己
 if (s->ch != NULL) { // 释放 s 原有空间
 free(s->ch); s->ch = NULL;
 }
 s->len = t->len;
 if (t->len == 0) {
 s->ch = NULL; return OK;
 }
 s->ch = (char*)malloc(t->len);
 if (s->ch == NULL) return ERROR;
 memcpy(s->ch, t->ch, t->len);
 return OK;
}
```

# 堆串：求长和清空、显示

- 求长：返回len成员（与顺序串一致）

```
int str_length(const HeapString* s) {
 if (s == NULL) return 0;
 return s->len;
}
```

- 清空：len成员置0，外加释放空间

```
void str_clear(HeapString* s) {
 if (s == NULL) return;
 if (s->ch != NULL) {
 free(s->ch);
 s->ch = NULL;
 }
 s->len = 0;
}
```

```
void str_display(const HeapString* s) {
 if (s == NULL || s->len == 0 || s->ch == NULL) {
 printf("(空串)");
 return;
 }
 for (int i = 0; i < s->len; i++) {
 putchar(s->ch[i]);
 }
}
```

# 堆串：比较

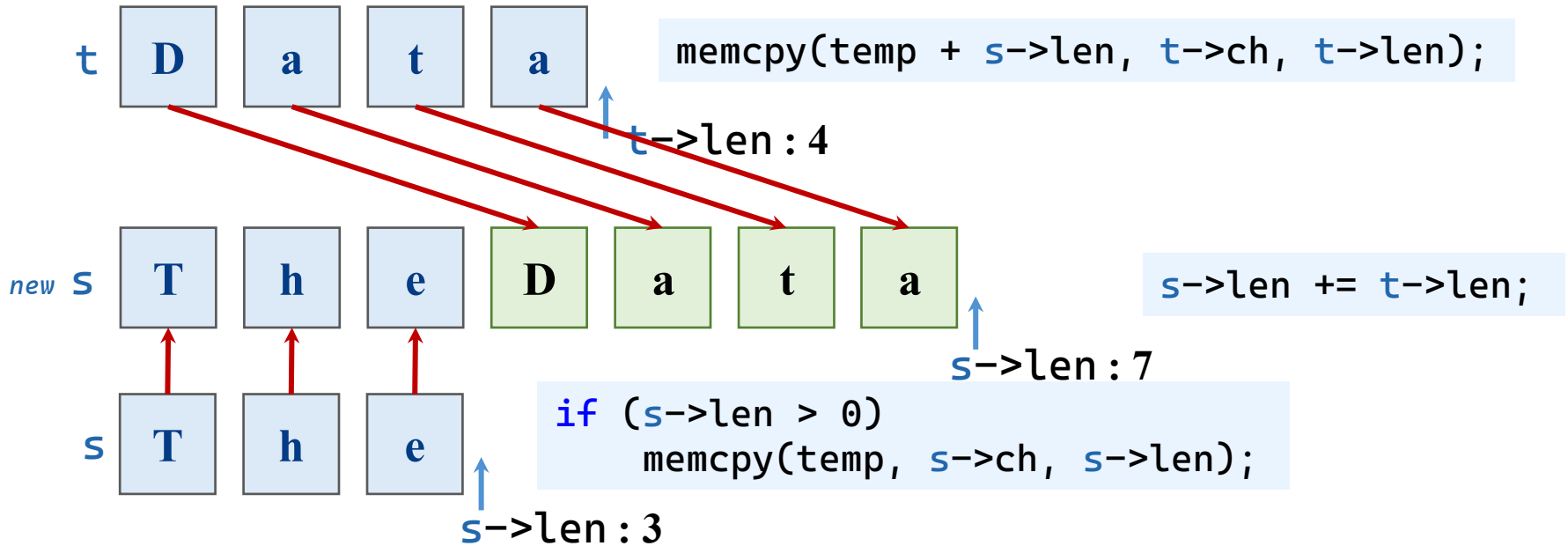
- 比较：与顺序栈一致

```
int str_compare(const HeapString* s, const HeapString* t) {
 if (s == NULL || t == NULL) return 0;
 int i = 0;
 while (i < s->len && i < t->len) {
 if (s->ch[i] != t->ch[i])
 return (s->ch[i] > t->ch[i]) ? 1 : -1;
 i++;
 }
 if (s->len == t->len) return 0;
 return (s->len > t->len) ? 1 : -1;
}
```

# 堆串：拼接

- 入串：赋值到顶部并前进

主体步骤



```
if (s == NULL || t == NULL) return ERROR;
```

边界判定

1. 串指针空
2. 位置越界

```
int str_cat(HeapString* s, const HeapString* t) {
 if (s == NULL || t == NULL) return ERROR;
 if (t->len == 0) return OK;

 int new_len = s->len + t->len;
 char* temp = (char*)malloc(new_len);
 if (temp == NULL) return ERROR;

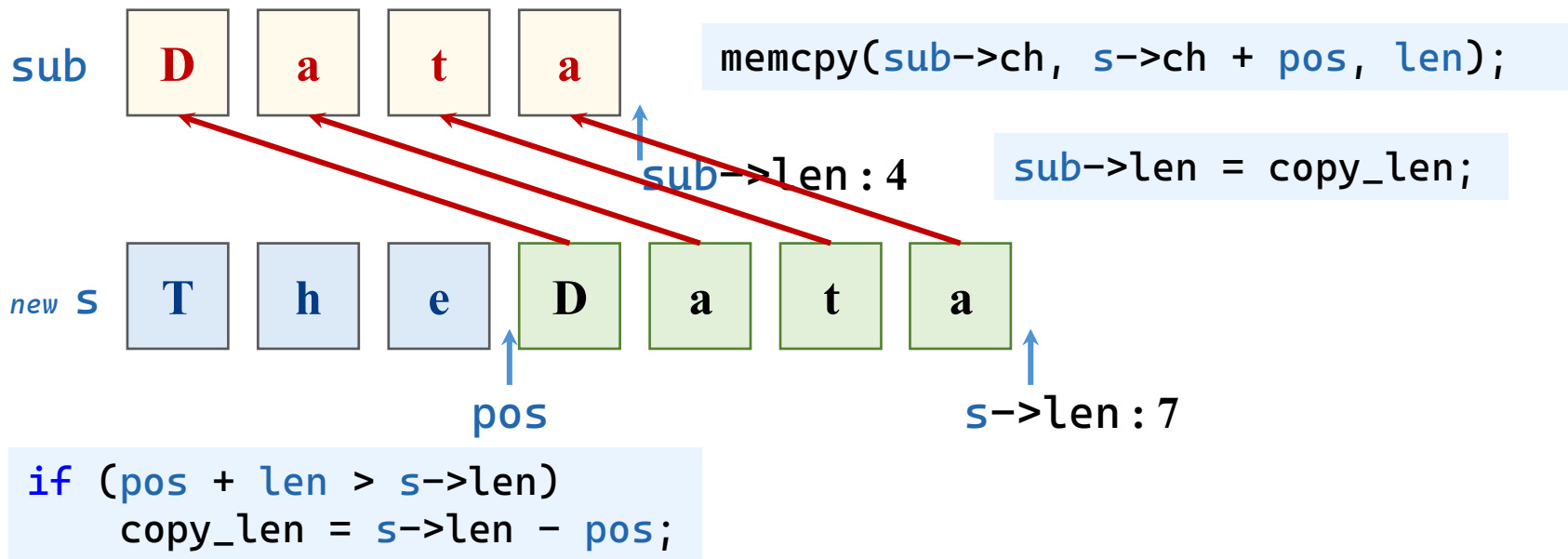
 if (s->len > 0) memcpy(temp, s->ch, s->len);
 memcpy(temp + s->len, t->ch, t->len);

 free(s->ch);
 s->ch = temp;
 s->len = new_len;
 return OK;
}
```

# 堆串：子串

• 入串：赋值到顶部并前进

主体步骤



```
if (s == NULL || t == NULL) return ERROR;
```

边界判定

1. 串指针空
2. 位置越界

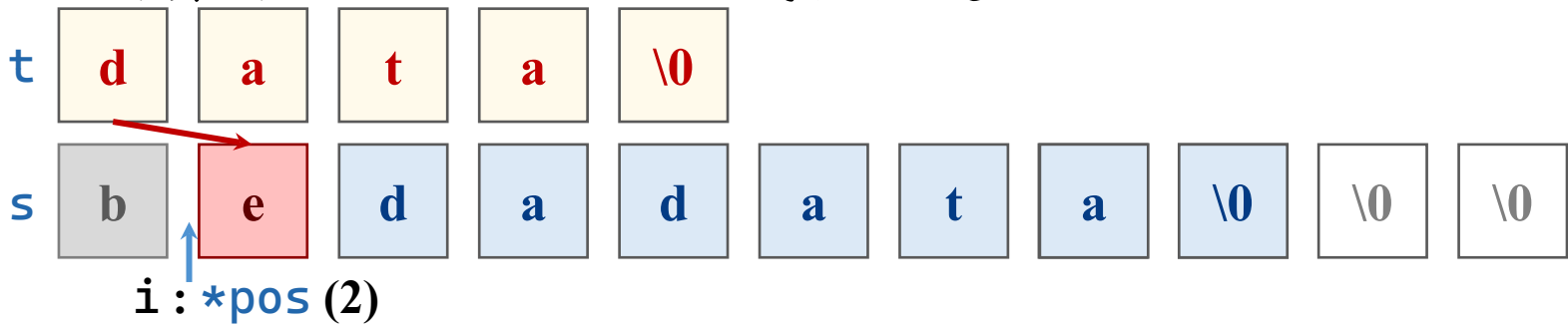


```
int str_substr(HeapString* sub, const HeapString* s,
 int pos, int len) {
 if (sub == NULL || s == NULL) return ERROR;
 pos--; // 0-based
 if (pos < 0 || pos >= s->len || len < 0)
 return ERROR;
 if (pos + len > s->len) len = s->len - pos;
 // 释放 sub 原有空间
 if (sub->ch != NULL) free(sub->ch);
 sub->ch = NULL;
 sub->len = len;
 if (len == 0) return OK;
 sub->ch = (char*)malloc(len);
 if (sub->ch == NULL) return ERROR;
 memcpy(sub->ch, s->ch + pos, len);
 return OK;
}
```

# 堆串：子串查找

- Brute-Force模式匹配算法

– 对齐开头，逐位比较；成功就走，失败重头



| i | j | 主串       | 子串   | i | j | 主串       | 子串   |
|---|---|----------|------|---|---|----------|------|
| 0 | 0 | bedadata | data | 3 | 0 | bedadata | data |
| 1 | 0 | bedadata | data | 4 | 0 | bedadata | data |
| 2 | 0 | bedadata | data | 5 | 1 | bedadata | data |
| 3 | 1 | bedadata | data | 6 | 2 | bedadata | data |
| 4 | 2 | bedadata | data | 7 | 3 | bedadata | data |

# 堆串：子串查找

- 堆串查找与顺序栈一致

```
int str_index(const HeapString* s, const HeapString* t, int* pos) {
 if (s == NULL || t == NULL || pos == NULL) return ERROR;
 if (t->len == 0) return ERROR;
 if (*pos < 1 || *pos > s->len) return ERROR;
 int i = *pos - 1; // 转为 0-based
 int j = 0;
 while (i < s->len && j < t->len)
 if (s->ch[i] == t->ch[j]) {
 i++; j++;
 } else {
 i = i - j + 1; j = 0;
 }
 if (j == t->len) {
 *pos = i - j + 1; // 转为 1-based
 return OK;
 }
 return ERROR;
}
```

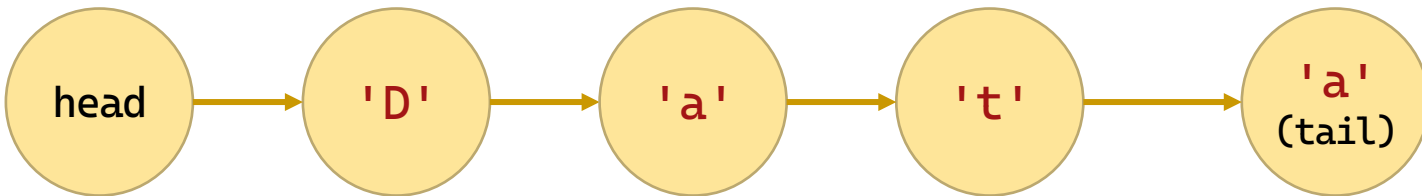
# 目录

|  |   |      |
|--|---|------|
|  | 1 | 基本概念 |
|  | 2 | 顺序串  |
|  | 3 | 堆串   |
|  | 4 | 块串   |
|  |   |      |

# 块串的存储结构

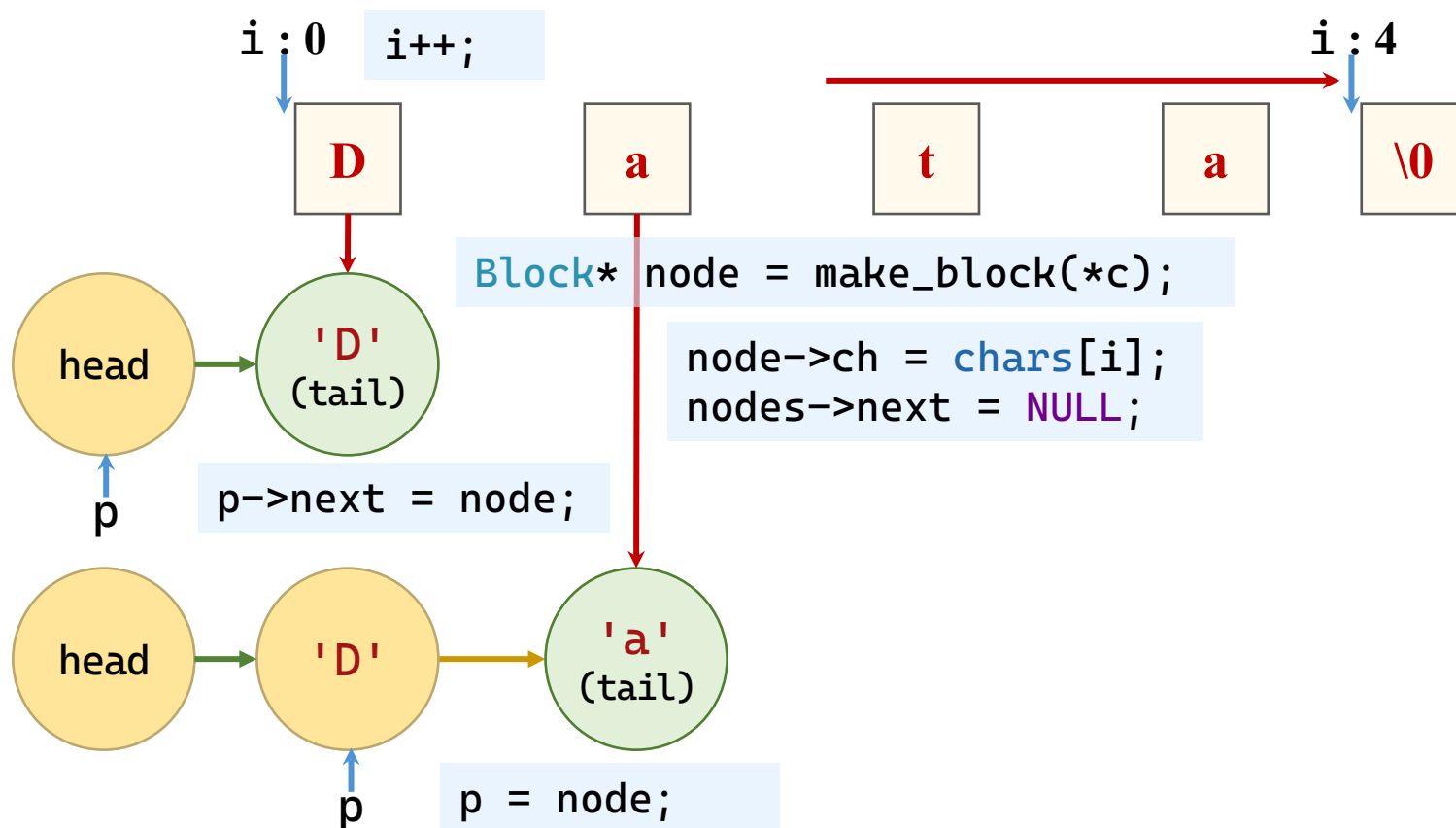
- 块串（链表串）的存储结构

```
typedef struct Block {
 char ch;
 struct Block* next;
} Block;
typedef struct {
 Block* head; // 指向头结点（空结点，不存数据，简化操作）
 Block* tail; // 指向最后一个数据结点
 int len; // 字符串长度
} BlockString;
```



# 块串：赋值

- 赋值：按顺序赋值元素，至\0或过长为止



```

static Block* make_block(char ch) {
 Block* p = (Block*)malloc(sizeof(Block));
 if (p) {
 p->ch = ch;
 p->next = NULL;
 }
 return p;
}

int str_assign(BlockString* s, const char* chars) {
 if (s == NULL || chars == NULL) return ERROR;
 str_clear(s);
 if (*chars == '\\0') {
 s->head = s->tail = NULL;
 s->len = 0;
 return OK;
 }
 Block* head = make_block(0);
 if (head == NULL) return ERROR;
 s->head = head;
 Block* p = head;

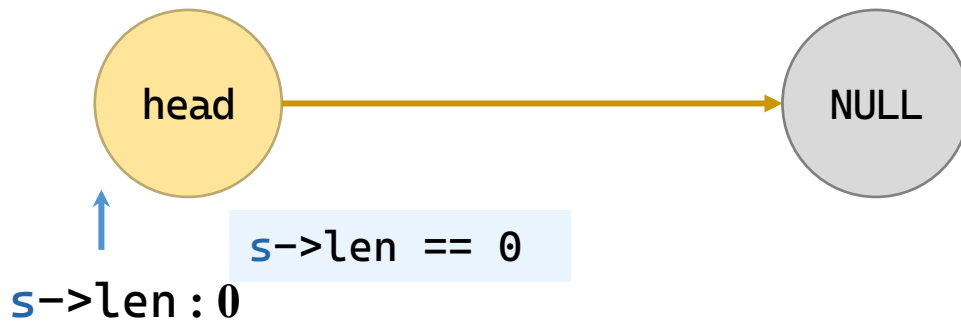
```

```
for (const char* c = chars; *c != '\0'; c++) {
 Block* node = make_block(*c);
 if (node == NULL) {
 str_clear(s);
 return ERROR;
 }
 p->next = node;
 p = node;
}
s->tail = p;
s->len = strlen(chars);
return OK;
}
```



# 块串：判断空

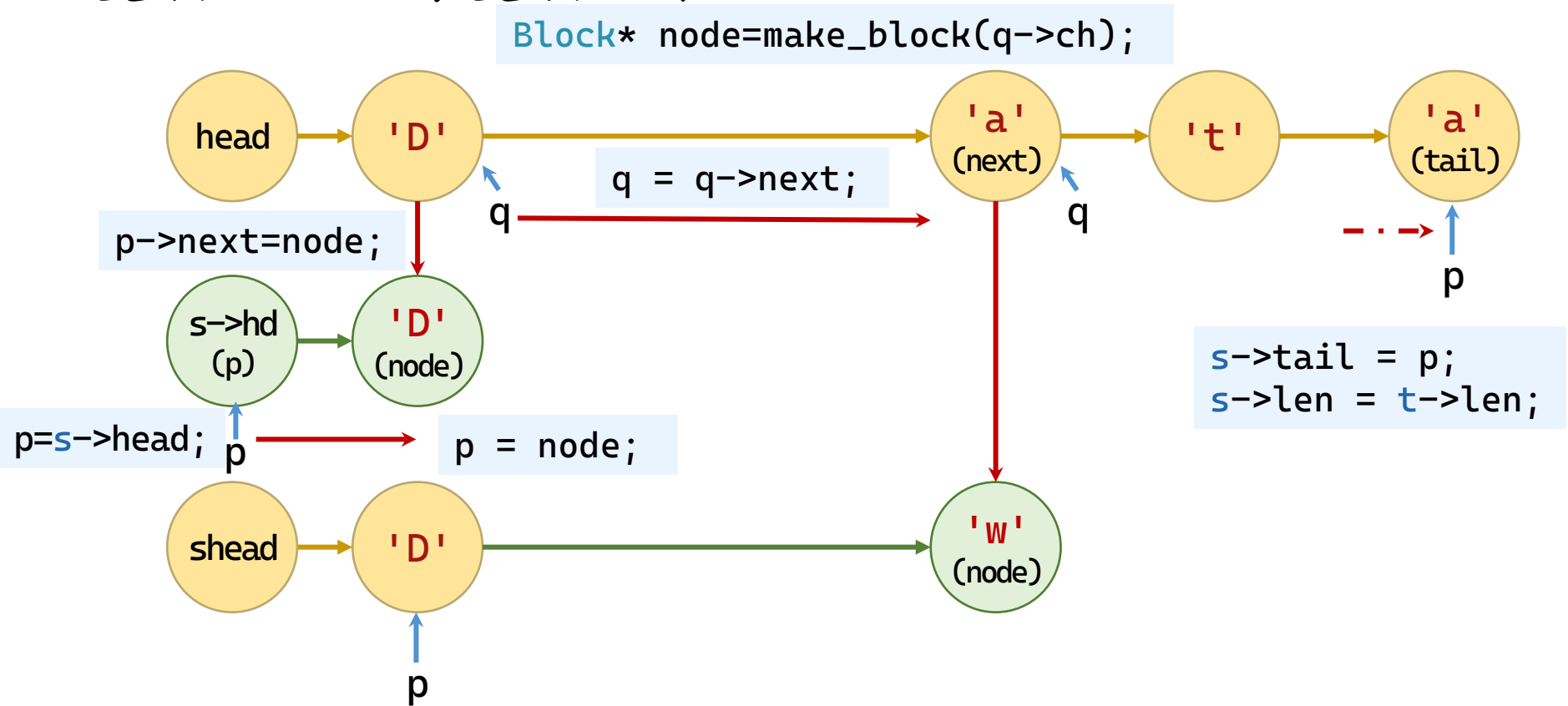
- 判断空：串长是否为0
- 判断满：开辟新节点空间失败



```
int str_empty(const SeqString* s) {
 if (s == NULL) return TRUE; // 视为空
 return (s->len == 0) ? TRUE : FALSE;
}
```

## 块串：复制

- 复制：按顺序复制元素



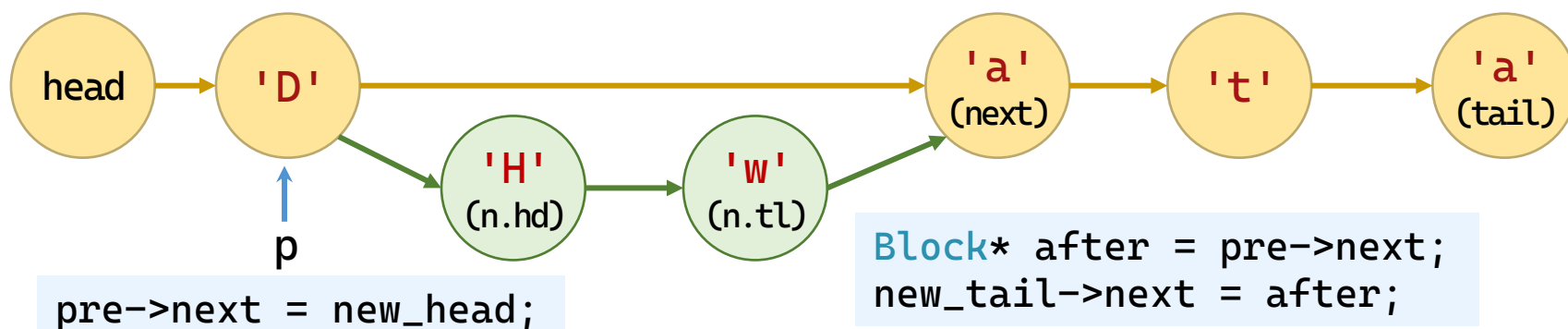
```
int str_copy(BlockString* s, const BlockString* t) {
 if (s == NULL || t == NULL) return ERROR;
 if (s == t) return OK;
 str_clear(s);
 if (t->len == 0) return OK;
 s->head = make_block(0);
 if (s->head == NULL) return ERROR;
 Block* p = s->head;
 Block* q = t->head;
 while (q != NULL) {
 Block* node = make_block(q->ch);
 if (node == NULL) {
 str_clear(s); return ERROR;
 }
 p->next = node;
 p = node;
 q = q->next;
 }
 s->tail = p;
 s->len = t->len;
 return OK;
}
```

# 块串：插入

• 入串：原串拆两半，复制新串，连成串

主体步骤

```
Block* pre = s->head;
for (i=0; i<pos && pre!=NULL; i++) {
 pre = pre->next;
```



```
if (s==NULL || t==NULL || t->len==0) return ERROR;
pos--; // 0-based
if (pos < 0 || pos > s->len) return ERROR;
...
if (pre == NULL && pos > 0) return ERROR;
```

边界判定

1. 串指针空
2. 位置越界

```

int str_insert(BlockString* s, const BlockString* t, int pos) {
 if (s == NULL || t == NULL || t->len == 0) return ERROR;
 pos--; // 0-based
 if (pos < 0 || pos > s->len) return ERROR;
 Block* pre = s->head; // 找到插入位置的前驱结点
 for (int i = 0; i < pos && pre != NULL; i++)
 pre = pre->next;
 if (pre == NULL && pos > 0) return ERROR; // 位置越界
 // 复制 t 的所有结点
 Block* new_head = NULL, *new_tail = NULL, *p = t->head;
 while (p != NULL) {
 Block* node = make_block(p->ch);
 if (node == NULL) {
 while (new_head) { // 释放已分配的结点
 Block* tmp = new_head;
 new_head = new_head->next;
 free(tmp);
 }
 return ERROR;
 }
 }
}

```

```
 if (new_head == NULL) new_head = node;
 else new_tail->next = node;
 new_tail = node;
 p = p->next;
 }
 // 将新链表插入到 pre 之后
 Block* after = pre->next;
 pre->next = new_head;
 new_tail->next = after;
 if (after == NULL)
 s->tail = new_tail; // 插入到末尾
 s->len += t->len;
 return OK;
}
```

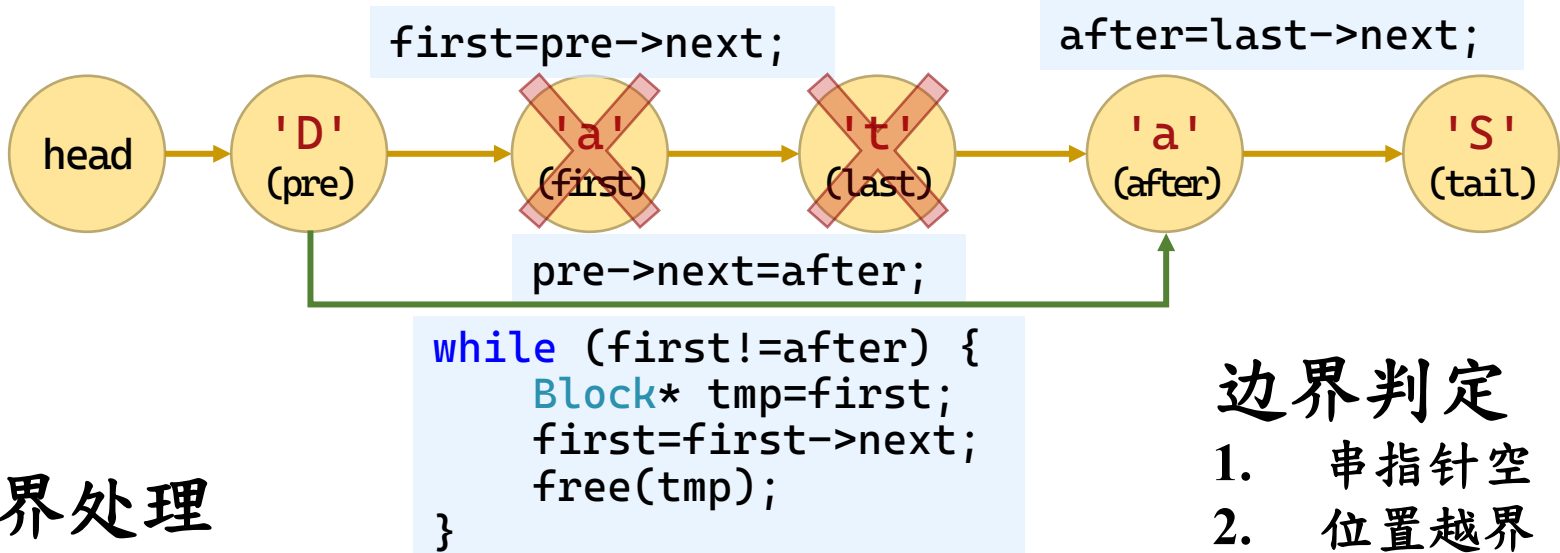
# 块串：删除

## • 删除：自顶部赋值并后退

## 主体步骤

```
Block* pre=s->head;
for (int i=0; i<pos; i++)
 pre=pre->next;
```

```
last= first;
for (i=0; i<len-1; i++)
 last=last->next;
```



## • 越界处理

## 边界判定

1. 串指针空
2. 位置越界

— 删除到串尾，串尾更新

```
if (after==NULL) s->tail=pre;
```

```

int str_delete(BlockString* s, int pos, int len) {
 if (s == NULL || s->len == 0) return ERROR;
 pos--; // 0-based
 if (pos < 0 || pos >= s->len) return ERROR;
 if (len <= 0) return OK;
 if (pos + len > s->len) len = s->len - pos;
 Block* pre = s->head; // 找到前驱
 for (int i = 0; i < pos; i++)
 pre = pre->next;
 Block* first = pre->next; // 待删除的第一个结点
 Block* last = first;
 for (int i = 0; i < len - 1; i++)
 last = last->next;
 Block* after = last->next;
 pre->next = after;
 if (after == NULL) s->tail = pre;
 while (first != after) { // 释放被删除的结点
 Block* tmp = first;
 first = first->next;
 free(tmp);
 }
 s->len -= len;
 return OK;
}

```



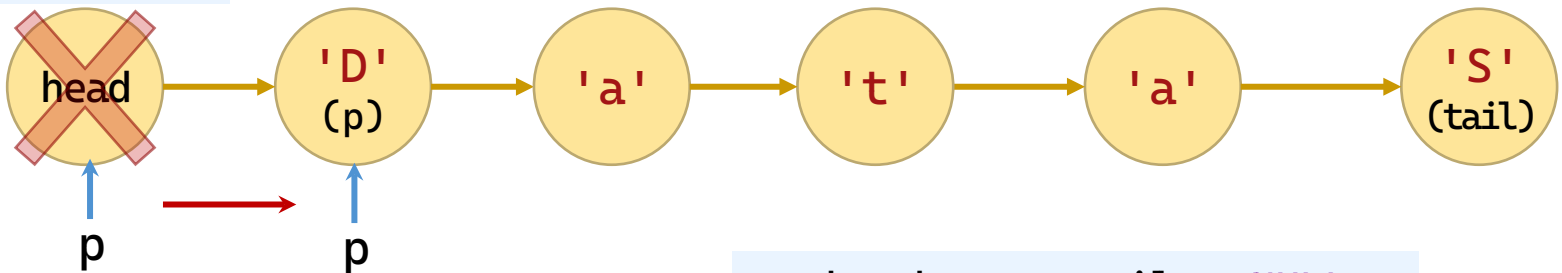
# 块串：求长和清空、显示

- 求长：返回len成员

```
int str_length(const SeqString* s) {
 if (s == NULL) return 0;
 return s->len;
}
```

- 清空：len成员置0，清除节点

```
free(tmp);
```



```
p = s->head;
Block* tmp = p;
```

```
p = p->next;
```

```
s->head = s->tail = NULL;
s->len = 0;
```

```

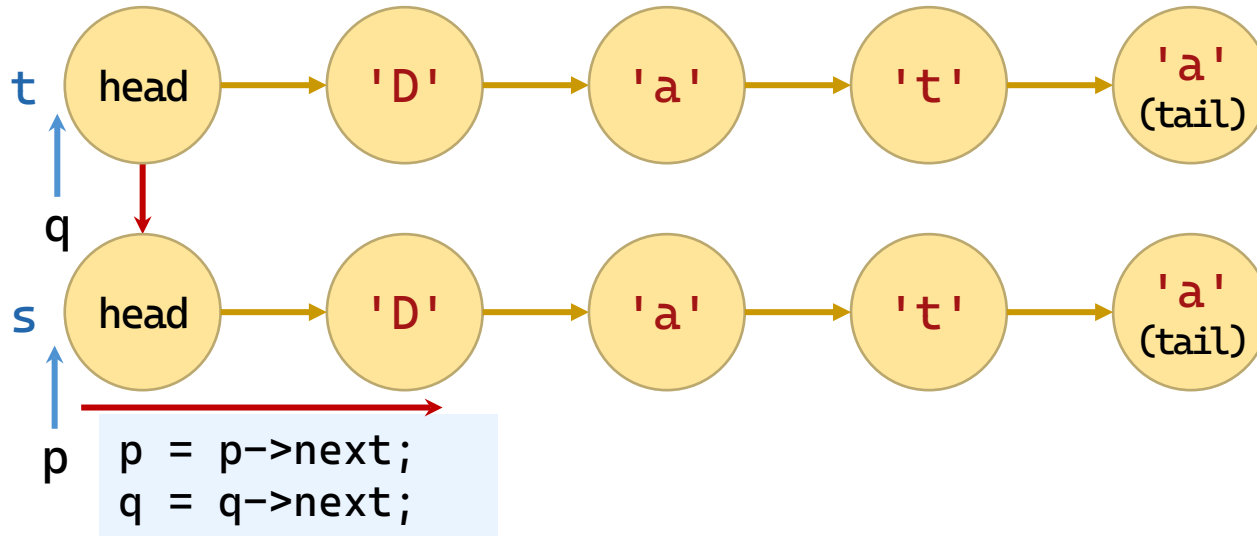
void str_clear(BlockString* s) {
 if (s == NULL) return;
 Block* p = s->head;
 while (p != NULL) {
 Block* tmp = p;
 p = p->next;
 free(tmp);
 }
 s->head = s->tail = NULL;
 s->len = 0;
}

void str_display(const BlockString* s) {
 if (s == NULL || s->len == 0 || s->head == NULL) {
 printf("(空串)"); return;
 }
 Block* p = s->head;
 while (p != NULL) {
 putchar(p->ch);
 p = p->next;
 }
}

```

# 块串：比较

- 比较：按顺序比较元素，相等继续，不等输出



```
while (p != NULL && q != NULL) {
 if (p->ch != q->ch)
 return (p->ch > q->ch) ? 1 : -1;
 p = p->next; q = q->next;
}
```

```
if (s->len == t->len) return 0;
return (s->len > t->len) ? 1 : -1;
```

## 边界判定

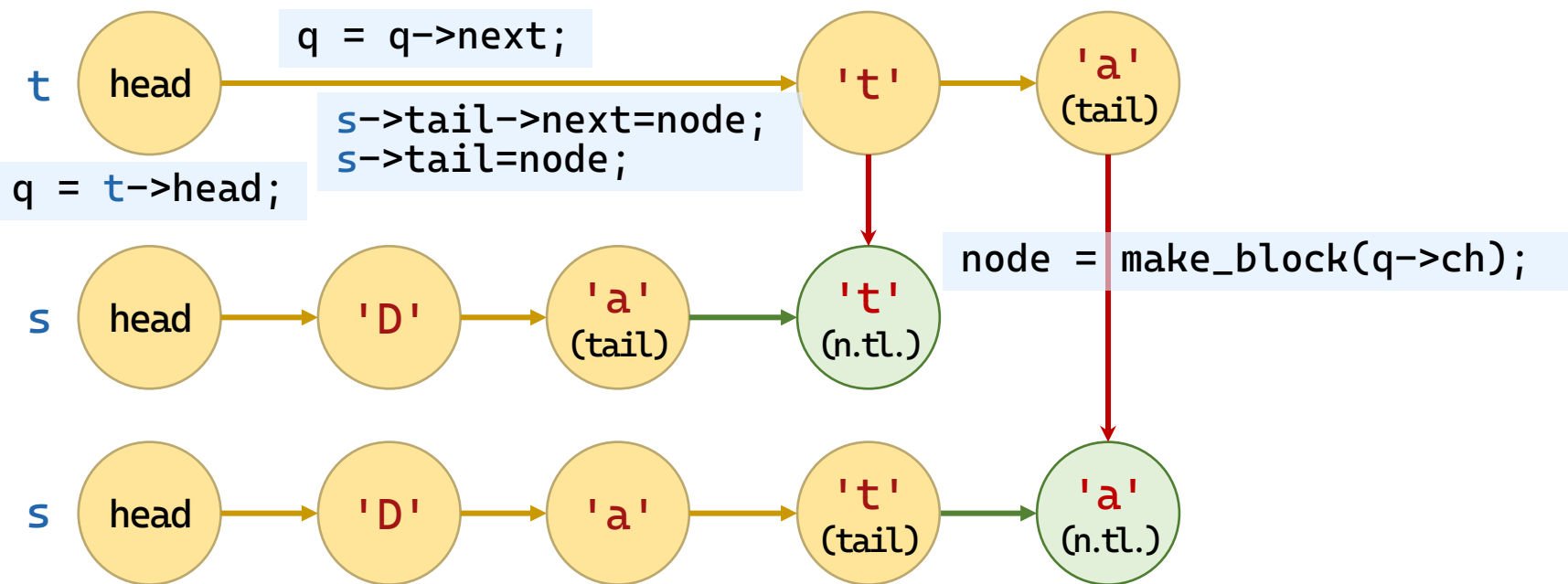
1. 串指针空

```
int str_compare(const BlockString* s, const BlockString* t) {
 if (s == NULL || t == NULL) return 0;
 Block* p = s->head, * q = t->head;
 while (p != NULL && q != NULL) {
 if (p->ch != q->ch)
 return (p->ch > q->ch) ? 1 : -1;
 p = p->next;
 q = q->next;
 }
 if (s->len == t->len) return 0;
 return (s->len > t->len) ? 1 : -1;
}
```

# 块串：拼接

- 入串：赋值到顶部并前进

主体步骤



```
if (s == NULL || t == NULL) return ERROR;
```

边界判定

- 串指针空
- 位置越界

```

int str_cat(BlockString* s, const BlockString* t) {
 if (s == NULL || t == NULL) return ERROR;
 if (t->len == 0) return OK;
 if (s->len == 0) // 若 s 为空, 直接复制 t
 return str_copy(s, t);
 Block* q = t->head; // 复制 t 的结点
 while (q != NULL) {
 Block* node = make_block(q->ch);
 if (node == NULL) return ERROR;
 s->tail->next = node;
 s->tail = node;
 q = q->next;
 }
 s->len += t->len;
 return OK;
}

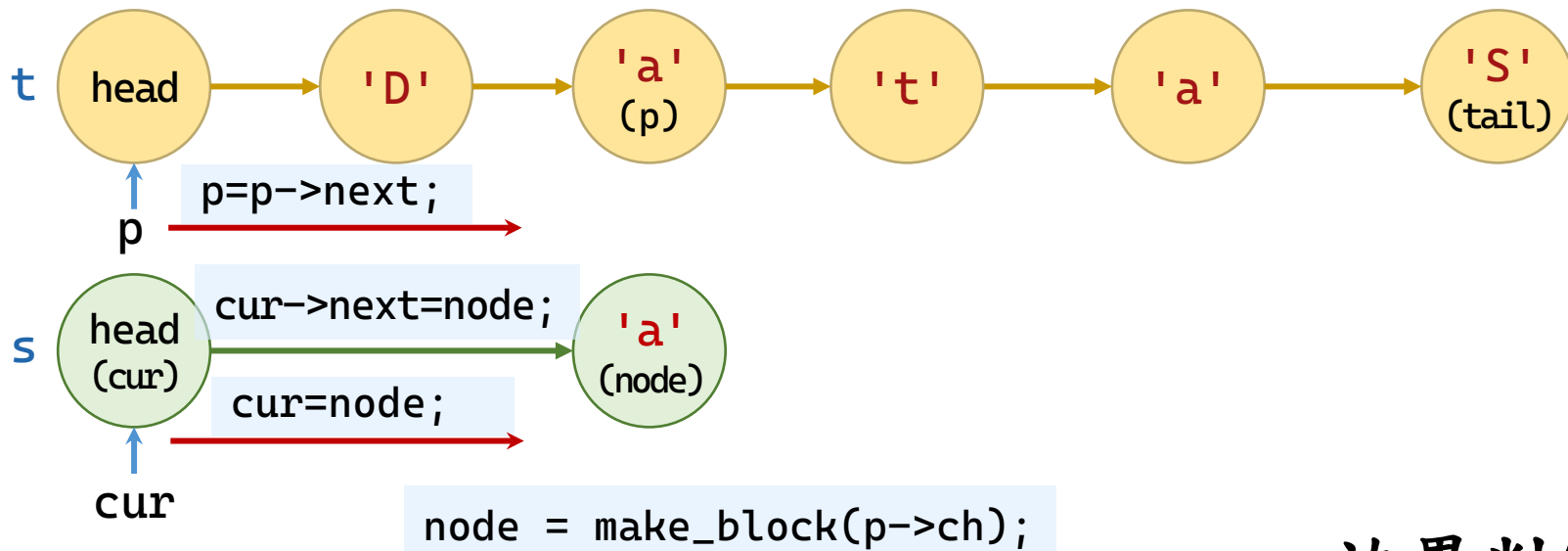
```

# 块串：子串

## • 入串：赋值到顶部并前进

## 主体步骤

```
Block* p = s->head;
for (i=1; i<pos && p!=NULL; i++)
 p = p->next;
```



```
cur = sub->head;
```

```
if (s == NULL || t == NULL) return ERROR;
```

## 边界判定

1. 串指针空
2. 位置越界

```
int str_substr(BlockString* sub, const BlockString* s,
int pos, int len) {
 if (sub == NULL || s == NULL) return ERROR;
 if (pos < 1 || pos > s->len || len < 0) return ERROR;
 if (pos + len - 1 > s->len) len = s->len - pos + 1;
 str_clear(sub);
 if (len == 0) return OK;
 Block* p = s->head; // 找到起始位置的前驱
 for (int i = 1; i < pos && p != NULL; i++)
 p = p->next;
 if (p == NULL) return ERROR; // 防御
 // 创建子串的头结点
 sub->head = make_block(0);
 if (sub->head == NULL) return ERROR;
 Block* cur = sub->head;
```



```
for (int i = 0; i < len; i++) {
 if (p == NULL) break;
 Block* node = make_block(p->ch);
 if (node == NULL) {
 str_clear(sub);
 return ERROR;
 }
 cur->next = node;
 cur = node;
 p = p->next;
}
sub->tail = cur;
sub->len = len;
return OK;
}
```

# 块串：子串查找

## • Brute-Force模式匹配算法

— 对齐开头

```
Block* s_cur = s->head->next;
for (int i = 1; i < *pos && s_cur != NULL; i++)
 s_cur = s_cur->next;
```

— 逐位比较

```
while (*pos <= s->len - t->len + 1) {
 for (int j = 0; j < t->len; j++) {
```

— 成功就走

```
 if (p == NULL || p->ch != q->ch) {
 match = 0; break;
 }
 p = p->next; q = q->next;
```

— 失败重头

```
s_cur = s_cur->next; // 不匹配, 移动到下一个起始位置
(*pos)++;
```

```
}
```

### 边界判定

1. 串指针空
2. 位置越界
3. 边界空

```

int str_index(const BlockString* s, const BlockString* t, int* pos) {
 if (s == NULL || t == NULL || pos == NULL) return ERROR;
 if (t->len == 0 || t->len > s->len) return ERROR;
 if (*pos < 1 || *pos > s->len) return ERROR;
 Block* s_cur = s->head->next;
 for (int i = 1; i < *pos && s_cur != NULL; i++)
 s_cur = s_cur->next;
 if (s_cur == NULL) return ERROR; // 起始位置无效
 while (*pos <= s->len - t->len + 1) {
 Block *p = s_cur, *q = t->head->next;
 int match = 1;
 for (int j = 0; j < t->len; j++) {
 if (p == NULL || p->ch != q->ch) {
 match = 0; break;
 }
 p = p->next; q = q->next;
 }
 if (match) // *pos 已经是匹配位置 (1-based)
 return OK;
 s_cur = s_cur->next; // 不匹配, 移动到下一个起始位置
 (*pos)++;
 }
 return ERROR;
}

```

4

谢谢观看



廈門大學  
XIAMEN UNIVERSITY



信息学院 黄 焯  
(特色化示范性软件学院) 博士, 副教授  
School of Informatics Wei Huang

数据结构与算法

Data Structures

5

# 数组和广义表

理论课程



廈門大學  
XIAMEN UNIVERSITY



信息学院 黄 焯  
(特色化示范性软件学院) 博士, 副教授  
School of Informatics Wei Huang

# 内容要点

- 数组
- 矩阵
  - 特殊矩阵
  - 稀疏矩阵
- 广义表

# 目录

|  |            |                   |
|--|------------|-------------------|
|  | <b>1</b>   | <b>数组</b>         |
|  | <b>1.1</b> | <b>数组的定义</b>      |
|  | <b>1.2</b> | <b>数组的顺序表示和实现</b> |
|  | <b>2</b>   | <b>矩阵</b>         |
|  | <b>3</b>   | <b>广义表</b>        |

# 数组

- 数组是一种数据结构，一般作为程序设计语言的固有类型。
- 数组可以看成是线性表的推广。
- 科学计算中涉及到大量的矩阵问题，被描述成一个二维数组。
  - 当矩阵规模很大且具有特殊结构（ 对角矩阵、三角矩阵、对称矩阵、稀疏矩阵等 ），为减少程序的时间和空间需求，采用自定义的描述方式。



# 基本概念

- 数组是一组偶对（下标值，数据元素值）的集合。
- 在数组中，对于一组有意义的下标，都存在一个与其对应的值。
- 一维数组对应着一个下标值，二维数组对应着两个下标值，如此类推。
- 数组是由  $n$  ( $n > 1$ ) 个具有相同数据类型的数据元素  $a_1, a_2, \dots, a_n$  组成的有序序列，且该序列必须存储在一块地址连续的存储单元中。

# 基本概念

- 特点

- 数组中的数据元素具有相同数据类型。
- 数组是一种随机存取结构，给定一组下标，就可以访问与其对应的数据元素。
- 数组中的数据元素个数是固定的。

# 数组的抽象数据类型定义

- 定义

ADT Array {

数据对象： $D = \{ a_{j_1, j_2, \dots, j_n} \mid a_{j_1, j_2, \dots, j_n} \in \text{ElemSet}, \text{数组元素第 } i \text{ 维下标 } j_i = 0, 1, \dots, b_i - 1, b_i \text{ 是数组第 } i \text{ 维的长度}, i = 1, 2, \dots, n, n > 0 \text{ 称为数组的维数} \}$

数据关系： $R = \{ R_1, R_2, \dots, R_n \}$

$R_i = \{ \langle a_{j_1, j_2, \dots, j_i, \dots, j_n}, a_{j_1, j_2, \dots, j_i + 1, \dots, j_n} \rangle \mid 0 \leq j_k \leq b_k - 1, 1 \leq k \leq n \text{ 且 } k \neq i, 0 \leq j_i \leq b_i - 2, \}$

$a_{j_1, j_2, \dots, j_i, \dots, j_n}, a_{j_1, j_2, \dots, j_i + 1, \dots, j_n} \in D \}$

基本操作：.....

} ADT Array

# 数组的抽象数据类型定义

- 由上述定义知， $n$ 维数组中有  $b_1 \times b_2 \times \dots \times b_n$  个数据元素，每个数据元素都受到  $n$  维关系的约束。
- 直观的  $n$  维数组
  - 以二维数组为例讨论。将二维数组看成是一个定长的线性表，其每个元素又是一个定长的线性表。
  - 设二维数组  $A = (a_{ij})_{m \times n}$ ，则  $A = (\alpha_1, \alpha_2, \dots, \alpha_p)$ ， $p = m$  或  $n$
  - 其中每个数据元素  $\alpha_j$  是一个列向量(线性表)：
    - $\alpha_j = (\alpha_{1,j}, \alpha_{2,j}, \dots, \alpha_{m,j}) \quad 1 \leq j \leq n$
    - 或是一个行向量： $\alpha_i = (\alpha_{i,1}, \alpha_{i,2}, \dots, \alpha_{i,n}) \quad 1 \leq i \leq m$
  - 如图5-1所示。

# 数组的抽象数据类型定义

- 二维数组图例形式

$$\mathbf{A} = \left( \begin{array}{|c|c|c|c|} \hline a_{11} & a_{12} & \dots & a_{1n} \\ \hline a_{21} & a_{22} & \dots & a_{2n} \\ \hline \vdots & \vdots & \ddots & \vdots \\ \hline a_{m1} & a_{m2} & \dots & a_{mn} \\ \hline \end{array} \right) \quad \mathbf{A} = \left( \begin{array}{l} \left( \begin{array}{|c|c|c|c|} \hline a_{11} & a_{12} & \dots & a_{1n} \\ \hline \end{array} \right) \\ \left( \begin{array}{|c|c|c|c|} \hline a_{21} & a_{22} & \dots & a_{2n} \\ \hline \end{array} \right) \\ \left( \begin{array}{|c|c|c|c|} \hline \vdots & \vdots & \ddots & \vdots \\ \hline \end{array} \right) \\ \left( \begin{array}{|c|c|c|c|} \hline a_{m1} & a_{m2} & \dots & a_{mn} \\ \hline \end{array} \right) \end{array} \right)$$

(a) 矩阵表示形式

(b) 行向量的一维数组形式

$$\mathbf{A} = \left( \begin{array}{|c|} \hline a_{11} \\ \hline a_{21} \\ \hline \vdots \\ \hline a_{m1} \\ \hline \end{array} \right) \left( \begin{array}{|c|} \hline a_{12} \\ \hline a_{22} \\ \hline \vdots \\ \hline a_{m2} \\ \hline \end{array} \right) \left( \begin{array}{|c|} \hline \dots \\ \hline \dots \\ \hline \ddots \\ \hline \dots \\ \hline \end{array} \right) \left( \begin{array}{|c|} \hline a_{1n} \\ \hline a_{2n} \\ \hline \vdots \\ \hline a_{mn} \\ \hline \end{array} \right)$$

(c) 列向量的一维数组形式

# 目录

|  |     |            |
|--|-----|------------|
|  | 1   | 数组         |
|  | 1.1 | 数组的定义      |
|  | 1.2 | 数组的顺序表示和实现 |
|  | 2   | 矩阵         |
|  | 3   | 广义表        |

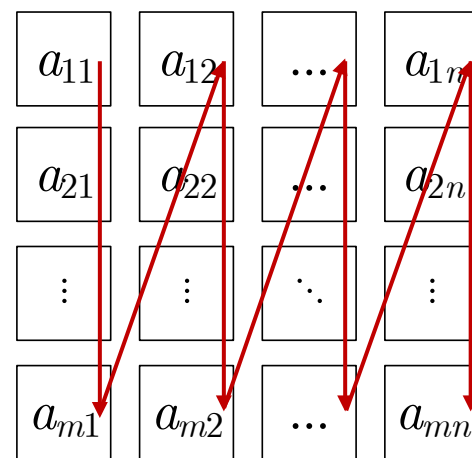
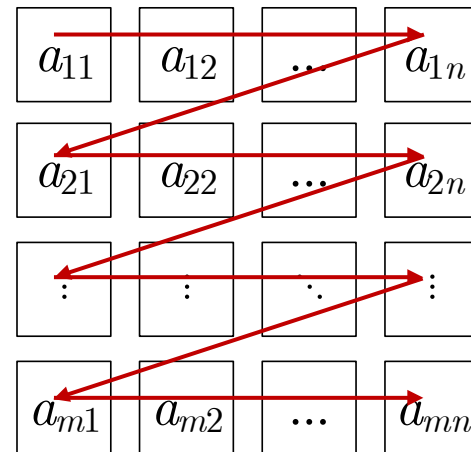
# 数组的顺序表示和实现

- 数组一般不做插入和删除操作，可采用顺序存储的方法来表示数组。
- 问题：计算机的内存结构是一维(线性)地址结构，对于多维数组，将其存放(映射)到内存一维结构时，有个次序约定问题。即必须按某种次序将数组元素排成一系列列，然后将这个线性序列存放到内存中。
- 二维数组是最简单的多维数组，以此为例说明多维数组存放(映射)到内存一维结构时的次序约定问题。

# 数组的顺序表示和实现

- 通常有两种顺序存储方式

- 行优先顺序(Row Major Order)：将数组元素按行排列，第 $i+1$ 个行向量紧接在第 $i$ 个行向量后面。
- PASCAL、C是按行优先顺序存储的。
- 列优先顺序(Column Major Order)：将数组元素按列向量排列，第 $j+1$ 个列向量紧接在第 $j$ 个列向量之后。
- FORTRAN、R、MATLAB是按列优先顺序存储的。



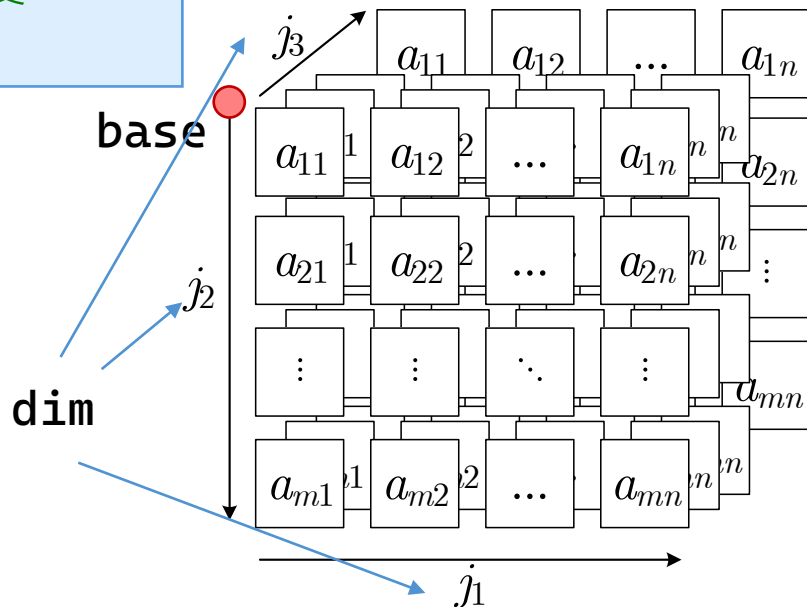


# 数组的存储结构

- 数组的存储结构

```
typedef struct {
 int* base; // 元素基址
 int dim; // 维数
 int* bounds; // 各维大小 (j)
 int* constants; // 该维的元素跨度
} Array;
```

`constants[i]`指的是第*i*维每前进1步，  
相当于最低一层的元素前进几步。  
用于定位的时候提高速度避免重复计算  
`bounds[i+1]*bounds[i+2]*...  
*bounds[dim]`。



# 数组：初始化

- 初始化

- 设置维度 `A->dim = dim;`

- 设置各维度长度

- 计算元素个数

- 开辟空间

```
A->bounds = (int*)malloc(dim * sizeof(int));
int elemtotal = 1;
for (int i = 0; i < dim; ++i) {
 A->bounds[i] = bounds[i];
 elemtotal *= bounds[i];
}
```

```
A->base = (int*)malloc(elemtotal * sizeof(int));
memset(A->base, 0, elemtotal * sizeof(int));
```

- 计算各维的底层跨度

```
A->constants = (int*)malloc(dim * sizeof(int));
A->constants[dim - 1] = 1;
for (int i = dim - 2; i >= 0; --i)
 A->constants[i] = A->bounds[i + 1] * A->constants[i + 1];
```

```

int InitArray(Array* A, int dim, int bounds[]) {
 if (dim < 1 || dim > MAX_ARRAY_DIM) return ERROR;
 A->dim = dim;
 A->bounds = (int*)malloc(dim * sizeof(int));
 if (!A->bounds) exit(OVERFLOW);
 int elemtotal = 1;
 for (int i = 0; i < dim; ++i) {
 if (bounds[i] <= 0) return UNDERFLOW;
 A->bounds[i] = bounds[i];
 elemtotal *= bounds[i];
 }
 A->base = (int*)malloc(elemtotal * sizeof(int));
 if (!A->base) exit(OVERFLOW);
 memset(A->base, 0, elemtotal * sizeof(int));
 A->constants = (int*)malloc(dim * sizeof(int));
 if (!A->constants) exit(OVERFLOW);
 A->constants[dim - 1] = 1;
 for (int i = dim - 2; i >= 0; --i)
 A->constants[i] = A->bounds[i + 1] * A->constants[i + 1];
 return OK;
}

```

# 数组：销毁数组

- 销毁：释放所有堆上的空间

```
int DestroyArray(Array* A) {
 if (!A->base) return ERROR;
 free(A->base); A->base = NULL;
 free(A->bounds); A->bounds = NULL;
 free(A->constants); A->constants = NULL;
 return OK;
}
```

# 数组：定位

- 定位：计算偏移量

$$I = \sum_{i=0}^d c_i \cdot i_i$$

```
int Locate(Array A, int indices[], int* off) {
 *off = 0;
 for (int i = 0; i < A.dim; ++i) {
 if (indices[i] < 0 || indices[i] >= A.bounds[i])
 return OVERFLOW;
 *off += A.constants[i] * indices[i];
 }
 return OK;
}
```

# 数组：赋值与取值

- 赋值与取值：按偏移量进行 
$$I = \sum_{i=0}^d c_i \cdot i_i$$

```
int Value(Array A, int* e, int indices[]) {
 int off;
 int s = Locate(A, indices, &off);
 if (s != OK) return s;
 *e = *(A.base + off);
 return OK;
}
```

```
int Assign(Array* A, int e, int indices[]) {
 int off;
 int s = Locate(*A, indices, &off);
 if (s != OK) return s;
 *(A->base + off) = e;
 return OK;
}
```

# 数组：显示

- 数组的显示是一个递归问题
  - 一维打印一行
  - 二维打印行列，显示行号
  - 三维以上带层次，为分隔的 $N-1$ 维问题

```

void print_array_rec(Array A, int cur_dim, int* indices, int depth) {
 if (cur_dim == A.dim - 1) {
 for (int i = 0; i < A.bounds[cur_dim]; ++i) {
 indices[cur_dim] = i;
 int val;
 Value(A, &val, indices);
 printf("%4d ", val);
 }
 printf("\n");
 } else if (cur_dim == A.dim - 2) {
 for (int i = 0; i < A.bounds[cur_dim]; ++i) {
 indices[cur_dim] = i;
 for (int d = 0; d < depth; ++d) printf(" ");
 printf("行%d: ", i);
 print_array_rec(A, cur_dim+1, indices, depth+1);
 }
 } else {
 for (int i = 0; i < A.bounds[cur_dim]; ++i) {
 indices[cur_dim] = i;
 for (int d = 0; d < depth; ++d) printf(" "); // 缩进
 printf("维%d[%d]:\n", cur_dim, i);
 print_array_rec(A, cur_dim + 1, indices, depth + 1);
 }
 }
}

```



```

void DisplayArray(Array A) {
 if (A.base == NULL) {
 printf("数组未初始化。\\n"); return;
 }
 int total = 1;
 for (int i = 0; i < A.dim; ++i) total *= A.bounds[i];
 printf("数组维度: %d, 总元素数: %d\\n", A.dim, total);
 if (total == 0) return;
 int* indices = (int*)malloc(A.dim * sizeof(int));
 if (!indices)
 printf("内存不足, 无法显示。\\n"); return;
 if (A.dim == 1) { // 一维: 直接打印一行
 printf("一维数组: \\n");
 for (int i = 0; i < A.bounds[0]; ++i) {
 indices[0] = i;
 int val;
 Value(A, &val, indices);
 printf("%4d ", val);
 }
 printf("\\n");
 } else if (A.dim == 2) { // 二维: 打印矩阵, 每行显示行号

```

```

printf("数组 (%d行 x %d列) : \n", A.bounds[0], A.bounds[1]);
for (int i = 0; i < A.bounds[0]; ++i) {
 indices[0] = i;
 printf("行%d: ", i);
 for (int j = 0; j < A.bounds[1]; ++j) {
 indices[1] = j;
 int val;
 Value(A, &val, indices);
 printf("%4d ", val);
 }
 printf("\n");
}
} else { // 三维及以上: 递归打印带层次结构
 printf("多维数组 (各维大小: ");
 for (int i = 0; i < A.dim; ++i) printf("%d%s",
A.bounds[i], (i == A.dim - 1 ? "): \n" : " x "));
 print_array_rec(A, 0, indices, 0);
}
free(indices);
}

```

# 目录

|  |     |         |
|--|-----|---------|
|  | 1   | 数组      |
|  | 2   | 矩阵的压缩存储 |
|  | 2.1 | 特殊矩阵    |
|  | 2.2 | 稀疏矩阵    |
|  | 3   | 广义表     |

# 矩阵的压缩存储

- 矩阵是一种常用的数学对象，在高级语言编程时，通常将一个矩阵描述为一个二维数组。
- 对于高阶矩阵，若其中非零元素呈某种规律分布或者矩阵中有大量的零元素，若仍然用常规方法存储，可能存储重复的非零元素或零元素，将造成存储空间的大量浪费。
- 对这类矩阵进行压缩存储：
  - 多个相同的非零元素只分配一个存储空间；
  - 零元素不分配空间。

# 对称矩阵

- 特殊矩阵：是指非零元素或零元素的分布有一定规律的矩阵。
- 对称矩阵：若一个 $n$ 阶方阵 $A=(a_{ij})_{n \times n}$ 中的元素满足性质： $a_{ij}=a_{ji}$ ,  $1 \leq i, j \leq n$ 且 $i \neq j$ ，则称 $A$ 为对称矩阵。
  - 对称矩阵中的元素关于主对角线对称，因此，让每一对对称元素 $a_{ij}$ 和 $a_{ji}$  ( $i \neq j$ ) 分配一个存储空间，则 $n^2$ 个元素压缩存储到 $n(n+1)/2$ 个存储空间，能节约近一半的存储空间。

$$A = \begin{pmatrix} 5 & 0 & 8 & 0 & 0 \\ 1 & 8 & 9 & 2 & 6 \\ 3 & 0 & 2 & 5 & 1 \\ 7 & 0 & 6 & 1 & 3 \end{pmatrix}$$

# 对称矩阵

- 对称矩阵

- 存储顺序（行优先顺序为例）：

- $a_{11} \ a_{21} \ a_{22} \ a_{31} \ a_{32} \ a_{33} \ \dots \ a_{n1} \ a_{n2} \ \dots \ a_{nn}$

$$k = \begin{cases} \frac{i(i-1)}{2} + (j-1), & i \geq j \\ \frac{j(j-1)}{2} + (i-1), & i < j \end{cases} \quad i = \left\lfloor \frac{1 + \sqrt{1 + 8k}}{2} \right\rfloor, j = k - \frac{i(i-1)}{2} + 1$$

# 三角矩阵

- 三角矩阵

- 以主对角线划分，三角矩阵有上三角和下三角两种。
- 上三角矩阵的下三角（不包括主对角线）中的元素均为常数 $c$ （一般为0）。下三角矩阵正好相反，它的主对角线上方均为常数。
- 三角矩阵中的重复元素 $c$ 可共享一个存储空间，其余的元素正好有 $n(n+1)/2$ 个。三角矩阵存储到向量 $[0, n(n+1)/2)$ 中，其中 $c$ 存放在向量的第 $n(n+1)/2$ 个分量中。
- 下三角的转换关系参考前页对称矩阵。上三角矩阵的情况将坐标对调。

# 对角矩阵

- 对角矩阵：矩阵中，除了主对角线和主对角线上或下方若干条对角线上的元素之外，其余元素皆为零。
  - 所有的非零元素集中在以主对角线为中心的带状区域中。

$$A = \begin{pmatrix} a_{11} & a_{12} & 0 & \dots & 0 \\ a_{21} & a_{22} & a_{23} & 0 & \dots & 0 \\ 0 & a_{32} & a_{33} & a_{34} & 0 & \dots & 0 \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ 0 & \dots & 0 & a_{n-1 \ n-2} & a_{n-1 \ n-1} & a_{n-1 \ n} \\ 0 & \dots & 0 & 0 & a_{n \ n-1} & a_{n \ n} \end{pmatrix}$$



# 对角矩阵

- 对角矩阵可按行优先顺序或对角线顺序，将其压缩存储到一个向量中，并且也能找到每个非零元素和向量下标的对应关系。
- 矩阵行列和偏移量的对应关系无需背诵，根据实际需要自行计算。

# 目录

|  |     |         |
|--|-----|---------|
|  | 1   | 数组      |
|  | 2   | 矩阵的压缩存储 |
|  | 2.1 | 特殊矩阵    |
|  | 2.2 | 稀疏矩阵    |
|  | 3   | 广义表     |

# 稀疏矩阵

- 稀疏矩阵(Sparse Matrix)：没有确切的定义。
  - 稀疏因子：设矩阵A是一个 $n \times m$ 的矩阵中有 $s$ 个非零元素，设  $\delta = s / (n \times m)$ ，称 $\delta$ 为稀疏因子。
  - 如果某一矩阵的稀疏因子 $\delta$ 满足 $\delta \leq 0.05$ 时称为稀疏矩阵。
- 压缩存储：只存储非0元素，行、列、元素值。
  - 一个三元组 $(i, j, a_{ij})$ 唯一确定稀疏矩阵的一个非零元素。

$$\mathbf{A} = \begin{pmatrix} 0 & 12 & 9 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 0 & 0 & 0 & 4 \\ 0 & 0 & 24 & 0 & 0 & 2 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & -7 & 0 \\ 0 & 0 & 0 & -6 & 0 & 0 & 0 & 0 \end{pmatrix}$$



$((1,2,12), (1,3,9), (3,1,-3), (3,8,4),$   
 $(4,3,24), (5,2,18), (6,7,-7), (7,4,-6) )$

# 三元组顺序表

- 若以行序为主序，稀疏矩阵中所有非0元素的三元组，就可以得构成该稀疏矩阵的一个三元组顺序表。
- 三元组顺序表定义

```
typedef struct {
 int row;
 int col;
 int e;
} Triple;
typedef struct {
 Triple data[MAXSIZE + 1]; // 三元组表，从下标1开始存储
 int m, n; // 行数、列数
 int len; // 非零元个数
} SparseMatrix;
```

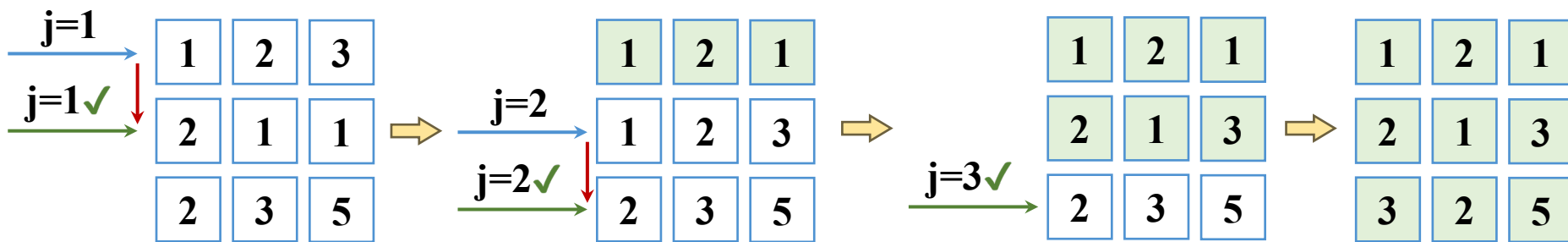
# 稀疏矩阵的转置（简单）

- 主要操作：行、列互换；问题：换完保持行主序排列

|   |   |   |
|---|---|---|
| 0 | 3 | 0 |
| 1 | 0 | 5 |
| 0 | 0 | 0 |



|   |   |   |
|---|---|---|
| 0 | 1 | 0 |
| 3 | 0 | 0 |
| 0 | 5 | 0 |



- 时间复杂度： $O(n \times L)$  行乘以长

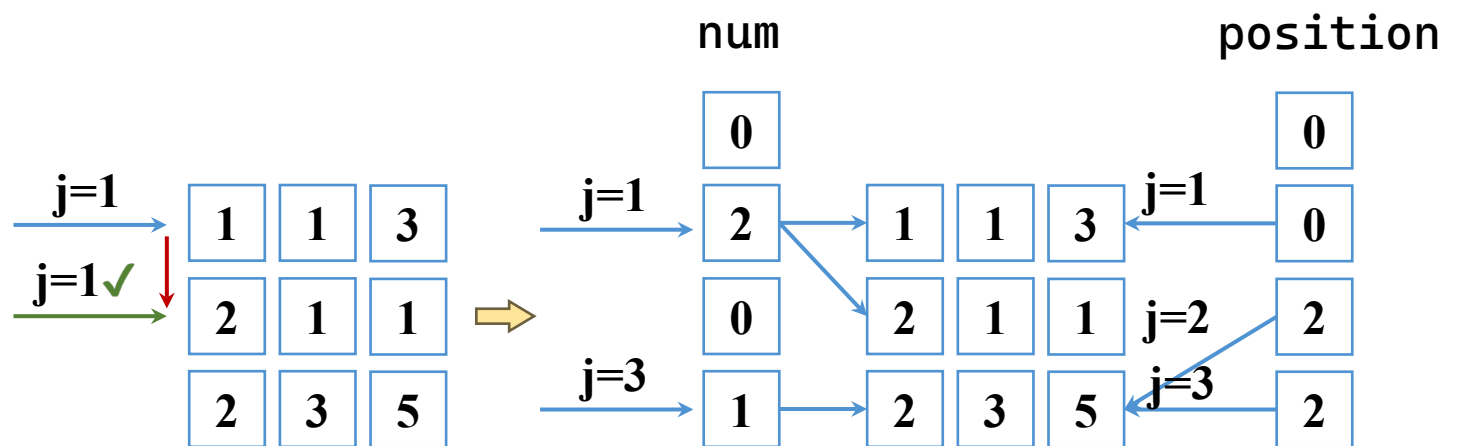
```

int transpose(const SparseMatrix* A, SparseMatrix* B) {
 if (A == NULL || B == NULL) return ERROR;
 B->m = A->n; B->n = A->m; B->len = A->len;
 if (B->len == 0) return OK;
 int j = 1; // B 中三元组当前存放位置
 for (int col = 1; col <= A->n; col++) {
 for (int i = 1; i <= A->len; i++) {
 if (A->data[i].col == col) {
 B->data[j].row = A->data[i].col;
 B->data[j].col = A->data[i].row;
 B->data[j].e = A->data[i].e;
 j++;
 }
 }
 }
 return OK;
}

```

# 稀疏矩阵的转置（快速）

- 简单转置的改进思路：只需要一层遍历
  - 算出每一列在目标三元组数组的偏移量
  - 每次处理一个数组，偏移量自增一
- 时间复杂度： $O(n + L)$  行加长



第j列存储，从  
position[j]开始

```

int fast_transpose(const SparseMatrix* A, SparseMatrix* B) {
 if (A == NULL || B == NULL) return ERROR;
 B->m = A->n; B->n = A->m; B->len = A->len;
 if (B->len == 0) return OK;
 int num[MAXROW + 2] = { 0 };
 int position[MAXROW + 2];
 for (int t = 1; t <= A->len; t++) // 统计每列非零元个数
 num[A->data[t].col]++;
 position[1] = 1; // 计算每列第一个元素在 B 中的起始位置
 for (int col = 2; col <= A->n; col++)
 position[col] = position[col - 1] + num[col - 1];
 for (int p = 1; p <= A->len; p++) { // 转置
 int col = A->data[p].col;
 int q = position[col];
 B->data[q].row = A->data[p].col;
 B->data[q].col = A->data[p].row;
 B->data[q].e = A->data[p].e;
 position[col]++;
 }
 return OK;
}

```



# 应用：矩阵乘法

- 于 A 的第  $i$  行与 B 的第  $j$  列对应元素乘积之和

$$c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj}, \quad 1 \leq i \leq m, 1 \leq j \leq p$$

- 方法

– 遍历每一行  $i$

```
for (row=1; row<=Q->m; row++) {
```

– 遍历该行存储的每一列  $k$

```
 for (p=1; p<=M->len; p++) {
 if (M->data[p].row!=row) continue;
```

– 对应计算  $a_{ik}b_{kj}$

```
 for (q=1; q<=N->len; q++) {
 if (N->data[q].row != M->data[p].col) continue;
```

– 并相加存入  $c_{ij}$

```
 temp[N->data[q].col]+=M->data[p].e*N->data[q].e;
```

```
for (int col = 1; col <= Q->n; col++)
 if (temp[col] != 0) {
 Q->len++; Q->data[Q->len].row=row;
 Q->data[Q->len].col=col; Q->data[Q->len].e=temp[col];
 }
```

# 应用：矩阵乘法

- 改进：记录每一行从data的偏移量到first

```
void build_first(SparseMatrix* A) {
 int num[MAXROW + 2] = { 0 };
 for (int i = 1; i <= A->len; i++) {
 num[A->data[i].row]++;
 }
 A->first[1] = 1;
 for (int i = 2; i <= A->m; i++) {
 A->first[i] = A->first[i - 1] + num[i - 1];
 }
 A->first[A->m + 1] = A->len + 1;
}
```

```

int multiply_matrix(const SparseMatrix* M,
 const SparseMatrix* N, SparseMatrix* Q) {
 if (M == NULL || N == NULL || Q == NULL) return ERROR;
 if (M->n != N->m) return ERROR;
 Q->m = M->m; Q->n = N->n; Q->len = 0;
 if (M->len == 0 || N->len == 0) return OK; // 结果为零矩阵
 int temp[MAXROW + 2] = { 0 };
 for (int row = 1; row <= Q->m; row++) {
 for (int col = 1; col <= Q->n; col++) temp[col] = 0;
 // 遍历 M 的第 row 行
 for (int p = M->first[row]; p < M->first[row + 1]; p++) {
 int col_m = M->data[p].col, val_m = M->data[p].e;
 // 遍历 N 的第 col_m 行
 for (int q = N->first[col_m]; q < N->first[col_m + 1]; q++) {
 int col_n = N->data[q].col, val_n = N->data[q].e;
 temp[col_n] += val_m * val_n;
 }
 }
 }
}

```

```

// 将非零结果存入 Q
for (int col = 1; col <= Q->n; col++) {
 if (temp[col] != 0) {
 if (Q->len >= MAXSIZE) {
 printf("乘积矩阵非零元数量超过 %d, 部分截断。
\n", MAXSIZE);
 return ERROR;
 }
 Q->len++;
 Q->data[Q->len].row = row;
 Q->data[Q->len].col = col;
 Q->data[Q->len].e = temp[col];
 }
}
return OK;
}

```

# 十字链表

- 当矩阵的非零元个数和位置在操作过程中变化较大时，就不宜采用顺序存储结构来表示三元组的线性表。
  - 例如，在作“将矩阵B加到矩阵A上”的操作时，由于非零元的插入或删除将会引起A.data中元素的移动。为此，对这种类型的矩阵，采用链式存储结构表示比三元组的线性表更方便。
- 矩阵中非0元素的结点所含的域有：行、列、值、行指针(指向同一行的下一个非0元)、列指针(指向同一列的下一个非0元)。其次，十字交叉链表还有一个头结点，结点的结构如图5-10所示。

# 十字链表

- 稀疏矩阵中同一行的非0元素的由right指针域链接成一个行链表，由down指针域链接成一个列链表。则每个非0元素既是某个行链表中的一个结点，同时又是某个列链表中的一个结点，所有的非0元素构成一个十字交叉的链表。称为十字链表。用两个一维数组分别存储行链表的头指针和列链表的头指针。
- 对于图5-11(a)的稀疏矩阵A，对应的十字交叉链表如图5-11(b)所示，结点的描述如下：

# 十字链表

- 十字链表的定义

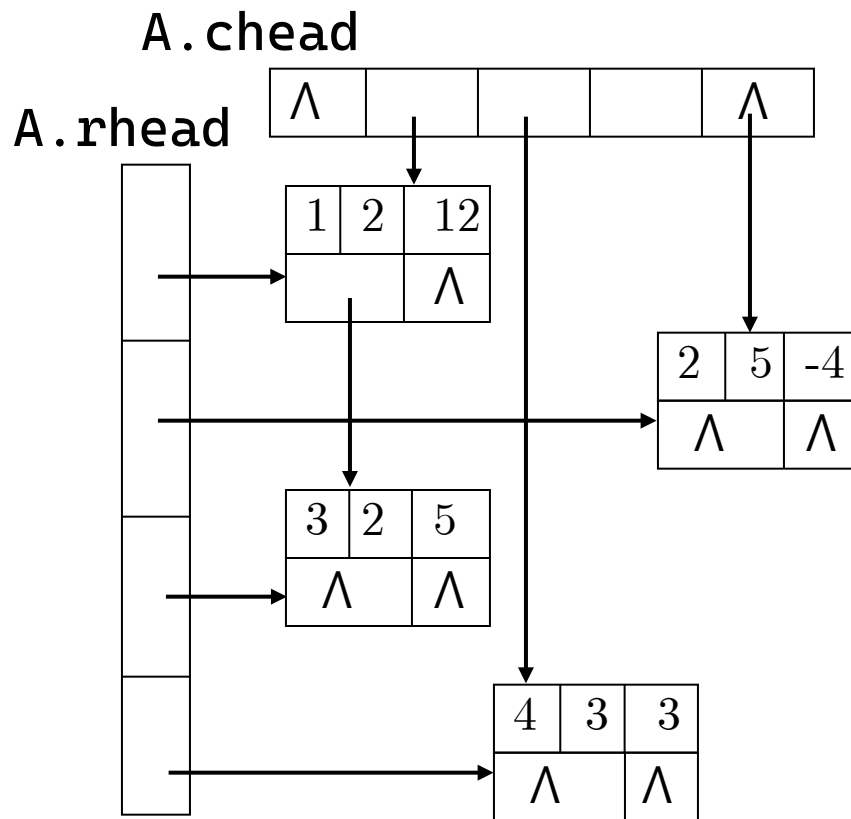
```
typedef struct OLNode {
 int row, col, value;
 struct OLNode* right, * down;
} OLNode, * OLink;

typedef struct {
 OLink* rhead, * chead; // 行、列头指针数组
 int m, n, len; // 行数、列数、非零元个数
} CrossList;
```

# 十字链表

$$\mathbf{A} = \begin{pmatrix} 0 & 12 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -4 \\ 0 & 5 & 0 & 0 & 0 \\ 0 & 0 & 3 & 0 & 0 \end{pmatrix}$$

(a) 稀疏矩阵



(b) 稀疏矩阵的十字交叉链表

图5-11 稀疏矩阵及其十字交叉链表



# 目录

|  |   |         |
|--|---|---------|
|  | 1 | 数组      |
|  | 2 | 矩阵的压缩存储 |
|  | 3 | 广义表     |
|  |   |         |
|  |   |         |

# 广义表

- 广义表(Lists，又称为列表)：是由 $n(n \geq 0)$ 个元素组成的有穷序列： $LS = (a_1, a_2, \dots, a_n)$ ，其中 $a_i$ 或者是原子项，或者是一个广义表。
  - 广义表是线性表的推广和扩充，在人工智能领域应用广泛。
  - 线性表定义为 $n$ 个元素 $a_1, a_2, \dots, a_n$ 的有穷序列，序列中的元素是具有相同数据类型的原子项(Atom)。
  - 所谓原子项可以是一个数或一个结构，是指结构上不可再分的。若放松对元素的这种限制，容许它们具有其自身结构，就产生了广义表的概念。

# 广义表

- 广义表： $LS$ ，长度： $n$ 
  - 习惯上：原子用小写字母，子表用大写字母。
  - 广义表中所包含的元素(包括原子和子表)的个数。
- 子表：
  - 若  $a_i$  是广义表，则称为  $LS$  的子表。
  - 广义表中括号的最大层数称为表深(度)。
- 表头：
  - 若广义表  $LS$  非空时  $a_1$  (表中第一个元素) 称为表头
  - 其余元素组成的子表  $(a_2, a_3, \dots, a_n)$  称为表尾

# 广义表实例

- 有关广义表的这些概念的例子如表5-2所示。

表5-2 广义表及其示例

| 广 义 表           | 表长n | 表深h      |
|-----------------|-----|----------|
| $A=()$          | 0   | 0        |
| $B=(e)$         | 1   | 1        |
| $C=(a,(b,c,d))$ | 2   | 2        |
| $D=(A,B,C)$     | 3   | 3        |
| $E=(a,E)$       | 2   | $\infty$ |
| $F=(( ))$       | 1   | 2        |

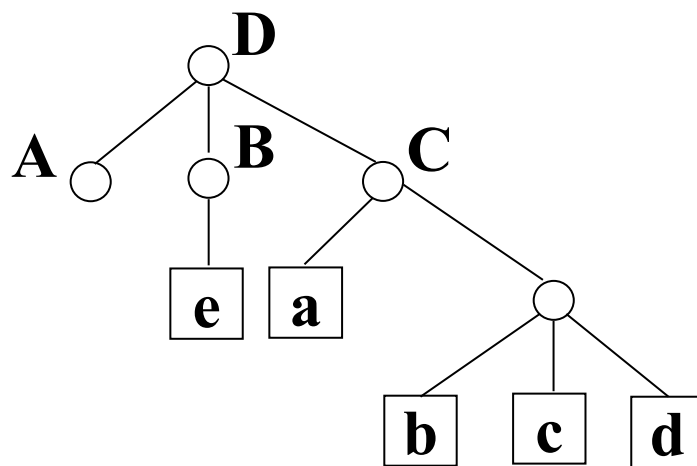


图5-12 广义表的图形表示

# 广义表的重要结论

- 广义表的重要结论：

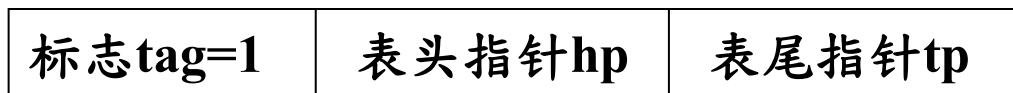
- 广义表的元素可以是原子，也可以是子表，子表的元素又可以是子表，...。即广义表是一个多层次的结构。
- 广义表可以被其它广义表所共享，也可以共享其它广义表。广义表共享其它广义表时通过表名引用。
- 广义表本身可以是一个递归表。
- 根据对表头、表尾的定义，任何一个非空广义表的表头可以是原子，也可以是子表，而表尾必定是广义表。

# 广义表的存储结构

- 由于广义表中的数据元素具有不同的结构，通常用链式存储结构表示，每个数据元素用一个结点表示。因此，广义表中就有两类结点：
  - 一类是表结点，用来表示广义表项，由标志域，表头指针域，表尾指针域组成；
  - 另一类是原子结点，用来表示原子项，由标志域，原子的值域组成。如图5-13所示。



(a) 原子结点



(b) 表结点

图5-13 广义表的链表结点结构示意图

# 广义表的定义

- 只要广义表非空，都是由表头和表尾组成。即一个确定的表头和表尾就唯一确定一个广义表。
- 广义表的定义

```
typedef struct GLNode {
 int tag; // 1-表结点;0-原子结点
 union {
 int value; // 原子结点的值域
 struct {
 struct GLNode* hp, * tp;
 } ptr; // ptr和atom两成员共用
 } Gdata;
} GLNode; // 广义表结点类型
```

```
typedef struct GLNode {
 int tag;
 union {
 int value;
 struct GLNode* hp;
 };
 int GLNode* tp;
} GLNode;
```

# 广义表

- 例：对 $A=()$ ， $B=(e)$ ， $C=(a, (b, c, d))$ ， $D=(A, B, C)$ ， $E=(a, E)$ 的广义表的存储结构如图5-14所示。

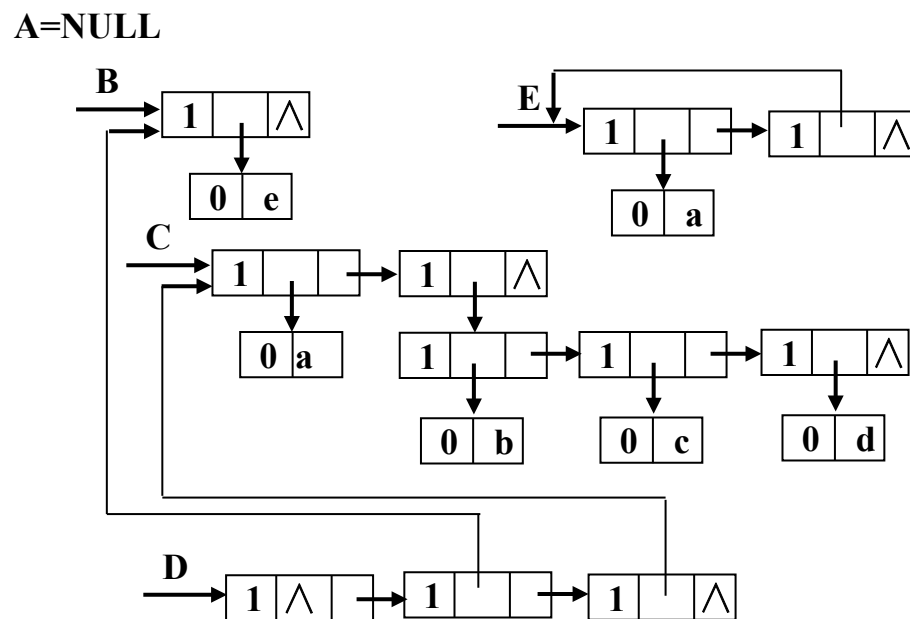
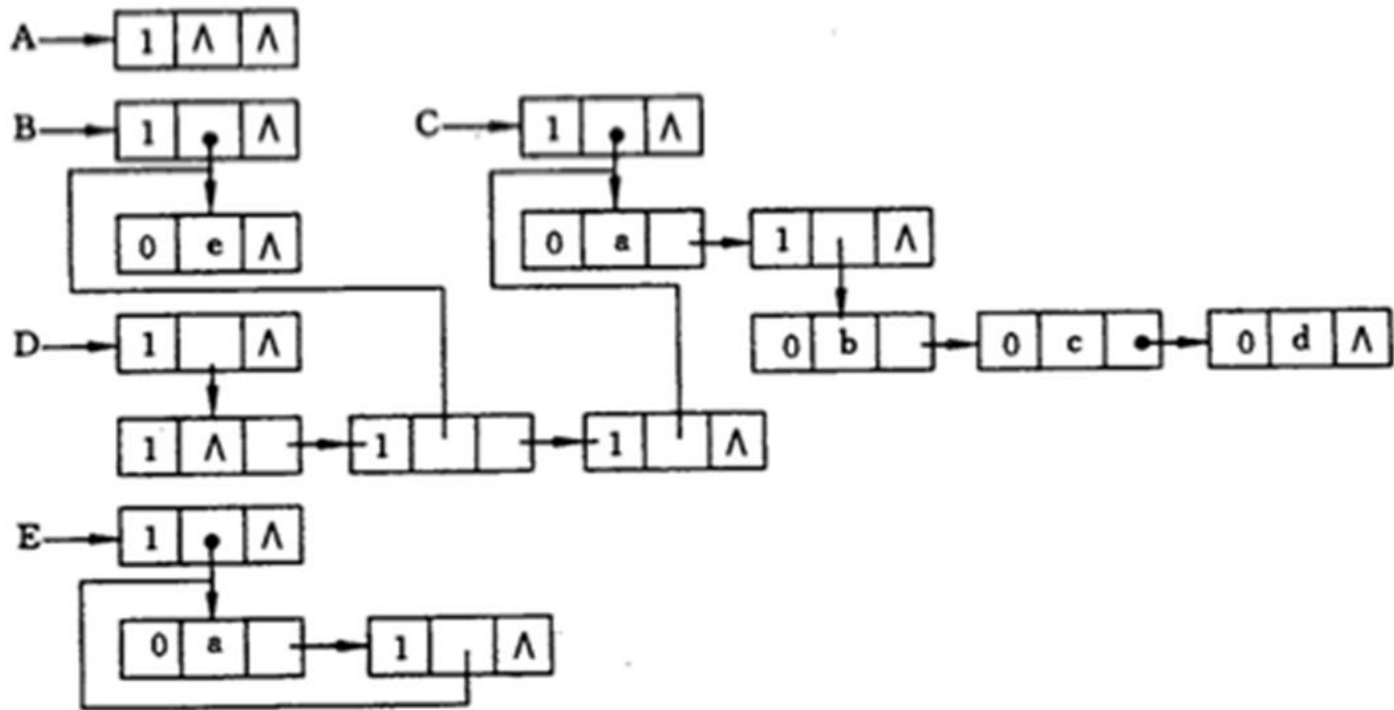


图5-14 广义表的存储结构示意图



# 广义表



# 广义表的结构特点

- 广义表中的数据元素有相对次序;
- 广义表的长度定义为最外层包含的元素个数;
- 广义表的深度定义为所包括弧的重数;
  - 注意:“原子”的深度为“0”;“空表”的深度为1
- 表头可以是原子或列表;表尾必定是列表。
- 广义表可以是一个递归的表;
  - 递归表的深度是无穷值,长度是有限值。
- 任何非空广义表 $LS$ 均可分解为表头  $\text{Head}(LS)=a_1$  和表尾  $\text{Tail}(LS)=(a_2, a_3, \dots, a_n)$  两部分

# 广义表的基本运算

- 广义表的基本运算，除包括线性表的基本运算外，还有求深度、取表头、取表尾、遍历等。
- 这些运算中大部分与对应的线性表、树或者图的运算类似，只是取表头和取表尾是广义表特有的运算。

5

谢谢观看

理论课程



廈門大學  
XIAMEN UNIVERSITY



信息学院 黄 焯  
(特色化示范性软件学院) 博士, 副教授  
School of Informatics Wei Huang

数据结构与算法

Data Structures

6

# 树和二叉树

理论课程



廈門大學  
XIAMEN UNIVERSITY



信息学院 黄 烽  
(特色化示范性软件学院) 博士, 副教授  
School of Informatics Wei Huang

# 内容要点

- 树的定义和基本术语
- 二叉树
- 遍历二叉树和线索二叉树
- 树和森林
- 哈夫曼树及其应用

# 目录

|  |   |             |
|--|---|-------------|
|  | 1 | 树的定义和基本术语   |
|  | 2 | 二叉树         |
|  | 3 | 遍历二叉树和线索二叉树 |
|  | 4 | 树和森林        |
|  | 5 | 哈夫曼树及其应用    |

# 基本概念

- 树(结构)：n个结点的有限集合( $n \geq 0$ )，
  - 空树： $n=0$ 的树(Tree)
- 根结点：没有前驱的结点
  - 对于非空树( $n > 0$ 时)，有且只有1个根结点；
- 前驱和后继
  - 当 $n > 1$ 时，除根结点之外，树中的其它任意1个结点有且只有1个前驱；
  - 树中每个结点可以有m个后继( $m \geq 0$ )。



# 基本概念

- 树的递归定义

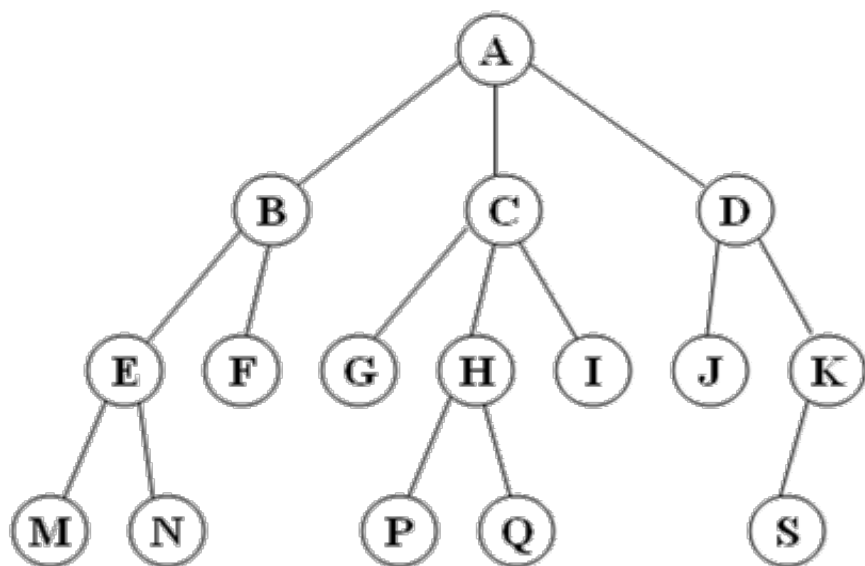
$$T = \begin{cases} \phi, & \text{当 } n=0 \text{ 时} \\ R, & \text{当 } n=1 \text{ 时} \\ R(T_1, \dots, T_m), & \text{当 } n>1 \text{ 时} \end{cases}$$

- 其中， $m>0$ ， $R$ 表示根结点；

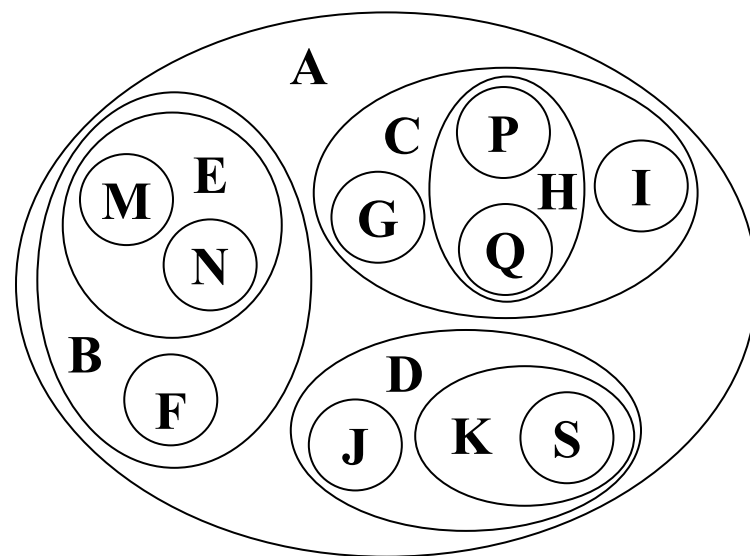
- $T_1, \dots, T_m$ 表示 $m$ 棵子树（ $m$ 个互不相交的有限集）。

# 树的表示

- 树的图形表示



- 树的嵌套集合形式表示

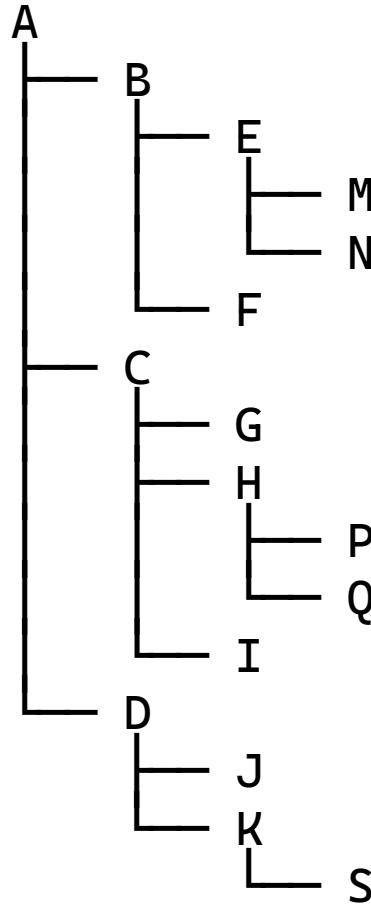


- 树的字符形式表示

$A(B(E(M, N), F), C(G, H(P, Q), I), D(J, K(S)))$ ;

# 树的表示

- 树的凹入表示



# 基本概念

- 树的结点：包含一个数据元素及若干个指针

- 如：ABCD...S

- 这些指针指向其拥有子树的根。

- 结点的度：结点拥有子树的个数。

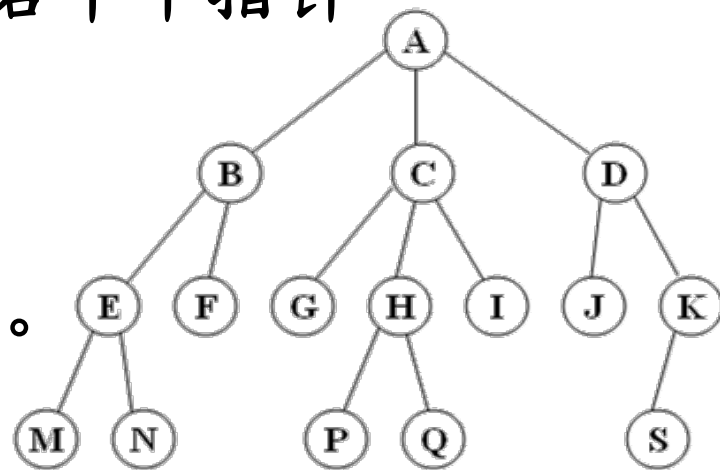
- C的度为3（孩子为G、H、I）。

- K的度为1（孩子为S）。

- F的度为0（没有孩子）。

- 叶子结点：度为0的结点即终端结点。

- 如：F、G、I、J、M、N、P、Q、S



# 基本概念

- 分支结点：度 $>0$ 的结点。

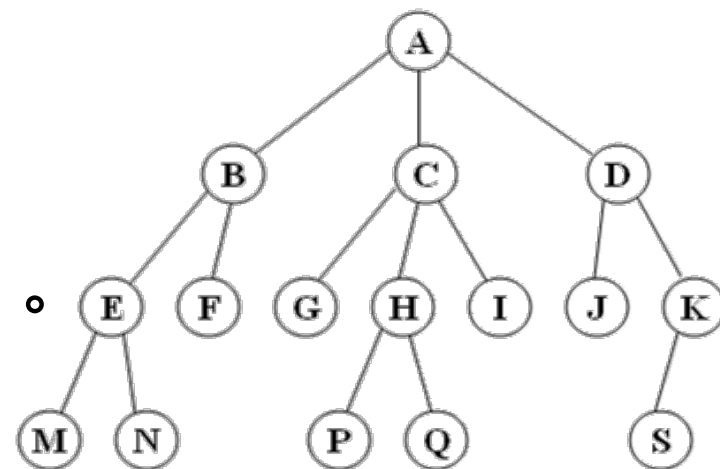
— 如：A、B、C、D、E、H、K。

- 树的度：树的度 $=\max(\text{结点的度})$ 。

— 如：结点 A、C 是 3，其余结点度均不超过 3，所以这棵树的度为 3。

- 结点的孩子：该结点指针指向的结点，结点拥有的（某棵）子树的根。

— 如：结点 B 的孩子是 E 和 F。结点 H 的孩子是 P 和 Q。结点 D 的孩子是 J 和 K。

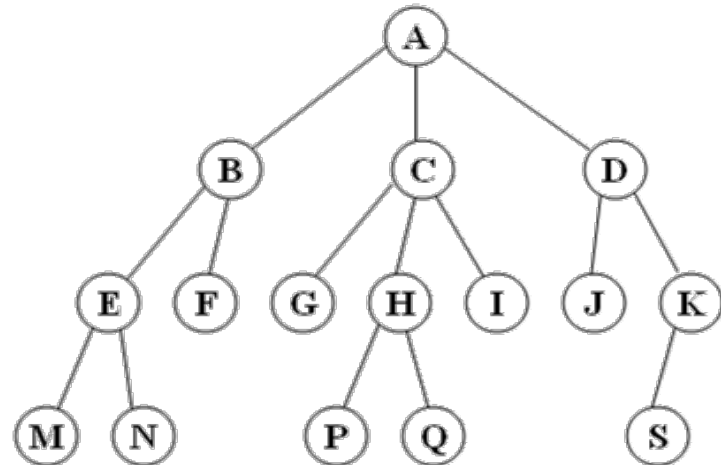


# 基本概念

- 父亲结点：孩子结点的前驱（结点）。
  - 结点 E 和 F 的父亲结点是 B；结点 S 的父亲是 K；结点 A 是根结点，没有父亲。
- 兄弟结点：同一个父亲结点的孩子。
  - B、C、D 互为兄弟（父亲都是 A）；E 和 F 互为兄弟（父亲都是 B）；G、H、I 互为兄弟（父亲都是 C）；M 和 N 互为兄弟（父亲都是 E）；P 和 Q 互为兄弟（父亲都是 H）；J 和 K 互为兄弟（父亲都是 D）。
- 结点的层次：根结点定义为第1层；根结点的孩子定义为第2层；第  $l$  层结点的孩子定义为第  $l+1$  层。

# 基本概念

- 树的高度(或深度) =  $\max$ (结点的层次)。
  - 树的高度是4
- 说明：树的高度  $\neq$  树的度(不同概念)。



# 有序树和无序树

- 有序树：(树中每一个)结点的各棵子树按一定的次序从左向右排列的树。
- 无序树：结点各子树与次序无关的树。
- 例如，对于有序树： $A(B, C) \neq A(C, B)$ ，
- 对于无序树： $A(B, C) = A(C, B)$ 。



# 森林

- 森林：m棵互不相交的树的有限集，
- 即  $F = \{ T_1, T_2, \dots, T_m \}$ ， $m \geq 0$ 。
- 当  $m=0$  时，F称为空森林。
- 对于森林F，如果将每棵树都赋予同一个父亲结点R，  
即  $F = R(T_1, \dots, T_m)$ ，则森林成为一棵树。

# 树的两个基本特点

- 树的两个基本特点：
  - (1) 递归性质
  - (2) 层次结构
- 一个组织机构可以看成是一棵树。
  - 结点：机构名称及相关信息
  - 关系：<上级机构，下级机构>
- 树是一种非线性结构

# 树的层次结构特性

- 存储器上的目录结构是树型结构。

结点：逻辑盘、文件夹、文件

关系：<逻辑盘，文件夹>

<文件夹，文件夹>

<文件夹，文件>

- 书目通常用树型结构描述。

结点：分类、书名、章节名称

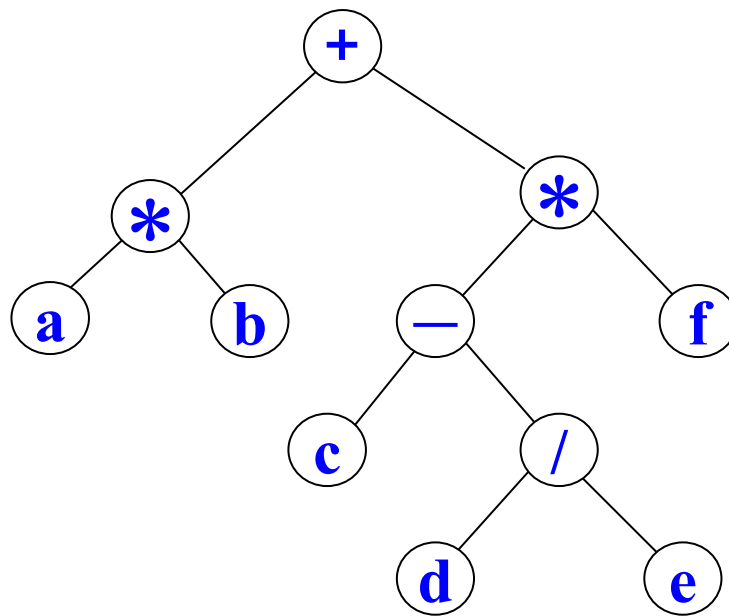
关系：<分类，书名>

<书名，章>

<章，节>

# 树的层次结构特性

- 一个表达式可以表示成一棵树。
  - 结点：运算符、操作数；
  - 关系：运算规则。
- 可以将表达式  $a*b+(c-d/e)*f$  表示成一棵树。



# 树的层次结构特性

- 树是一种非线性结构，具有递归性质和层次结构两种基本特点。
- 树的图形表示比较直观，常被用于算法的设计与分析，而树的字符形式表示易于存储，常被用于构建树的输入。

# 目录

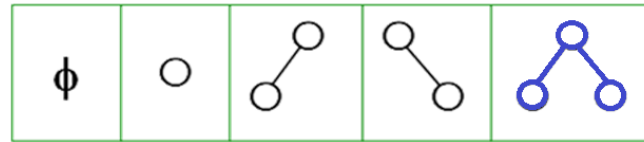
|  |   |             |
|--|---|-------------|
|  | 1 | 树的定义和基本术语   |
|  | 2 | 二叉树         |
|  | 3 | 遍历二叉树和线索二叉树 |
|  | 4 | 树和森林        |
|  | 5 | 哈夫曼树及其应用    |

# 二叉树的定义

- 二叉树：结点的度 $\leq 2$ 的(有序)树。

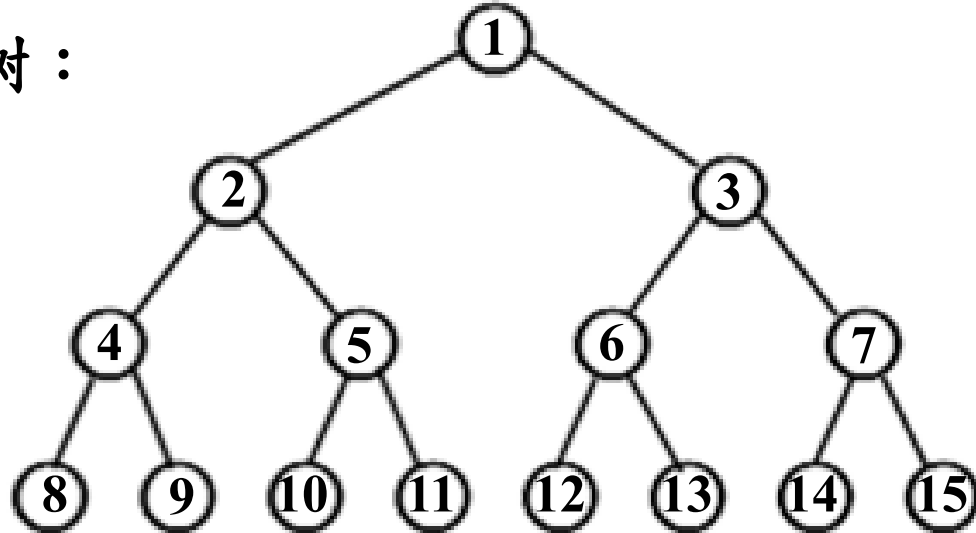
- 二叉树中每个结点的子树最多2棵。

- 二叉树的5种基本形态



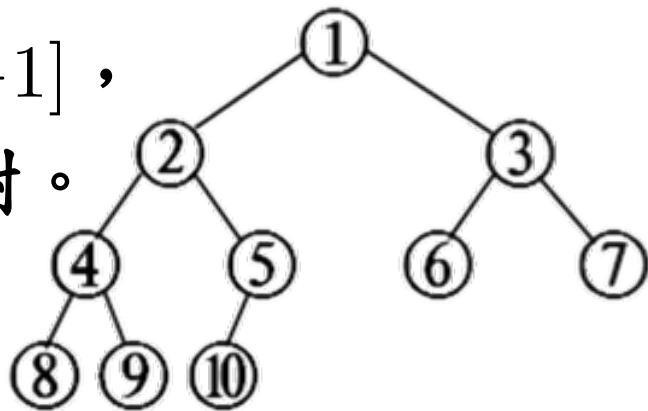
- 满二叉树：一棵高度为 $k$ 且具有 $2^k-1$ 个结点的二叉树。

- 如 $k=4$ 的满二叉树：



# 完全二叉树

- 完全二叉树
- 一棵高度为  $k$ 、结点个数  $\in [2^{k-1}, 2^k - 1]$ ，且第  $k$  层结点都集中在左侧的二叉树。
- 满二叉树也是完全二叉树。
- 性质：
  - 在二叉树的第  $i$  层上最多有  $2^{i-1}$  个结点 ( $i \geq 1$ )。
  - 高度为  $k$  的二叉树最多有  $2^k - 1$  个结点 ( $k \geq 1$ )。
  - 如果二叉树  $T$  的叶子结点数为  $n_0$ ，度为 2 的结点数为  $n_2$ ，则  $n_0 = n_2 + 1$ 。





# 二叉树的性质

- 如果二叉树  $T$  的叶子结点数为  $n_0$ ，度为2的结点数为  $n_2$ ，则  $n_0 = n_2 + 1$ 。
  - 证明：设二叉树  $T$  共有  $n$  个结点，叶子结点数为  $n_0$ ，度=1的结点数为  $n_1$ ，度=2的结点数为  $n_2$ ， $T$  的分支数为  $m$ 。
    - (1) 由于二叉树中所有结点的度  $\leq 2$ ，则有  $n = n_0 + n_1 + n_2$
    - (2) 除根结点外，其余结点都有唯一前驱，则有  $n - 1 = m$
    - (3) 由于度 =  $i$  的结点有  $i$  个分支，则有  $m = 0 + 1 \times n_1 + 2 \times n_2$
- 即 
$$n_0 = n_2 + 1$$

# 二叉树的性质

- 性质：具有 $n$ 个结点的完全二叉树 $T$ 高度为 $\lfloor \log_2 n \rfloor + 1$ 。

– 证明：设 $T$ 的高度为 $k$ 。

根据完全二叉树的定义可知，

$T$ 的结点个数 $n \in [2^{k-1}, 2^k - 1]$ ，即

$$2^{k-1} \leq n \leq 2^k - 1 < 2^k$$

$$k-1 \leq \log_2 n < k$$

$$k-1 \leq \lfloor \log_2 n \rfloor \leq k-1$$

$$k = \lfloor \log_2 n \rfloor + 1, \text{ 证毕}$$

# 二叉树的性质

- 性质5：设完全二叉树含有 $n$ 个结点。

- (1) 如果编号 $i=1$ ，则结点 $i$ 是根结点；

- 如果编号 $i>1$ ，则结点 $i$ 的父亲结点的编号是 $\lfloor i/2 \rfloor$ 。

- (2) 结点 $i$ 的左孩子的编号为 $2i$ ；

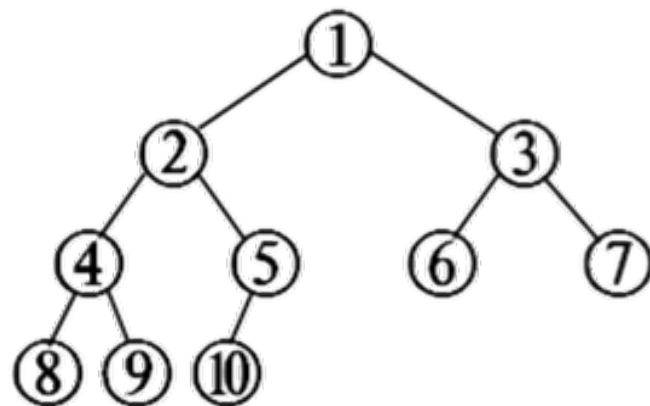
- 结点 $i$ 的右孩子的编号为 $2i+1$ 。

- (3) 如果 $2i \leq n$ ，则结点 $i$ 为分支结点；

- 如果 $2i > n$ ，则结点 $i$ 为叶子结点。

- (4) 如果 $n$ 为奇数，则每个分支结点都有左孩子和右孩子；

- 如果 $n$ 为偶数，则编号为 $n/2$ 的分支结点(最大的分支结点)只有左孩子，没有右孩子。

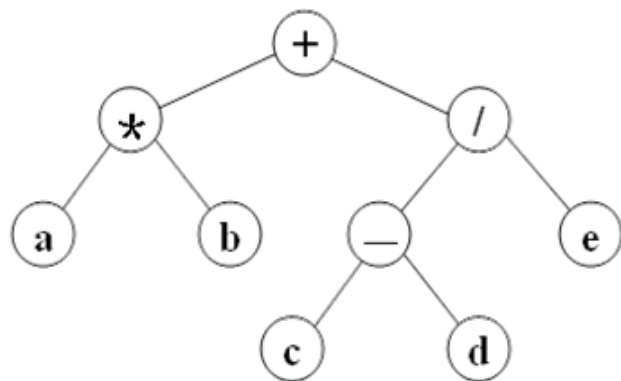


# 二叉树的定义

- 二叉树的顺序存储结构：

- 将二叉树的所有结点按照一定的次序存放的一组地址连续的存储单元中。
- 一般约定：从根结点开始，自上而下，自左而右排成一个线性序列。

- 例如，可以将下面的二叉树这样存



|   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |
|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 |
| + | * | / | a | b | - | e |   |   |    |    | c  | d  |    |    |

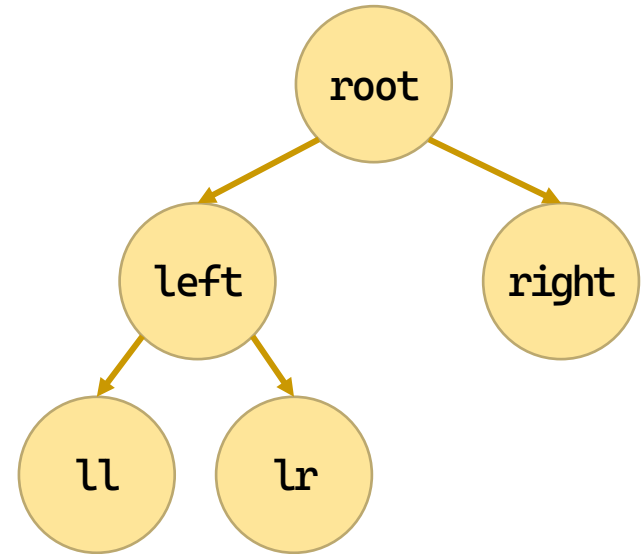
# 二叉树的定义

- 二叉树的顺序存储可以直接用顺序表描述
  - 将结点的编号和数组下标一一对应；
  - 结点的编号必须能反映其逻辑关系。
- 顺序存储结构适用于完全二叉树
  - 结点的编号容易和数组下标对应，且浪费的存储空间较小。

# 二叉树的定义

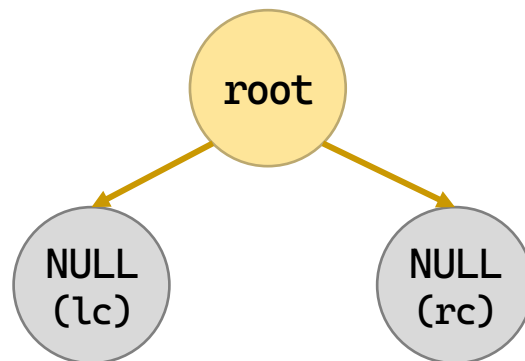
- 二叉树的存储结构

```
typedef struct Node {
 char data;
 struct Node* LChild;
 struct Node* RChild;
} BiTNode, * BiTree;
```



# 二叉树：初始化

- 初始化：只生成头指针。



```
int TreeInit(BiTree* T) {
 *T = (BiTree)malloc(sizeof(BiTNode));
 if (*T == NULL)
 return 0;
 // 初始化头结点：左右孩子置空，数据域可置为特殊值（如 '\0'）
 (*T)->LChild = NULL;
 (*T)->RChild = NULL;
 (*T)->data = '\0';
 return 1;
}
```

// 头结点数据域不使用，置空  
// 成功

# 二叉树：建立

- 扩展先序创建（递归）
  - 建结点，录入左子结点，录入右子结点

```
void create_bi_tree(BiTree* bt) {
 char c;
 c = getchar();
 if (c == '.') {
 (*bt) = NULL;
 } else {
 (*bt) = (BiTNode*)malloc(sizeof(BiTNode));
 (*bt)->data = c;
 create_bi_tree(&((*bt)->LChild));
 create_bi_tree(&((*bt)->RChild));
 }
}
```



# 二叉树：建立

## • 广义表创建（递归）

- “(”：入栈，入子左结点
- “)”：出栈，到父节点
- “,”：左结点入右结点
- 其它：建结点保存

```
if (c == '(') {
 push(S, p); parent=p; left=0;
```

```
} else if (c == ')') {
 pop(S, &parent);
```

```
} else if (c == ',') {
 left = 1;
```

```
} else if (c >= 'A' && c <= 'Z') {
 p=(BiTNode*)malloc(sizeof(BiTNode));
 p->data=c; p->LChild=p->RChild=NULL;
 if (*root == NULL)
 *root = p;
 else
 if (left == 0)
 parent->LChild = p;
 else
 parent->RChild = p;
}
```

```

int create_bi_tree_bsl(BiTree* root, char* str) {
 SeqStack* S = (SeqStack*)malloc(sizeof(SeqStack));
 init_stack(S);
 BiTree p = NULL;
 BiTree parent = NULL;
 int left = 0;
 *root = NULL;
 for (; *str; str++) {
 char c = *str;
 if (c == '(') {
 push(S, p);
 parent = p;
 left = 0;
 } else if (c == ',') {
 left = 1;
 } else if (c == ')') {
 pop(S, &parent);
 }
 }
}

```

```

 } else if (c >= 'A' && c <= 'Z') {
 p = (BiTNode*)malloc(sizeof(BiTNode));
 p->data = c;
 p->LChild = p->RChild = NULL;
 if (*root == NULL) {
 *root = p;
 } else {
 if (left == 0)
 parent->LChild = p;
 else
 parent->RChild = p;
 }
 }
}
int empty = is_empty(S);
free(S);
return empty;
}

```

# 二叉树：显示

- 竖向树状打印

- 处理显示右子树（左转 $90^\circ$ 时，右在上）
- 显示缩进
- 处理显示左子树

```
void print_btree_vertical(BiTree root, int nLayer) {
 int i;
 if (root) {
 print_btree_vertical(root->RChild, nLayer + 1);
 for (i = 0; i < nLayer; i++)
 printf(" ");
 printf("%c \n", root->data);
 print_btree_vertical(root->LChild, nLayer + 1);
 }
}
```

# 二叉树：显示

- 广义表状打印

- 显示做左括号，处理左子树，显示逗号，处理右子树，显示右括号

```
void print_btree_bsl(BiTree bt) {
 if (bt) {
 printf("%c", bt->data);
 if (bt->LChild || bt->RChild) {
 printf("(");
 print_btree_bsl(bt->LChild);
 if (bt->RChild) {
 printf(",");
 print_btree_bsl(bt->RChild);
 }
 printf(")");
 }
 }
}
```

# 目录

|  |     |             |
|--|-----|-------------|
|  | 3   | 遍历二叉树和线索二叉树 |
|  | 3.1 | 遍历二叉树       |
|  | 3.2 | 线索二叉树       |
|  | 4   | 树和森林        |
|  | 5   | 哈夫曼树及其应用    |

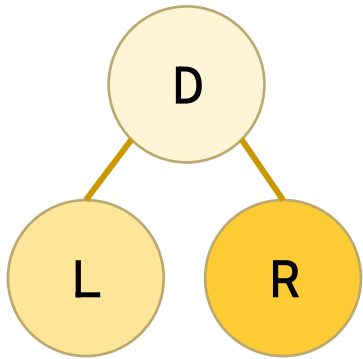
# 遍历二叉树

- 遍历二叉树：按照一定规则访问二叉树中的每个结点，使得每个结点均能被访问一次，且仅被访问一次。
- 通过一次遍历，可以使二叉树中的结点由非线性排列变为线性排列。遍历操作可以使二叉树线性化。
- 二叉树由3个基本单元组成：根结点、左子树和右子树，因此，遍历二叉树的问题可归结为解决：
  - (1) 访问根结点(D)；
  - (2) 遍历左子树(L)；
  - (3) 遍历右子树(R)。

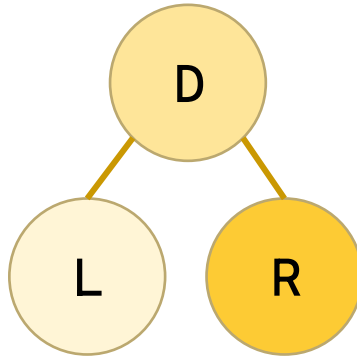
# 遍历二叉树

- 遍历二叉树共有六种方案：

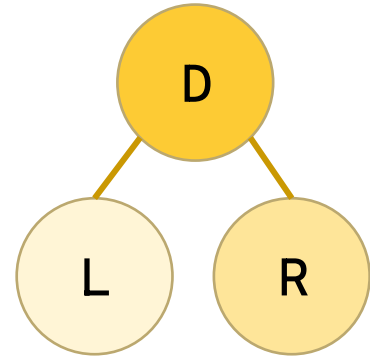
- DLR，LDR，LRD —— L先于R（常用）
- DRL，RDL，RLD —— R先于L



前序遍历 DLR  
PreOrder



中序遍历 LDR  
InOrder



后序遍历 LRD  
PostOrder



# 二叉树：递归版本

- 递归版本的遍历直接按照定义编写

```
void pre_order(BiTree root) {
 if (root) {
 visit(root);
 pre_order(root->LChild);
 pre_order(root->RChild);
 }
}
```

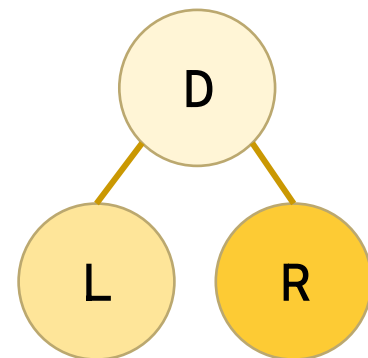
```
void post_order(BiTree root) {
 if (root) {
 post_order(root->LChild);
 post_order(root->RChild);
 visit(root);
 }
}
```

```
void in_order(BiTree root) {
 if (root) {
 in_order(root->LChild);
 visit(root);
 in_order(root->RChild);
 }
}
```

# 二叉树：栈版本

- 栈版本的前序遍历

- 访问根结点，压入根结点
- 如果有左结点，访问左结点
- 如果已无左结点，弹出父结点，访问右节点



- 实现方式

- 当前结点非空，访问、压入并进入左结点
- 否则，弹出栈，进入右节点

```
while (root || !is_empty(S)) {
 if (root) {
 visit(root);
 push(S, root);
 root = root->LChild;
 } else {
 pop(S, &root);
 root = root->RChild;
 }
}
```

```

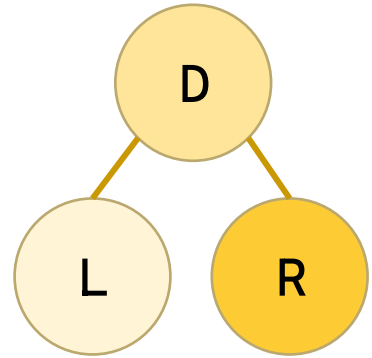
void pre_order_stack(BiTree root) {
 SeqStack* S = (SeqStack*)malloc(sizeof(SeqStack));
 init_stack(S);
 while (root || !is_empty(S)) {
 if (root) {
 visit(root);
 push(S, root);
 root = root->LChild;
 } else {
 pop(S, &root);
 root = root->RChild;
 }
 }
 free(S);
}

```

# 二叉树：栈版本

- 栈版本的前序遍历

- 不断压入左结点，进入最底层左结点
- 弹出结点，访问结点
- 进入右节点



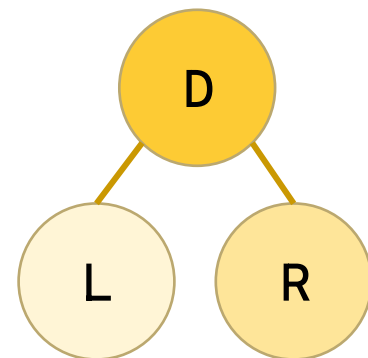
```
while (root || !is_empty(S)) {
 while (root) {
 push(S, root);
 root = root->LChild;
 }
 pop(S, &root);
 visit(root);
 root = root->RChild;
}
```

```
void in_order_stack(BiTree root) {
 SeqStack* S = (SeqStack*)malloc(sizeof(SeqStack));
 init_stack(S);
 while (root || !is_empty(S)) {
 while (root) {
 push(S, root);
 root = root->LChild;
 }
 pop(S, &root);
 visit(root);
 root = root->RChild;
 }
 free(S);
}
```

# 二叉树：栈版本

- 栈版本的后序遍历

- 不断压入，进入左结点（不访问）
- 访问栈顶，如果其有右结点右结点未访问，  
进入右节点
- 否则，访问当前结点，  
弹出结点，保存旧结点



```
while (root || !is_empty(S)) {
 if (root) {
 push(S, root);
 root = root->LChild;
 } else {
 get_top(S, &root);
 if (root->RChild == NULL
 || root->RChild == node) {
 visit(root);
 pop(S, &root);
 node = root;
 root = NULL;
 } else
 root = root->RChild;
 }
}
```

```

void post_order_stack(BiTree root) {
 BiTNode* node = root;
 SeqStack* S = (SeqStack*)malloc(sizeof(SeqStack));
 init_stack(S);
 while (root || !is_empty(S)) {
 if (root) {
 push(S, root);
 root = root->LChild;
 } else {
 get_top(S, &root);
 if (root->RChild == NULL || root->RChild == node) {
 visit(root);
 pop(S, &root);
 node = root;
 root = NULL;
 } else {
 root = root->RChild;
 }
 }
 }
 free(S);
}

```

# 二叉树遍历的应用

- 例：假设二叉树 $T = "+(* (a, b), /(- (c, d), e))"$ ，二叉链表 $T$ 已建立。写出：前序、中序和后序遍历。

- 前序： $+*ab/-cde$

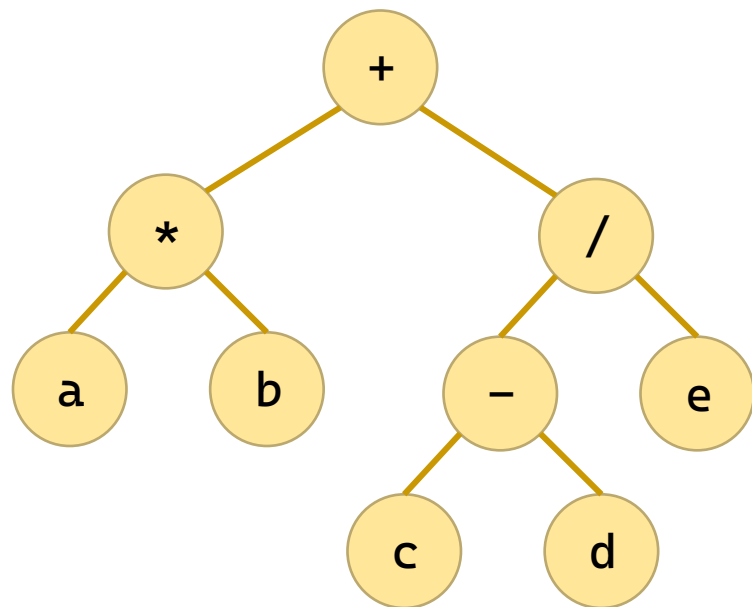
–  $+(*ab, /(-cd, e)) \rightarrow +(*ab, /-cde)$

- 中序： $a*b+c-d/e$

–  $+(a*b, /(c-d, e)) \rightarrow +(a*b, c-d/e)$

- 后序： $ab*cd-e/+$

–  $+(ab*, /(cd-, e)) \rightarrow +(ab*, cd-e/)$





# 二叉树遍历的应用

- **例3-1** 已知二叉树T的先序序列和中序序列分别为 **ABDFGCEH** 和 **BFDGACEH**，试画出T。
  - 前序A最前，划分BFDG-A-CEH，即： $A(BDFG, CEH)$
  - 前序B最前，所以划分  $\phi$ -B-FDG，即： $B(, DFG)$
  - 前序D最前，所以划分 F-D-G，即： $D(F, G)$
  - 前序C最前，所以划分  $\phi$ -EH，即： $C(, EH)$
  - 前序E最前，所以划分  $\phi$ -E-H，即： $E(, H)$
- 综上， $T=A(B(, D(F, G)), C(, E(, H)))$

# 二叉树遍历的应用

- 例3-2 设二叉树T的中序序列和后序序列分别为：中序：3, 7, 11, 14, 18, 22, 27, 35、后序：3, 11, 7, 14, 27, 35, 22, 18，试画出二叉树T。
  - 后序18最后，划分3, 11, 7, 14-18-27, 35, 22
  - 后序14最后，所以划分3, 11, 7-14- $\phi$
  - 后序7最后，所以划分 3-7-11
  - 后序22最后，所以划分  $\phi$ -22-27, 35
  - 后序35最后，所以划分 27-35- $\phi$
- 综上， $T=18(14(7(3,11)),22(,35(27)))$

# 目录

|  |     |             |
|--|-----|-------------|
|  | 3   | 遍历二叉树和线索二叉树 |
|  | 3.1 | 遍历二叉树       |
|  | 3.2 | 线索二叉树       |
|  | 4   | 树和森林        |
|  | 5   | 哈夫曼树及其应用    |

# 线索二叉树

- 线索：在结点的空指针域中存放指向某次遍历得到的后继或前驱的指针。
- 在具有 $n$ 个结点的二叉链表(二叉树)中，存在 $n+1$ 个空指针域。
- 后继线索(右线索)：在空右指针域中存放指向某次遍历得到的后继结点的指针。
- 前驱线索(左线索)：在空左指针域中存放指向某次遍历得到的前驱结点的指针。

# 线索二叉树

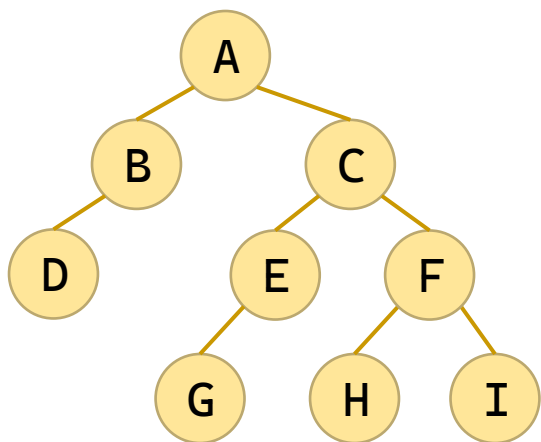
- 线索二叉树：根据某次遍历，在二叉树中的相关空指针域都写入线索(后继线索或前驱线索)，即成为线索二叉树。
- 线索二叉树可理解为已经线索化的二叉树。
- 先序后继：先序遍历中得到的后继(先序前驱, 中序后继, 中序前驱, 后序后继, 后序前驱)。

# 线索二叉树

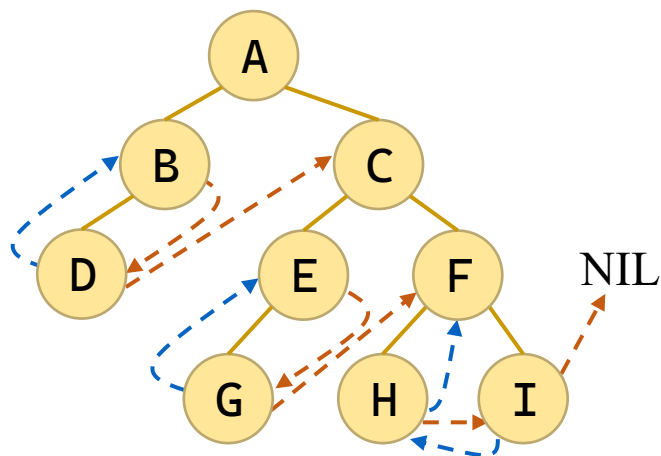
- 线索化原则：

- 先序后继：将右子树的根结点链入左子树的最后一个叶子结点。
- 中序后继：给定一个结点 $x$ ，它是其左子树最后一个叶子结点的后继结点。
- 后序后继：
  - (1) 若结点 $x$ 是二叉树的根，则其后继为空；
  - (2) 若结点 $x$ 是其父亲的右孩子或是其父亲的左孩子且其父亲没有右孩子，则其后继即为父亲结点；
  - (3) 若结点 $x$ 是其父亲的左孩子，且其父亲有右子树，则其后继为父亲的右子树上按后序遍历列出的第一个结点。

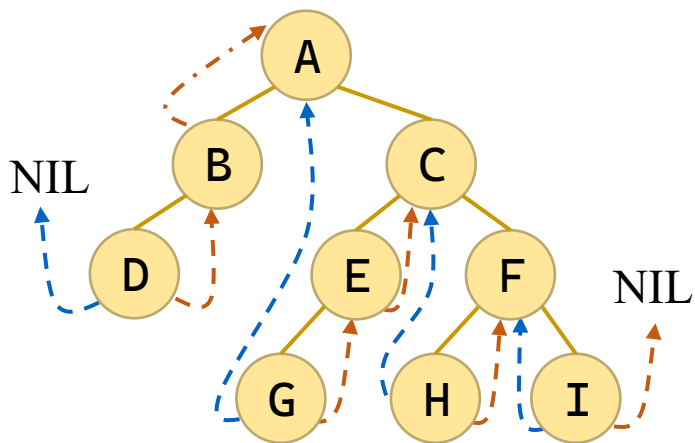
# 线索二叉树



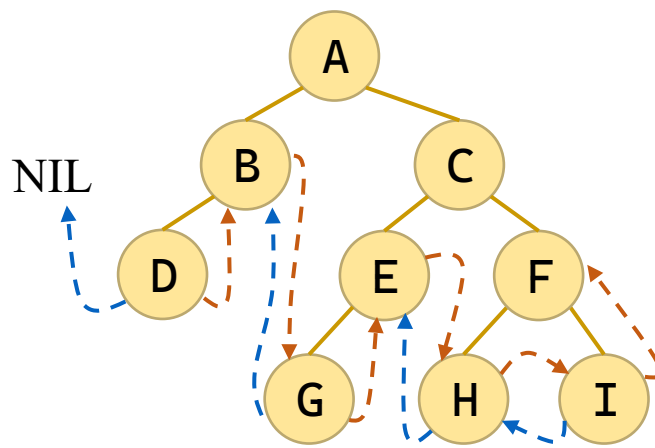
(a) 二叉树



(b) 先序线索树: ABDCEGFHI



(c) 中序线索树: DBAGECHFI



(d) 后序线索树: DBGEHIFCA

# 线索二叉树的定义

- 线索二叉树的存储结构

```
typedef struct Node {
 char data;
 struct Node* LChild;
 struct Node* RChild;
 int LTag; // 0: 左孩子, 1: 左线索
 int RTag; // 0: 右孩子, 1: 右线索
} BiTNode, * BiTree;
typedef struct {
 BiTree data[STACK_SIZE];
 int top;
} SeqStack;
```



# 线索二叉树：中序建立线索

- 中序：线索化左子树，处理当前结点，线索化右子树

- 记录下前一结点

```
*pre = p;
```

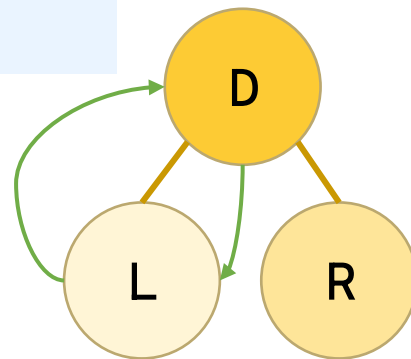
- 如果左子树空，

- 左结点指向前一结点

```
if (!p->LChild) {
 p->LChild = *pre;
 p->LTag = 1;
}
```

- 如果前一结点右子树空，

- 其右结点指向当前结点



```
if (*pre && !(*pre)->RChild) {
 (*pre)->RChild = p;
 (*pre)->RTag = 1;
}
```

```

void InThread(BiTree p, BiTree* pre) {
 if (p) {
 InThread(p->LChild, pre); // 1. 线索化左子树
 if (!p->LChild) { // 2. 处理当前结点
 p->LChild = *pre; p->LTag = 1;
 }
 if (*pre && !(*pre)->RChild) {
 (*pre)->RChild = p; (*pre)->RTag = 1;
 }
 *pre = p;
 InThread(p->RChild, pre); // 3. 线索化右子树
 }
}

void InThreading(BiTree* root) { // 中序线索化入口
 BiTree pre = NULL;
 if (*root) {
 InThread(*root, &pre);
 if (pre && !pre->RChild) // 处理最后一个结点的右线索
 pre->RTag = 1; // 置为线索 (后继为空)
 }
}

```

# 线索二叉树：中序遍历

- 线索二叉树的中序遍历

- 遍历直至退出

- 找到最左子结点

- 先访问它

- 跳到其右节点（发挥索引作用）

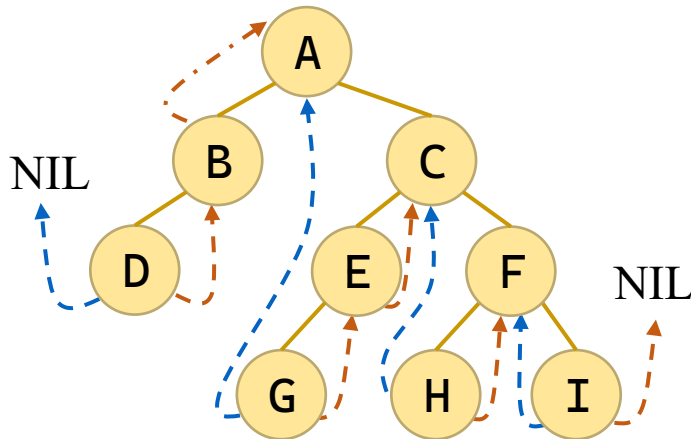
```
while (p) {
```

```
 while (p && p->LTag == 0)
 p = p->LChild;
```

```
 visit(p);
```

```
 p = p->RChild;
```

```
}
```



# 线索二叉树：中序遍历

- 线索二叉树的中序遍历（优化）

- 找到最左子结点

- 退出时，为最左子的左（父）中结点

```
while (p && p->LTag == 0)
 p = p->LChild;
```

- 遍历直至退出

```
while (p) {
```

- 先访问它（中结点优先）

```
 visit(p);
```

- 跳到其右节点

```
 p = p->RChild;
```

- 如果是右子结点（而非线索），访问该右子节点的最左结点

```
 if (p->RTag == 0) {
 while (p && p->LTag == 0)
 p = p->LChild;
 }
```

```
}
```

```

void InOrderThreaded(BiTree root) {
 BiTree p = root;
 if (!p) return;
 // 先找到最左下结点
 while (p && p->LTag == 0)
 p = p->LChild;

 while (p) {
 visit(p);
 p = p->RChild;
 // 如果右指针不是线索, 到右子树的最左下结点
 if (p->RTag == 0) {
 p = p->RChild;
 while (p && p->LTag == 0)
 p = p->LChild;
 }
 }
}

```

# 目录

|  |   |             |
|--|---|-------------|
|  | 2 | 二叉树         |
|  | 3 | 遍历二叉树和线索二叉树 |
|  | 4 | 树和森林        |
|  | 5 | 哈夫曼树及其应用    |
|  | 6 | 小结          |

# 树的存储结构

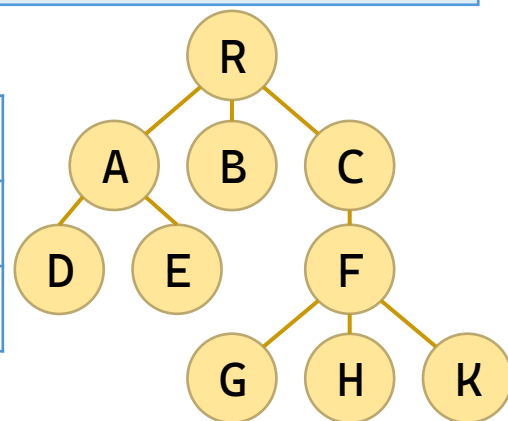
- 树的父亲表示法，用顺序表存储树的结点，每个结点包含数据域和指针域(父亲位置)。
- 树的孩子表示法，每个结点建立1个链表，用于存储该结点的所有孩子结点。
- 树的孩子兄弟表示法，用二叉树的存储结构表示树，第1个孩子为左孩子，其它兄弟依次存储为右孩子。

# 树的父亲表示法定义

- 树的父亲表示法存储结构

```
typedef struct {
 char data; // 数据域（可根据需要修改类型）
 int parent; // 父结点在数组中的位置（下标），根为 -1
} PTNode;
typedef struct {
 PTNode nodes[MAX_TREE_SIZE]; // 结点数组
 int n; // 当前结点个数
} PTree;
```

| 位置  | 0  | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|-----|----|---|---|---|---|---|---|---|---|---|
| 数据域 | R  | A | B | C | D | E | F | G | H | K |
| 指针域 | -1 | 0 | 0 | 0 | 1 | 1 | 3 | 6 | 6 | 6 |





# 树的儿子表示法定义

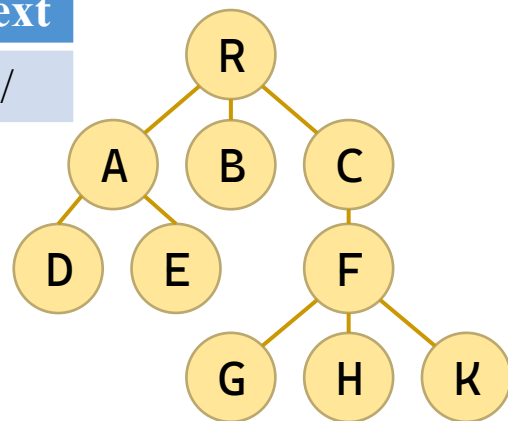
- 树的儿子表示法存储结构

```
typedef struct ChildNode { // 孩子链表结点
 int child; // 孩子结点在数组中的下标
 struct ChildNode* next; // 指向下一个孩子
} ChildNode;
typedef struct { // 树结点
 char data; // 数据域
 ChildNode* firstChild; // 指向第一个孩子的链表结点
} CTNode;
typedef struct { // 树定义
 CTNode nodes[MAX_TREE_SIZE]; // 结点数组
 int n; // 结点个数
} CTree;
```

# 树的儿子表示法定义

- n : 6
- nodes : []

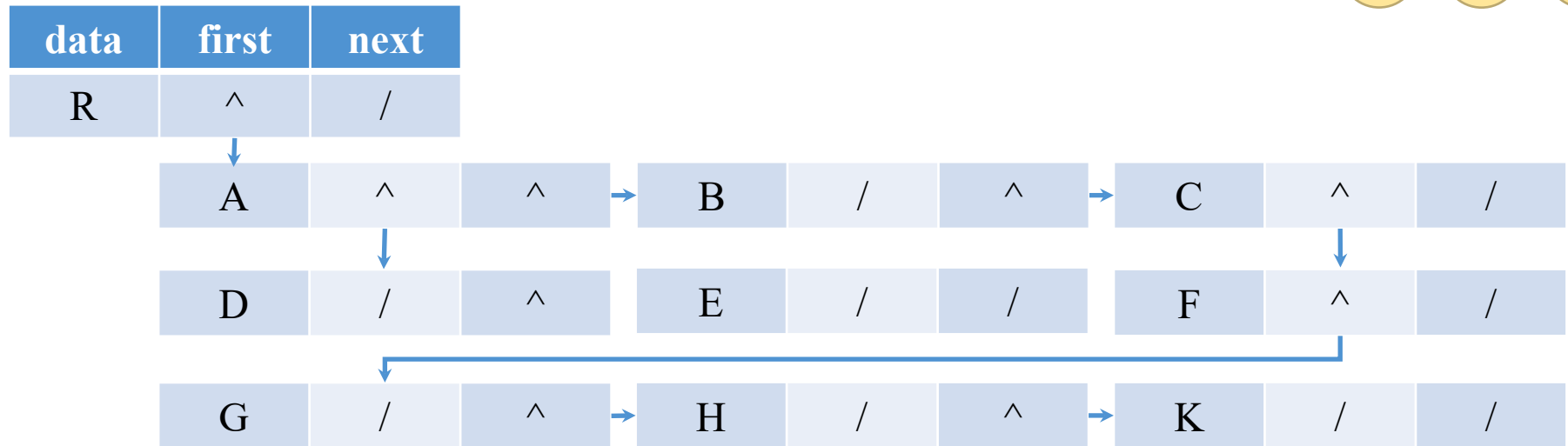
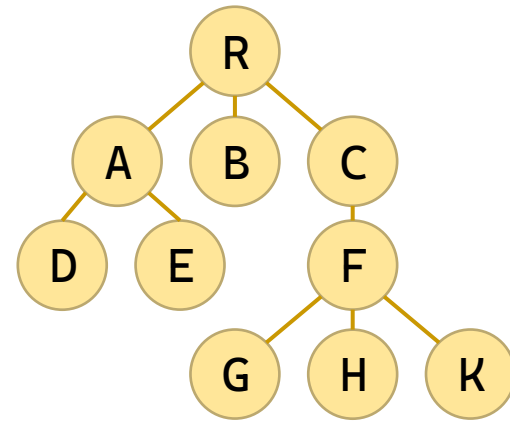
| id | data | firstChd | child | next | child | next | child | next |
|----|------|----------|-------|------|-------|------|-------|------|
| 0  | R    | ^        | 1     | ^    | 2     | ^    | 3     | /    |
| 1  | A    | ^        |       |      |       |      |       |      |
| 2  | B    | /        |       |      |       |      |       |      |
| 3  | C    | ^        | 6     | /    |       |      |       |      |
| 4  | D    | /        |       |      |       |      |       |      |
| 5  | E    | /        |       |      |       |      |       |      |
| 6  | F    | ^        | 7     | ^    | 8     | ^    | 9     | /    |
| 7  | G    | /        |       |      |       |      |       |      |
| 8  | H    | /        |       |      |       |      |       |      |
| 9  | K    | /        |       |      |       |      |       |      |



# 树的兄弟表示法定义

## • 树的兄弟表示法存储结构

```
typedef struct CSNode {
 char data; // 数据域
 struct CSNode* firstChild; // 第一个孩子
 struct CSNode* nextSibling; // 下一个兄弟
} CSNode, *CSTree;
```



# 树转换为二叉树

- 树（尤其是多叉树）的存储结构比较复杂（如孩子链表、孩子兄弟等）。而“孩子兄弟表示法”本质上就是一种二叉树。将其转为标准的二叉树后，内存布局更规整。
- 树和森林没有统一的“中序遍历”定义，但二叉树有成熟且唯一的先序、中序、后序遍历算法。转换后，我们可以直接用二叉树的遍历算法去处理树和森林，无需为树重新设计复杂的遍历逻辑。

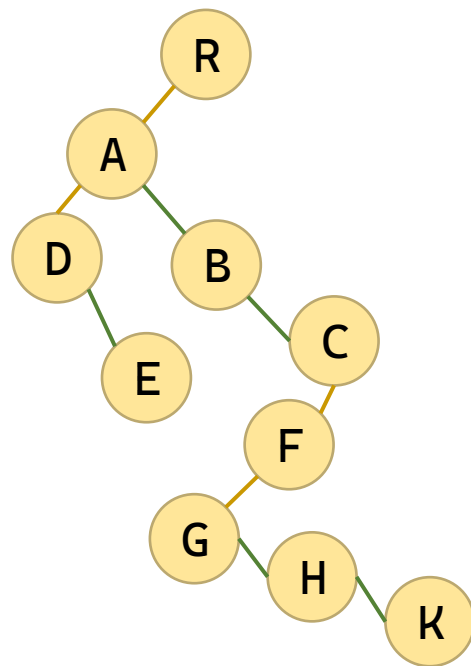
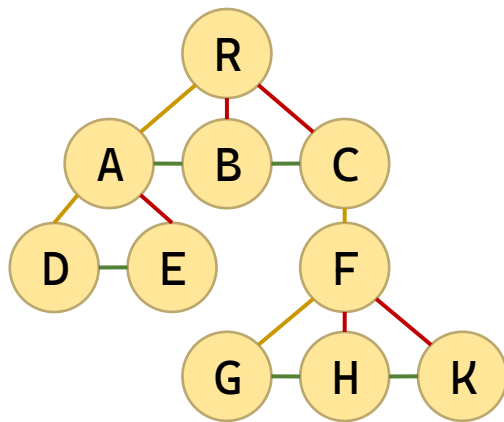
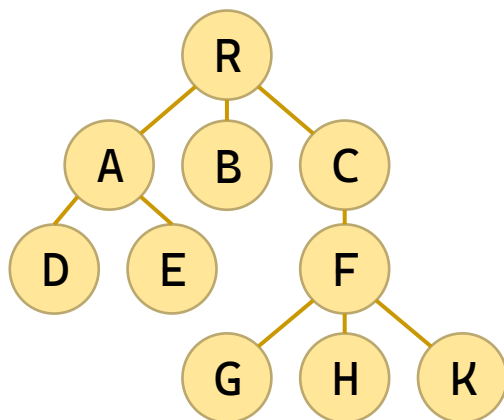
# 树转换为二叉树

- 由于二叉树和树都可用二叉链表作为存储结构，对比各自的结点结构可以看出，以二叉链表作为媒介可以导出树和二叉树之间的一个对应关系。
  - 从物理结构来看，树和二叉树的二叉链表是相同的，只是对指针的逻辑解释不同而已。
  - 从树的二叉链表表示的定义可知，任何一棵和树对应的二叉树，其右子树一定为空。
- 下图直观地展示了树和二叉树之间的对应关系。

# 树转换为二叉树

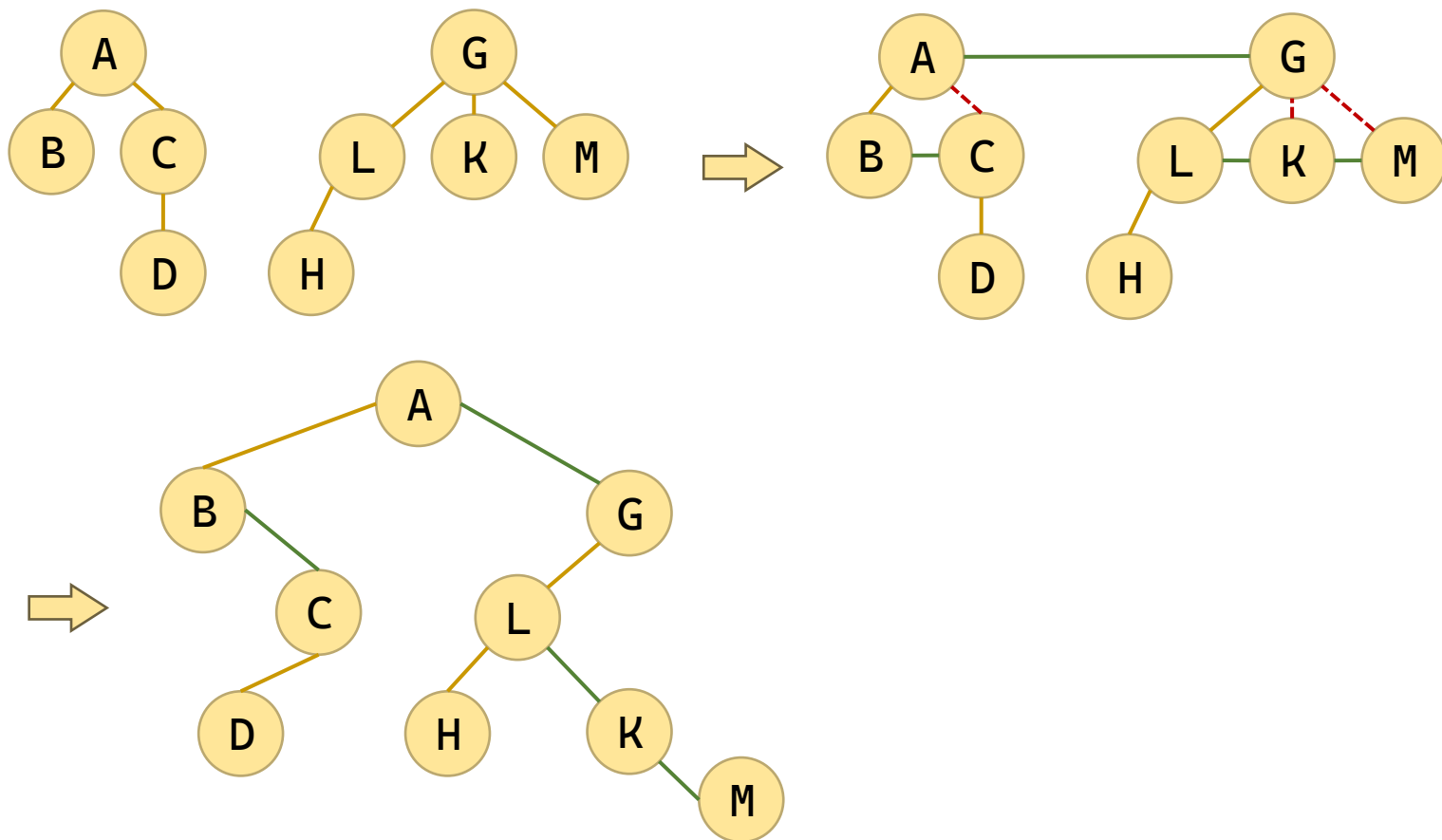
- 树与二叉树的转换

- 兄弟变右孩子
- 删除非长子
- 整理图形



# 森林转换为二叉树

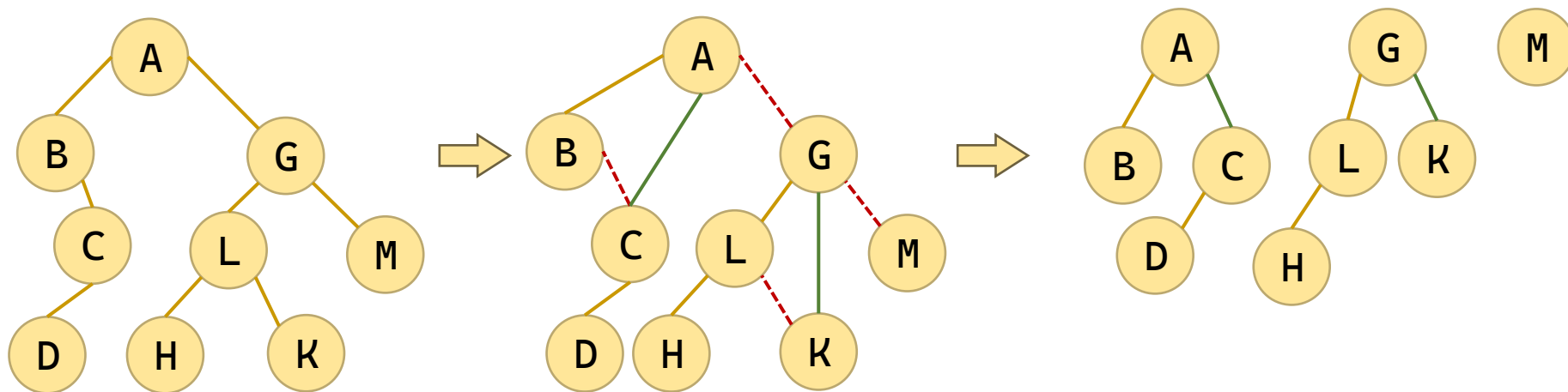
- 把森林中的树，相互视为兄弟



# 二叉树转换成森林

- 二叉树还原为森林

- 原“兄弟变右孩子”，改为“右孩子变兄弟”
- 原“删除非长子”，改为“删除右孩子”





# 树的遍历

- 树的先根遍历相当于其二叉树的前序遍历
- 树的后根遍历相当于其二叉树的中序遍历
- 树的层序遍历使用队列实现
  - 出队列，直到队列为空
  - 访问出队列的结点
  - 将其子结点（长子、兄弟结点）加入队列

```
void preOrderRoot(CSTree root) {
 if (root == NULL) return;
 printf("%c ", root->data);
 preOrderRoot(root->firstChild);
 preOrderRoot(root->nextSibling);
}

void postOrderRoot(CSTree root) {
 if (root == NULL) return;
 postOrderRoot(root->firstChild);
 printf("%c ", root->data);
 postOrderRoot(root->nextSibling);
}
```

```

void levelOrder(CSTree root) {
 if (root == NULL) return;
 Queue q;
 init_queue(&q);
 enqueue(&q, root);
 while (!is_empty(&q)) {
 CSTree cur = dequeue(&q);
 printf("%c ", cur->data);

 // 将当前结点的所有孩子依次入队
 CSTree child = cur->firstChild;
 while (child != NULL) {
 enqueue(&q, child);
 child = child->nextSibling;
 }
 }
}

```

# 树的应用

- 例3-6 采用树的儿子兄弟表示法描述组织机构。

# 子集树

- 所有的分支结点都含有相同个数的子树。
- 解决n个数，每个数有m中选择的组合问题。

## 遍历m叉子集树算法

```
for (i = 0; i < m; i++)
 for (j = 0; j < m; j++)
 for (k = 0; k < m; k++)
 visit(path);
```

n固定 ( 如 : 3 )

```
void dfs(int depth, int n, int m, int* path) {
 if (depth == n) {
 visit(path);
 return;
 }
 for (int i = 0; i < m; i++) {
 path[depth] = i;
 dfs(depth + 1, n, m, path);
 }
}
```

n不固定 ( 时间复杂度 :  $O(m^n)$  )

# 子集树的应用：输出二进制

- 例3-7 设计算法，输出所有4位二进制数。

```
void SubTree(int k) {
 if (k > n) {
 for (int i = 1; i <= n; i++)
 printf("%d", x[i]);
 printf(" ");
 return;
 }
 for (int i = 0; i < m; i++) {
 x[k] = i; // 第 k 位赋值为 i (0 或 1)
 SubTree(k + 1); // 递归处理下一位
 }
}
```

# 子集树的应用：n皇后问题

- **例3-8** n皇后问题找可行方案数的递归算法。
- 解决方法：增加一项，如果能放棋子则继续

```
void nQueen(int k) {
 if (k == n) {
 total++; return;
 }
 for (int col = 0; col < n; col++) {
 if (isSafe(k, col)) { // 如果第 k 行第 col 列安全
 queenPos[k] = col; // 放置皇后
 nQueen(k + 1); // 递归放置下一行
 }
 }
}
```

```

int isSafe(int row, int col) {
 for (int i = 0; i < row; i++) {
 if (queenPos[i] == col) // 列冲突: 同一列
 return 0;
 if (abs(row - i) == abs(col - queenPos[i])) // 对角线冲突
 return 0;
 }
 return 1; // 安全
}

void nQueen(int k) {
 if (k == n) {
 total++; return;
 }
 for (int col = 0; col < n; col++) {
 if (isSafe(k, col)) { // 如果第 k 行第 col 列安全
 queenPos[k] = col; // 放置皇后
 nQueen(k + 1); // 递归放置下一行
 }
 }
}

```



# 子集树的应用：背包问题

- 例3-9 背包问题递归算法。
- 解决方法：增加一项，如果能放棋子则继续

```
void nQueen(int k) {
 if (k == n) {
 total++; return;
 }
 for (int col = 0; col < n; col++) {
 if (isSafe(k, col)) { // 如果第 k 行第 col 列安全
 queenPos[k] = col; // 放置皇后
 nQueen(k + 1); // 递归放置下一行
 }
 }
}
```

```

void dfs(int depth, int n, int m, int* path) {
 // 1. 到达叶子结点 (所有物品决策完毕)
 if (depth == n) {
 // 更新最优解
 if (currentValue > bestValue) {
 bestValue = currentValue;
 }
 return;
 }
 // 2. 遍历 m 种状态 (m=2, 即 0:不选, 1:选)
 for (int i = 0; i < m; i++) {
 // 剪枝条件: 如果 i==1 (选), 但放不下当前物品, 则跳过该分支
 if (i == 1 && currentWeight + w[depth] > capacity) {
 continue; // 相当于这棵子树被剪掉了, 不往下递归
 }
 // ----- 做选择 -----
 path[depth] = i; // 记录当前决策 (完全符合模板)
 if (i == 1) { // 执行“选”的操作
 currentWeight += w[depth];
 currentValue += val[depth];
 }
 }
}

```

```

// 如果 i==0, 什么也不做 (不选)

// ----- 递归进入下一层 -----
dfs(depth + 1, n, m, path);

// ----- 回溯 (撤销选择) -----
if (i == 1) { // 撤销“选”的操作
 currentWeight -= w[depth];
 currentValue -= val[depth];
}
// 注意: path[depth] 不需要显式重置, 下一轮循环会直接覆盖
}
}

```

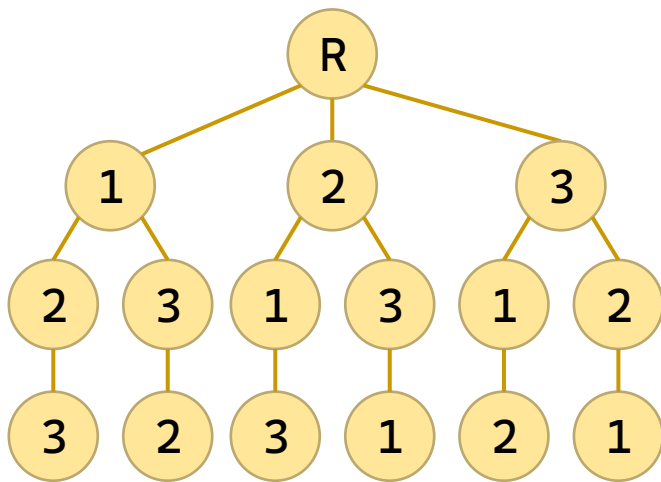
```

void knapSack(int k) {
 if (k == n) { // 1. 到达叶子结点 (所有物品决策完毕)
 if (currentValue > bestValue) // 当前方案价值最优
 bestValue = currentValue; // 则更新
 return;
 }
 // 2. 尝试“不选”第 k 个物品 (左分支)
 knapSack(k + 1);
 // 3. 尝试“选”第 k 个物品 (右分支)
 // 前提：加上当前物品后，重量不能超过背包容量
 if (currentWeight + w[k] <= capacity) {
 // 做选择：放入背包
 currentWeight += w[k];
 currentValue += v[k];
 // 递归处理下一个物品
 knapSack(k + 1);
 // 回溯：撤销选择，恢复现场 (这是必须的！)
 currentWeight -= w[k];
 currentValue -= v[k];
 }
}

```

# 排列树

- “排列树”是组合数学和算法设计中的一个概念，它指的是一种用于描述和解决排列问题的树形结构。
- 第 $i$ 层分支结点拥有与其祖先都不同的 $n-i+1$ 棵子树。
- 解决 $n$ 个数的全排列问题。



# 排列树

- 例3-10 遍历排列树算法(输出全排列)。

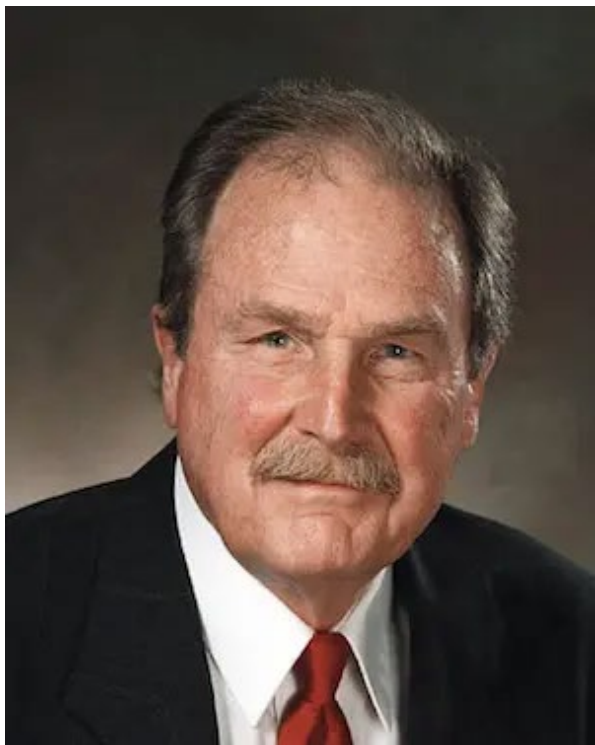
```
void backtrack(int pos) {
 if (pos == n) { // 所有位置已填满
 process();
 return;
 }
 // 尝试将 a[i] 放到 pos 位置 (i 从 pos 到 n-1)
 for (int i = pos; i < n; i++) {
 int temp = a[pos]; // 交换 a[pos] 和 a[i]
 a[pos] = a[i];
 a[i] = temp;
 backtrack(pos + 1); // 递归处理下一个位置
 temp = a[pos]; // 回溯：恢复交换
 a[pos] = a[i];
 a[i] = temp;
 }
}
```

# 目录

|  |   |             |
|--|---|-------------|
|  | 2 | 二叉树         |
|  | 3 | 遍历二叉树和线索二叉树 |
|  | 4 | 树和森林        |
|  | 5 | 哈夫曼树及其应用    |
|  | 6 | 小结          |

# David A. Huffman

1951年，哈夫曼和他在MIT信息论的同学需要选择是完成学期报告还是期末考试。导师Robert M. Fano给他们的学期报告的题目是，寻找最有效的二进制编码。由于无法证明哪个已有编码是最有效的，哈夫曼放弃对已有编码的研究，转向新的探索，最终发现了基于有序频率二叉树编码的想法，并很快证明了这个方法是最有效的。



1952年，David A. Huffman在麻省理工攻读博士时发表了《一种构建极小冗余编码的方法》（A Method for the Construction of Minimum-Redundancy Codes）一文，它一般就叫做Huffman编码。



# 哈夫曼编码

- 前缀编码：一种不等长的字符编码，要求字符集中任何一个字符的编码都不是另一个字符编码的前缀。

例如，设字符集={ a, b, c, d, e }，则

a=00，b=01，c=10，d=110，e=111

是一种前缀编码，编成0001001011100110。

|    |    |    |    |    |     |    |     |
|----|----|----|----|----|-----|----|-----|
| 编码 | 00 | 01 | 00 | 10 | 111 | 00 | 110 |
| 消息 | a  | b  | a  | c  | e   | a  | d   |

# 哈夫曼编码

- 哈夫曼编码：一种带权总量最小(如传送的二进制代码串最短)的前缀编码。
- 设计哈夫曼编码时应考虑：
  - (1) 数据对象，如字符集、符号集；
  - (2) 权值，如各个字符使用的频率；
  - (3) 哈夫曼树——用于带权总量最短；
  - (4) 前缀编码——使得能唯一译码。

# 哈夫曼编码

- 设计哈夫曼编码的一般步骤：

- (1)以权值集为叶子结点构造一颗哈夫曼树；
- (2)将哈夫曼树的左支标示0，右支标示1；
- (3)写出哈夫曼编码：从哈夫曼树根结点到各个叶子结点的路径代码。

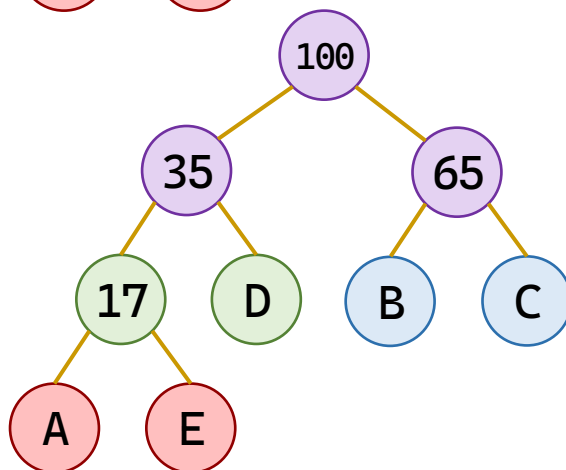
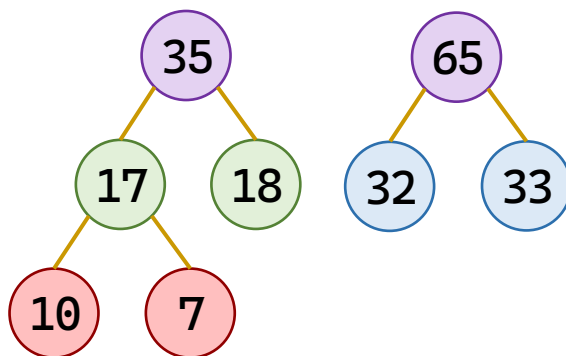
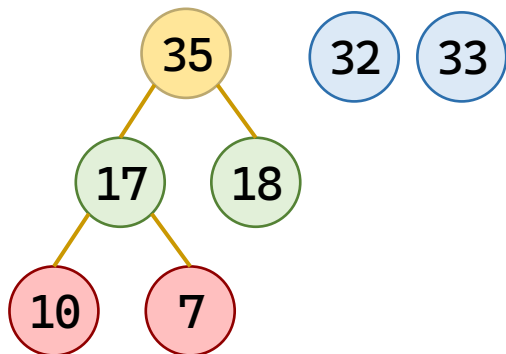
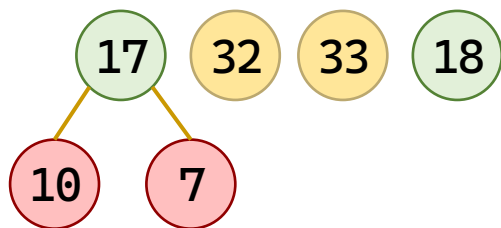
- 构造哈夫曼树

- 选择最轻的两堆合并
- 删掉合并前的两个权重，加上合并后的权重
- 剩下一堆时停止

把几堆不同重量的石子合并成一堆，每次只能合并两堆，搬石头的力气等于两堆重量之和，怎么合并最省力？

# 哈夫曼编码

- 例 已知字符集={A, B, C, D, E}，它们的权值={10, 32, 33, 18, 7}，试设计哈夫曼编码。



|   |     |
|---|-----|
| A | 000 |
| B | 10  |
| C | 11  |
| D | 01  |
| E | 001 |

# 哈夫曼编码说明

- 几点说明：

- (1)在选取结点权值最小的两棵子树时,如果出现权值相同,可以任选其中一棵子树;
- (2)两棵结点权值最小的二叉树组成新的二叉树时,没有规定左右子树次序;
- (3)在哈夫曼树中,权值越大的叶子结点离根结点越近——哈夫曼树的应用依据;
- (4)具有 $n$ 个叶子结点的哈夫曼树,共有 $2n-1$ 个结点(满足二叉树的性质)。

# 哈夫曼编码说明

– (4) 具有  $n$  个叶子结点的哈夫曼树，共有  $2n-1$  个结点(满足二叉树的性质)。

∵ 在二叉树中，结点总数为  $n_0 + n_1 + n_2$ ，

在哈夫曼树中，没有度为1的结点，

即哈夫曼树的结点总数为  $n_0 + 0 + n_2$ ，

根据二叉树性质  $n_0 = n_2 + 1$ ，即  $n_2 = n_0 - 1$ ，

∴ 哈夫曼树的结点总数  $n_0 + n_2 = 2n_0 - 1 = 2n - 1$ 。

这个结论常用来在编程时为哈夫曼树静态分配数组空间，直接开  $2n$  大小就够了。

# 哈夫曼树的实现

- 选择最轻的两堆合并

- 设最小值下标的初值为0
- 如果只考虑树根
- 如果最小值下标是初值0
- 或者找到更小
- 更新最小值
- 次小值下标多一个条件：  
不等于最小值下标

```
int min_1 = 0, min_2 = 0;
```

```
for (i = 1; i <= range; i++)
 if (htree[i].parent == 0)
```

```
 if (min_1 == 0 ||
```

```
 htree[i].weight < htree[min_1].weight)
```

```
 min_1 = i;
```

```
*s1 = min_1;
```

```
if (htree[i].parent == 0 && i != *s1)
```

# 哈夫曼树的实现

- 删掉合并前的两个权重

- 将最小值和次小值的父结点指向新结点

```
htree[s1].parent = i;
htree[s2].parent = i;
```

- 加上合并后的权重

- 更新新结点

```
htree[i].lchild = s1;
htree[i].rchild = s2;
htree[i].weight = htree[s1].weight +
 htree[s2].weight;
```

- 剩下一堆时停止

- 循环下标从  $n+1$  到  $2n-1$ ，合并生成新节点

```
for (i = n + 1; i <= 2 * n - 1; i++)
```



```

#include <stdio.h>
#include <string.h>
#define MAX_LEAVES 20 // 最大叶子数（可调）
#define MAX_NODES (2 * MAX_LEAVES) // 最大总节点数
typedef struct { // 哈夫曼树节点结构（静态三叉链表）
 int weight; // 权值
 int parent; // 双亲下标（0表示根）
 int lchild; // 左孩子下标
 int rchild; // 右孩子下标
} HNode;
// 全局变量
HNode htree[MAX_NODES];
int n; // 实际叶子个数

/* ===== 查找两个最小权值的根节点 ===== */
/*
* 功能：在 htree[1..range] 范围内，找出两个 parent==0 的节点，
* 分别作为最小权值 s1 和次小权值 s2。
* 算法：采用“打擂台”思想，用哨兵变量 min_1, min_2 记录下标，
* 一次遍历完成查找，避免先找最小再找次小的重复遍历。
*/

```

```

void selectMin(int range, int* s1, int* s2) {
 int i;
 int min_1 = 0; // 最小值下标, 0表示未找到
 int min_2 = 0; // 次小值下标
 for (i = 1; i <= range; i++) // 第一趟: 找最小值
 if (htree[i].parent == 0) // 只考虑森林中的树根
 if (min_1 == 0
 || htree[i].weight < htree[min_1].weight)
 min_1 = i; // 发现更小的, 更新
 *s1 = min_1; // 此时 min_1 一定不为0 (因为至少有两个根)

 for (i = 1; i <= range; i++) { // 第二趟: 找次小值 (跳过 s1)
 if (htree[i].parent == 0 && i != *s1)
 if (min_2 == 0
 || htree[i].weight < htree[min_2].weight)
 min_2 = i;
 }
 *s2 = min_2;
}

/* ===== 构建哈夫曼树 ===== */
void createHuffmanTree(int weights[]) {
 int i;

```

```

int totalNodes = 2 * n - 1; // 哈夫曼树总节点数
memset(htree, 0, sizeof(htree)); // 1. 初始化数组 (0下标不用)
for (i = 1; i <= n; i++) // 2. 将前 n 个位置设为叶子节点
 htree[i].weight = weights[i - 1];
for (i = n + 1; i <= totalNodes; i++) { // 3. 合并生成新节点
 int s1, s2;
 selectMin(i - 1, &s1, &s2); // 在前i-1个节点中找最小
 htree[s1].parent = i; htree[s2].parent = i; // 父节点指向
 htree[i].lchild = s1; htree[i].rchild = s2;
 htree[i].weight = htree[s1].weight + htree[s2].weight;
 printf("第%2d次合并: 选[%d]权=%2d和[%d]权=%2d, 建[%d]权=%d\n",
 i - n, s1, htree[s1].weight, s2, htree[s2].weight,
 i, htree[i].weight);
}
// 打印数组存储表 (便于画树验证)
printf("\n===== 哈夫曼数组存储表 =====\n");
printf("下标\t权值\t父节点\t左孩子\t右孩子\n");
for (i = 1; i <= totalNodes; i++)
 printf("%d\t%d\t%d\t%d\t%d\n", i, htree[i].weight,
 htree[i].parent, htree[i].lchild, htree[i].rchild);
}

```

```

/* ===== 生成哈夫曼编码 ===== */
void getHuffmanCodes(char codes[][MAX_LEAVES]) {
 int i, cur, p, idx;
 char temp[MAX_LEAVES];
 for (i = 1; i <= n; i++) { // 对每个叶子
 idx = 0; cur = i;
 while (htree[cur].parent != 0) { // 从叶子向上爬到根
 p = htree[cur].parent;
 temp[idx++] = (htree[p].lchild == cur) ? '0' : '1';
 cur = p;
 }
 temp[idx] = '\0';
 int len = idx;
 for (int j = 0; j < len; j++) // 叶子往上走的编码需要反转
 codes[i - 1][j] = temp[len - 1 - j];
 codes[i - 1][len] = '\0';
 }
 if (n == 1) { // 特判：如果只有一个叶子，编码规定为 "0"
 codes[0][0] = '0'; codes[0][1] = '\0';
 }
}

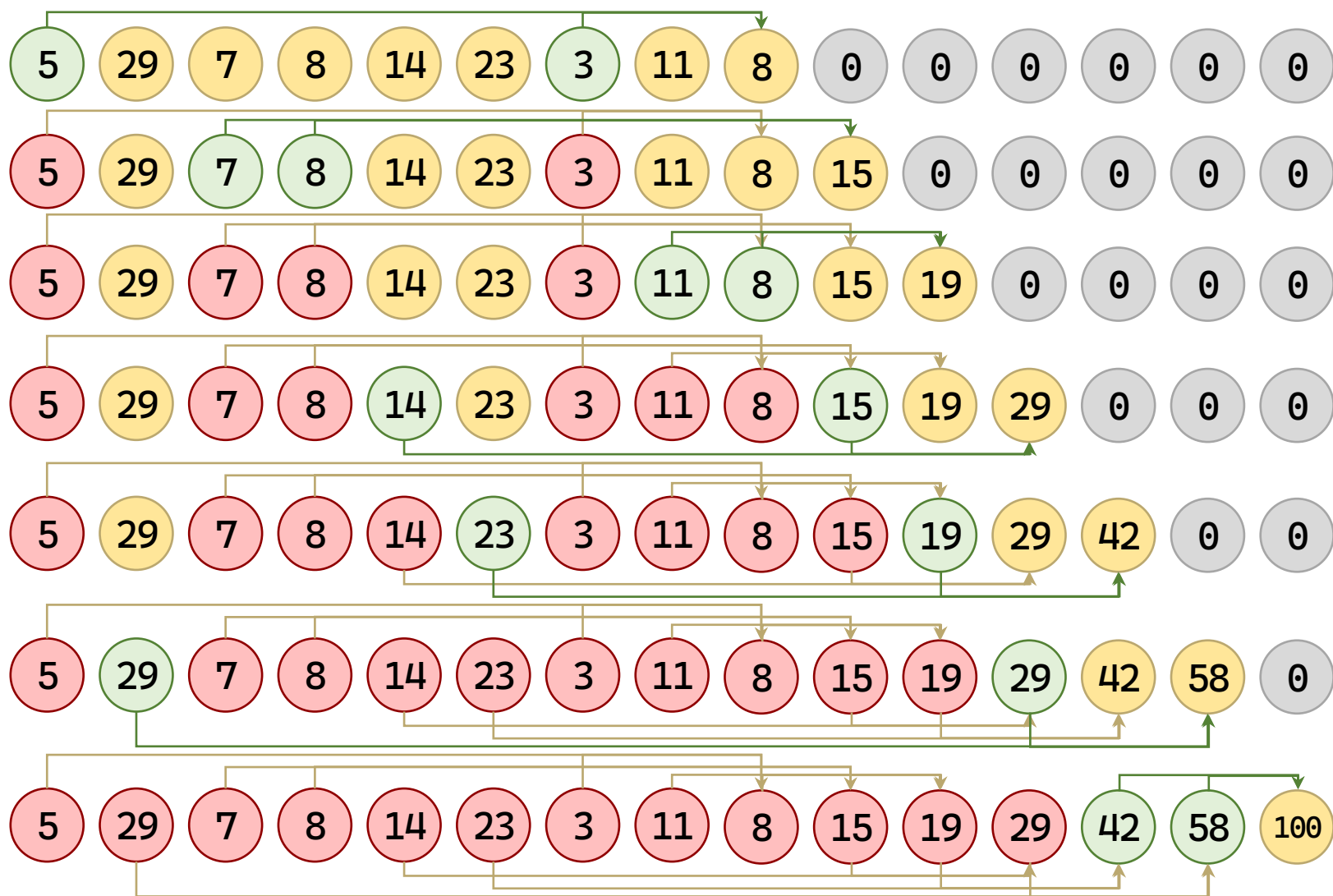
```

```

int main() {
 int weights[MAX_LEAVES];
 char huffmanCodes[MAX_LEAVES][MAX_LEAVES];
 printf("请输入叶子节点个数 n (1~%d): ", MAX_LEAVES);
 scanf("%d", &n);
 if (n < 1 || n > MAX_LEAVES) {
 printf("输入不合法! \n"); return -1;
 }
 printf("请输入 %d 个权值 (空格分隔) : ", n);
 for (int i = 0; i < n; i++)
 scanf("%d", &weights[i]);
 createHuffmanTree(weights);
 getHuffmanCodes(huffmanCodes);
 printf("\n===== 哈夫曼编码表 =====\n");
 for (int i = 0; i < n; i++)
 printf("权值 %2d 的编码: %s\n", weights[i], huffmanCodes[i]);
 int wpl = 0;
 for (int i = 1; i <= n; i++) {
 int len = strlen(huffmanCodes[i - 1]);
 wpl += htree[i].weight * len;
 }
 printf("该哈夫曼树的带权路径长度 WPL = %d\n", wpl);
 return 0;
}

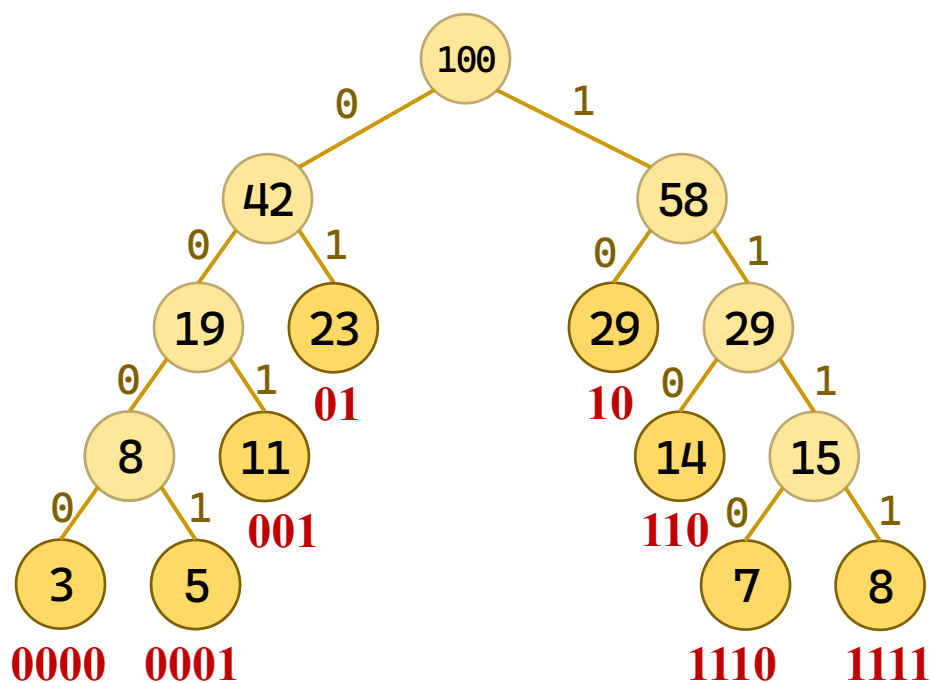
```

# 哈夫曼树的实现



# 哈夫曼树的实现

$$\begin{aligned} W &= \sum_i w_i \cdot l_i \\ &= 3 \times 4 + 5 \times 4 + 11 \times 3 + 23 \times 2 + 29 \times 2 + 14 \times 3 + 7 \times 4 + 8 \times 4 \\ &= 12 + 20 + 33 + 46 + 58 + 42 + 28 + 32 \\ &= 271 \end{aligned}$$



# 哈夫曼树的实现

请输入叶子节点个数 n (1~20): 8

请输入权值 (空格分隔) : 5 29 7 8 14 23 3 11

>>>>> 开始合并过程 <<<<<

第1次合并:选[7]权=3和[1]权=5, 建[9]权=8

第2次合并:选[3]权=7和[4]权=8, 建[10]权=15

第3次合并:选[9]权=8和[8]权=11, 建[11]权=19

第4次合并:选[5]权=14和[10]权=15, 建[12]权=29

第5次合并:选[11]权=19和[6]权=23, 建[13]权=42

第6次合并:选[2]权=29和[12]权=29, 建[14]权=58

第7次合并:选[13]权=42和[14]权=58, 建[15]权=100

===== 哈夫曼数组存储表 =====

| 下标 | 权值 | 父节点 | 左孩子 | 右孩子 |
|----|----|-----|-----|-----|
| 1  | 5  | 9   | 0   | 0   |
| 2  | 29 | 14  | 0   | 0   |
| 3  | 7  | 10  | 0   | 0   |
| 4  | 8  | 10  | 0   | 0   |
| 5  | 14 | 12  | 0   | 0   |
| 6  | 23 | 13  | 0   | 0   |
| 7  | 3  | 9   | 0   | 0   |
| 8  | 11 | 11  | 0   | 0   |

|    |     |    |    |    |
|----|-----|----|----|----|
| 9  | 8   | 11 | 7  | 1  |
| 10 | 15  | 12 | 3  | 4  |
| 11 | 19  | 13 | 9  | 8  |
| 12 | 29  | 14 | 5  | 10 |
| 13 | 42  | 15 | 11 | 6  |
| 14 | 58  | 15 | 2  | 12 |
| 15 | 100 | 0  | 13 | 14 |

=====

===== 哈夫曼编码表 =====

权值 5 的编码: 0001

权值 29 的编码: 10

权值 7 的编码: 1110

权值 8 的编码: 1111

权值 14 的编码: 110

权值 23 的编码: 01

权值 3 的编码: 0000

权值 11 的编码: 001

该哈夫曼树的带权路径长度 WPL = 271



# 目录

|  |   |             |
|--|---|-------------|
|  | 2 | 二叉树         |
|  | 3 | 遍历二叉树和线索二叉树 |
|  | 4 | 树和森林        |
|  | 5 | 哈夫曼树及其应用    |
|  | 6 | 小结          |

# 本章小结

- 树的定义、满二叉树、完全二叉树。
- 二叉树性质。
- 二叉树存储结构：
  - (1) 顺序存储结构(适用于完全二叉树)。
  - (2) 链式存储结构及其基本操作。
- 二叉树遍历递归算法(3个)
- 二叉树遍历非递归算法(3个)

# 本章小结

- 先序后继线索化算法
- 树的存储结构及其应用
- 子集树
- 排列树
- 哈夫曼树、哈夫曼编码

# 本章小结

**例3-11** 编写递归算法：对于二叉树中每一个元素值为x的结点，删去以它为根的子树，并释放相应的空间。

```
void freeTree(BiTree T) {
 if (T == NULL) return;
 freeTree(T->lchild); freeTree(T->rchild);
 free(T);
}

BiTree deleteSubtree(BiTree T, int x) {
 if (T == NULL) return NULL;
 if (T->data == x) {
 freeTree(T); return NULL;
 }
 T->lchild = deleteSubtree(T->lchild, x);
 T->rchild = deleteSubtree(T->rchild, x);
 return T;
}
```

# 本章小结

- 例3-12 求二叉链表T中值为x的数据元素所在的层次。

```
int findLevelRecur(BiTree T, int x, int level) {
 if (T == NULL) return 0; // 空树未找到
 if (T->data == x) return level; // 找到, 返回当前层次
 // 先查左子树
 int leftRes = findLevelRecur(T->lchild, x, level + 1);
 if (leftRes != 0) return leftRes; // 左子树找到了, 直接返回
 // 左子树没找到, 再查右子树
 return findLevelRecur(T->rchild, x, level + 1);
}

int getLevel(BiTree T, int x) { // 对外接口: 求值为x的结点的层次
 return findLevelRecur(T, x, 1); // 根结点层次为1
}
```

# 本章小结

- 例3-13 判断二叉树是否相似。

```
bool isSimilar(BiTree T1, BiTree T2) {
 // 情况1: 两棵树都为空 → 形状相同 (相似)
 if (T1 == NULL && T2 == NULL) {
 return true;
 }
 // 情况2: 一棵为空, 另一棵不为空 → 形状不同 (不相似)
 if (T1 == NULL || T2 == NULL) {
 return false;
 }
 // 情况3: 两棵都不为空 → 递归判断左子树和右子树是否都相似
 // 注意: 这里完全没有比较 T1->data 和 T2->data, 只比较结构!
 return isSimilar(T1->lchild, T2->lchild) &&
 isSimilar(T1->rchild, T2->rchild);
}
```

6

谢谢观看

理论课程



廈門大學  
XIAMEN UNIVERSITY



信息学院 黄 焯  
(特色化示范性软件学院) 博士, 副教授  
School of Informatics Wei Huang

# 数据结构与算法

## Data Structures

7

图

理论课程



廈門大學  
XIAMEN UNIVERSITY



信息学院 黃 煒  
(特色化示范性软件学院) 博士, 副教授  
School of Informatics Wei Huang



# 内容要点

- 图的定义和术语
- 图的存储结构
- 图的遍历
- 最小生成树
- 有向无环图及其应用
- 最短路径

# 目录

|  |   |           |
|--|---|-----------|
|  | 1 | 图的定义和术语   |
|  | 2 | 图的存储结构    |
|  | 3 | 图的遍历      |
|  | 4 | 最小生成树     |
|  | 5 | 有向无环图及其应用 |

# 图的定义和术语

- 图（Graph）是一种多对多的非线性数据结构。

- 树有根，链表有头尾

- 图研究的核心是顶点之间的连通关系。

- 图的数据元素是顶点。如：a, b。

- 图的逻辑结构是关系。如： $\langle a, b \rangle$ 。

- 边（Edge）：指的是两个顶点之间直接的连接。

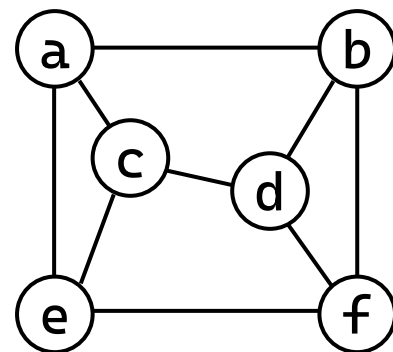
- 如果 $\langle v, w \rangle \in VR$ 必有 $\langle w, v \rangle \in VR$ ，则称 $(v, w)$ 是 $v$ 和 $w$ 之间的一条边

- 边没有次序关系，即 $(v, w)$ 和 $(w, v)$ 表示同一条边。

设A和B是两个集合。 $A \times B$ 的任意一个子集R，称为从A到B的一个二元关系。

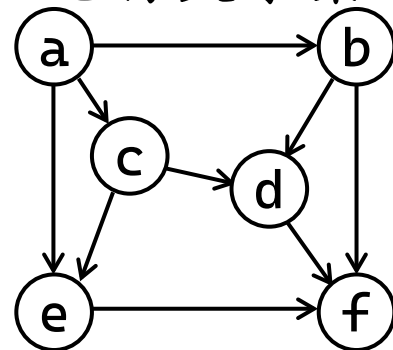
- 如果  $A=B$ ，则称 R 为 A 上的二元关系。

- 若有序对  $\langle x, y \rangle \in R$ ，则称  $x$  与  $y$  具有关系 R。



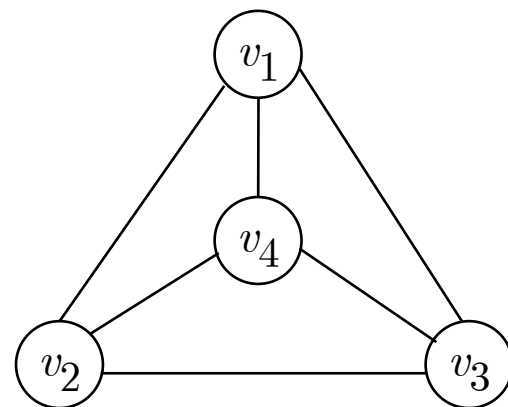
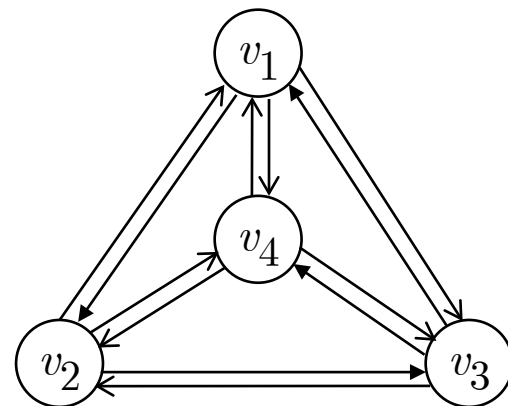
# 图的定义和术语

- 图  $G$  是由两个集合组成的二元组  $G = (V, E)$ 。
  - $V$  (Vertex Set) : 顶点的非空有限集。如  $V = \{a, b, c, d, e, f\}$ 。
  - $E$  (Edge/Arc Set) : 顶点之间关系的有限集，也称关系集。
    - 是  $E \subseteq V \times V$  的一个子集。
    - 如 :  $E = VR = \{ \langle a, b \rangle, \langle b, c \rangle, \langle c, a \rangle, \langle d, f \rangle, \dots \}$
- 无向图 : 由顶点集和边集构成的图。
- 有向图 : 由顶点集和弧集 (有次序关系) 构成的图。
  - 弧 (Arc) : 如果  $\langle v, w \rangle \in VR$  , 则  $\langle v, w \rangle$  表示从  $v$  到  $w$  的一条弧。
  - 弧头 (Head) : 箭头端顶点 ; 弧尾 (Tail) : 箭头另一顶点。



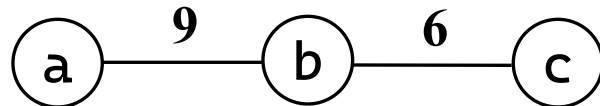
# 图的定义和术语

- 设  $\langle v_i, v_j \rangle \in VR$ ,  $v_i \neq v_j$ ; 图中顶点数目  $n$ ; 图中边 (或弧) 的数目  $m$ ; 则:
  - 对于有向图:  $M=n(n-1)$ ;
  - 对于无向图:  $M=n(n-1)/2$ 。
  - 对有向图和无向图有  $0 \leq m \leq M$ 。
- 完全图 等定义
  - 完全图:  $m=M$ 。
  - 稀疏图:  $m \ll M$ 。
  - 稠密图:  $m$  接近于  $M$ 。



# 图的定义和术语

- 权值 ( Weight ) : 为图中的边或弧所赋予的一个数值 , 用于量化其某种特性或成本。

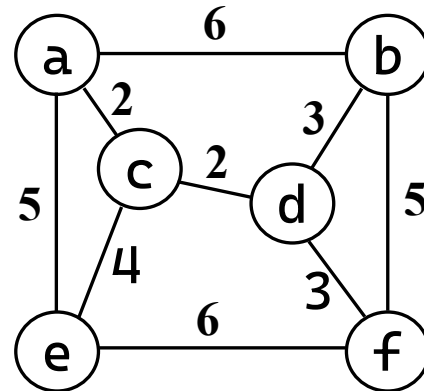
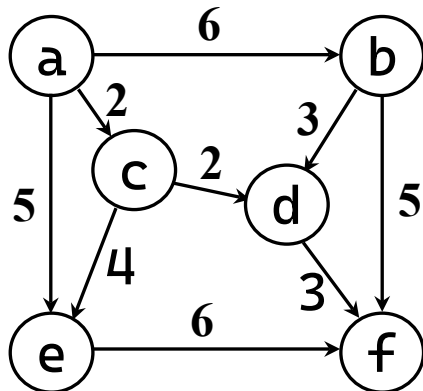


- 网 ( Network ) : 指带有权值的图。

– 网是边集  $E$  上定义了一个权值函数  $W:E \rightarrow R$  的图  $G=(V, E)$ 。

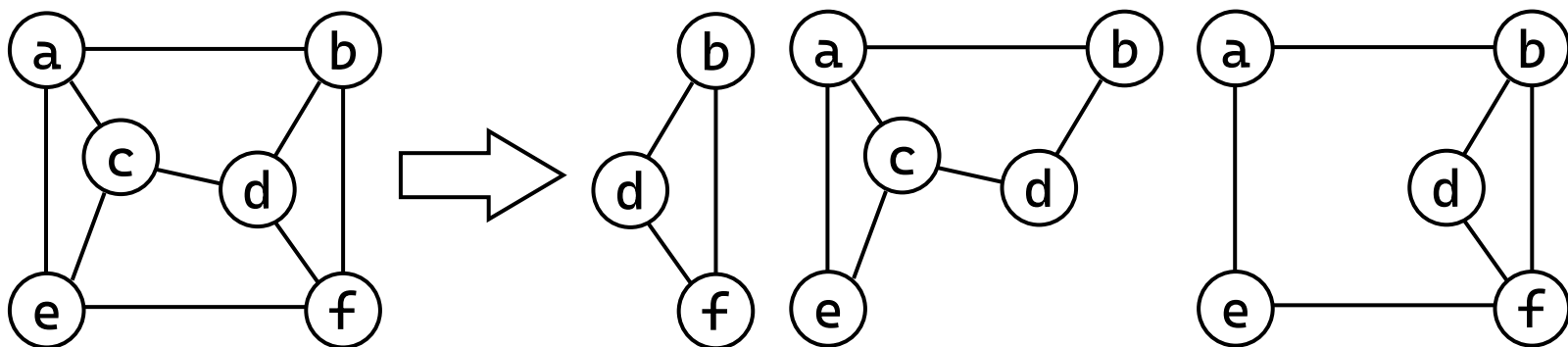
– 在不会引起歧义时，网也直接称为图。

– 有向网：带权的有向图；无向网：带权的无向图。



# 图的定义和术语

- 子图：设图  $G=(V, E)$ ， $G'=(V', E')$ ，若  $V' \subseteq V$  且  $E' \subseteq E$ ，则称  $G'$  为  $G$  的子图。

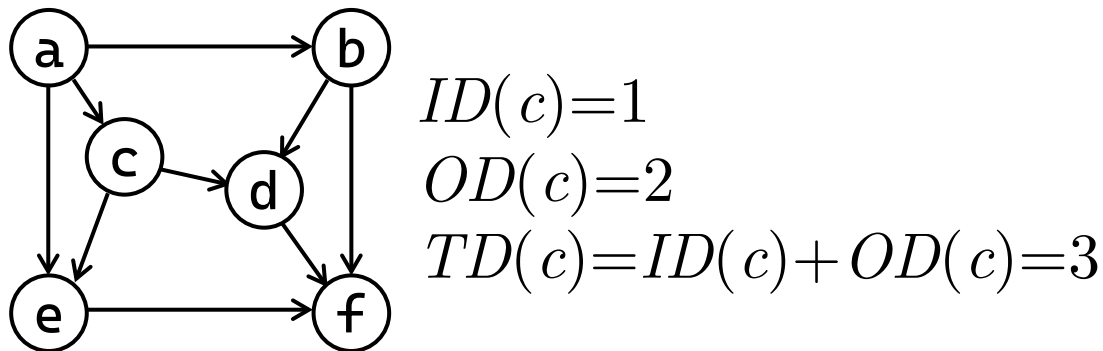
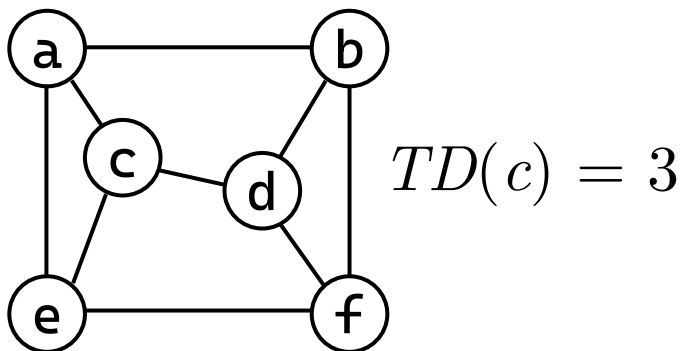


## • 邻接点 ( Adjacent )

- 对于有向图  $G=(V, E)$ ，如果边  $(v, v') \in E$ ，则称顶点  $v$  和  $v'$  互为邻接点(即相邻接)，边  $(v, v')$  与  $v$  和  $v'$  相关联。
- 对于无向图  $G=(V, E)$ ，如果弧  $\langle v, v' \rangle \in E$ ，则称  $v$  邻接到  $v'$ ， $v'$  邻接自  $v$ ；弧  $\langle v, v' \rangle$  和顶点  $v, v'$  相关联。

# 图的定义和术语

- 顶点的度：与顶点 $v$ 相邻接的边的数目，记为 $TD(v)$ 。
  - 有向图顶点 $v$ 的入度 $ID(v)$ ：以 $v$ 为弧头的弧的数目
  - 有向图顶点 $v$ 的出度 $OD(v)$ ：以 $v$ 为弧尾的弧的数目
  - 有向图顶点 $v$ 的度： $TD(v) = ID(v) + OD(v)$ 。
  - 无向图顶点 $v$ 的度：记为 $TD(v)$ 。
  - 边长  $m = \frac{1}{2} \sum_{i=1}^n TD(v_i)$





# 图的定义和术语

## • 路径

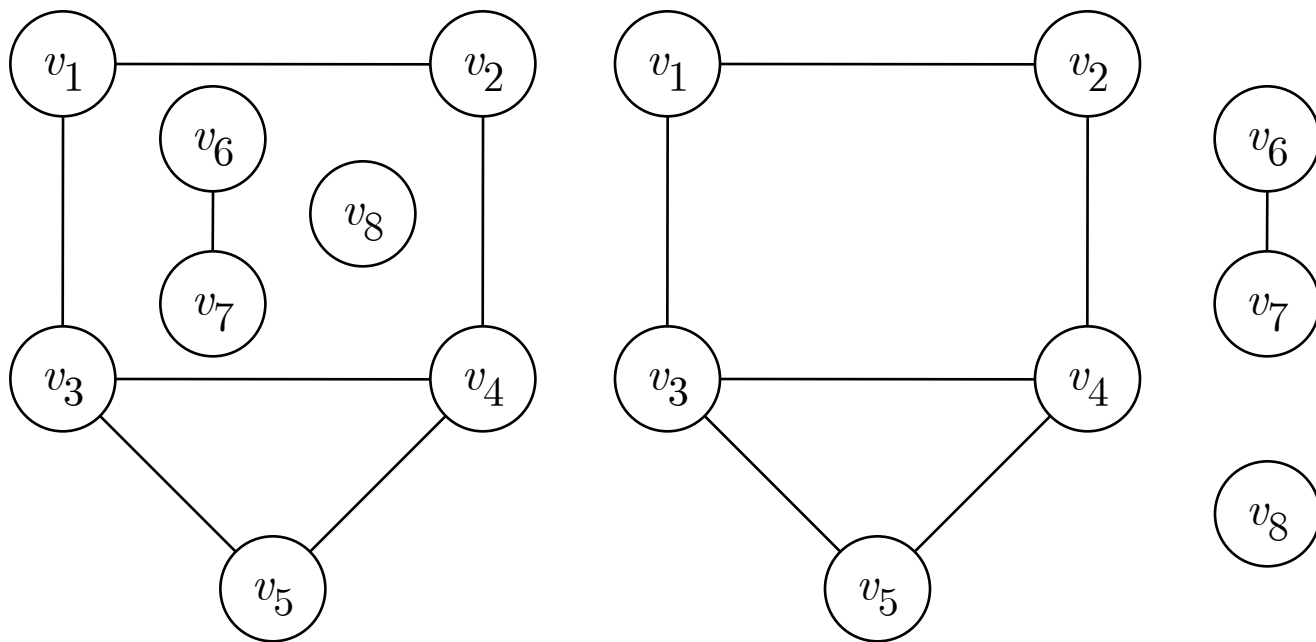
- 设图  $G=(V, E)$  为一个无向图或有向图，若存在一个顶点序列  $v=v_0, v_1, \dots, v_k=v'$ ，满足：
- 边的存在性：对于任意  $j \in \{1, 2, \dots, k\}$ ，顶点对  $(v_{j-1}, v_j) \in E$ 。
  - 方向性：在无向图中，边  $(v_{j-1}, v_j)$  无方向；在有向图中，边必须为有向边  $\langle v_{j-1}, v_j \rangle$  且方向从  $v_{j-1}$  指向  $v_j$ 。
- 起点与终点：序列首顶点  $v_0$  为路径的起点，末顶点  $v_k$  为终点。
- 路径的长度：路径中边的数量（或跳数）为  $|P| = k$ 。
- 路径的构成：路径  $P$  可以表示为顶点序列  $v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_k$ ，或简记为  $P = \langle v_0, v_1, \dots, v_k \rangle$ 。

# 图的定义和术语

- 简单路径与回路
  - 简单路径：路径中所有顶点各不相同（即 $\forall i \neq j, v_i \neq v_j$ ）。
  - 回路（Cycle）或环：起点与终点相同（ $v_0 = v_k$ ）
    - 简单回路：简单路径中的回路。
- $v$ 和 $v'$ 连通： $v$ 和 $v'$ 之间有路径。
- 连通图：无向图中任意两个顶点都连通。
  - 强连通图：有向图中， $\forall i \neq j, v_i$ 到 $v_j$ 、 $v_j$ 到 $v_i$ 都有路径。
- 连通分量：无向图中的极大连通子图。
  - 强连通分量：有向图中的极大连通子图。

# 图的定义和术语

- 例如，非连通图及其3个连通分量。

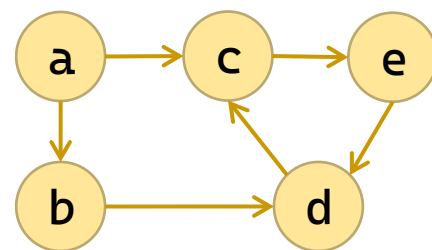


# 图的定义和术语

- 例 设图  $G=(V, E)$  ,  $V=\{a, b, c, d, e\}$  ,  
 $E=\{ \langle a, b \rangle, \langle a, c \rangle, \langle b, d \rangle, \langle c, e \rangle, \langle d, c \rangle, \langle e, d \rangle \}$

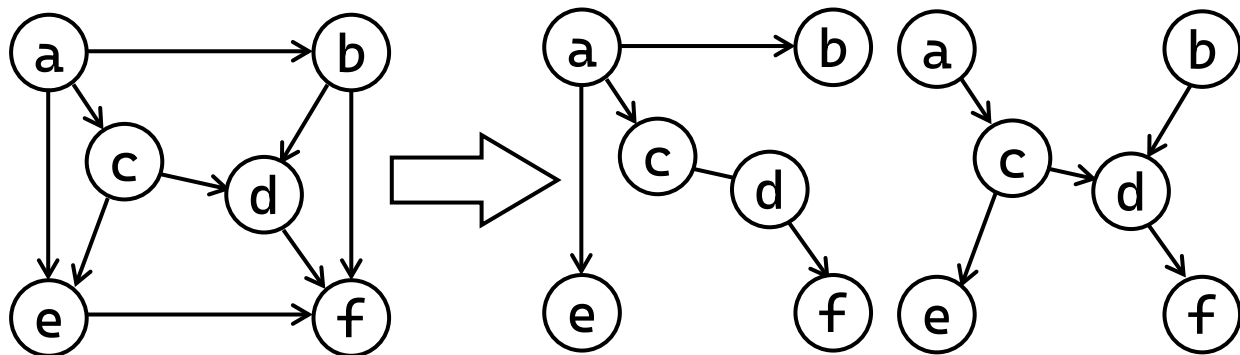
- 回答下列问题：

- (1) 求与顶点b相关联的弧；  $\langle a, b \rangle, \langle b, d \rangle$
- (2) 是否存在从c到b的路径？ 不存在，  $\langle c, e \rangle, \langle e, d \rangle, \langle d, c \rangle$
- (3) 求ID(d)、OD(d)、TD(d) ; ID(d)=2、OD(d)=1、TD(d)=3
- (4) 该有向图是否是强连通图？ 不是，不存在c到b的路径。
- (5) 画出各个强连通分量。 ①  $\langle a \rangle$  ; ②  $\langle b \rangle$  ;  
③  $\langle c, e \rangle, \langle e, d \rangle, \langle d, c \rangle$  。



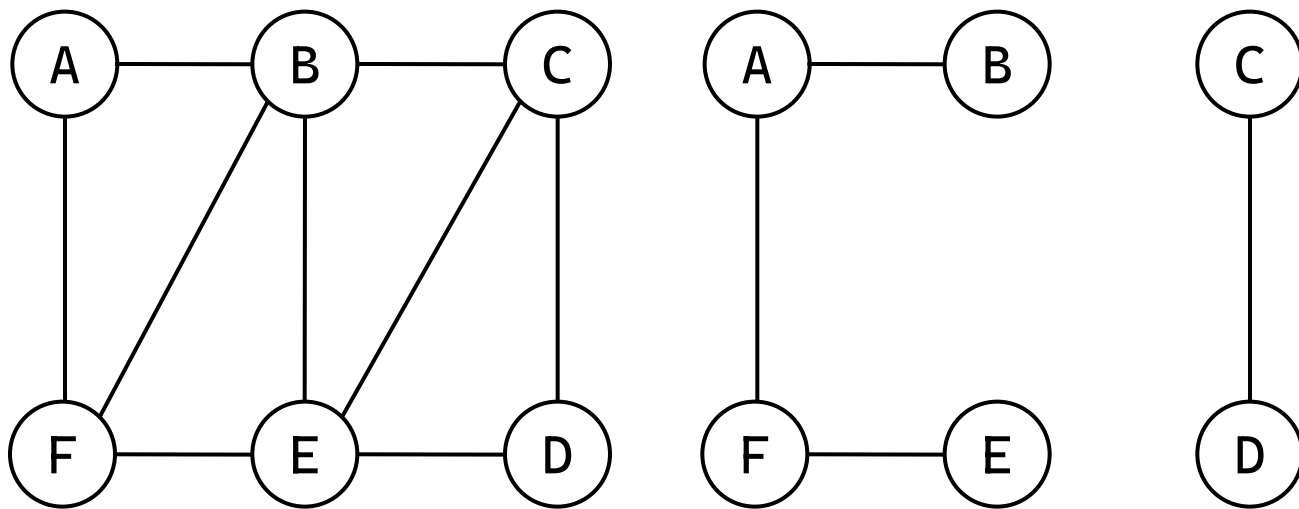
# 图的定义和术语

- 生成树：设连通图  $G$  共有  $n$  个顶点，则含有图  $G$  的  $n$  个顶点且有  $n-1$  条边的连通图，称为图  $G$  的一棵生成树。
  - 有  $n$  个顶点和  $n-1$  条边的图不一定是生成树。
  - 在有  $n$  个顶点的图中，如果边多于  $n-1$  条，则一定有环；如果边少于  $n-1$  条，则是非连通图。
- 有向树：只有一个顶点的入度为 0，其它顶点的入度都为 1 的有向图。



# 图的定义和术语

- 生成森林：含有图中全部 $n$ 个顶点的 $m$ 棵互不相交的有向树(共有 $n-m$ 条弧)。

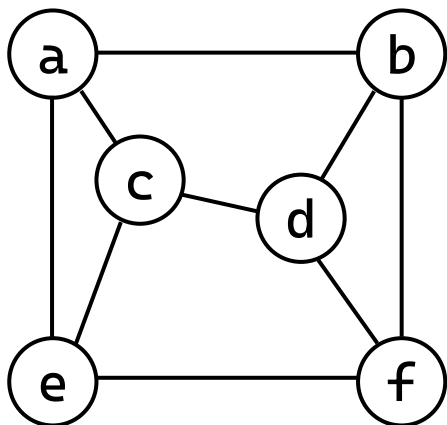


# 目录

|  |   |           |
|--|---|-----------|
|  | 1 | 图的定义和术语   |
|  | 2 | 图的存储结构    |
|  | 3 | 图的遍历      |
|  | 4 | 最小生成树     |
|  | 5 | 有向无环图及其应用 |

# 图的邻接矩阵

- 例 对于无向图



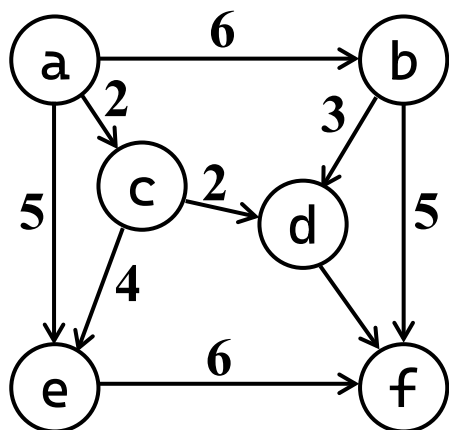
$V_e = \{ a, b, c, d, e, f \}$

| <b>Vr</b> | <b>a</b> | <b>b</b> | <b>c</b> | <b>d</b> | <b>e</b> | <b>f</b> |
|-----------|----------|----------|----------|----------|----------|----------|
| <b>a</b>  | 0        | 1        | 1        | 0        | 1        | 0        |
| <b>b</b>  | 1        | 0        | 0        | 1        | 0        | 1        |
| <b>c</b>  | 1        | 0        | 0        | 1        | 1        | 0        |
| <b>d</b>  | 0        | 1        | 1        | 0        | 0        | 1        |
| <b>e</b>  | 1        | 0        | 1        | 0        | 0        | 1        |
| <b>f</b>  | 0        | 1        | 0        | 1        | 1        | 0        |



# 图的邻接矩阵

- 例 对于有向网



$V_e = \{ a, b, c, d, e, f \}$

| <b>Vr</b> | <b>a</b> | <b>b</b> | <b>c</b> | <b>d</b> | <b>e</b> | <b>f</b> |
|-----------|----------|----------|----------|----------|----------|----------|
| <b>a</b>  | 0        | 6        | 2        | 0        | 5        | 0        |
| <b>b</b>  | 0        | 0        | 0        | 3        | 0        | 5        |
| <b>c</b>  | 0        | 0        | 0        | 2        | 4        | 0        |
| <b>d</b>  | 0        | 0        | 0        | 0        | 0        | 3        |
| <b>e</b>  | 0        | 0        | 0        | 0        | 0        | 6        |
| <b>f</b>  | 0        | 0        | 0        | 0        | 0        | 0        |

# 图的邻接矩阵

- 邻接矩阵，也称为数组表示法

- 对于无权图：adj取0表示无边/不邻接，取1表示有边/邻接。
- 对于带权图（网）：adj直接存储权值（如距离、花费）。此时，通常用0或MAX表示“不连通”。

```
typedef struct {
 int adj;
} ArcNode;
typedef struct AdjMatrix {
 char vertex[MAX_VERTEX];
 ArcNode arcs[MAX_VERTEX][MAX_VERTEX];
 int vexnum;
} AdjMatrix;
```

# 图的邻接矩阵：初始化和定位

- 初始化：将邻接矩阵置0或者INF

```
void init_graph(AdjMatrix* G) {
 G->vexnum = 0;
 for (int i = 0; i < MAX_VERTEX; i++)
 for (int j = 0; j < MAX_VERTEX; j++)
 G->arcs[i][j].adj = INFINITY;
}
```

- 定位顶点：根据顶点名字查询顶点下表

```
int locate_vertex(AdjMatrix* G, char v) {
 for (int i = 0; i < G->vexnum; i++)
 if (G->vertex[i] == v)
 return i;
 return ERROR;
}
```

# 图的邻接矩阵：插入顶点

- 插入顶点：新建顶点，与其它顶点的权重置为最大

```
int insert_vertex(AdjMatrix* G, char v) {
 if (locate_vertex(G, v) != ERROR) // 已存在
 return ERROR;
 if (G->vexnum >= MAX_VERTEX) // 容量不足
 return ERROR;
 G->vertex[G->vexnum] = v;
 // 新顶点与其他顶点的连接初始化为 INFINITY
 for (int i = 0; i <= G->vexnum; i++) {
 G->arcs[i][G->vexnum].adj = INFINITY;
 G->arcs[G->vexnum][i].adj = INFINITY;
 }
 G->vexnum++;
 return TRUE;
}
```

# 图的邻接矩阵：删除顶点

- 删除顶点

- 删除出边：后面的边（行）依次往前覆盖

```
for (int i = loc; i < G->vexnum - 1; i++)
 for (int j = 0; j < G->vexnum; j++)
 G->arcs[i][j].adj = G->arcs[i + 1][j].adj;
```

- 删除入边：后面的边（列）依次往前覆盖

```
for (int i = loc; i < G->vexnum - 1; i++) {
 for (int j = 0; j < G->vexnum; j++) {
 G->arcs[j][i].adj = G->arcs[j][i + 1].adj;
```

- 删除顶点：后面的顶点依次往前覆盖

```
for (int i = loc; i < G->vexnum - 1; i++)
 G->vertex[i] = G->vertex[i + 1];
```

- 顶点长度收缩

```
G->vexnum--;
```

```

int delete_vertex(AdjMatrix* G, char v) {
 int loc = locate_vertex(G, v);
 if (loc == ERROR)
 return ERROR;
 // 行上移 (将后面的行覆盖到前面)
 for (int i = loc; i < G->vexnum - 1; i++)
 for (int j = 0; j < G->vexnum; j++)
 G->arcs[i][j].adj = G->arcs[i + 1][j].adj;
 // 列左移 (将后面的列覆盖到前面)
 for (int i = loc; i < G->vexnum - 1; i++)
 for (int j = 0; j < G->vexnum; j++)
 G->arcs[j][i].adj = G->arcs[j][i + 1].adj;
 // 顶点表前移
 for (int i = loc; i < G->vexnum - 1; i++)
 G->vertex[i] = G->vertex[i + 1];
 G->vexnum--;
 return TRUE;
}

```

# 图的邻接矩阵：插入弧

- 插入弧：找到点，更新权重

```
int insert_arc(AdjMatrix* G, char vi, char vt, int weight) {
 int row = locate_vertex(G, vi);
 int col = locate_vertex(G, vt);
 if (row == ERROR || col == ERROR)
 return ERROR;
 G->arcs[row][col].adj = weight;
 return TRUE;
}
```

# 图的邻接矩阵：删除弧

- 删除弧：找到点，更新权重为无限大

```
int delete_arc(AdjMatrix* G, char vi, char vt) {
 int row = locate_vertex(G, vi);
 int col = locate_vertex(G, vt);
 if (row == ERROR || col == ERROR
 || G->arcs[row][col].adj == INFINITY)
 return ERROR;
 G->arcs[row][col].adj = INFINITY;
 return TRUE;
}
```



# 图的邻接矩阵：创建图

- 创建图

- 初始化

```
init_graph(G);
```

- 循环接受输入点

```
for (i = 0; input[i]!='\0' && i<MAX_VERTEX; i++)
```

- 插入点

```
insert_vertex(G, input[i])
```

- 初始化对角线

```
for (int i = 0; i < G->vexnum; i++) {
 G->arcs[i][i].adj = 0;
```

- 循环接受输入弧

```
while (1) {
 scanf(" %c %c %d", &v1, &v2, &weight);
```

- 插入弧

```
insert_arc(G, v1, v2, weight);
```

```
}
```

```

void create_graph(AdjMatrix* G) {
 char input[MAX_VERTEX + 1]; // 存放输入的顶点字符串
 char v1, v2;
 int weight;
 init_graph(G);
 printf("请在一行内输入所有顶点（如 'ABCDE'）：");
 scanf("%s", input);
 getchar(); // 吸收换行
 // 将输入的每个字符作为一个顶点插入
 for (int i = 0; input[i] != '\0' && i < MAX_VERTEX; i++)
 if (insert_vertex(G, input[i]) == ERROR)
 printf("警告：顶点 %c 插入失败。\\n", input[i]);
 // 初始化矩阵：所有元素为 INFINITY，对角线为 0
 for (int i = 0; i < G->vexnum; i++)
 G->arcs[i][i].adj = 0;
 printf("请输入弧（格式：起点 终点 权重），输入 '#' 结束：\\n");
 while (1) {
 printf("> ");
 scanf(" %c", &v1);
 }
}

```

```
 if (v1 == '#')
 break;
 scanf("%c %d", &v2, &weight);
 int result = insert_arc(G, v1, v2, weight);
 if (result == ERROR)
 printf("插入弧 %c->%c 失败 (顶点不存在) \n", v1, v2);
 else
 printf("插入成功\n");
}
printf("弧输入结束。 \n");
}
```

# 图的邻接矩阵：计算出入度

- 计算出入度

```
void print_all_degrees(AdjMatrix* G) {
 printf("顶点\t出度\t入度\t度\n");
 for (int i = 0; i < G->vexnum; i++) {
 char v = G->vertex[i];
 int out_deg = 0, in_deg = 0;
 for (int j = 0; j < G->vexnum; j++) {
 if (G->arcs[i][j].adj!=INFINITY && G->arcs[i][j].adj!=0)
 out_deg++;
 if (G->arcs[j][i].adj!=INFINITY && G->arcs[j][i].adj!=0)
 in_deg++;
 }
 printf(" %c\t%d\t%d\t%d\n", v, out_deg, in_deg, out_deg+in_deg);
 }
 printf("=====\n");
}
```

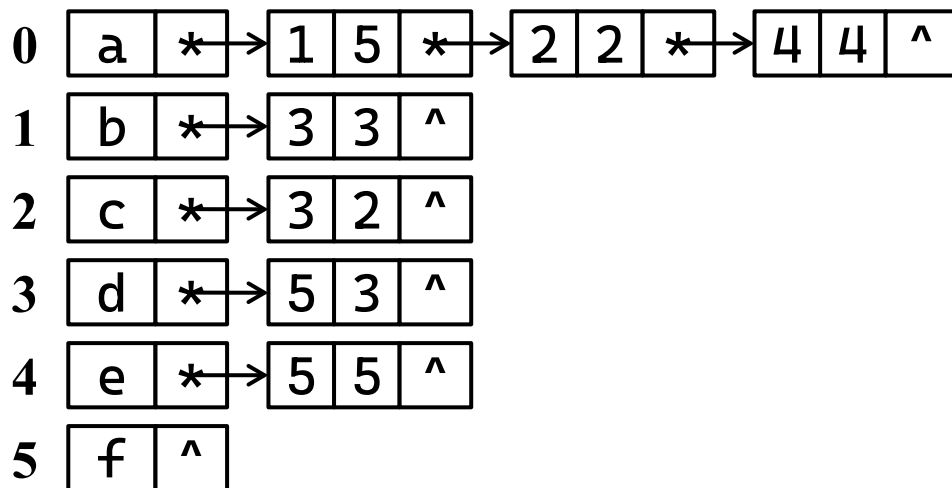
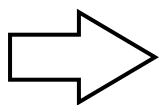
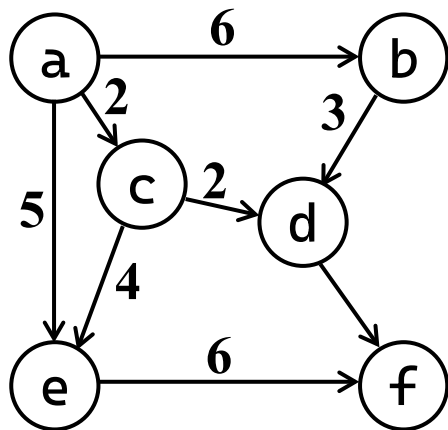
# 图的邻接表

- 图的邻接表存储表示：

- 图的一种链式存储结构；

- 图中每一个顶点对应1个结点，并由所有顶点结点构成一个顺序表(结构)；

- 图中每一个顶点建立一条链，该链由它的所有邻接点构成。



# 图的邻接表

- 无向图邻接表

- 1条边对应2个表结点，即结点总数是边数的2倍；
- 顶点的度=邻接点链表中结点的数目。

- 有向图邻接表

- 1条弧对应1个表结点，即结点总数=弧的数目；
- 顶点的出度=邻接点链表中结点数目。
- 在有向图邻接表中，求顶点的入度比较复杂，需搜索所有的邻接点链表后才能确定。
- 对有向图建立逆邻接链表，则顶点的入度=逆邻接点链表中结点的数目。

# 图的邻接表

- 图的邻接表的存储结构

```
typedef struct ArcNode { // 边结点（链表结点）
 int adjvex; // 邻接点下标
 int weight; // 权值
 struct ArcNode* next; // 下一条边
} ArcNode;
typedef struct { // 顶点结点
 char data; // 顶点数据
 ArcNode* firstarc; // 第一条边
} VertexNode;
typedef struct { // 邻接表表示的图
 VertexNode vertex[MAX_VERTEX];
 int vexnum; // 顶点数
} AdjList;
```

# 图的邻接表：初始化和定位

- 初始化：顶点数归零，所有顶点的边置空

```
void init_graph(AdjList* G) {
 G->vexnum = 0;
 for (int i = 0; i < MAX_VERTEX; i++) {
 G->vertex[i].firstarc = NULL;
 }
}
```

- 定位顶点：根据顶点名字查询顶点下表

```
int locate_vertex(AdjList* G, char v) {
 for (int i = 0; i < G->vexnum; i++)
 if (G->vertex[i].data == v)
 return i;
 return ERROR;
}
```



# 图的邻接表：插入顶点

- 插入顶点：新建顶点，该顶点的边置空

```
int insert_vertex(AdjList* G, char v) {
 if (locate_vertex(G, v) != ERROR)
 return ERROR;
 if (G->vexnum >= MAX_VERTEX)
 return ERROR;

 G->vertex[G->vexnum].data = v;
 G->vertex[G->vexnum].firstarc = NULL;
 G->vexnum++;
 return OK;
}
```

# 图的邻接表：删除顶点

- 删除顶点

- 删除出边：顶点下所有边

```
ArcNode* p = G->vertex[j].firstarc;
while (p != NULL) {
 ArcNode* q = p;
 p = p->next;
 free(q);
}
```

- 顶点下的边置空

```
G->vertex[j].firstarc = NULL;
```

- 删除入边：循环各点

```
for (int i = 0; i < G->vexnum; i++) {
```

- 从首边开始

```
ArcNode* pre=NULL, * cur=G->vertex[i].firstarc;
```

- 找到当前边的前结点

```
while (cur != NULL) {
 if (cur->adjvex == j) {
 if (pre == NULL)
 G->vertex[i].firstarc=cur->next;
 else
 pre->next=cur->next;
 free(cur);
 break; // 不会有重复边
 }
 pre = cur; cur = cur->next;
}
```

- 删除当前结点

# 图的邻接表：删除顶点

- 删除顶点

- 删除顶点：

- 后面的顶点

- 依次往前覆盖

- 顶点长度收缩

```
for (int i = j; i < G->vexnum - 1; i++) {
 G->vertex[i].data = G->vertex[i + 1].data;
 G->vertex[i].firstarc = G->vertex[i + 1].firstarc;
}
```

```
G->vexnum--;
```

```

int delete_vertex(AdjList* G, char v) {
 int j = locate_vertex(G, v);
 if (j == ERROR)
 return ERROR;
 // 1. 删除所有以 v 为起点的边 (出边)
 ArcNode* p = G->vertex[j].firstarc;
 while (p != NULL) {
 ArcNode* q = p;
 p = p->next;
 free(q);
 }
 G->vertex[j].firstarc = NULL;
 // 2. 删除所有指向 v 的边 (入边) : 遍历所有顶点, 删除其链表中
 // adjvex == j 的结点
 for (int i = 0; i < G->vexnum; i++) {
 ArcNode* pre = NULL, * cur = G->vertex[i].firstarc;
 while (cur != NULL) {
 if (cur->adjvex == j) {
 if (pre == NULL)
 G->vertex[i].firstarc = cur->next;
 else
 pre->next = cur->next;
 free(cur);
 break; // 无重边
 }
 pre = cur;
 cur = cur->next;
 }
 }
}

```

```

 pre = cur;
 cur = cur->next;
 }
}

// 3. 将顶点表中后面的顶点前移, 并更新所有邻接点下标 (>j 的减 1)
// 先将顶点数据前移
for (int i = j; i < G->vexnum - 1; i++) {
 G->vertex[i].data = G->vertex[i + 1].data;
 G->vertex[i].firstarc = G->vertex[i + 1].firstarc;
}
G->vexnum--;

// 4. 更新所有链表中 adjvex > j 的结点, 将其减 1
for (int i = 0; i < G->vexnum; i++) {
 ArcNode* cur = G->vertex[i].firstarc;
 while (cur != NULL) {
 if (cur->adjvex > j)
 cur->adjvex--;
 cur = cur->next;
 }
}
return OK;
}

```

# 图的邻接表：插入弧

- 插入弧

- 找到插入位置

- 邻接点升序

```
ArcNode* pre=NULL, * cur=G->vertex[i].firstarc;
while (cur != NULL && cur->adjvex < j) {
 pre = cur;
 cur = cur->next;
}
```

- 如果已有则更新

```
if (cur != NULL && cur->adjvex == j) {
 cur->weight = weight;
 return OK;
}
```

- 新建边

```
ArcNode* p = (ArcNode*)malloc(sizeof(ArcNode));
p->adjvex = j; p->weight = weight; p->next = cur;
```

- 接上

- 判断首结点

```
if (pre == NULL)
 G->vertex[i].firstarc = p;
else
 pre->next = p;
```

```

int insert_arc(AdjList* G, char vi, char vt, int weight) {
 int i = locate_vertex(G, vi);
 int j = locate_vertex(G, vt);
 if (i == ERROR || j == ERROR)
 return ERROR;
 ArcNode* pre = NULL, * cur = G->vertex[i].firstarc;
 while (cur != NULL && cur->adjvex < j) { // 查找前结点
 pre = cur;
 cur = cur->next;
 }
 if (cur != NULL && cur->adjvex == j) { // 已存在: 更新权重
 cur->weight = weight;
 return OK;
 }
 // 不存在: 创建新结点插入
 ArcNode* p = (ArcNode*)malloc(sizeof(ArcNode));
 p->adjvex = j; p->weight = weight; p->next = cur;
 if (pre == NULL)
 G->vertex[i].firstarc = p;
 else
 pre->next = p;
 return OK;
}

```

# 图的邻接表：删除弧

- 删除弧

- 找到前一位置

```
ArcNode* pre=NULL, * cur=G->vertex[i].firstarc;
while (cur != NULL && cur->adjvex != j) {
 pre = cur;
 cur = cur->next;
}
if (cur == NULL) return ERROR;
```

- 删除边

- 判断首结点

```
if (pre == NULL)
 G->vertex[i].firstarc = cur->next;
else
 pre->next = cur->next;
free(cur);
```



```

int delete_arc(AdjList* G, char vi, char vt) {
 int i = locate_vertex(G, vi);
 int j = locate_vertex(G, vt);
 if (i == ERROR || j == ERROR)
 return ERROR;
 ArcNode* pre = NULL, * cur = G->vertex[i].firstarc;
 while (cur != NULL && cur->adjvex != j) {
 pre = cur;
 cur = cur->next;
 }
 if (cur == NULL) // 未找到
 return ERROR;

 if (pre == NULL)
 G->vertex[i].firstarc = cur->next;
 else
 pre->next = cur->next;
 free(cur);
 return OK;
}

```

# 图的邻接表：创建图

- 创建图

- 初始化

```
init_graph(G);
```

- 循环接受输入点

```
for (i = 0; input[i]!='\0' && i<MAX_VERTEX; i++)
```

- 插入点

```
insert_vertex(G, input[i])
```

- 循环接受输入弧

```
while (1) {
 scanf(" %c %c %d", &v1, &v2, &weight);
```

- 插入弧

```
 insert_arc(G, v1, v2, weight);
```

```
}
```

```

void create_graph(AdjList* G) {
 char input[MAX_VERTEX + 1];
 char vi, vt;
 int weight;
 init_graph(G);
 printf("请在一行内输入所有顶点 (如 'ABCDE') : ");
 scanf("%s", input); getchar(); // 吸收换行
 for (int i = 0; input[i] != '\0' && i < MAX_VERTEX; i++)
 if (insert_vertex(G, input[i]) == ERROR)
 printf("警告: 顶点 %c 插入失败.\n", input[i]);
 printf("请输入弧 (格式: 起点 终点 权重), 输入 '#' 结束: \n");
 while (1) {
 printf("> "); scanf(" %c", &vi);
 if (vi == '#') break;
 scanf("%c %d", &vt, &weight);
 if (insert_arc(G, vi, vt, weight) == ERROR)
 printf("插入弧 %c->%c 失败 (顶点不存在) \n", vi, vt);
 else
 printf("插入成功\n");
 }
 printf("弧输入结束. \n");
}

```

# 图的邻接表：计算出入度

- 计算出入度

- 出度由顶点的链表算

```
ArcNode* p = G->vertex[i].firstarc;
while (p != NULL) {
 out_deg++;
 p = p->next;
}
```

- 入度从顶点链表查找

```
for (int i = 0; i < G->vexnum; i++) {
 p = G->vertex[i].firstarc;
 while (p != NULL) {
 if (p->adjvex == loc)
 in_deg++;
 p = p->next;
 }
}
```

```

void print_all_degrees(AdjMatrix* G) {
 printf("顶点\t出度\t入度\t度\n");
 for (int i = 0; i < G->vexnum; i++) {
 char v = G->vertex[i].data;
 int out_deg = 0, in_deg = 0;
 ArcNode* p = G->vertex[i].firstarc;
 while (p != NULL) {
 out_deg++;
 p = p->next;
 }
 for (int j = 0; j < G->vexnum; j++) {
 p = G->vertex[j].firstarc;
 while (p != NULL) {
 if (p->adjvex == i)
 in_deg++;
 p = p->next;
 }
 }
 printf(" %c\t%d\t%d\t%d\n", v, out_deg, in_deg,
 out_deg+in_deg);
 }
 printf("=====\n");
}

```

# 目录

|  |   |           |
|--|---|-----------|
|  | 2 | 图的存储结构    |
|  | 3 | 图的遍历      |
|  | 4 | 最小生成树     |
|  | 5 | 有向无环图及其应用 |
|  | 6 | 最短路径      |

# 图的遍历

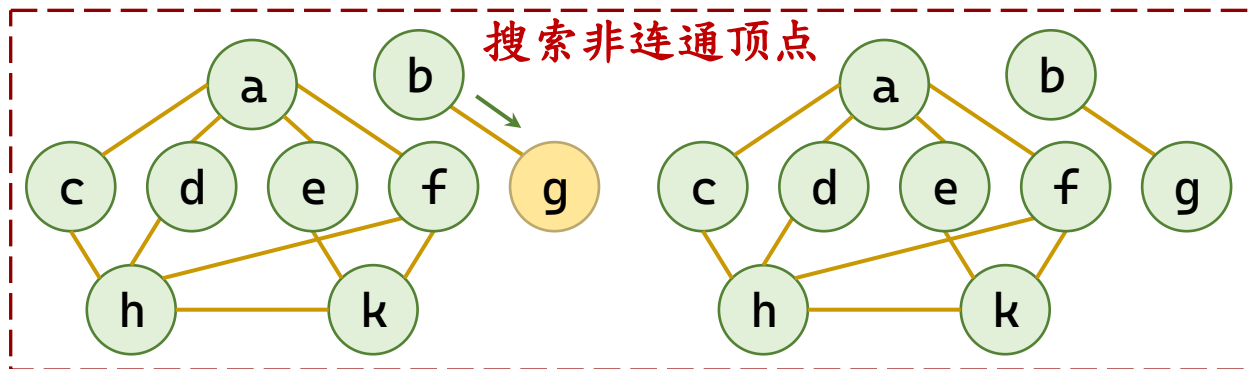
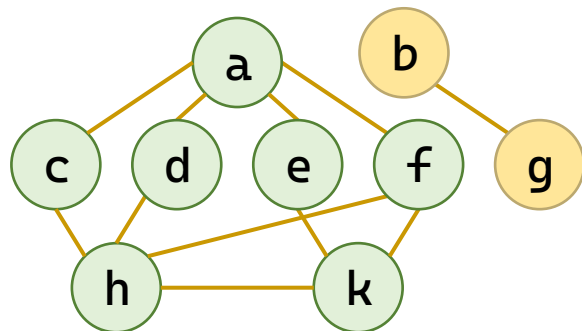
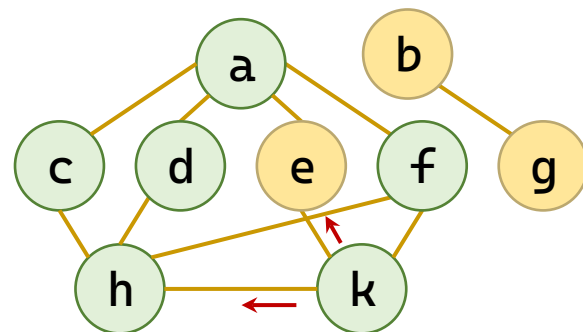
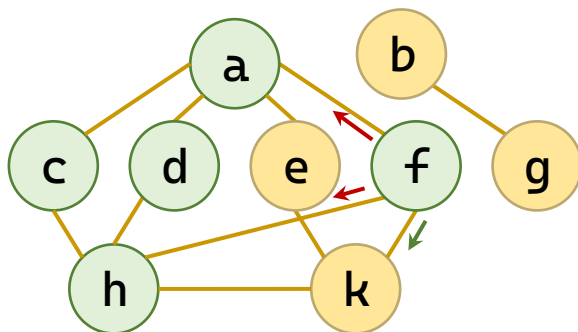
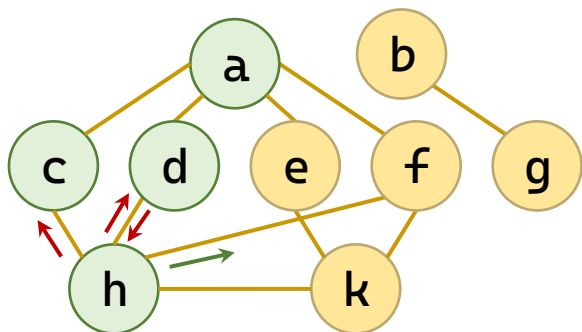
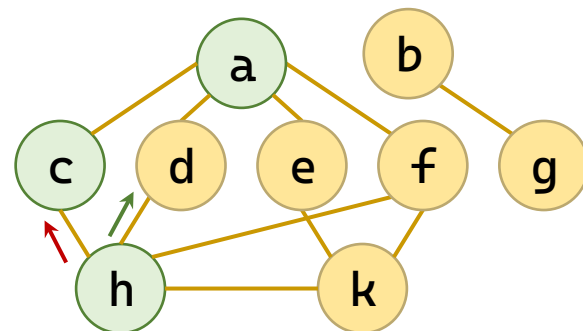
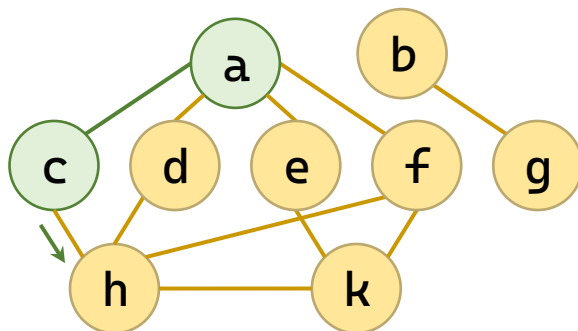
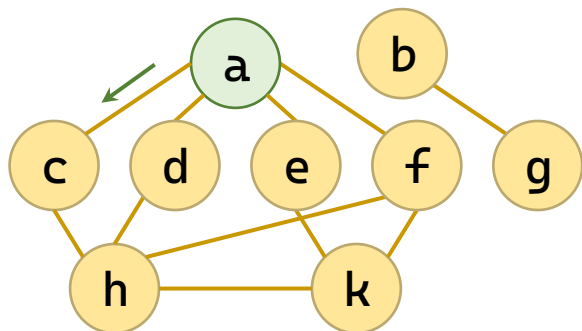
- 从图中某个顶点出发，按照一定规则访问图中的所有顶点，且图中的每个顶点仅被访问一次。
- 基本思想
  - 1、从图中某个顶点 $v$ 出发，访问该顶点；
  - 2、查找顶点 $v$ 的第一个未被访问的邻接点 $v_1$ ，访问该顶点， $v \leftarrow v_1$ ；
  - 3、重复第2步操作，直到图中没有未被访问的顶点为止。
- 深度优先搜索(Depth First Search)
- 广度优先搜索(Breadth First Search)

# 深（广）度优先遍历

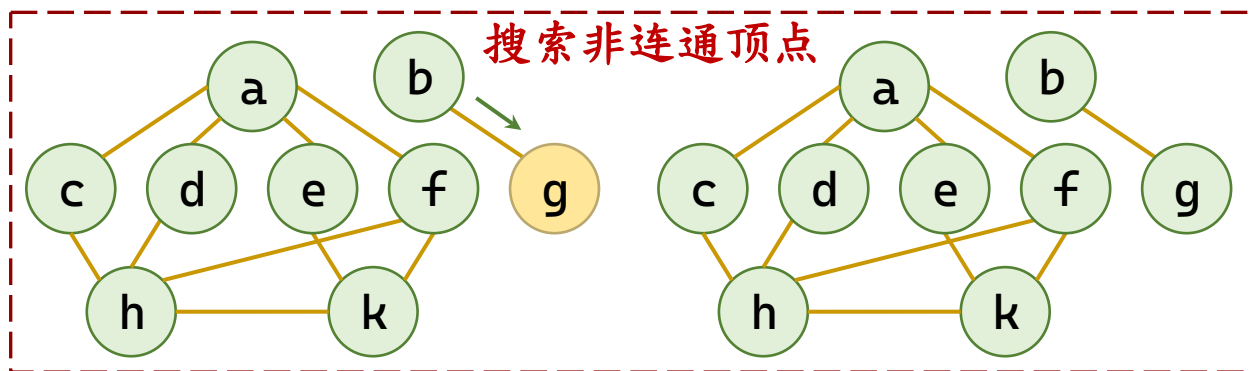
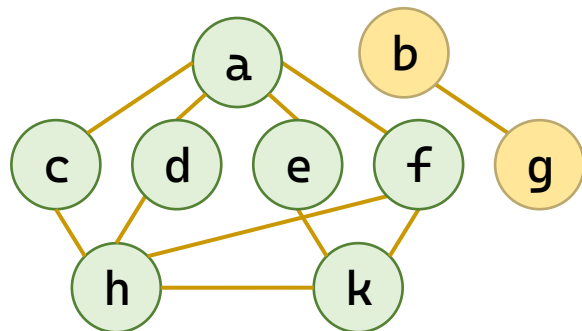
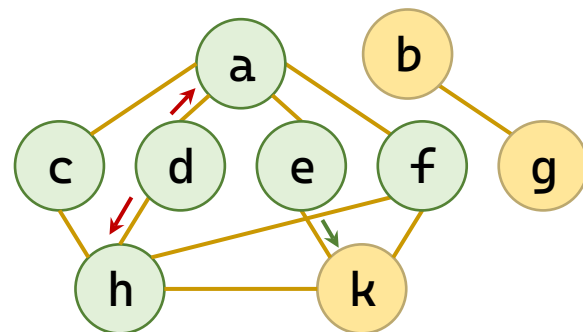
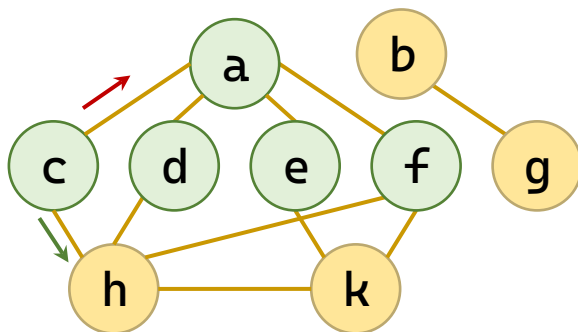
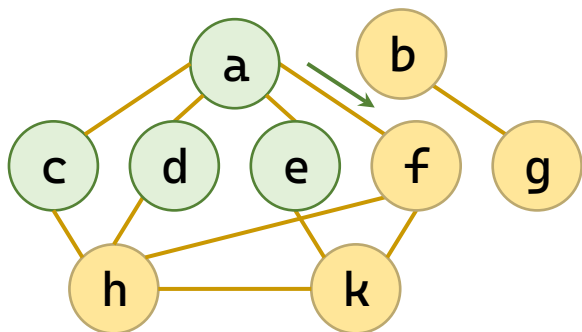
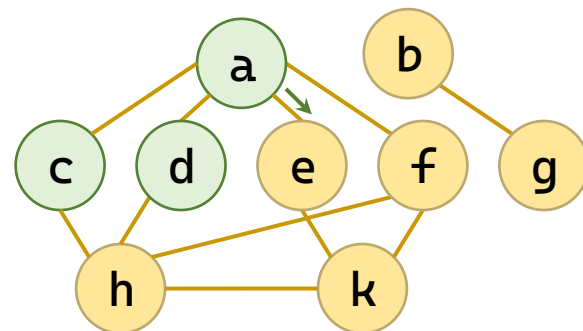
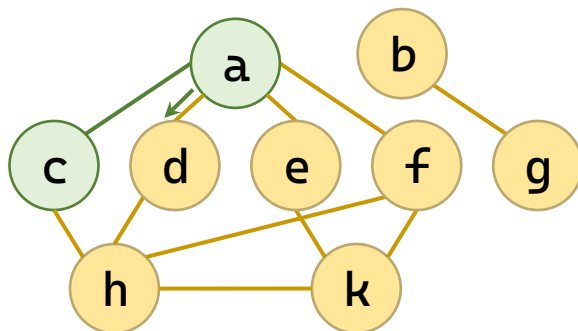
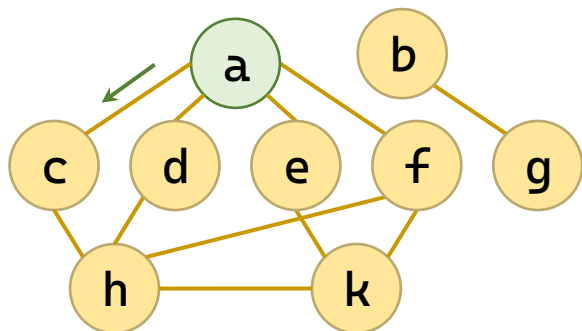
- 初始化：防重复数组visited[]清零。
- 启动：遇 visited[v]==0的v启动。
- 取出：从栈（DFS）或队列（BFS）中弹出一个顶点。
- 访问：处理该顶点，并将对应visited[v]标记置1。
- 扩展
  - 深度优先：扫描其邻接点，将首个未访问的邻接点压入栈。
  - 广度优先：扫描其邻接点，将所有未访问的邻接点进队列。
- 循环：重复“取出、访问、扩展”直至容器为空。
- 非连通处理：回到“启动”，直至全图标记完毕。



# 深度优先遍历



# 广度优先遍历



# 深度优先和广度优先

- 相同点：取出、访问、扩展
- 不同点：深度优先用栈、广度优先用队列

```
void dfs_stack(int start) {
 int vst[SIZE] = {}; // 已访问标记
 Stack s;
 init_stack(&s);
 push(&s, start);
 vst[start] = 1; // 入栈即标记
 while (!is_stack_empty(&s)) {
 int v = pop(&s);
 printf("%d ", v);
 for (int w = 1; w <= n; w++)
 if (g[v][w]==1 && !vst[w]) {
 v[w] = 1; // 入栈即标记
 push(&s, w);
 }
 }
 printf("\n");
}
```

```
void bfs_queue(int start) {
 int v[SIZE] = {}; // 已访问标记
 Queue q;
 init_queue(&q);
 enqueue(&q, start);
 vst[start] = 1; // 入队即标记
 while (!is_queue_empty(&q)) {
 int v = dequeue(&q);
 printf("%d ", v);
 for (int w = 1; w <= n; w++)
 if (g[v][w]==1 && !vst[w]) {
 v[w] = 1; // 入队即标记
 enqueue(&q, w);
 }
 }
 printf("\n");
}
```

# 应用：判断两个顶点是否连通

- 例 判断v1和v2是否连通(邻接矩阵存储)

```
bool isConnected(AdjMatrix* G, int b, int c) {
 if (b == c) return true;
 int visited[MAX_VERTEX] = { 0 }; // 访问标示数组
 SeqQueue q; init_queue(&q);
 enqueue(&q, b); visited[b] = 1; // 起始顶点入队并标记
 while (!is_empty(&q)) {
 int v;
 dequeue(&q, &v); // 取出：从队首取出一个顶点
 if (v == c) return true; // 访问：检查是否为目标顶点
 for (int w = 0; w < G->vexnum; w++) // 扩展：遍历邻接点
 if (G->arcs[v][w].adj != INFINITY && visited[w] == 0) {
 visited[w] = 1; enqueue(&q, w); // 入队时立即标记
 }
 }
 return false; // 队列空仍未找到，不连通
}
```

# 目录

|  |   |           |
|--|---|-----------|
|  | 2 | 图的存储结构    |
|  | 3 | 图的遍历      |
|  | 4 | 最小生成树     |
|  | 5 | 有向无环图及其应用 |
|  | 6 | 最短路径      |

# 生成树

- 在遍历过程中，所有经过的边记为  $T$ ，没有经过的边(即回边)记为  $B$ 。图  $G'=(V, T)$  是图  $G$  的一棵生成树。
  - 深度优先生成树，广度优先生成树
- 最小生成树(Minimum Cost Spanning Tree):
  - 在一个连通网的所有生成树中，代价之和最小的生成树。
  - 例：在  $n$  个城市间建立一个通信网，修建  $n-1$  条费用总和最小的通信线路，等价于：构建一棵最小代价生成树。

# 最小生成树

- 最小生成树(Minimum Cost Spanning Tree)

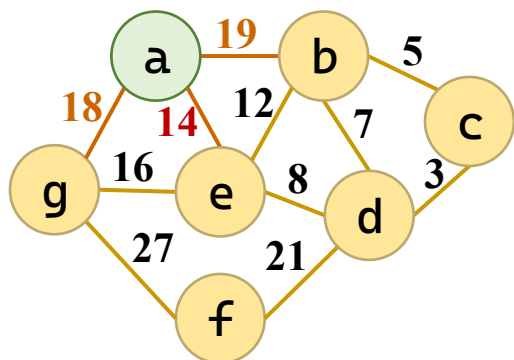
- 性质：设  $G=(V, E)$  是一个连通网， $U$  是顶点集  $V$  的一个非空子集。若  $(u, v)$  是一条具有最小权值的边，其中， $u \in U$ ， $v \in V - U$ ，则存在一棵包含边  $(u, v)$  的最小生成树。

- 方法

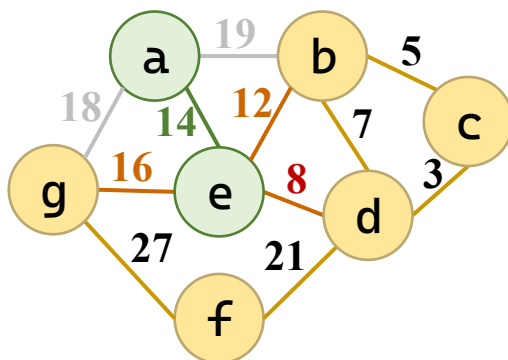
- 算法一：普里姆 (Prim) 算法

- 算法二：克鲁斯卡尔 (Kruskal) 算法

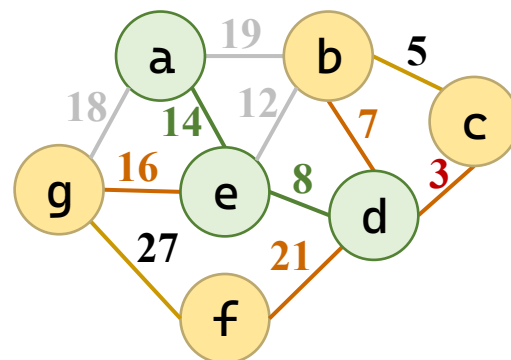
# Prim算法——加点法



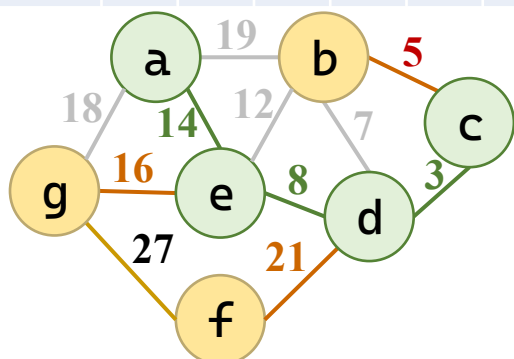
|       | b  | c | d | e  | f | g  |
|-------|----|---|---|----|---|----|
| $V_i$ | a  |   |   | a  |   | a  |
| 代价    | 19 |   |   | 14 |   | 18 |



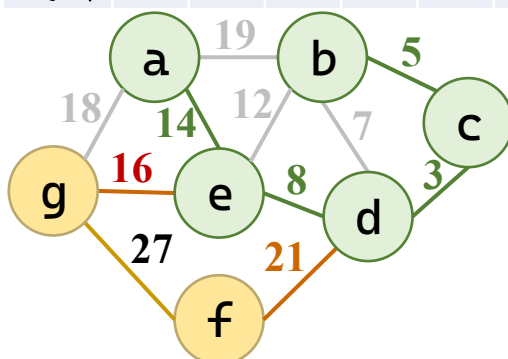
|       | b  | c | d | e  | f | g  |
|-------|----|---|---|----|---|----|
| $V_i$ | e  |   | e | a  |   | e  |
| 代价    | 12 |   | 8 | 14 |   | 16 |



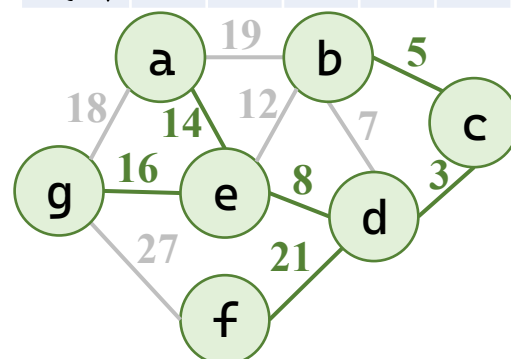
|       | b | c | d | e  | f  | g  |
|-------|---|---|---|----|----|----|
| $V_i$ | d | d | e | a  | d  | e  |
| 代价    | 7 | 3 | 8 | 14 | 21 | 16 |



|       | b | c | d | e  | f  | g  |
|-------|---|---|---|----|----|----|
| $V_i$ | c | d | e | a  | d  | e  |
| 代价    | 5 | 3 | 8 | 14 | 21 | 16 |



|       | b | c | d | e  | f  | g  |
|-------|---|---|---|----|----|----|
| $V_i$ | c | d | e | a  | d  | e  |
| 代价    | 5 | 3 | 8 | 14 | 21 | 16 |



|       | b | c | d | e  | f  | g  |
|-------|---|---|---|----|----|----|
| $V_i$ | c | d | e | a  | d  | e  |
| 代价    | 5 | 3 | 8 | 14 | 21 | 16 |



# Prim算法——加点法

- 算法

- 初始化

```
for (i = 0; i < G->vexnum; i++) {
 lowcost[i] = G->arcs[start][i].adj;
 adjvex[i] = start;
}
visited[start] = 1; lowcost[start] = 0;
```

- 重复  $N-1$  次加顶点

```
for (int k = 1; k < G->vexnum; k++) {
```

- 选出最小的未访问顶点

```
for (int i = 0; i < G->vexnum; i++)
 if (!visited[i] && lowcost[i] < min) {
 min = lowcost[i]; j = i;
 }
```

- 选中边

```
mst_edges[mst_count].u = adjvex[j];
mst_edges[mst_count].v = j;
mst_edges[mst_count].w = min;
mst_count++;
```

- 更新代价和  
 连接点数组

```
visited[j] = 1;
for (int i = 0; i < G->vexnum; i++)
 if (!visited[i] && G->arcs[j][i].adj < lowcost[i]) {
 lowcost[i] = G->arcs[j][i].adj;
 adjvex[i] = j;
 }
```

```

int Prim(AdjMatrix* G, int start) {
 int lowcost[MAX_VERTEX]; // 记录各顶点到当前生成树的最小权值
 int adjvex[MAX_VERTEX]; // 记录该最小权值边对应的树中顶点
 int visited[MAX_VERTEX] = { 0 }; // 标记顶点是否已在生成树中
 // 初始化
 for (int i = 0; i < G->vexnum; i++) {
 lowcost[i] = G->arcs[start][i].adj;
 adjvex[i] = start;
 }
 visited[start] = 1; lowcost[start] = 0; // 权值为0, 避免选中
 int total_weight = 0, mst_count = 0;
 Edge mst_edges[MAX_VERTEX]; // 存储选中边
 // 循环 n-1 次, 每次加入一个顶点
 for (int k = 1; k < G->vexnum; k++) {
 // 1. 从 lowcost 中选出最小的未访问顶点
 int min = INFINITY;
 int j = -1;
 for (int i = 0; i < G->vexnum; i++)
 if (!visited[i] && lowcost[i] < min) {
 min = lowcost[i];
 j = i;
 }
 }
}

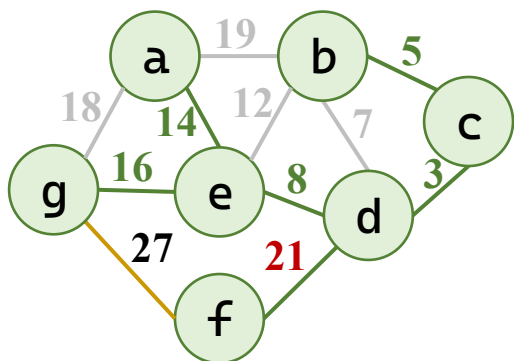
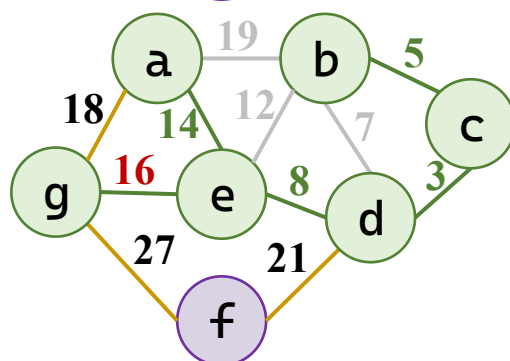
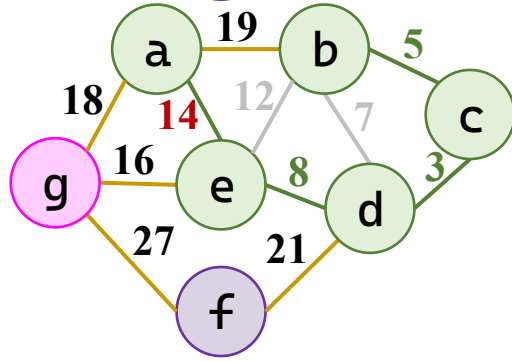
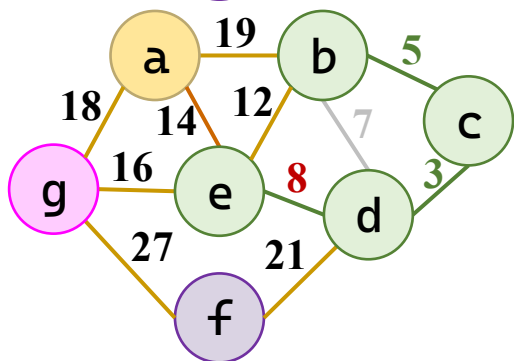
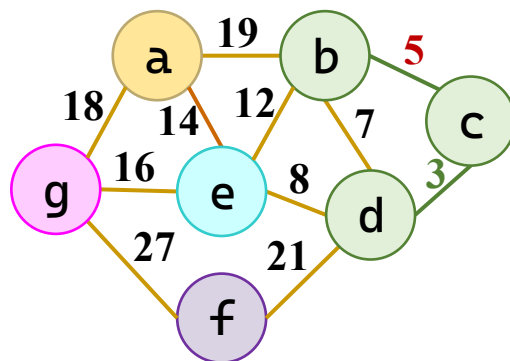
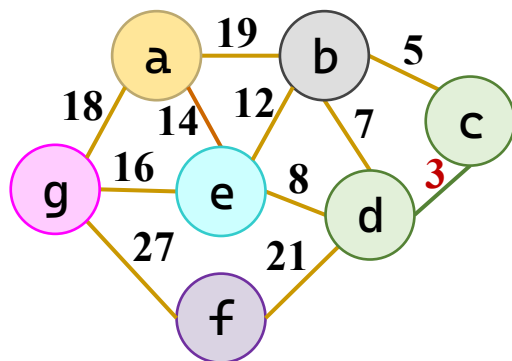
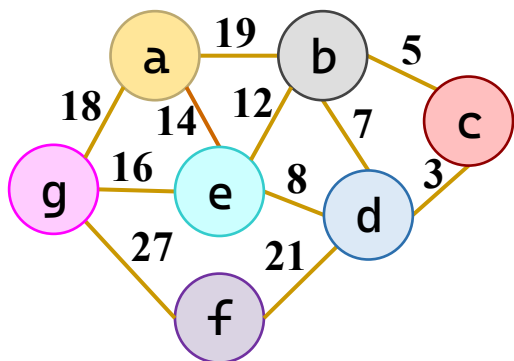
```

```

 if (j == -1) {
 printf("图不连通, 无法生成最小生成树! \n");
 return FAILURE;
 }
 // 2. 选中边
 visited[j] = 1;
 mst_edges[mst_count].u = adjvex[j];
 mst_edges[mst_count].v = j;
 mst_edges[mst_count].w = min;
 mst_count++;
 total_weight += min;
 // 3. 更新 lowcost 和 adjvex
 for (int i = 0; i < G->vexnum; i++)
 if (!visited[i] && G->arcs[j][i].adj < lowcost[i]) {
 lowcost[i] = G->arcs[j][i].adj;
 adjvex[i] = j;
 }
 }
 return SUCCESS;
}

```

# Kruskal算法——加边法



| 始 | 终 | 权  |
|---|---|----|
| c | d | 3  |
| b | c | 5  |
| b | d | 7  |
| d | e | 8  |
| b | e | 12 |
| a | e | 14 |
| e | g | 16 |
| a | g | 18 |
| a | b | 19 |
| d | f | 21 |
| f | g | 27 |

# Kruskal算法——加边法

- 算法

- 提取所有边

```
Edge edges[MAX_VERTEX * MAX_VERTEX];
int edge_num = 0;
for (i = 0; i < G->vexnum; i++)
 for (j = i + 1; j < G->vexnum; j++)
 if (G->arcs[i][j].adj != INFINITY) {
 edges[edge_num].u = i;
 edges[edge_num].v = j;
 edges[edge_num].w = G->arcs[i][j].adj;
 edge_num++;
 }
```

- 按权重升序排列

```
for (i = 0; i < edge_num - 1; i++)
 for (j = 0; j < edge_num - 1 - i; j++)
 if (edges[j].w > edges[j + 1].w) {
 Edge tmp = edges[j];
 edges[j] = edges[j + 1];
 edges[j + 1] = tmp;
 }
```

- 初始化，每个点各自染色

```
int comp[MAX_VERTEX];
for (int i = 0; i < G->vexnum; i++)
 comp[i] = i;
```

# Kruskal算法——加边法

- 算法

- 每次添一条边

- 判断是否同色成环

- 如不成环，选中

- 染色

```
for (int i = 0; i < edge_num; i++) {
 int u = edges[i].u, v = edges[i].v;
 if (comp[u] != comp[v]) {
 mst_edges[mst_count] = edges[i];
 mst_count++;

 int old_id = comp[v];
 int new_id = comp[u];
 for (int k = 0; k < G->vexnum; k++)
 if (comp[k] == old_id)
 comp[k] = new_id;
 total_weight += edges[i].w;
 if (mst_count == G->vexnum - 1) break;
 }
}
```

```

void Kruskal(AdjMatrix* G) {
 // 1. 提取所有边
 Edge edges[MAX_VERTEX * MAX_VERTEX];
 int edge_num = 0;
 for (int i = 0; i < G->vexnum; i++)
 for (int j = i + 1; j < G->vexnum; j++)
 if (G->arcs[i][j].adj != INFINITY) {
 edges[edge_num].u = i;
 edges[edge_num].v = j;
 edges[edge_num].w = G->arcs[i][j].adj;
 edge_num++;
 }
 // 2. 按权值排序 (冒泡)
 for (int i = 0; i < edge_num - 1; i++)
 for (int j = 0; j < edge_num - 1 - i; j++)
 if (edges[j].w > edges[j + 1].w) {
 Edge tmp = edges[j];
 edges[j] = edges[j + 1];
 edges[j + 1] = tmp;
 }
}

```

// 3. 初始化连通分量数组

```
int comp[MAX_VERTEX];
for (int i = 0; i < G->vexnum; i++) comp[i] = i;
Edge mst_edges[MAX_VERTEX]; // 存储选中边
int mst_count = 0, total_weight = 0;
for (int i = 0; i < edge_num; i++) {
 int u = edges[i].u, v = edges[i].v;
 if (comp[u] != comp[v]) {
 mst_edges[mst_count] = edges[i]; // 存储选中边
 mst_count++;
 int old_id = comp[v], new_id = comp[u];
 for (int k = 0; k < G->vexnum; k++) // 合并分量
 if (comp[k] == old_id) comp[k] = new_id;
 total_weight += edges[i].w;
 if (mst_count == G->vexnum - 1) break;
 }
}
if (mst_count != G->vexnum - 1) {
 printf("图不连通，无法生成最小生成树! \n");
 return FAILURE;
}
return SUCCESS;
}
```



# Kruskal算法的并查集改进

- 并查集：改为染色为记录共同的祖先

- 查找根节点

```
int find(int parent[], int x) {
 if (parent[x] != x)
 parent[x] = find(parent, parent[x]);
 return parent[x];
}
```

- 合并两个集合

```
void union_set(int parent[], int rank[], int x, int y) {
 int rx = find(parent, x), ry = find(parent, y);
 if (rx == ry) return;
 if (rank[rx] < rank[ry])
 parent[rx] = ry;
 else if (rank[rx] > rank[ry])
 parent[ry] = rx;
 else {
 parent[ry] = rx; rank[rx]++;
 }
}
```

# Kruskal算法的并查集改进

- 并查集：改为染色为记录共同的祖先

- 并查集初始化替代  
原染色数组

```
int parent[MAX_VERTEX], rank[MAX_VERTEX];
for (int i = 0; i < G->vexnum; i++) {
 parent[i] = i; rank[i] = 0;
}
```

- 使用并查集的查找判断替代判断同色

```
if (find(parent, u) != find(parent, v)) {
```

- 使用并查集的合并替代染色

```
union_set(parent, rank, u, v);
```

```

// 查找根节点 (路径压缩)
int find(int parent[], int x) {
 if (parent[x] != x)
 parent[x] = find(parent, parent[x]);
 return parent[x];
}

// 合并两个集合 (按秩合并)
void union_set(int parent[], int rank[], int x, int y) {
 int rx = find(parent, x);
 int ry = find(parent, y);
 if (rx == ry) return;
 if (rank[rx] < rank[ry])
 parent[rx] = ry;
 else if (rank[rx] > rank[ry])
 parent[ry] = rx;
 else {
 parent[ry] = rx;
 rank[rx]++;
 }
}

```

```

int Kruskal_DisjointSet(AdjMatrix* G) {
 Edge edges[MAX_VERTEX * MAX_VERTEX];
 int edge_num = 0;
 for (int i = 0; i < G->vexnum; i++)
 for (int j = i + 1; j < G->vexnum; j++)
 if (G->arcs[i][j].adj != INFINITY) {
 edges[edge_num].u = i; edges[edge_num].v = j;
 edges[edge_num].w = G->arcs[i][j].adj;
 edge_num++;
 }
 for (int i = 0; i < edge_num - 1; i++) // 冒泡排序
 for (int j = 0; j < edge_num - 1 - i; j++)
 if (edges[j].w > edges[j + 1].w) {
 Edge tmp = edges[j];
 edges[j] = edges[j + 1];
 edges[j + 1] = tmp;
 }
 // 并查集初始化
 int parent[MAX_VERTEX], rank[MAX_VERTEX];
 for (int i = 0; i < G->vexnum; i++) {
 parent[i] = i;
 rank[i] = 0;
 }
}

```

```

Edge mst_edges[MAX_VERTEX];
int mst_count = 0, total_weight = 0;
for (int i = 0; i < edge_num; i++) {
 int u = edges[i].u, v = edges[i].v;
 if (find(parent, u) != find(parent, v)) {
 mst_edges[mst_count] = edges[i];
 mst_count++;
 union_set(parent, rank, u, v);
 total_weight += edges[i].w;
 if (mst_count == G->vexnum - 1) break;
 }
}
if (mst_count != G->vexnum - 1) {
 printf("\n图不连通，无法生成最小生成树! \n");
 return FAILURE;
}
return SUCCESS;
}

```

# 目录

|  |   |           |
|--|---|-----------|
|  | 2 | 图的存储结构    |
|  | 3 | 图的遍历      |
|  | 4 | 最小生成树     |
|  | 5 | 有向无环图及其应用 |
|  | 6 | 最短路径      |

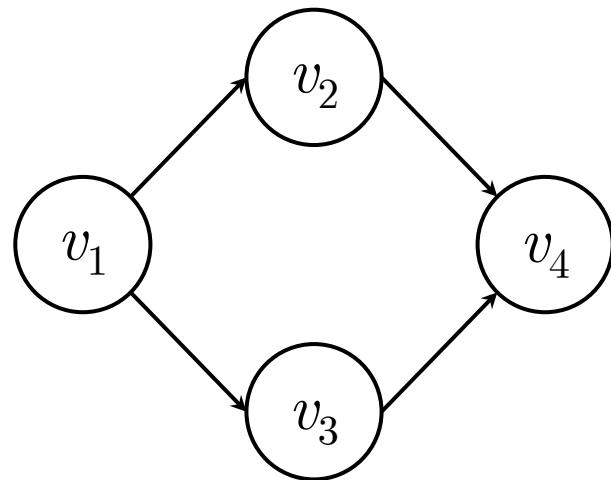
# 有向无环图

- 有向无环图 ( Directed Acyclic Graph )

- 一个没有回路 ( 即无环 ) 的有向图。
- 主要用于研究工程项目的工序问题、工程时间进度问题等。

一个**工程**(project)都可分为若干个称为**活动**(active)的**子工程(或工序)**，各个子工程受到一定的条件约束：某个子工程必须开始于另一个子工程完成之后；整个工程有一个开始点(起点)和一个终点。人们关心：

- ◆ 工程能否顺利完成？（拓扑排序问题）
- ◆ 估算整个工程完成所必须的最短时间是多少？影响工程的关键活动是什么？（关键路径问题）



# 有向无环图

- **AOV网**：图中顶点表示活动，有向边表示活动之间的优先关系，这样的有向图称为顶点表示活动的网 (Activity On Vertex Network，AOV网)。
- **检查有向图中是否存在环的方法**：
  - 方法一：如果有向图 $G$ 的某个顶点 $v$ 出发，进行深度优先搜索遍历时产生顶点 $u$ 到顶点 $v$ 的回边，则图 $G$ 存在包含顶点 $u$ 和顶点 $v$ 的环。
  - 方法二：对有向图 $G$ 进行拓扑排序。如果所有顶点都包含在“拓扑序列”中，则图 $G$ 不存在环。

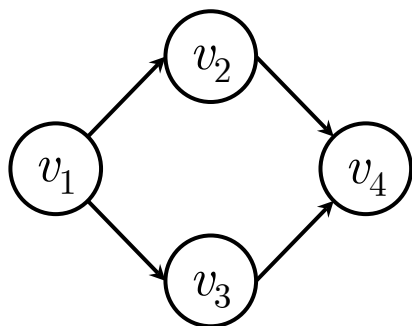


# 拓扑序列

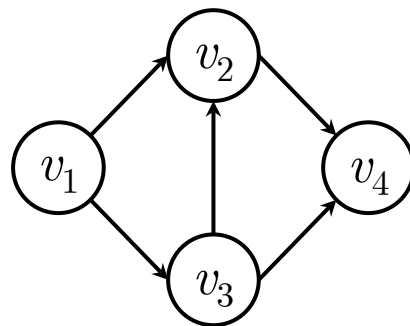
- 拓扑序列

- 按照有向图的弧 (次序关系)，将图中顶点排成一个线性序列 (对于图中没有限定次序关系的顶点，可以人为地添加上任意的次序关系)，由此得到的顶点序列称为拓扑序列。

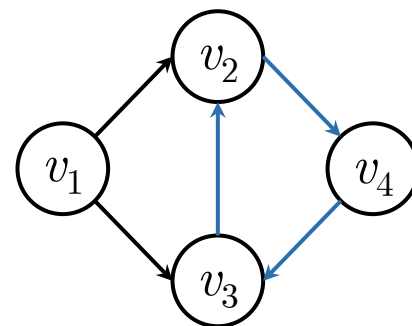
- 例如，在由顶点  $v_1$   $v_2$   $v_3$   $v_4$  组成的有向图中，得到拓扑序列。



$v_1$   $v_2$   $v_3$   $v_4$  和  $v_1$   $v_3$   $v_2$   $v_4$



$v_1$   $v_3$   $v_2$   $v_4$



无 ( 因为有环 )

# 拓扑序列

- 拓扑排序的一般操作方法：

- (1)从有向图中选取一个入度为0的顶点，输出它；
- (2)从有向图中删去该顶点以及所有以它为尾的弧；
- 重复上述两步，直至图空或者找不到入度为0的顶点为止。

- 拓扑排序算法的基本操作步骤：

- 求各顶点的入度，将入度=0的顶点入队；
- 当队列非空时，进行下列操作：
- (1) 输出队首元素 $v$ ；
- (2)  $v$ 的所有邻接点入度减1。如果出现入度0的点，点入队。

# 拓扑序列的算法

- 拓扑序列的算法

- 计算各点入度

- 零入度的点入栈

- 不断处理至栈空

- 出栈

- 加入解

- 删除顶点*i*及其出边，  
更新邻接点入度

```
FindID(G, indegree);
```

```
for (i = 0; i < G->vexnum; i++)
 if (indegree[i] == 0)
 Push(&S, i);
```

```
while (!IsEmpty(&S)) {
```

```
 Pop(&S, &i);
```

```
 res[count++] = i;
```

```
 for (j = 0; j < G->vexnum; j++)
 if (G->arcs[i][j].adj != 0) {
 indegree[j]--;
 if (indegree[j] == 0)
 Push(&S, j);
 }
}
```

```
}
```

# 拓扑序列的算法

- 计算各点入度的算法

- 遍历各点

```
for (i = 0; i < G->vexnum; i++) {
```

- 初始化

```
indegree[i] = 0;
```

- 遍历各点

```
for (j = 0; j < G->vexnum; j++)
```

- 如果存在边，入度自增1

```
if (G->arcs[j][i].adj != 0)
 indegree[i]++;
```

```
}
```

```

void FindID(AdjMatrix* G, int indegree[]) {
 int i, j;
 for (i = 0; i < G->vexnum; i++) {
 indegree[i] = 0;
 for (j = 0; j < G->vexnum; j++)
 if (G->arcs[j][i].adj != 0)
 indegree[i]++;
 }
}

int* Toposort(AdjMatrix* G) {
 int indegree[MAX_VERTEX];
 int i, j, count = 0;
 SeqStack S;
 int* res = (int*)malloc(sizeof(int) * G->vexnum);
 FindID(G, indegree);
 InitStack(&S);
 // 入度为0的顶点入栈
 for (i = 0; i < G->vexnum; i++)
 if (indegree[i] == 0)
 Push(&S, i);
}

```

```

while (!IsEmpty(&S)) {
 Pop(&S, &i);
 res[count++] = i;
 // 删除顶点i及其出边, 更新邻接点入度
 for (j = 0; j < G->vexnum; j++) {
 if (G->arcs[i][j].adj != 0) {
 indegree[j]--;
 if (indegree[j] == 0)
 Push(&S, j);
 }
 }
}
if (count == G->vexnum)
 return res;
else {
 free(res);
 return NULL;
}
}

```

# 关键路径

- AOE网 (Activity on Edge)

- AOE网是一个带权有向无环图。
- 其中，顶点表示事件，用弧表示活动，权表示活动持续的时间。
- 例如，

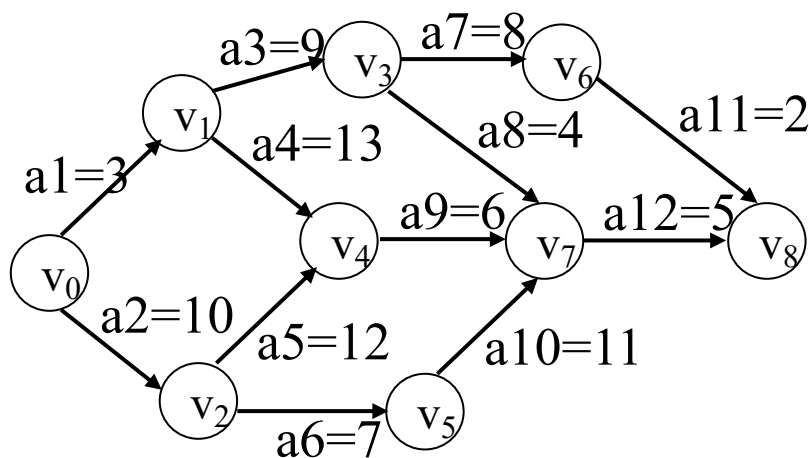


图4-1 一个AOE网

# 关键路径

- 问题描述：假设以AOE网表示1个施工图,权值表示完成该项子工程（或称活动）所需时间，则
  - （1）完成整个工程需要多少时间？
  - （2）哪些活动是影响工程进度的关键？
- 源点：入度=0的顶点(工程的开始点)。
- 汇点：出度=0的顶点(工程的结束点)。



# 关键路径

- 关键路径(工程完成最短时间)：从起点到终点的最长路径长度(路径上各活动持续时间之和)。长度最长的路径称为关键路径
- 关键活动：关键路径上的活动称为关键活动。关键活动是影响整个工程的关键。
- 事件 $v_i$ 的最早发生时间：设 $v_0$ 是起点，从 $v_0$ 到 $v_i$ 的最长路径长度称为事件 $v_i$ 的最早发生时间，即是以 $v_i$ 为尾的所有活动的最早发生时间。

# 关键路径

- 若活动  $a_i$  是弧  $\langle j, k \rangle$ ，持续时间是  $\text{dut}(\langle j, k \rangle)$ ，设：
  - $e(i)$ ：表示活动  $a_i$  的最早开始时间；
  - $l(i)$ ：在不影响进度的前提下，表示活动  $a_i$  的最晚开始时间；  
则  $l(i) - e(i)$  表示活动  $a_i$  的时间余量，
  - 若  $l(i) - e(i) = 0$ ，表示活动  $a_i$  是关键活动。
  - $\text{ve}(i)$ ：表示事件  $v_i$  的最早发生时间，即从起点到顶点  $v_i$  的最长路径长度；
  - $\text{vl}(i)$ ：表示事件  $v_i$  的最晚发生时间。
  - 则有以下关系：
$$\begin{cases} e(i) = \text{ve}(j) \\ l(i) = \text{vl}(k) - \text{dut}(\langle j, k \rangle) \end{cases} \quad (4-1)$$

# 关键路径

$$ve(j) = \begin{cases} 0, & j=0, \text{表示} v_j \text{是起点} \\ \max\{ve(i) + dut(<i, j>) \mid <v_i, v_j> \text{是网中的弧}\} \end{cases} \quad (4-2)$$

- 含义是：源点事件的最早发生时间设为0；除源点外，只有进入顶点 $v_j$ 的所有弧所代表的活动全部结束后，事件 $v_j$ 才能发生。即只有 $v_j$ 的所有前驱事件 $v_i$ 的最早发生时间 $ve(i)$ 计算出来后，才能计算 $ve(j)$ 。
- 方法是：对所有事件进行拓扑排序，然后依次按拓扑顺序计算每个事件的最早发生时间。

# 关键路径

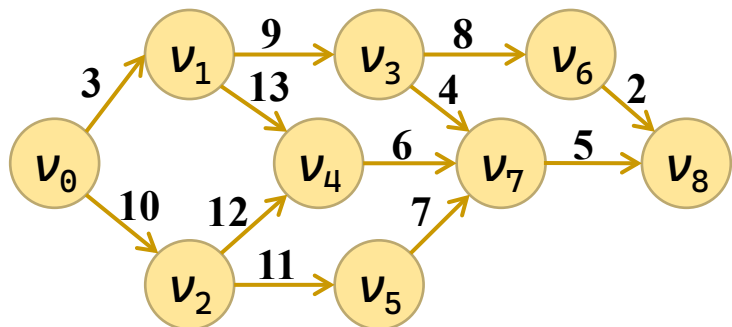
$$vl(j) = \begin{cases} ve(n-1) & j=n-1, \text{ 表示 } v_j \text{ 是终点} \\ \min\{vl(k) - dut(<j, k>) \mid <\underline{v}_j, v_k> \text{ 是网中的弧} \} \end{cases} \quad (4-3)$$

- 含义是：只有 $v_j$ 的所有后继事件 $v_k$ 的最晚发生时间 $vl(k)$ 计算出来后，才能计算 $vl(j)$ 。
- 方法是：按拓扑排序的逆顺序，依次计算每个事件的最晚发生时间。

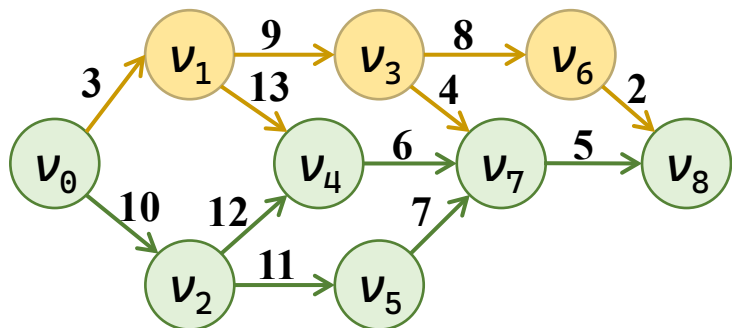
# 关键路径

- 求AOE中关键路径和关键活动算法思想：
  - 1、利用拓扑排序求出AOE网的一个拓扑序列；
  - 2、从拓扑排序的序列的第一个顶点(源点)开始，按拓扑顺序依次计算每个事件的最早发生时间 $ve(i)$ ；
  - 3、从拓扑排序的序列的最后一个顶点(汇点)开始，按逆拓扑顺序依次计算每个事件的最晚发生时间 $vl(i)$ ；
  - 4、根据公式4-1计算每个关键活动。

# 求关键路径



| 拓扑  | 0   | 2   | 5   | 1   | 4   | 3   | 7  | 6  | 8  |
|-----|-----|-----|-----|-----|-----|-----|----|----|----|
| 顶点  | 0   | 1   | 2   | 3   | 4   | 5   | 6  | 7  | 8  |
| ve  | 0   | 3   | 10  | 12  | 22  | 17  | 20 | 28 | 33 |
| vl反 | -33 | -24 | -23 | -10 | -11 | -12 | -2 | -5 | 0  |
| vl  | 0   | 9   | 10  | 23  | 22  | 21  | 31 | 28 | 33 |



| 起点 | 终点 | ei | li | vl | dt | 关键 |
|----|----|----|----|----|----|----|
| 0  | 1  | 0  | 6  | 9  | 3  |    |
| 0  | 2  | 0  | 0  | 10 | 10 | *  |
| 1  | 3  | 3  | 14 | 23 | 9  |    |
| 1  | 4  | 3  | 9  | 22 | 13 |    |
| 2  | 4  | 10 | 10 | 22 | 12 | *  |
| 2  | 5  | 10 | 10 | 21 | 11 | *  |
| 3  | 6  | 12 | 23 | 31 | 8  |    |
| 3  | 7  | 12 | 24 | 28 | 4  |    |
| 4  | 7  | 22 | 22 | 28 | 6  | *  |
| 5  | 7  | 21 | 21 | 28 | 7  | *  |
| 6  | 8  | 20 | 31 | 33 | 2  |    |
| 7  | 8  | 28 | 28 | 33 | 5  | *  |

# 求关键路径

- 关键路径算法

- 拓扑排序

```
topoSeq = Toposort(G);
```

- 计算最早发生时间

- 初始化

```
for (i = 0; i < G->vexnum; i++) ve[i] = 0;
```

- 对所有的下一条边

```
for (i = 0; i < G->vexnum; i++) {
 int u = topoSeq[i];
```

```
 for (p=FirstAdjVtx(G, u); p!=ERROR; p=NextAdjVtx(G, u, p)) {
```

- 如果ve更短则更新

```
 if (ve[p] < ve[u] + G->arcs[u][p].adj)
 ve[p] = ve[u] + G->arcs[u][p].adj;
```

```
 }
```

- 逆序计算最迟发生时间

- 初始化

```
int max_ve = ve[topoSeq[G->vexnum - 1]];
for (i = 0; i < G->vexnum; i++) vl[i] = max_ve;
```

# 求关键路径

## • 关键路径算法

### – 逆序计算最迟发生时间

- 初始化

```
int max_ve = ve[topoSeq[G->vexnum - 1]];
for (i = 0; i < G->vexnum; i++) vl[i] = max_ve;
```

- 逆拓扑序计算

```
for (i = G->vexnum - 1; i >= 0; i--) {
 int u = topoSeq[i];
```

- 对所有的下一条边

```
for (p=FirstAdjVtx(G, u); p!=ERROR; p=NextAdjVtx(G, u, p)) {
```

- 如果vl更短则更新

```
if (vl[u] > vl[p] - G->arcs[u][p].adj)
 vl[u] = vl[p] - G->arcs[u][p].adj;
```

### – 输出结果

```
}
```

- 如果 $ve_i = vl_p - a_{ip}$ ，则为关键路径

```
ei = ve[i];
li = vl[p] - G->arcs[i][p].adj;
tag = (ei == li) ? '*' : ' ';
```



```

int FirstAdjVtx(AdjMatrix* G, int v) { // 获取第一个邻接点
 int i;
 for (i = 0; i < G->vexnum; i++)
 if (v != i && G->arcs[v][i].adj != INFINITY)
 return i;
 return ERROR;
}

int NextAdjVtx(AdjMatrix* G, int v, int w) { // 获取下一个邻接点
 int i;
 for (i = w + 1; i < G->vexnum; i++)
 if (v != i && G->arcs[v][i].adj != INFINITY)
 return i;
 return ERROR;
}

int CriticalPath(AdjMatrix* G) {
 int i, p, ei, li;
 char tag;
 int* ve, * vl, * topoSeq;
 ve = (int*)malloc(sizeof(int) * G->vexnum);
 vl = (int*)malloc(sizeof(int) * G->vexnum);
 if (!ve || !vl) return ERROR;

```

```

/* 拓扑排序 */
topoSeq = Toposort(G);
if (topoSeq == NULL) {
 printf("图存在环, 无法计算关键路径! \n");
 free(ve); free(vl);
 return ERROR;
}
/* 1. 按拓扑序计算 ve (最早发生时间) */
for (i = 0; i < G->vexnum; i++) ve[i] = 0;
for (i = 0; i < G->vexnum; i++) {
 int u = topoSeq[i];
 for (p=FirstAdjVtx(G, u); p!=ERROR; p=NextAdjVtx(G, u, p))
 if (ve[p] < ve[u] + G->arcs[u][p].adj)
 ve[p] = ve[u] + G->arcs[u][p].adj;
}
/* 2. 初始化 vl 为汇点的 ve (取所有 ve 的最大值更通用) */
int max_ve = ve[topoSeq[G->vexnum - 1]]; // 拓扑序尾顶点即汇点
for (i = 0; i < G->vexnum; i++) vl[i] = max_ve;
/* 3. 逆拓扑序计算 vl (最迟发生时间) */
for (i = G->vexnum - 1; i >= 0; i--) {
 int u = topoSeq[i];

```

```

 for (p=FirstAdjVtx(G, u); p!=ERROR; p=NextAdjVtx(G, u, p))
 if (vl[u] > vl[p] - G->arcs[u][p].adj)
 vl[u] = vl[p] - G->arcs[u][p].adj;
 }
 free(topoSeq);
 /* 输出关键活动 */
 printf("\n v1 v2 ei li vl dt 关键\n");
 printf("=====\n");
 for (i = 0; i < G->vexnum; i++) {
 for (p=FirstAdjVtx(G,i); p!=ERROR; p=NextAdjVtx(G,i,p)) {
 ei = ve[i];
 li = vl[p] - G->arcs[i][p].adj;
 tag = (ei == li) ? '*' : ' ';
 printf(" %c %c %2d %2d %2d %2d %c\n",
 G->vertex[i], G->vertex[p], ei, li, vl[p],
 G->arcs[i][p].adj, tag);
 }
 }
 free(ve); free(vl);
 return OK;
}

```

# 目录

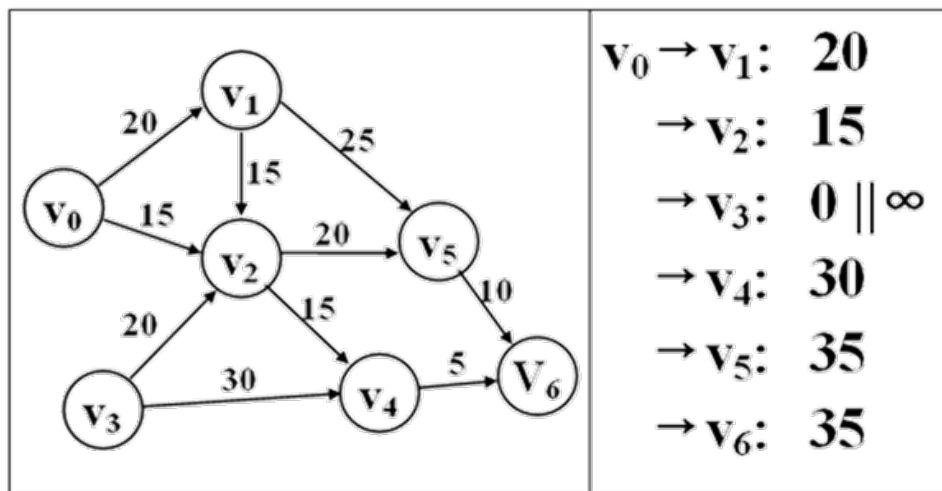
|  |   |           |
|--|---|-----------|
|  | 2 | 图的存储结构    |
|  | 3 | 图的遍历      |
|  | 4 | 最小生成树     |
|  | 5 | 有向无环图及其应用 |
|  | 6 | 最短路径      |

# 求带权图中的最短路径问题

- 用顶点表示城市，边表示城市之间的通路，权值表示城市之间的距离。
  - 如何判断两个城市之间是否有通路？
  - 如果有通路，走哪条路最短？
- 求最短路径时，路径长度为路径上各边的权值之和。
  - 如果城市之间有通路，其权值 $>0$ ，否则，其权值 $=0$  (或 $\infty$ )。
- 单源最短路径问题：
  - 设顶点 $v$ 是指定源点, 要求在有向网 $G$ 中找到从源点 $v$ 到其它各顶点的最短路径。

# 单源最短路径问题

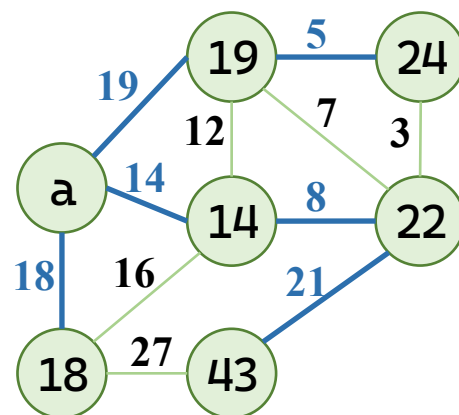
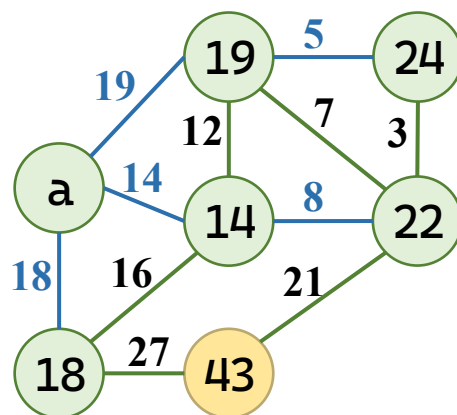
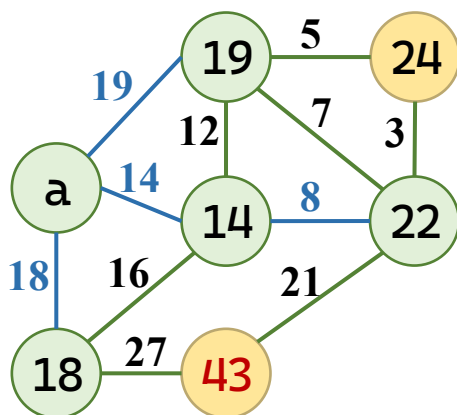
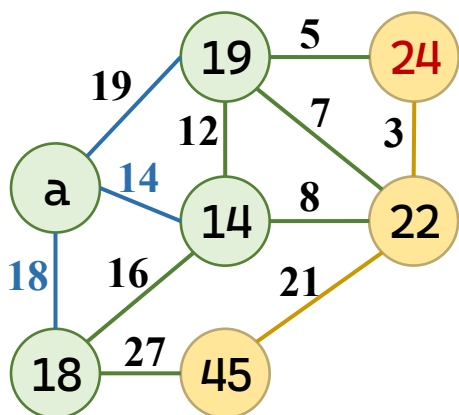
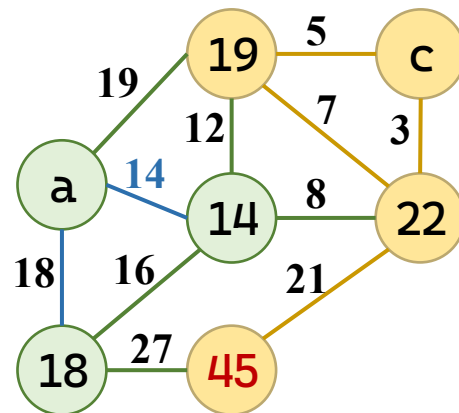
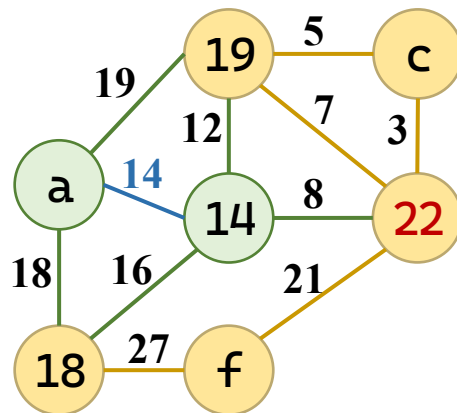
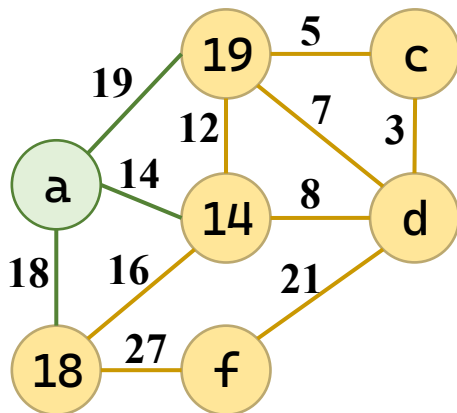
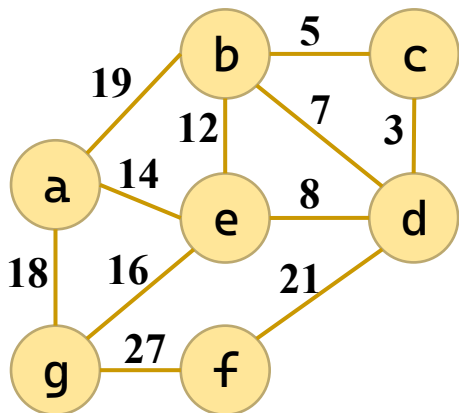
- 例：求下图中 $v_0$ 到其它各顶点的最短路径。



# 狄克斯特拉（ Dijkstra ）算法

- 假设所有道路的长度都非负，如果从已确定集合S找到了s到它们的最短路径，每次从未确定集合U中找到的最小顶点u，加入S，新的集合S仍是最短路径集合。
- 反证：假设存在w,u属于U，s属于S，s...w...u比当前s...u更近，路径长度小于  $\text{dist}[u]$ 。s...w除了w以外都在S中。则，s...w的长度不小于已知的最小长度  $\text{dist}[w]$ ，s...w...u的路径长度必然不小于  $\text{dist}[w]$ ，也应该不小于当前U中最小长度  $\text{dist}[u]$ 。这一来，与u是最小长度的顶点这一规则相矛盾。

# 狄克斯特拉 ( Dijkstra ) 算法





# 狄克斯特拉 ( Dijkstra ) 算法

- 算法

- 初始化：以起始点更新各数组

```
for (i = 0; i < G->vexnum; i++) {
 dist[i] = G->arcs[v0][i]; //
 visited[i] = 0; // 所有顶点均未确定最短路径
 if (G->arcs[v0][i] != INFINITY && i != v0)
 prev[i] = v0; // 有直接边，前驱为源点
 else
 prev[i] = -1; // 无直接边，前驱未知
}
// 源点到自身的距离为 0，并标记为已访问
dist[v0] = 0; visited[v0] = 1; prev[v0] = -1;
```

# 狄克斯特拉 ( Dijkstra ) 算法

## • 算法

– 循环确定最短路径

– 在U中，选择距离最小的顶点

– 标记已找到

– 更新其余未确定顶点的最短距离

```
for (i = 1; i < G->vexnum; i++) {
 int minDist = INFINITY; u = -1;
 for (j = 0; j < G->vexnum; j++)
 if (!visited[j] && dist[j] < minDist) {
 minDist = dist[j]; u = j;
 }

 visited[u] = 1;

 for (j = 0; j < G->vexnum; j++) {
 if (!visited[j] && G->arcs[u][j] != INFINITY) {
 int newDist = dist[u] + G->arcs[u][j];
 if (newDist < dist[j]) {
 dist[j] = newDist; prev[j] = u; // 更新前驱
 }
 }
 }
}
```

```

void Dijkstra(AdjMatrix* G, int v0, int dist[], int prev[]) {
 int i, j, u;
 int visited[MAX_VERTEX]; // 1 表示顶点 i 已确定最短路径
 // ----- 1. 初始化 -----
 for (i = 0; i < G->vexnum; i++) {
 dist[i] = G->arcs[v0][i]; // 源点到各顶点的直接距离
 visited[i] = 0; // 所有顶点均未确定最短路径
 if (G->arcs[v0][i] != INFINITY && i != v0)
 prev[i] = v0; // 有直接边, 前驱为源点
 else
 prev[i] = -1; // 无直接边, 前驱未知
 }
 // 源点到自身的距离为 0, 并标记为已访问
 dist[v0] = 0; visited[v0] = 1; prev[v0] = -1;
 // ----- 2. 主循环: 每次确定一个顶点的最短路径 -----
 for (i = 1; i < G->vexnum; i++) {
 // 2.1 在未确定最短路径的顶点中, 选择距离最小的顶点 u
 int minDist = INFINITY;
 u = -1;
 }
}

```

```

for (j = 0; j < G->vexnum; j++)
 if (!visited[j] && dist[j] < minDist) {
 minDist = dist[j];
 u = j;
 }
// 如果 u == -1, 说明剩余顶点均不可达, 提前结束
if (u == -1) break;
// 标记 u 的最短路径已确定
visited[u] = 1;
// 2.2 以 u 为中间点, 更新其余未确定顶点的最短距离
for (j = 0; j < G->vexnum; j++) {
 if (!visited[j] && G->arcs[u][j] != INFINITY) {
 int newDist = dist[u] + G->arcs[u][j];
 if (newDist < dist[j]) {
 dist[j] = newDist;
 prev[j] = u; // 更新前驱
 }
 }
}
}
}
}
}

```

# 每一对顶点最短路径问题

- 问题描述

- 求图中任意一对顶点之间的最短路径。

# 弗洛伊德 ( Floyd ) 算法

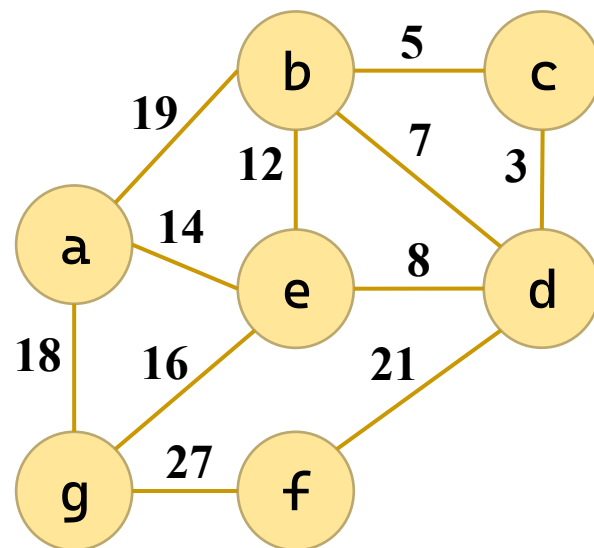
•

| dist | a        | b        | c        | d        | e        | f        | g        |
|------|----------|----------|----------|----------|----------|----------|----------|
| a    | 0        | 19       | $\infty$ | $\infty$ | 14       | $\infty$ | 18       |
| b    | 19       | 0        | 5        | 7        | 12       | $\infty$ | $\infty$ |
| c    | $\infty$ | 5        | 0        | 3        | $\infty$ | $\infty$ | $\infty$ |
| d    | $\infty$ | 7        | 3        | 0        | 8        | 21       | $\infty$ |
| e    | 14       | 12       | $\infty$ | 8        | 0        | $\infty$ | 16       |
| f    | $\infty$ | $\infty$ | $\infty$ | 21       | $\infty$ | 0        | 27       |
| g    | 18       | $\infty$ | $\infty$ | $\infty$ | 16       | 27       | 0        |

| path | a | b | c | d | e | f | g |
|------|---|---|---|---|---|---|---|
| a    | a | a | / | / | a | / | a |
| b    | b | b | b | b | b | / | / |
| c    | / | c | c | c | / | / | / |
| d    | / | d | d | d | d | d | / |
| e    | e | e | / | e | e | / | e |
| f    | / | / | / | f | / | f | f |
| g    | g | / | / | / | g | g | g |

更新：

1. a: bag比bg短；
2. b: abc比ac短；  
abd比ad短；  
cbe比ce短；  
cbg比cg短；  
dbg比dg短；
3. c: 无；
4. d: adf比af短；  
bdf比bf短；  
cde比ce短；  
cdf比cf短；  
edf比ef短；
5. e: aed比ad短；  
aef比af短；  
beg比bg短；  
ceg比cg短；  
deg比dg短；



# 弗洛伊德 ( Floyd ) 算法

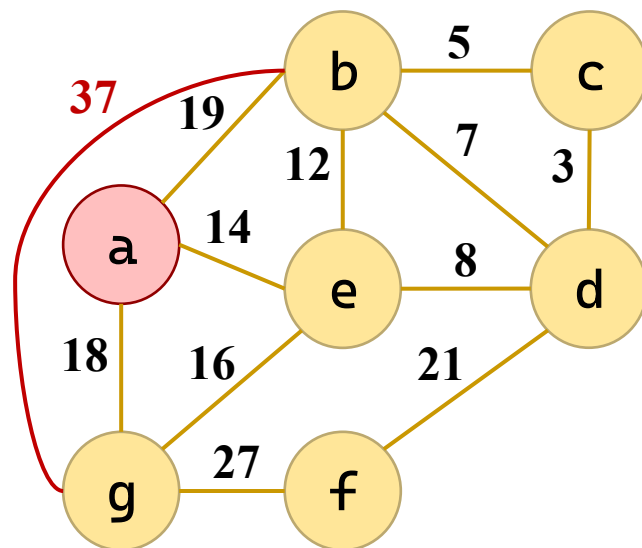
•

| dist | a        | b        | c        | d        | e        | f        | g        |
|------|----------|----------|----------|----------|----------|----------|----------|
| a    | 0        | 19       | $\infty$ | $\infty$ | 14       | $\infty$ | 18       |
| b    | 19       | 0        | 5        | 7        | 12       | $\infty$ | 37       |
| c    | $\infty$ | 5        | 0        | 3        | $\infty$ | $\infty$ | $\infty$ |
| d    | $\infty$ | 7        | 3        | 0        | 8        | 21       | $\infty$ |
| e    | 14       | 12       | $\infty$ | 8        | 0        | $\infty$ | 16       |
| f    | $\infty$ | $\infty$ | $\infty$ | 21       | $\infty$ | 0        | 27       |
| g    | 18       | $\infty$ | $\infty$ | $\infty$ | 16       | 27       | 0        |

| path | a | b | c | d | e | f | g |
|------|---|---|---|---|---|---|---|
| a    | a | a | / | / | a | / | a |
| b    | b | b | b | b | b | / | a |
| c    | / | c | c | c | / | / | / |
| d    | / | d | d | d | d | d | / |
| e    | e | e | / | e | e | / | e |
| f    | / | / | / | f | / | f | f |
| g    | g | / | / | / | g | g | g |

更新：

1. **a**: **bag**比**bg**短；
2. **b**: **abc**比**ac**短；  
abd比ad短；  
cbe比ce短；  
cbg比cg短；  
dbg比dg短；
3. **c**: 无；
4. **d**: **adf**比**af**短；  
bdf比bf短；  
cde比ce短；  
cdf比cf短；  
edf比ef短；
5. **e**: **aed**比**ad**短；  
aef比af短；  
beg比bg短；  
ceg比cg短；  
deg比dg短；



# 弗洛伊德 ( Floyd ) 算法

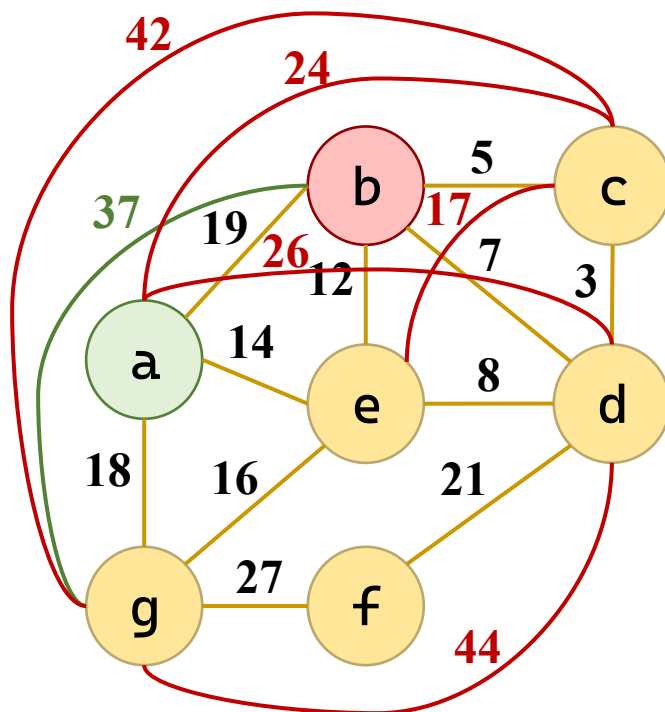
•

| dist | a  | b  | c  | d  | e  | f  | g  |
|------|----|----|----|----|----|----|----|
| a    | 0  | 19 | 24 | 26 | 14 | ∞  | 18 |
| b    | 19 | 0  | 5  | 7  | 12 | ∞  | 37 |
| c    | ∞  | 5  | 0  | 3  | 17 | ∞  | 42 |
| d    | ∞  | 7  | 3  | 0  | 8  | 21 | 44 |
| e    | 14 | 12 | ∞  | 8  | 0  | ∞  | 16 |
| f    | ∞  | ∞  | ∞  | 21 | ∞  | 0  | 27 |
| g    | 18 | ∞  | ∞  | ∞  | 16 | 27 | 0  |

| path | a | b | c | d | e | f | g |
|------|---|---|---|---|---|---|---|
| a    | a | a | b | b | a | / | a |
| b    | b | b | b | b | b | / | a |
| c    | / | c | c | c | b | / | a |
| d    | / | d | d | d | d | d | a |
| e    | e | e | / | e | e | / | e |
| f    | / | / | / | f | / | f | f |
| g    | g | / | / | / | g | g | g |

更新：

1. a: bag比bg短；
2. b: abc比ac短；  
abd比ad短；  
cbe比ce短；  
cbg比cg短；  
dbg比dg短；
3. c: 无；
4. d: adf比af短；  
bdf比bf短；  
cde比ce短；  
cdf比cf短；  
edf比ef短；
5. e: aed比ad短；  
aef比af短；  
beg比bg短；  
ceg比cg短；  
deg比dg短；





# 弗洛伊德 ( Floyd ) 算法

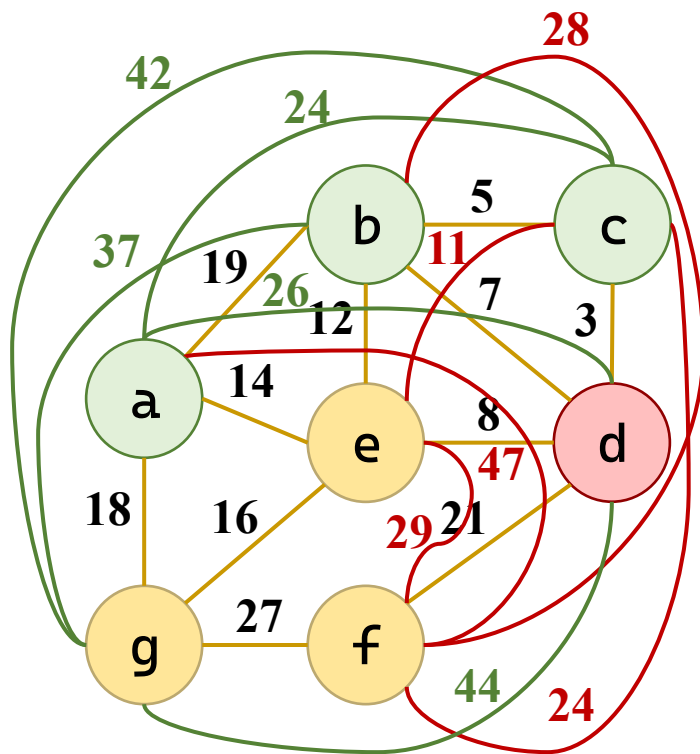
•

| dist | a        | b        | c        | d        | e        | f  | g  |
|------|----------|----------|----------|----------|----------|----|----|
| a    | 0        | 19       | 24       | 26       | 14       | 47 | 18 |
| b    | 19       | 0        | 5        | 7        | 12       | 28 | 37 |
| c    | $\infty$ | 5        | 0        | 3        | 11       | 24 | 42 |
| d    | $\infty$ | 7        | 3        | 0        | 8        | 21 | 44 |
| e    | 14       | 12       | $\infty$ | 8        | 0        | 29 | 16 |
| f    | $\infty$ | $\infty$ | $\infty$ | 21       | $\infty$ | 0  | 27 |
| g    | 18       | $\infty$ | $\infty$ | $\infty$ | 16       | 27 | 0  |

| path | a | b | c | d | e | f | g |
|------|---|---|---|---|---|---|---|
| a    | a | a | b | b | a | d | a |
| b    | b | b | b | b | b | d | a |
| c    | / | c | c | c | d | d | a |
| d    | / | d | d | d | d | d | a |
| e    | e | e | / | e | e | d | e |
| f    | / | / | / | f | / | f | f |
| g    | g | / | / | / | g | g | g |

更新：

1. a: bag比bg短；
2. b: abc比ac短；  
abd比ad短；  
cbe比ce短；  
cbg比cg短；  
dbg比dg短；
3. c: 无；
4. d: adf比af短；  
bdf比bf短；  
cde比ce短；  
cdf比cf短；  
edf比ef短；
5. e: aed比ad短；  
aef比af短；  
beg比bg短；  
ceg比cg短；  
deg比dg短；



# 弗洛伊德 ( Floyd ) 算法

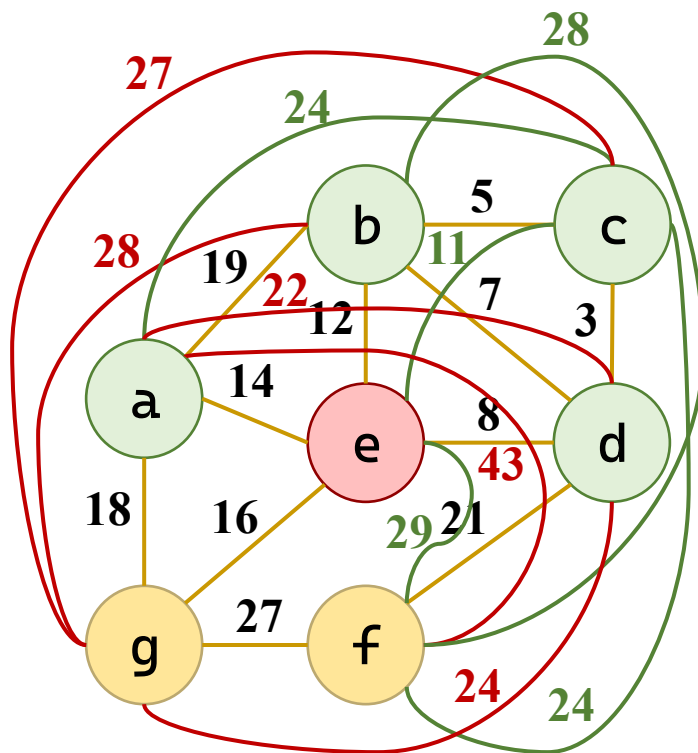
•

| dist | a        | b        | c        | d        | e        | f  | g  |
|------|----------|----------|----------|----------|----------|----|----|
| a    | 0        | 19       | 24       | 22       | 14       | 43 | 18 |
| b    | 19       | 0        | 5        | 7        | 12       | 28 | 28 |
| c    | $\infty$ | 5        | 0        | 3        | 11       | 24 | 27 |
| d    | $\infty$ | 7        | 3        | 0        | 8        | 21 | 24 |
| e    | 14       | 12       | $\infty$ | 8        | 0        | 29 | 16 |
| f    | $\infty$ | $\infty$ | $\infty$ | 21       | $\infty$ | 0  | 27 |
| g    | 18       | $\infty$ | $\infty$ | $\infty$ | 16       | 27 | 0  |

| path | a | b | c | d | e | f | g |
|------|---|---|---|---|---|---|---|
| a    | a | a | b | e | a | d | a |
| b    | b | b | b | b | b | d | e |
| c    | / | c | c | c | d | d | e |
| d    | / | d | d | d | d | d | e |
| e    | e | e | / | e | e | d | e |
| f    | / | / | / | f | / | f | f |
| g    | g | / | / | / | g | g | g |

更新：

1. a: bag比bg短；
2. b: abc比ac短；  
abd比ad短；  
cbe比ce短；  
cbg比cg短；  
dbg比dg短；
3. c: 无；
4. d: adf比af短；  
bdf比bf短；  
cde比ce短；  
cdf比cf短；  
edf比ef短；
5. e: aed比ad短；  
aef比af短；  
beg比bg短；  
ceg比cg短；  
deg比dg短；



# 弗洛伊德 ( Floyd ) 算法

| 起始  | 距离 | 路径   | 起始  | 距离 | 路径   | 起始  | 距离 | 路径   |
|-----|----|------|-----|----|------|-----|----|------|
| a→b | 19 | ab   | c→d | 3  | cd   | e→f | 29 | edf  |
| a→c | 24 | abc  | c→e | 11 | cde  | e→g | 16 | eg   |
| a→d | 22 | aed  | c→f | 24 | cdf  | f→a | 43 | fdea |
| a→e | 14 | ae   | c→g | 27 | cdeg | f→b | 28 | fdb  |
| a→f | 43 | aedf | d→a | 22 | dea  | f→c | 24 | fdc  |
| a→g | 18 | ag   | d→b | 7  | db   | f→d | 21 | fd   |
| b→a | 19 | ba   | d→c | 3  | dc   | f→e | 29 | fde  |
| b→c | 5  | bc   | d→e | 8  | de   | f→g | 27 | fg   |
| b→d | 7  | bd   | d→f | 21 | df   | g→a | 18 | ga   |
| b→e | 12 | be   | d→g | 24 | deg  | g→b | 28 | geb  |
| b→f | 28 | bdf  | e→a | 14 | ea   | g→c | 27 | gedc |
| b→g | 28 | beg  | e→b | 12 | eb   | g→d | 24 | ged  |
| c→a | 24 | cba  | e→c | 11 | edc  | g→e | 16 | ge   |
| c→b | 5  | cb   | e→d | 8  | ed   | g→f | 27 | gf   |

| path | a | b | c | d | e | f | g |
|------|---|---|---|---|---|---|---|
| a    | a | a | b | e | a | d | a |
| b    | b | b | b | b | b | d | e |
| c    | / | c | c | c | d | d | e |
| d    | / | d | d | d | d | d | e |
| e    | e | e | / | e | e | d | e |
| f    | / | / | / | f | / | f | f |
| g    | g | / | / | / | g | g | g |

# 弗洛伊德 ( Floyd ) 算法

## • 算法

### — 初始化

- 记录路径 $i \rightarrow j$ 的前驱
- 能到达的记 $i$   
无法到达记-1
- 自己的路径记 $i$

```
for (i = 0; i < G->vexnum; i++) {
 for (j = 0; j < G->vexnum; j++) {
 d[i][j] = G->arcs[i][j].adj;
 if (i!=j && G->arcs[i][j].adj<INFINITY)
 p[i][j] = i;
 else
 p[i][j] = -1;
 }
 p[i][i] = i;
}
```

### — 核心三重循环

- 遍历中间点、  
起点、终点
- 找更短路径更新  
距离和前驱

```
for (k = 0; k < G->vexnum; k++)
 for (i = 0; i < G->vexnum; i++)
 for (j = 0; j < G->vexnum; j++)
 if (d[i][k] + d[k][j] < d[i][j]) {
 d[i][j] = d[i][k] + dist[k][j];
 p[i][j] = p[k][j];
 }
```

```

void Floyd(AdjMatrix* G) {
 int dist[MAX_VERTEX][MAX_VERTEX]; // 最短距离矩阵
 int path[MAX_VERTEX][MAX_VERTEX]; // 前驱矩阵: path[i][j]
 表示 i→j 路径上 j 的直接前驱
 int i, j, k;

 // ===== 1. 初始化 =====
 for (i = 0; i < G->vexnum; i++) {
 for (j = 0; j < G->vexnum; j++) {
 dist[i][j] = G->arcs[i][j].adj;
 if (i != j && G->arcs[i][j].adj < INFINITY) {
 path[i][j] = i; // i 直接到 j, 前驱为 i
 } else {
 path[i][j] = -1; // 不可达
 }
 }
 path[i][i] = i; // 自己到自己, 前驱为自己
 }

 // ===== 2. Floyd 核心三重循环 =====

```

```

for (k = 0; k < G->vexnum; k++)
 for (i = 0; i < G->vexnum; i++)
 for (j = 0; j < G->vexnum; j++)
 // 如果经过 k 点路径更短, 则更新
 if (dist[i][k] + dist[k][j] < dist[i][j]) {
 dist[i][j] = dist[i][k] + dist[k][j];
 // 更新前驱: i→j 的路径中, j 的前驱等于 k→j 路径中 j 的前驱
 path[i][j] = path[k][j];
 }
// ===== 3. 输出结果 =====
printf("\n===== Floyd 算法结果 =====\n");
printf("\n各对顶点间的最短路径: \n");
for (i = 0; i < G->vexnum; i++)
 for (j = 0; j < G->vexnum; j++)
 if (i != j && dist[i][j] < INFINITY) {
 printf(" %c → %c : 距离 = %3d, 路径 = ",
 G->vertex[i], G->vertex[j], dist[i][j]);
 printPath(i, j, path, G);
 printf("\n");
 }
printf("\n");
}

```

# 本节小结

- 单源最短路径
  - 从指定顶点  $v$ ，要求在图  $G$  中找到从顶点  $v$  到其它各顶点的最短路径。
  - Dijkstra 算法时间复杂度为  $O(n^2)$ 。
- 求图  $G$  中任意一对顶点之间的最短路径。
  - Floyd 算法时间复杂度为  $O(n^3)$ 。

7

谢谢观看

理论课程



廈門大學  
XIAMEN UNIVERSITY



信息学院 黄 焯  
(特色化示范性软件学院) 博士, 副教授  
School of Informatics Wei Huang



8

# 查找

理论课程



廈門大學  
XIAMEN UNIVERSITY



信息学院 黃 煒  
(特色化示范性软件学院) 博士, 副教授  
School of Informatics Wei Huang

# 内容要点

- 静态查找
  - 顺序查找
  - 折半查找
  - 分块查找
- 动态查找
  - 平衡二叉树
  - B树和B<sup>+</sup>树
- 哈希表

# 目录

|  |   |         |
|--|---|---------|
|  | 1 | 基本概念和分类 |
|  | 2 | 静态查找    |
|  | 3 | 动态查找    |
|  | 4 | 哈希表     |
|  |   |         |

# 基本概念和分类

- 查找：在数据表中查找满足给定值的第一个记录或全部记录。
- 查找成功：在数据表中查找到满足条件的记录——返回查找结果所在记录的指针。
- 查找不成功：在数据表中没有查找到满足条件的记录——返回空指针。
- 静态查找：没有改变数据表的查找。
- 动态查找：可能需要在数据表中进行插入操作或删除操作的查找。

# 基本概念和分类

- 关键字：记录中某个成员(数据项)的值，用以识别一个记录。
  - 若关键字可以唯一识别一个记录，则称：主关键字；
  - 若关键字可能识别若干个记录，则称：次关键字。
- 平均查找长度(Average Search Length)

$$ASL = \sum_{i=1}^n P_i C_i$$

- 其中， $n$ 为数据表长度， $P_i$ 为查找数据表中第 $i$ 个记录的概率， $\sum_{i=1}^n P_i = 1$ ， $C_i$ 为找到第 $i$ 个记录时已经进行比较的次数。

# 目录

|  |   |         |
|--|---|---------|
|  | 1 | 基本概念和分类 |
|  | 2 | 静态查找    |
|  | 3 | 动态查找    |
|  | 4 | 哈希表     |
|  |   |         |

# 顺序查找

- 顺序查找法：依据顺序查找的方法实现对数据表的查找算法。
- 顺序查找过程：从数据表中的最后一个记录(或第一个记录)开始，逐个比较关键字和给定值，并判断查找是否成功。
- 例：在数据表 $L[1..n]$ 中，顺序查找关键字=key的记录。若找到，返回该记录在数据表中的位置，否则返回0。

# 顺序查找

- 普通算法

- 遍历整个数组
- 如果找到则返下标
- 找不到则告错

```
for (int i = 0; i < n; i++)

 if (arr[i] == key) {
 return i;
 }

return -1;
```

- 哨兵算法

- 设置哨兵
- 遍历整个数组
- 如果找到则返下标
  - 返回越界则为告错

```
arr[n] = key;

int i = 0;
while (arr[i] != key)
 i++;

return i;
```



```
int sequentialSearch(int arr[], int n, int key) {
 for (int i = 0; i < n; i++) {
 if (arr[i] == key) {
 return i; // 找到了, 立即返回位置
 }
 }
 return -1; // 遍历完都没找到
}

int sequentialSearchWithSentinel(int arr[], int n, int key) {
 arr[n] = key; // 把目标值放在最后一个位置 (哨兵)
 int i = 0;
 while (arr[i] != key) { // 少了 i < n 的判断, 循环更快
 i++;
 }
 return i; // 如果 i == n, 说明原数组里根本没有
}
```

# 图的定义和术语

- 顺序查找的时间复杂度  $O(n)$
- 对数据表而言， $C_i = i$ ，即

$$ASL = P_n + 2P_{n-1} + \dots + (n-1)P_2 + nP_1$$

- 在等概率查找的情况下， $P_i = \frac{1}{n}$
- 即顺序表查找的平均查找长度为：

$$ASL = \frac{1}{n} \sum_{i=1}^n i = \frac{n+1}{2}$$

# 折半查找算法

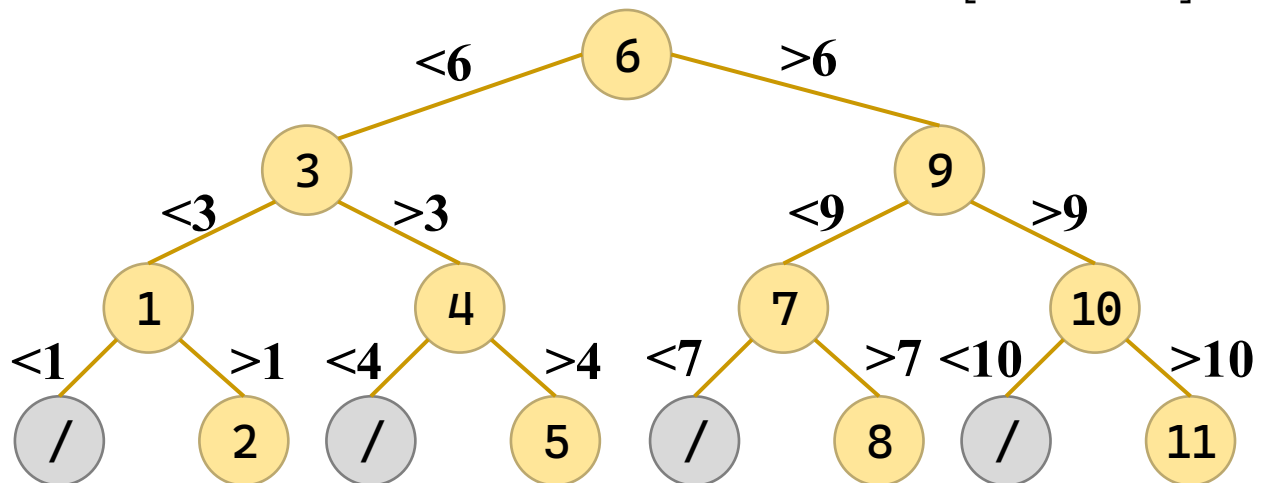
- 有序表：在数据表中，记录按关键字值从小到大递增（或从大到小递减）有序。
- 折半查找法：用折半查找的方法实现对有序表的查找算法，也称为二分查找法。
- 时间复杂度为  $O(\log n)$ ，最多查找次数： $\lfloor \log_2 n \rfloor + 1$

查找长度：1

查找长度：2

查找长度：3

查找长度：4



# 折半查找算法

- 递归算法

- 如果区间空，退出

```
if (low > high)
 return -1;
```

- 如果中数找到，  
则返回

```
int mid = low + (high - low) / 2;
if (arr[mid] == key) {
 return mid;
```

- 如果中数太小，  
找左半边

```
} else if (arr[mid] < key) {
 return binSearch(arr, mid+1, high, key);
```

- 如果中数太大，  
找右半边

```
} else {
 return binSearch(arr, low, mid-1, key);
}
```

# 折半查找算法

- 迭代算法

- 如果区间空，退出

- 如果中数找到，  
则返回

- 如果中数太小，  
找左半边

- 如果中数太大，  
找右半边

```
while (low <= high) {
```

```
 int mid = low + (high - low) / 2;
 if (arr[mid] == key) {
 return mid;
```

```
 } else if (arr[mid] < key) {
 low = mid + 1;
```

```
 } else {
 high = mid - 1;
 }
```

```
int binarySearchRecHelper(int arr[], int low, int high, int key) {
 if (low > high) {
 return -1; // 递归出口: 区间无效, 未找到
 }
 int mid = low + (high - low) / 2;

 if (arr[mid] == key) {
 return mid;
 } else if (arr[mid] < key) {
 return binarySearchRecHelper(arr, mid+1, high, key); // 查右半
 } else {
 return binarySearchRecHelper(arr, low, mid-1, key); // 查左半
 }
}

int binarySearchRecursive(int arr[], int n, int key) {
 return binarySearchRecursiveHelper(arr, 0, n - 1, key);
}
```

```
int binarySearchIterative(int arr[], int n, int key) {
 int low = 0;
 int high = n - 1;
 while (low <= high) {
 // 防止 (low + high) 过大导致整数溢出, 用减法替代加法
 int mid = low + (high - low) / 2;
 if (arr[mid] == key) {
 return mid; // 找到了
 } else if (arr[mid] < key) {
 low = mid + 1; // 目标在右半区
 } else {
 high = mid - 1; // 目标在左半区
 }
 }
 return -1; // 区间无效, 未找到
}
```

# 分块查找

- 将数据按照一定原则存入数据表，使数据表划分为若干个逻辑子表。逻辑子表由索引表划分。
  - 索引表：由每个逻辑子表的最大关键字项构成的有序表。
  - 分块查找：先用顺序查找法在索引表中查找索引记录，再依据该记录值在数据表中的相应逻辑子表查找所需的记录。
- 例如，数据表 $L=\{22, 12, 8, 18, 33, 42, 23, 35, 48, 55, 60, 86, 77\}$ 包含三个逻辑子表。
  - 索引表 $LI=\{22, 1, 4; 42, 5, 4; 86, 9, 5\};$



# 分块查找

- 索引分块的存储

```
typedef struct {
 int max; // 该块内的最大值
 int start; // 该块在数组中的起始下标
 int length; // 该块的长度（元素个数）
} IndexBlock;
```

# 分块查找

- 建立索引表 时间复杂度： $O(n)$

- 遍历块

- 设置起始
    - 设置长度

```
for (i = 0; i < *numBlocks; i++) {
```

```
 index[i].start = i * blockSize;
```

```
 if (i == *numBlocks - 1) // 最后一块可能不满
 index[i].length = n - index[i].start;
 else
 index[i].length = blockSize;
```

- 遍历块内元素

```
 int maxVal = arr[index[i].start];
 for (j=index[i].start+1;
 j<index[i].start+index[i].length; j++)
```

- 找出最大值

```
 if (arr[j] > maxVal)
 maxVal = arr[j];
 index[i].max = maxVal;
```

```
}
```

# 分块查找

- 分块查找 时间复杂度： $O(\log b + s)$

- 在索引表中折半查找，定位到块
- 整理块号

```
int low = 0, high = numBlocks - 1, targetBlock = -1;
while (low <= high) {
 int mid = low + (high - low) / 2;
 if (index[mid].max == key) {
 targetBlock = mid; break;
 } else if (index[mid].max < key)
 low = mid + 1; // key 比当前块最大值还大，往后找
 else
 high = mid - 1; // key 比当前块最大值小，往前找
}
```

- 遍历块内元素
- 找出值

```
if (targetBlock == -1)
 if (low < numBlocks)
 targetBlock = low;
 else
 return -1; // key比所有块的最大值都大，不存在

int start = index[targetBlock].start;
int end = start + index[targetBlock].length;
for (int i = start; i < end; i++)
 if (arr[i] == key)
 return i;
```

```

typedef struct {
 int max; // 该块内的最大值
 int start; // 该块在数组中的起始下标
 int length; // 该块的长度 (元素个数)
} IndexBlock;

void buildIndex(int arr[], int n, IndexBlock index[], int*
 numBlocks, int blockSize) {
 *numBlocks = (n + blockSize - 1) / blockSize; // 总块数上取整
 for (int i = 0; i < *numBlocks; i++) {
 index[i].start = i * blockSize;
 if (i == *numBlocks - 1) // 最后一块可能不满, 单独处理长度
 index[i].length = n - index[i].start;
 else
 index[i].length = blockSize;
 // 遍历块内元素, 找出最大值 (因为块内完全无序, 必须遍历找最大)
 int maxVal = arr[index[i].start];
 for (int j = index[i].start + 1;
 j < index[i].start + index[i].length; j++)
 if (arr[j] > maxVal)
 maxVal = arr[j];
 index[i].max = maxVal;
 }
}

```

```

int blockSearch(int arr[], IndexBlock index[], int numBlocks,
 int key) {
 // 第一步：在索引表中折半查找，定位到具体块
 int low = 0, high = numBlocks - 1;
 int targetBlock = -1;
 while (low <= high) {
 int mid = low + (high - low) / 2;
 if (index[mid].max == key) {
 targetBlock = mid;
 break;
 } else if (index[mid].max < key)
 low = mid + 1; // key 比当前块最大值还大，往后找
 else
 high = mid - 1; // key 比当前块最大值小，往前找
 }
 // 如果没精确命中，low 指向第一个 max > key 的块，key 就在那一块里
 if (targetBlock == -1)
 if (low < numBlocks)
 targetBlock = low;
 else
 return -1; // key 比所有块的最大值都大，肯定不存在
}

```

// 第二步：在确定的块内进行顺序查找

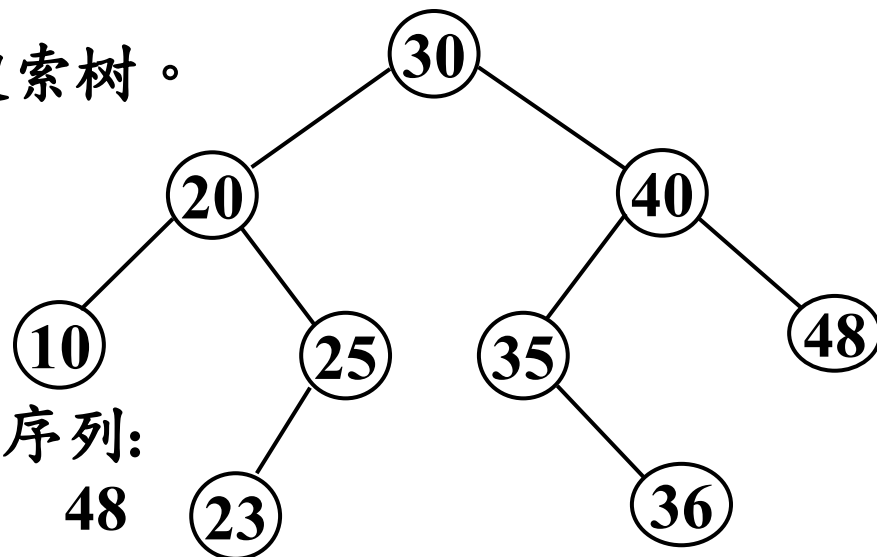
```
int start = index[targetBlock].start;
int end = start + index[targetBlock].length;
for (int i = start; i < end; i++) {
 if (arr[i] == key) {
 return i;
 }
}
return -1; // 块内没找到
}
```

# 目录

|  |   |         |
|--|---|---------|
|  | 1 | 基本概念和分类 |
|  | 2 | 静态查找    |
|  | 3 | 动态查找    |
|  | 4 | 哈希表     |
|  |   |         |

# 二叉搜索树

- 定义：或是一棵空树，或是具有如下特性的二叉树：
  - (1) 若左子树不空，则左子树上所有结点的值均小于根结点的值；
  - (2) 若右子树不空，则右子树上所有结点的值均大于(或等于)根结点的值；
  - (3) 左、右子树分别是二叉搜索树。



中序遍历二叉搜索树可以得到排序序列:

10 20 23 25 30 35 36 40 48



# 二叉搜索树：插入

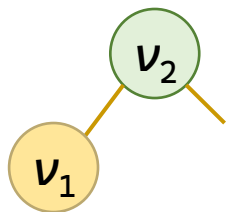
## • 算法

– 如果空树则为根



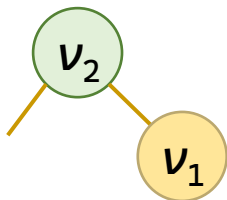
```
if (root == NULL) {
 return createNode(key);
}
```

– 如果当前值比树根大，



```
if (key < root->data) {
 root->lchild = insertBST(root->lchild, key);
}
```

– 如果当前值比树根小，



```
else if (key > root->data) {
 root->rchild = insertBST(root->rchild, key);
}
```

```

BSTNode* createNode(int key) {
 BSTNode* node = (BSTNode*)malloc(sizeof(BSTNode));
 node->data = key; node->lchild = NULL; node->rchild = NULL;
 return node;
}

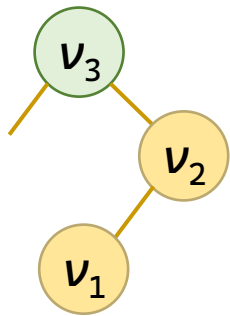
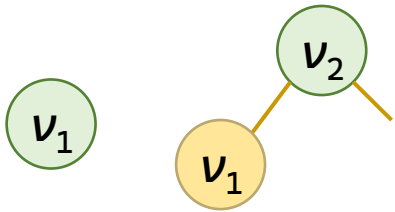
BSTNode* insertBST(BSTNode* root, int key) {
 // 递归出口：找到空位置，创建新节点并返回
 if (root == NULL)
 return createNode(key);
 // 如果关键字小于根节点，插入左子树
 if (key < root->data)
 root->lchild = insertBST(root->lchild, key);
 // 如果关键字大于根节点，插入右子树
 else if (key > root->data)
 root->rchild = insertBST(root->rchild, key);
 else
 printf("提示：关键字 %d 已存在于树中。\\n", key);
 return root;
}

```

# 二叉搜索树：创建

- 算法

- 遍历数组
- 插入元素



```
BSTNode* createBST(int arr[], int n) {
 BSTNode* root = NULL;
 for (int i = 0; i < n; i++) {
 root = insertBST(root, arr[i]);
 }
 return root;
}
```

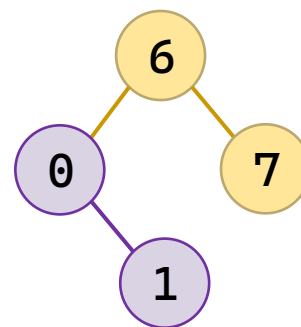
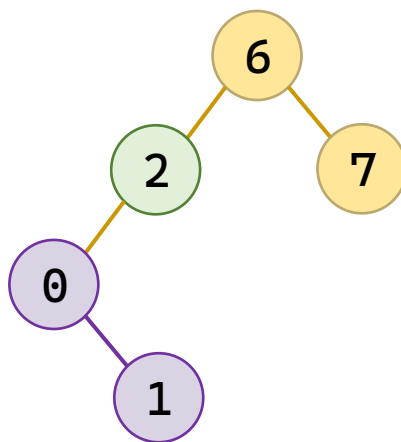
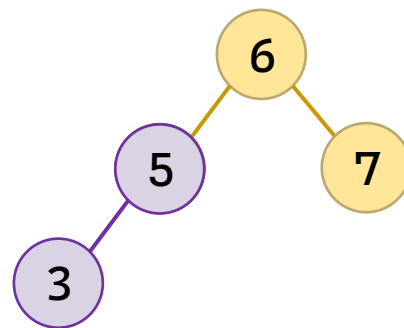
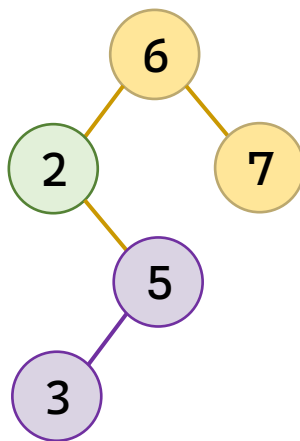
# 二叉搜索树：删除

- 算法

- 递归查找，找到元素

- 左子树为空，找右孩子

- 右子树为空，找左孩子

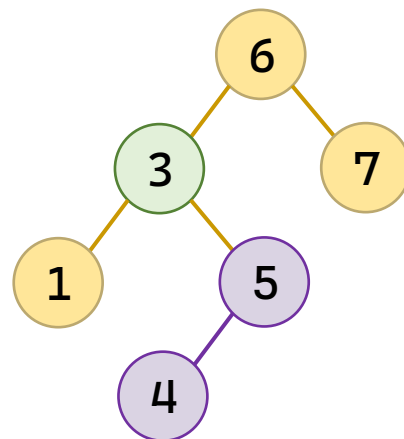
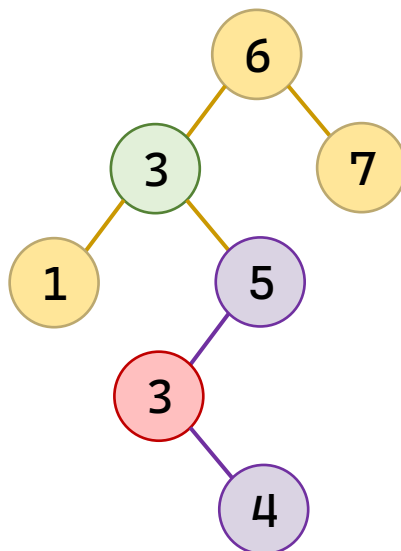
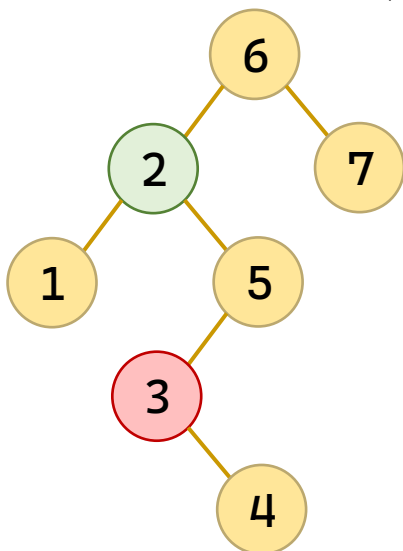


# 二叉搜索树：删除

- 算法

- 左右子树不为空

- 找到右子树最小值，即最往左的左子树
    - 替代当前值
    - 删除右子树里的最小值点



# 二叉搜索树：删除

- 算法

- 递归查找  
找到元素

```
BSTNode* deleteBST(BSTNode* root, int key) {
 if (key < root->data)
 root->lchild = deleteBST(root->lchild, key);
 else if (key > root->data)
 root->rchild = deleteBST(root->rchild, key);
```

- 左子树为空，找右孩子

```
 if (root->lchild == NULL) {
 BSTNode* temp = root->rchild;
 free(root);
 return temp;
 }
```

- 右子树为空，找左孩子

```
 if (root->rchild == NULL) {
 BSTNode* temp = root->lchild;
 free(root);
 return temp;
 }
```

# 二叉搜索树：删除

- 算法

- 左右子树不为空

- 找到右子树最小值，即最往左的左子树

```
BSTNode* mostLeft = root->rchild;
while (mostLeft->lchild != NULL) {
 mostLeft = mostLeft->lchild;
}
```

- 替代当前值

```
root->data = mostLeft->data;
```

- 删除右子树里的最小值点

```
root->rchild = deleteBST(root->rchild, mostLeft->data);
```

```

BSTNode* deleteBST(BSTNode* root, int key) {
 if (root == NULL) {
 printf("删除失败: 关键字 %d 不在树中.\n", key);
 return NULL;
 }
 // 第一步: 递归找到要删除的节点
 if (key < root->data) {
 root->lchild = deleteBST(root->lchild, key);
 } else if (key > root->data) {
 root->rchild = deleteBST(root->rchild, key);
 }
 else {
 // 找到了! 开始处理删除逻辑
 if (root->lchild == NULL) { // 情况 1: 左子树为空, 或叶子
 BSTNode* temp = root->rchild;
 free(root); // 释放当前节点
 return temp; // 让父节点直接指向右孩子
 }
 else if (root->rchild == NULL) { // 情况 2: 右子树为空
 BSTNode* temp = root->lchild;
 free(root);
 return temp; // 让父节点直接指向左孩子
 }
 }
}

```



```

else { // 情况 3: 左右子树都不为空 (最经典的情况)
 // 找到右子树中最小的节点 (即中序后继)
 BSTNode* mostLeft = root->rchild;
 while (mostLeft->lchild != NULL) {
 mostLeft = mostLeft->lchild;
 }
 // 用该最小节点的值覆盖当前要删除的节点
 root->data = mostLeft->data;
 // 在右子树中递归删除那个值相同的最小节点
 // (它一定没有左孩子, 退化为情况1)
 root->rchild = deleteBST(root->rchild, mostLeft->data);
}
}
return root;
}

```

# 二叉搜索树：查找

- 算法

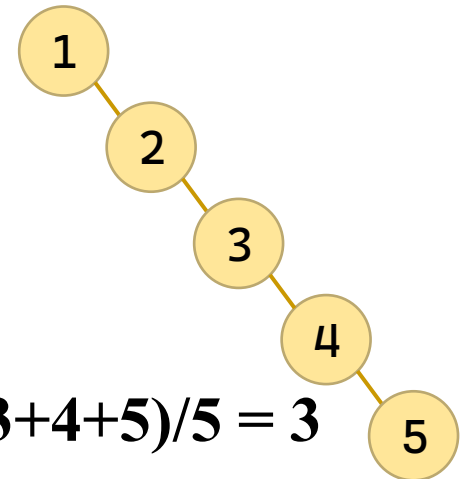
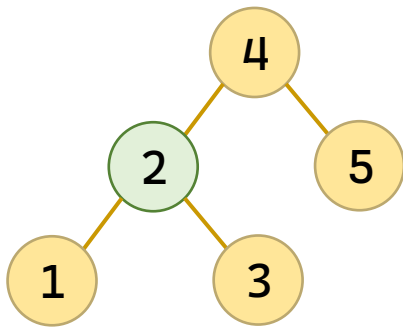
- 中序遍历算法

```
BSTNode* searchBST(BSTNode* root, int key) {
 // 递归出口：要么找到空（没找到），要么找到了
 if (root == NULL || root->data == key) {
 return root;
 }
 // 根据大小关系决定去左子树还是右子树继续找
 if (key < root->data) {
 return searchBST(root->lchild, key);
 } else {
 return searchBST(root->rchild, key);
 }
}
```

# 二叉排序树-查找性能分析

- 查找性能分析

- 对于每一棵二叉排序树，均可按照平均查找长度的定义来计算它的ASL值。
- 由n个关键字值构造所得的不同形态的二叉排序树，其平均查找长度不一定相同，甚至可能差别很大。



$$ASL_1 = (3 + 2 + 3 + 1 + 2) / 5 = 2.2 \quad ASL_2 = (1 + 2 + 3 + 4 + 5) / 5 = 3$$

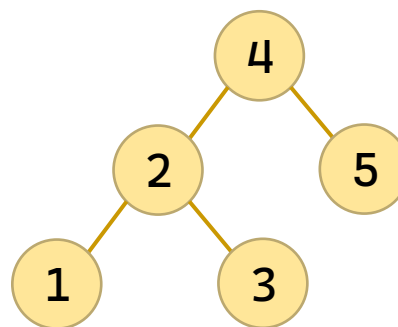
# 平衡二叉树

- 平衡二叉树 ( **Balanced Binary Tree** )
  - 别称：AVL树，Adelson-Velskii and Landis Tree
- 定义
  - 或者是一棵空树，或者是具有下列性质的二叉树：
  - (1)左子树和右子树都是平衡二叉树；
  - (2)左子树和右子树的深度之差 $\leq 1$ 。

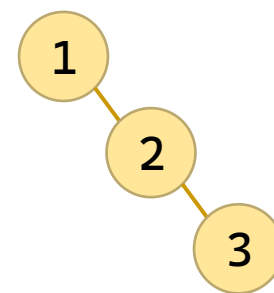
# 平衡二叉树：定义

- 平衡二叉树的存储

```
typedef struct AVLNode {
 int data; // 数据域
 int height; // 当前节点的高度
 struct AVLNode* lchild, * rchild;
} AVLNode;
```



平衡



不平衡

# 平衡二叉树：判断平衡

- 更新高度

- 后续遍历获得左右子树高度
- 当前高度是左右子树高度加1

```
int getHeight(AVLNode* node) {
 return node ? node->height : 0;
}
void updateHeight(AVLNode* node) {
 if (node) {
 int leftH = getHeight(node->lchild);
 int rightH = getHeight(node->rchild);
 node->height = 1 +
 (leftH > rightH ? leftH : rightH);
 }
}
```

- 平衡因子：左右子树高度差

```
int getBalanceFactor(AVLNode* node) {
 return node ? getHeight(node->lchild) - getHeight(node->rchild) : 0;
}
```

# 平衡二叉树：左旋转、右旋转

- 右旋转

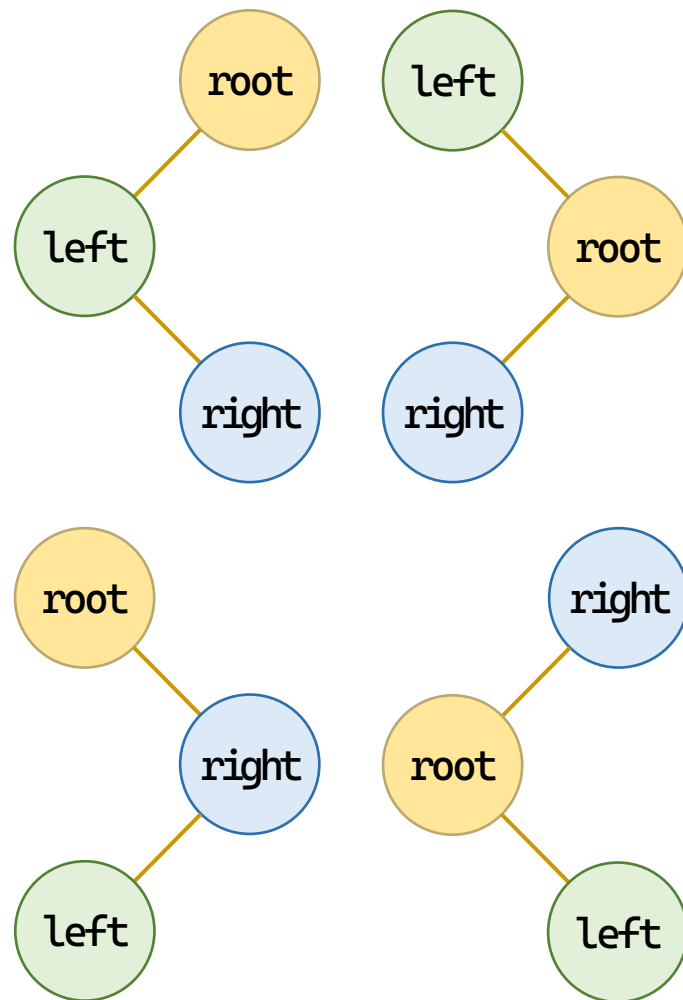
```
AVLNode* left = root->lchild;
AVLNode* left_right = left->rchild;
left->rchild = root;
root->lchild = left_right;
```

- 左旋转

```
AVLNode* right = root->rchild;
AVLNode* right_left = right->lchild;
right->lchild = root;
root->rchild = right_left;
```

- 旋转后更新涉及各树

```
updateHeight(root);
updateHeight(left);
```



```

AVLNode* rightRotate(AVLNode* root) {
 AVLNode* left = root->lchild;
 AVLNode* left_right = left->rchild;
 left->rchild = root;
 root->lchild = left_right;
 updateHeight(root);
 updateHeight(left);
 return left;
}

AVLNode* leftRotate(AVLNode* root) {
 AVLNode* right = root->rchild;
 AVLNode* right_left = right->lchild;
 right->lchild = root;
 root->rchild = right_left;
 updateHeight(root);
 updateHeight(right);
 return right;
}

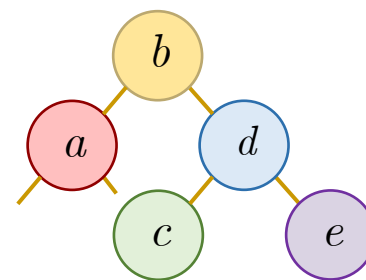
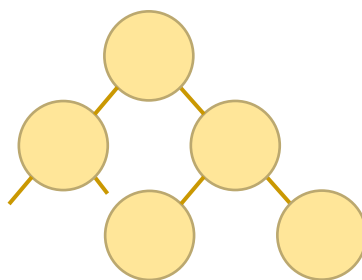
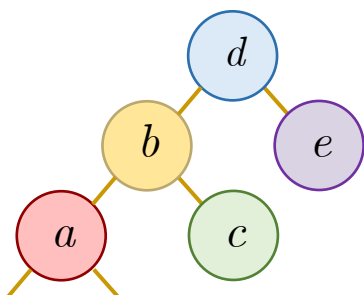
```



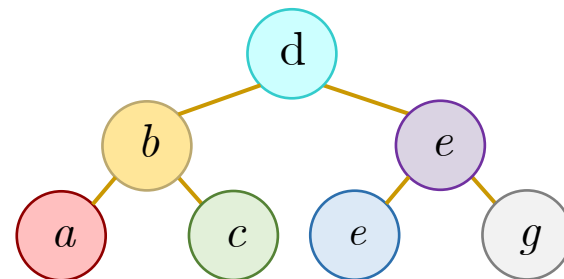
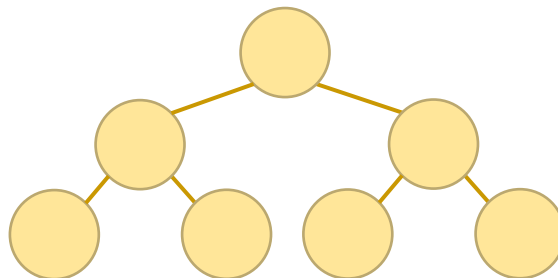
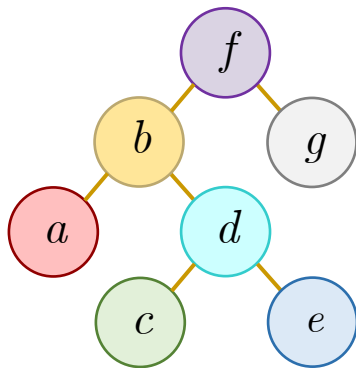
# 平衡二叉树：平衡

- 根结点左重（先画图，再填空）

- LL：左子树左重（或平衡）



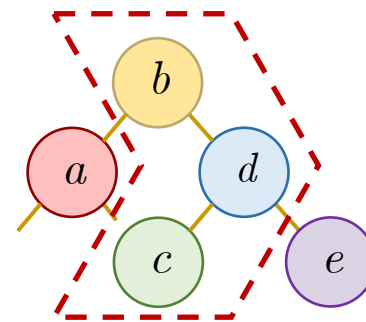
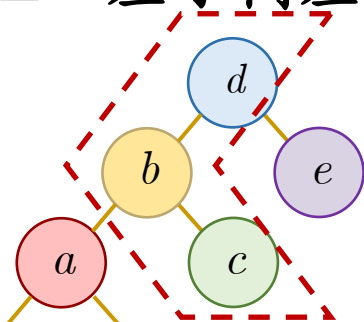
- LR：左子树右重



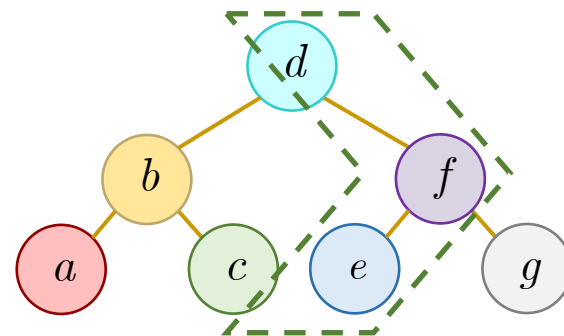
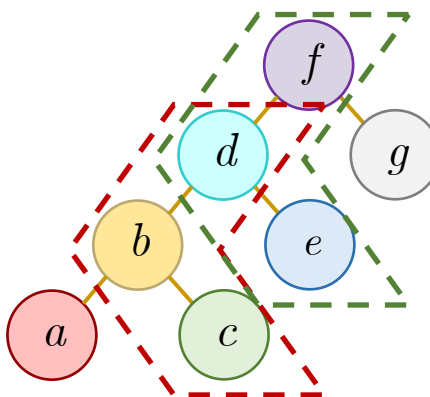
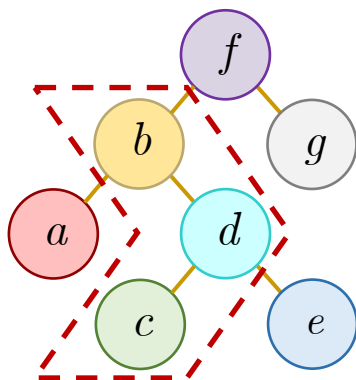
# 平衡二叉树：平衡

- 根结点左重

- LL：左子树左重（或平衡）



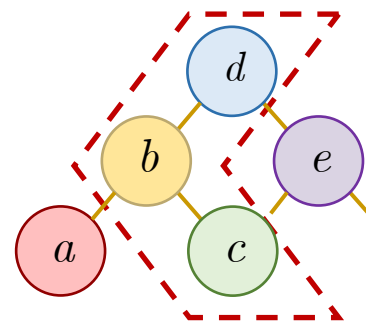
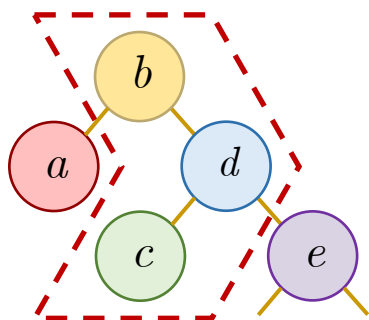
- LR：左子树右重



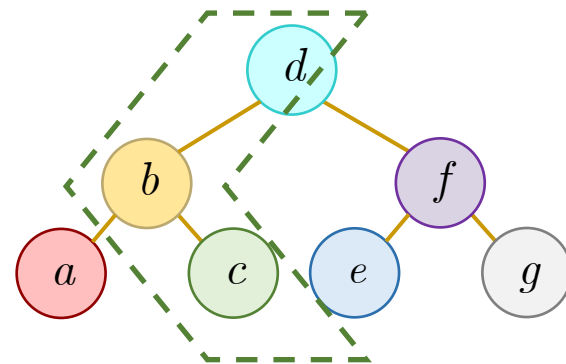
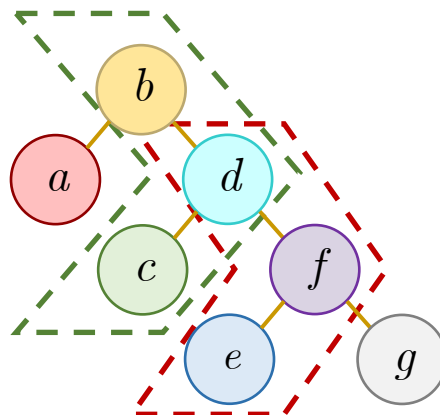
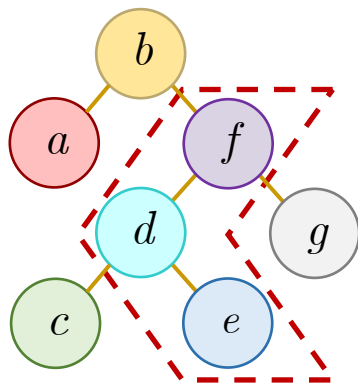
# 平衡二叉树：平衡

- 根结点右重（按根结点左重的情况镜像解决）

– RR：左子树右重（或平衡）



– RL：左子树左重



# 平衡二叉树：平衡

- 根结点左重

```
int bf = getBalanceFactor(node);
```

- LL：根结点右转

```
if (bf>1 && getBalanceFactor(node->lchild)>=0)
 return rightRotate(node);
```

- LR：左子树左转，根结点右转

```
if (bf > 1 && getBalanceFactor(node->lchild) < 0) {
 node->lchild = leftRotate(node->lchild);
 return rightRotate(node);
}
```

- 根结点右重

- RR：根结点左转

```
if (bf<-1 && getBalanceFactor(node->rchild)>=0)
 return leftRotate(node);
```

- RL：右子树右转，根结点左转

```
if (bf < -1 && getBalanceFactor(node->rchild) > 0) {
 node->rchild = rightRotate(node->rchild);
 return leftRotate(node);
}
```

```

AVLNode* balanceNode(AVLNode* node) {
 if (node == NULL) return NULL;
 int bf = getBalanceFactor(node);
 // LL 型: 左重且左孩子非右重 (>=0 表示左孩子本身左重或平衡)
 if (bf > 1 && getBalanceFactor(node->lchild) >= 0)
 return rightRotate(node);
 // LR 型: 左重且左孩子右重 (<0)
 if (bf > 1 && getBalanceFactor(node->lchild) < 0) {
 node->lchild = leftRotate(node->lchild);
 return rightRotate(node);
 }
 // RR 型: 右重且右孩子非左重 (<=0)
 if (bf < -1 && getBalanceFactor(node->rchild) <= 0)
 return leftRotate(node);
 // RL 型: 右重且右孩子左重 (>0)
 if (bf < -1 && getBalanceFactor(node->rchild) > 0) {
 node->rchild = rightRotate(node->rchild);
 return leftRotate(node);
 }
 return node;
}

```

# 平衡二叉树：插入

## • 算法

— 后续遍历，找到插入位置

```
if (root == NULL)
 return createNode(key);
```

▪ 因为当前结点是否平衡取决于左右子树是否平衡

— 插入数据小于当前结点，在左子树插入

▪ 因为当前结点可能左右子树，不可能在当前结点插入

```
if (key < root->data)
 root->lchild = insertAVL(root->lchild, key);
```

— 插入数据大于当前结点，在右子树插入

```
else if (key > root->data)
 root->rchild = insertAVL(root->rchild, key);
```

— 插入后更新高度，重新平衡

```
updateHeight(root);
return balanceNode(root);
```

```
AVLNode* insertAVL(AVLNode* root, int key) {
 if (root == NULL) {
 return createNode(key);
 }

 if (key < root->data) {
 root->lchild = insertAVL(root->lchild, key);
 } else if (key > root->data) {
 root->rchild = insertAVL(root->rchild, key);
 } else {
 printf("提示: 关键字 %d 已存在, 插入忽略.\n", key);
 return root;
 }

 updateHeight(root);
 return balanceNode(root);
}
```

# 平衡二叉树：删除

## • 算法

– 后续遍历，找到插入位置

```
if (root == NULL)
 return NULL;
```

– 删除数据小于当前结点，在左子树删除

▪ 因为当前结点可能左右子树，不可能在当前结点插入

```
if (key < root->data)
 root->lchild = deleteAVL(root->lchild, key);
```

– 删除数据大于当前结点，在右子树删除

```
else if (key > root->data)
 root->rchild = deleteAVL(root->rchild, key);
```

– 执行删除（见后页）

– 删除后更新高度，重新平衡

```
updateHeight(root);
return balanceNode(root);
```



# 平衡二叉树：删除

- 算法：执行删除（此时已找到）

- 左子结点空，  
右子结点升级

```
if (root->lchild == NULL) {
 AVLNode* temp = root->rchild;
 free(root);
 return temp;
}
```

- 右子结点空，  
左子结点升级

```
} else if (root->rchild == NULL) {
 AVLNode* temp = root->lchild;
 free(root);
 return temp;
}
```

- 左右结点都在，  
用右节点的最小  
结点顶替

```
} else {
 AVLNode* temp = root->rchild;
 while (temp->lchild != NULL)
 temp = temp->lchild;
 root->data = temp->data;
 root->rchild = deleteAVL(
 root->rchild, temp->data);
}
```

```

AVLNode* deleteAVL(AVLNode* root, int key) {
 if (root == NULL) {
 printf("删除失败: 关键字 %d 不在树中.\n", key);
 return NULL;
 }
 if (key < root->data)
 root->lchild = deleteAVL(root->lchild, key);
 else if (key > root->data)
 root->rchild = deleteAVL(root->rchild, key);
 else { // 找到待删除节点
 if (root->lchild == NULL) {
 AVLNode* temp = root->rchild;
 free(root);
 return temp;
 } else if (root->rchild == NULL) {
 AVLNode* temp = root->lchild;
 free(root);
 return temp;
 } else {

```

```
// 双孩子：用右子树最小值（中序后继）顶替
```

```
AVLNode* temp = root->rchild;
```

```
while (temp->lchild != NULL)
```

```
 temp = temp->lchild;
```

```
root->data = temp->data;
```

```
root->rchild = deleteAVL(root->rchild, temp->data);
```

```
 }
```

```
}
```

```
if (root == NULL) return NULL;
```

```
updateHeight(root);
```

```
return balanceNode(root);
```

```
}
```

# 平衡二叉树：查找

- 算法

- 如相等，则停止
- 如过大，找左支
- 如过小，找右支

```
AVLNode* searchAVL(AVLNode* root, int key) {
 if (root == NULL || root->data == key) {
 return root;
 }
 if (key < root->data) {
 return searchAVL(root->lchild, key);
 } else {
 return searchAVL(root->rchild, key);
 }
}
```

# 平衡二叉树：性能分析

- 平衡二叉树的查找性能分析：
  - 在平衡树上进行查找的过程和二叉排序树相同，因此，查找过程中和给定值进行关键字的比较次数不超过平衡树的深度。
- 在平衡二叉树上进行查找的时间复杂度为  $O(\log n)$
- 高度为  $h$  的 AVL 树所含的最少节点数  $N(h)$  满足：
  - $$N(h) = N(h-1) + N(h-2) + 1$$

# 平衡二叉树：性能分析

- 问：含有  $n$  个关键字的平衡二叉树可能达到的最小和最大深度是多少？

$$h_{\min} = \lfloor \log_2 n \rfloor + 1, \quad h_{\max} \leq 1.44 \log_2 (n + 2) - 0.328$$

$n=0$



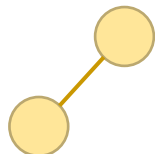
$h_{\max}=0$

$n=1$



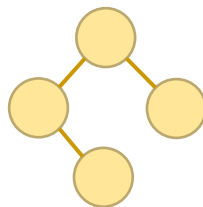
$h_{\max}=1$

$n=2$



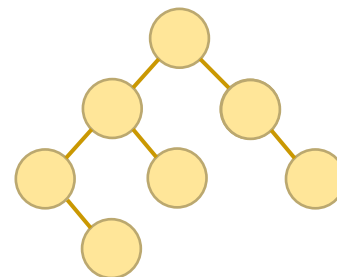
$h_{\max}=2$

$n=4$



$h_{\max}=3$

$n=7$



$h_{\max}=4$

# B树和B<sup>+</sup>树

- 平衡二叉排序树便于动态查找，因此用平衡二叉排序树来组织索引表是一种可行的选择。当用于大型数据库时，所有数据及索引都存储在外存，因此，涉及到内、外存之间频繁的数据交换，这种交换速度的快慢成为制约动态查找的瓶颈。若以二叉树的结点作为内、外存之间数据交换单位，则查找给定关键字时对磁盘平均进行 $\log_2 n$ 次访问是不能容忍的，因此，必须选择一种能尽可能降低磁盘I/O次数的索引组织方式。树结点的大小尽可能地接近页的大小。

# B树

- R. Bayer和E. McCreight在1972年提出了一种多路平衡查找树，称为B树(其变型体是B+树)。
- 一个 $m$ 阶的B树是一个有以下属性的树：
  - 每个节点最多有  $m$  个子节点。
  - 每个非叶子、非根节点至少有  $\lceil m/2 \rceil$  个子节点。
  - 如果根节点不是叶子，则它至少有2个子节点。
  - 一个有  $k$  个子节点的非叶子节点，包含  $k-1$  个键值。
  - 所有叶子节点都在同一层。

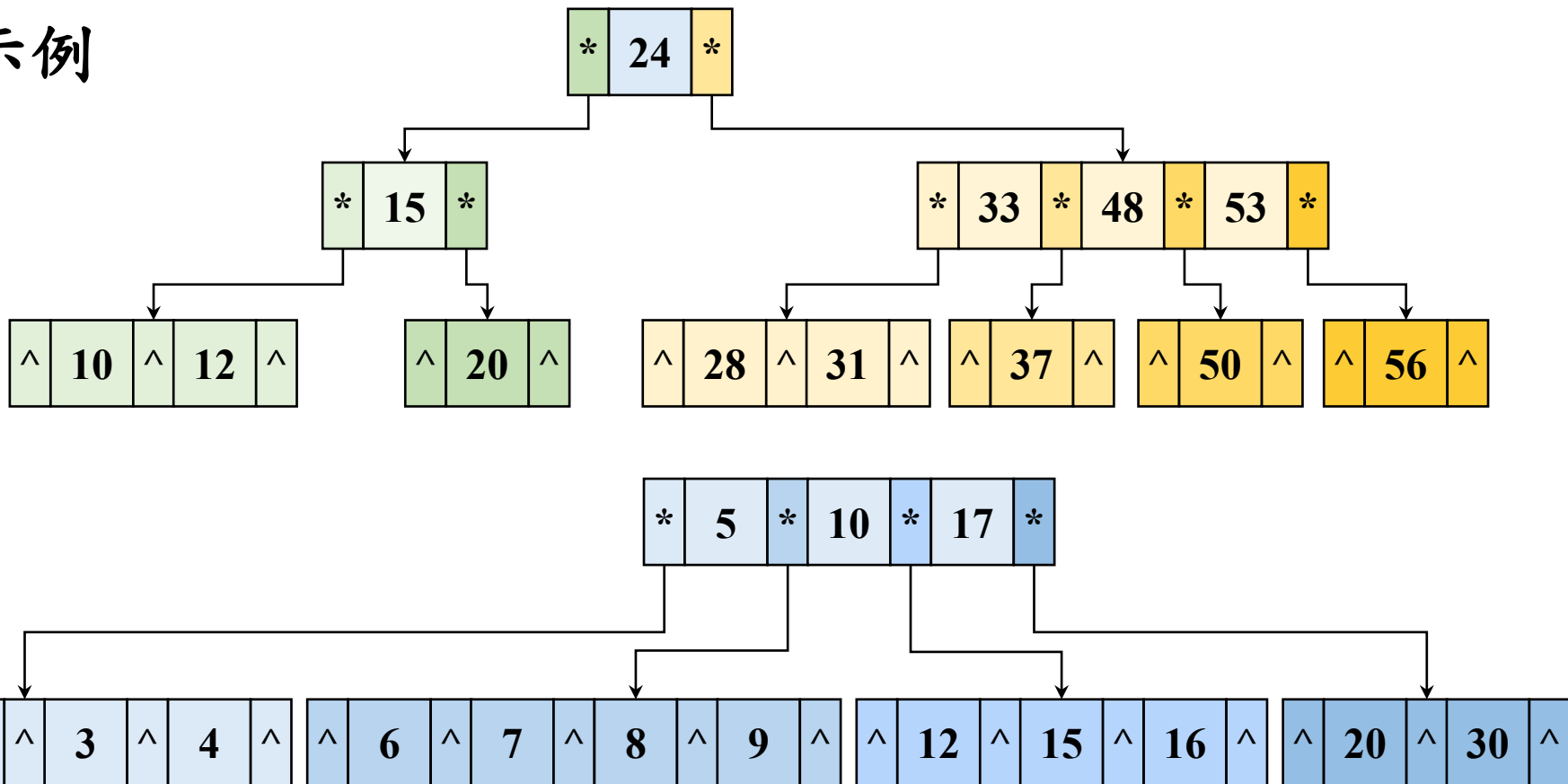


# B树

- R. Bayer和E. McCreight在1972年提出了一种多路平衡查找树，称为B树(其变型体是B+树)。
- 一个 $m$ 阶的B树是一个有以下属性的树：
  - 每个节点最多有  $m$  个子节点。
  - 每个非叶子、非根节点至少有  $\lceil m/2 \rceil$  个子节点。
  - 如果根节点不是叶子，则它至少有2个子节点。
  - 一个有  $k$  个子节点的非叶子节点，包含  $k-1$  个键值。
  - 所有叶子节点都在同一层。

# B树

- 示例



# B树

## • B树的存储结构

```
#define T 3
#define MAX_KEYS (2 * T - 1)
#define MAX_CHILDREN (2 * T)

typedef struct BTreeNode {
 int keyCount; /* 当前节点中关键字的数量 */
 int keys[MAX_KEYS]; /* 始终按升序存放关键字 */
 void *recptr[MAX_KEYS]; /* 与关键字对应的记录指针 */
 bool isLeaf; /* 叶子节点没有孩子 */
 struct BTreeNode *children[MAX_CHILDREN];
} BTreeNode;
```

# B树：查找

- 算法

- 找到首次小于关键字的索引

```
while (index < node->keyCount && key > node->keys[index])
 ++index;
```

- 找到就返回

```
if (index < node->keyCount && key == node->keys[index])
 return node;
```

- 否则搜索下一级区间

- 记录叶子节点，提前停止

```
return node->isLeaf ? NULL : searchNode(node->children[index], key);
```

```

bool contains(const BTreeNode *root, int key) {
 return root != NULL
 && searchNode((BTreeNode *)root, key) != NULL;
}

BTreeNode *searchNode(BTreeNode *node, int key) {
 int index = 0;

 while (index < node->keyCount && key > node->keys[index]) {
 ++index;
 }

 if (index < node->keyCount && key == node->keys[index]) {
 return node;
 }

 return node->isLeaf ? NULL :
 searchNode(node->children[index], key);
}

```

# B树：建立结点

- 算法

```
BTreeNode *createNode(bool isLeaf) {
 BTreeNode *node = (BTreeNode *)calloc(1, sizeof(BTreeNode));
 if (node == NULL) {
 perror("无法分配 B 树节点");
 exit(EXIT_FAILURE);
 }
 node->isLeaf = isLeaf;
 return node;
}
```

# B树：插入

- 算法

- 如果已有关键字则报错
- 如果空树则建立根

- 如果根满则分裂根

- 执行非空树插入

```
if (contains(*root, key))
 return false;
```

```
if (*root == NULL) {
 *root = createNode(true);
 (*root)->keys[0] = key;
 (*root)->recptr[0] = NULL;
 (*root)->keyCount = 1;
 return true;
}
```

```
if ((*root)->keyCount == MAX_KEYS) {
 newRoot = createNode(false);
 newRoot->children[0] = *root;
 splitChild(newRoot, 0);
 *root = newRoot;
}
```

```
insertNonFull(*root, key);
```

```

bool insert(BTreeNode **root, int key) {
 BTreeNode *newRoot;
 if (contains(*root, key))
 return false;
 if (*root == NULL) {
 *root = createNode(true);
 (*root)->keys[0] = key;
 (*root)->recptr[0] = NULL;
 (*root)->keyCount = 1;
 return true;
 }
 if ((*root)->keyCount == MAX_KEYS) {
 newRoot = createNode(false);
 newRoot->children[0] = *root;
 splitChild(newRoot, 0);
 *root = newRoot;
 }
 insertNonFull(*root, key);
 return true;
}

```



# B树：插入

- 非空树插入

- 如果在叶子层插入，像顺序表一样后移、插入

```
for (i = list->length
- 1; i >= index; i--)
```

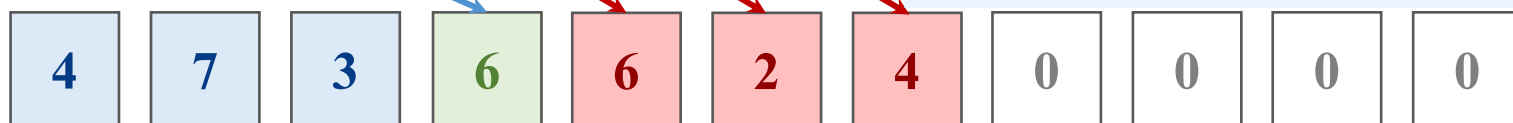
`list->length: 6`

```
list->length++;
```



`index: 3`

```
list->data[i+1] = list->data[i];
```



data



```
list->data[index] = data;
```

`SL_MAXSIZE`

- 如果非叶子节点，找到插入处，分裂满分支，再向下进入

# B树：插入

- 非空树插入

- 如果在叶子层插入，

```
if (node->isLeaf) {
```

- 像顺序表  
后移

```
while (index >= 0 && key < node->keys[index]) {
 node->keys[index + 1] = node->keys[index];
 node->recptr[index + 1] = node->recptr[index];
 --index;
}
```

- 插入

```
node->keys[index + 1] = key;
node->recptr[index + 1] = NULL;
++node->keyCount;
return;
```

```
}
```

# B树：插入

- 算法

- 如果非叶子节点，找到插入处

```
while (index >= 0 && key < node->keys[index])
 --index;
++index;
```

- 分裂满分支，避免回溯

```
if (node->children[index]->keyCount == MAX_KEYS) {
 splitChild(node, index);
 if (key > node->keys[index])
 ++index;
}
```

- 再向下进入

```
insertNonFull(node->children[index], key);
```

```

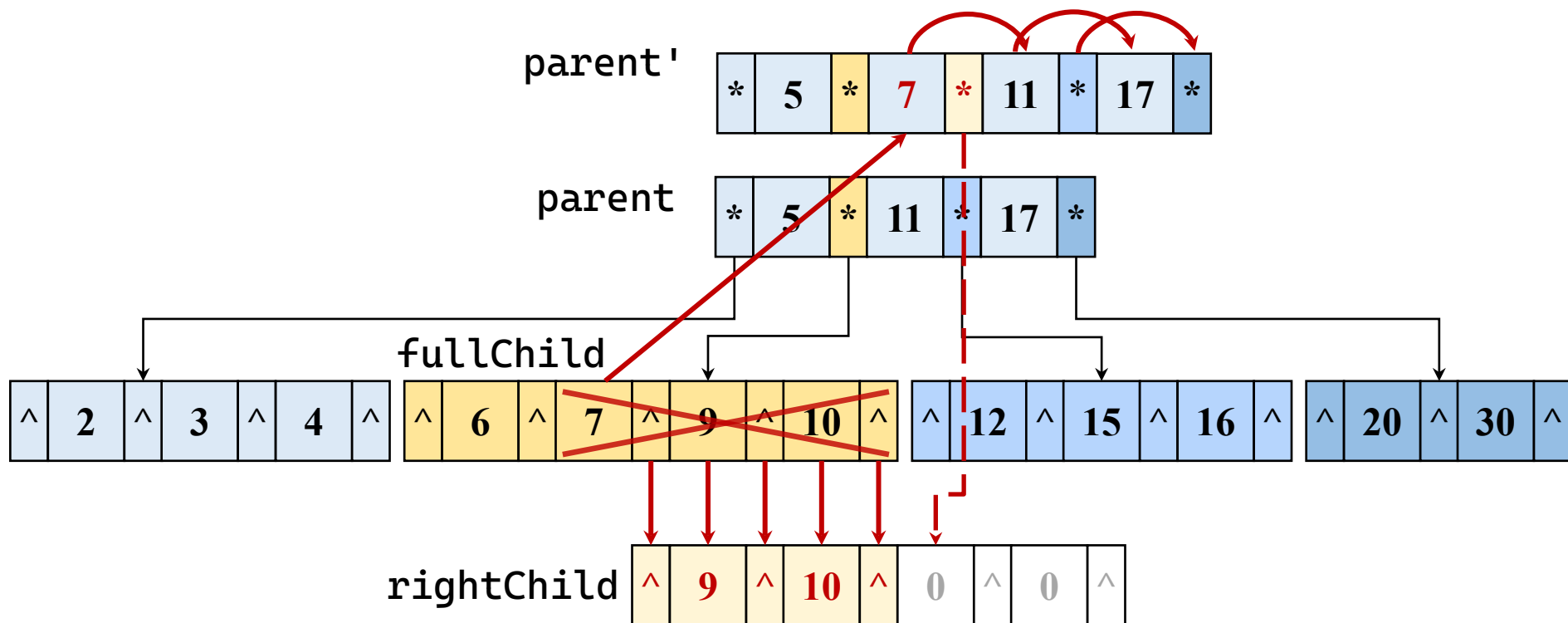
void insertNonFull(BTreeNode *node, int key) {
 int index = node->keyCount - 1;
 if (node->isLeaf) {
 while (index >= 0 && key < node->keys[index]) {
 node->keys[index + 1] = node->keys[index];
 node->recptr[index + 1] = node->recptr[index];
 --index;
 }
 node->keys[index + 1] = key;
 node->recptr[index + 1] = NULL;
 ++node->keyCount;
 return;
 }
 while (index >= 0 && key < node->keys[index])
 --index;
 ++index;
 /* 向下进入前先分裂满孩子，避免回溯。 */
 if (node->children[index]->keyCount == MAX_KEYS) {
 splitChild(node, index);
 if (key > node->keys[index])
 ++index;
 }
 insertNonFull(node->children[index], key);
}

```

# B树：分裂

- 算法

- 建右支，满支挪一半，父节点后半部右移，中子提一级



# B树：分裂

## • 算法

### – 建右支

```
BTreeNode *full = parent->children[index];
BTreeNode *right = createNode(fullChild->isLeaf);
```

### – 满支挪一半

#### ▪ 设置长度

```
right->keyCount = T - 1;
full->keyCount = T - 1;
```

#### ▪ 挪动主键、 记录

```
for (i = 0; i < T - 1; ++i) {
 right->keys[i] = fullChild->keys[i + T];
 right->recptr[i] = fullChild->recptr[i + T];
}
```

#### ▪ 非叶子挪 子分支

```
if (!fullChild->isLeaf)
 for (i = 0; i < T; ++i)
 right->children[i] = full->children[i + T];
```

### – 父节点后半部右移

### – 中子提一级

# B树：分裂

- 算法

- 父节点后半部右移

- 挪分支

```
for (i = parent->keyCount; i >= index + 1; --i)
 parent->children[i + 1] = parent->children[i];
parent->children[index + 1] = rightChild;
```

- 挪主键

```
for (i = parent->keyCount - 1; i >= index; --i) {
 parent->keys[i + 1] = parent->keys[i];
 parent->recptr[i + 1] = parent->recptr[i];
}
```

- 中子提一级

```
parent->keys[index] = full->keys[T - 1];
parent->recptr[index] = full->recptr[T - 1];
++parent->keyCount;
```

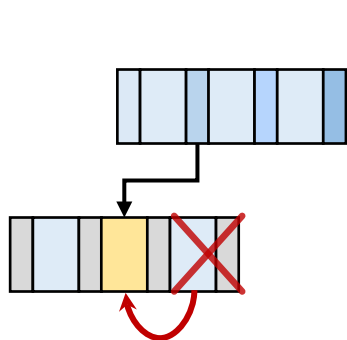
```

void splitChild(BTreeNode *parent, int index) {
 BTreeNode *fullChild = parent->children[index];
 BTreeNode *rightChild = createNode(fullChild->isLeaf);
 int i;
 rightChild->keyCount = T - 1;
 for (i = 0; i < T - 1; ++i) {
 rightChild->keys[i] = fullChild->keys[i + T];
 rightChild->recptr[i] = fullChild->recptr[i + T];
 }
 if (!fullChild->isLeaf)
 for (i = 0; i < T; ++i)
 rightChild->children[i] = fullChild->children[i+T];
 fullChild->keyCount = T - 1;
 for (i = parent->keyCount; i >= index + 1; --i)
 parent->children[i + 1] = parent->children[i];
 parent->children[index + 1] = rightChild;
 for (i = parent->keyCount - 1; i >= index; --i) {
 parent->keys[i + 1] = parent->keys[i];
 parent->recptr[i + 1] = parent->recptr[i];
 }
 parent->keys[index] = fullChild->keys[T - 1];
 parent->recptr[index] = fullChild->recptr[T - 1];
 ++parent->keyCount;
}

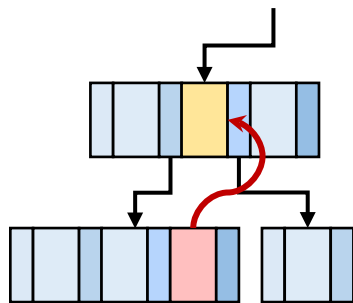
```



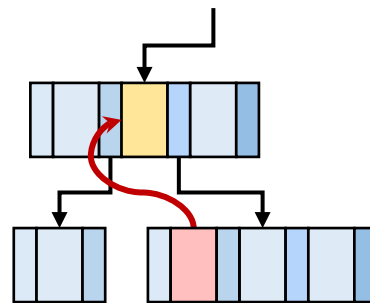
# B树：删除



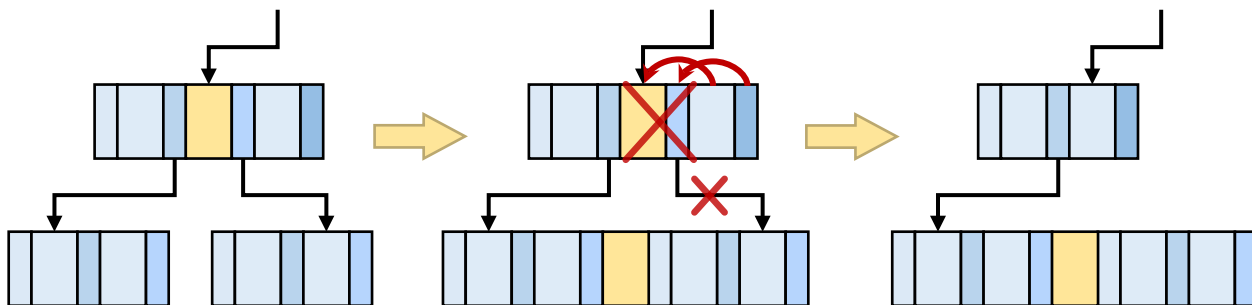
A1：命中叶子  
直接删除



A2：命中非叶，左富右穷  
左支幼子上移  
(移：删除原节点)

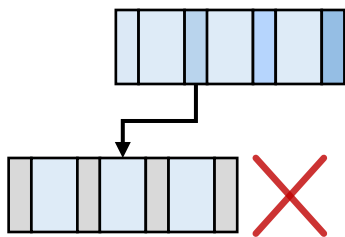


A3：命中非叶，左穷右富  
右支长子上移

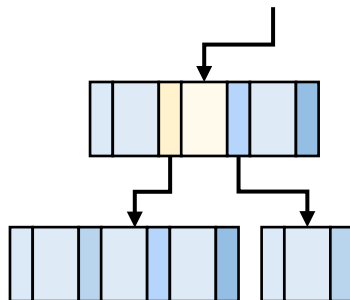


A4：命中非叶，左右都穷，左右合并  
合并：主键下沉为中子，右支复制到左支

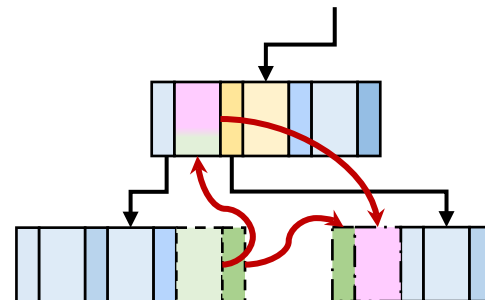
# B树：删除



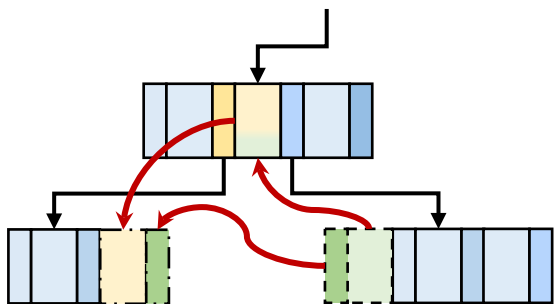
**B1：**未命中，停在叶子  
直接报错



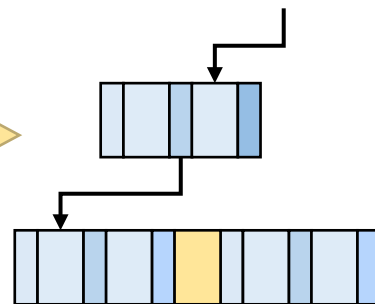
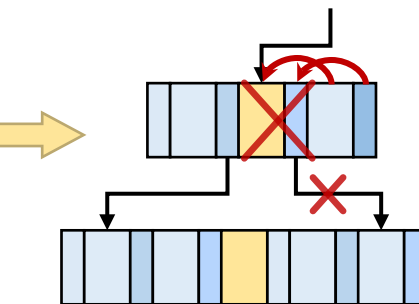
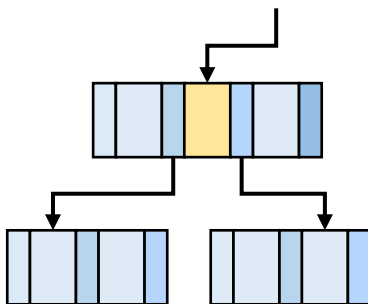
**B2：**未命中，非叶  
进入左支继续删



**B21：**删左支前左兄富  
借左兄的幼子



**B22：**删左支前左兄富  
借左兄的幼子



**B23：**删左支前左右穷

合并：非最后一支：主键下沉为中子，右支复制到本支  
最后一支：主键左兄下沉为中子，本支复制到左支

# B树：删除

- 算法

- 找到位置：第一个大于等于key的位置

```
int index = 0;
while (index < node->keyCount && key > node->keys[index])
 ++index;
```

- A：如果命中

```
if (index < node->keyCount && node->keys[index] == key) {
```

- A1：叶子，if (node->isLeaf) {

直接删除

```
while (index < node->keyCount - 1) {
 node->keys[index] = node->keys[index + 1];
 node->recptr[index] = node->recptr[index + 1];
 ++index;
}
--node->keyCount;
```

```
}
```

# B树：删除

## • 算法

– A2：命中非叶，左富右穷，左支幼子上移，删除原节点

```
if (node->children[index]->keyCount >= T) {
 int replacement = predecessor(node->children[index]);
 node->keys[index] = replacement; // 替换当前关键字
 removeFromNode(node->children[index], replacement); // 递归删除前驱
```

– A3：命中非叶，左穷右富，右支长子上移

```
} else if (node->children[index + 1]->keyCount >= T) {
 int replacement = successor(node->children[index + 1]);
 node->keys[index] = replacement;
 removeFromNode(node->children[index + 1], replacement);
```

– A4：命中非叶，左右都穷，左右合并

```
else {
 mergeChildren(node, index);
 removeFromNode(node->children[index], key);
}
```

# B树：删除

## • 算法

– B1：未命中，停在叶子，直接报错

```
if (node->isLeaf)
 return;
```

– B2：未命中，非叶，进入左支继续删

```
...
removeFromNode(node->children[index], key);
```

▪ 删之前要保证孩子至少有T个关键字

– B21：删左支前左兄富，借左兄的幼子

```
if (node->children[index]->keyCount == T - 1) {
 if (index > 0 && node->children[index - 1]->keyCount >= T)
 borrowFromPrevious(node, index); // 从左兄弟借
```

– B22：删左支前左兄富，借左兄的幼子

```
else if (index < node->keyCount
 && node->children[index + 1]->keyCount >= T)
 borrowFromNext(node, index); // 从右兄弟借
```

# B树：删除

- 算法

- B23：删左支前左右穷

- 合并：非最后一支：主键下沉为中子，右支复制到本支

```
else if (index < node->keyCount)
 mergeChildren(node, index);
```

- 合并：最后一支：主键左兄下沉为中子，本支复制到左支

```
else {
 mergeChildren(node, index - 1);
 --index; // 合并后，要调整索引
}
```

```

void removeFromNode(BTreeNode* node, int key) {
 int index = 0;
 /* 在当前节点中查找 key 的位置：找到第一个 >= key 的关键字位置 */
 while (index < node->keyCount && key > node->keys[index])
 ++index;
 /* ===== 情况 1：在当前节点找到了 key ===== */
 if (index < node->keyCount && node->keys[index] == key) {
 /* 1a. 叶子节点：直接删除 */
 if (node->isLeaf) {
 /* 将 index 后面的关键字前移覆盖 */
 while (index < node->keyCount - 1) {
 node->keys[index] = node->keys[index + 1];
 node->recptr[index] = node->recptr[index + 1];
 ++index;
 }
 --node->keyCount;
 }
 /* 1b. 内部节点：用前驱或后继替换 */
 else {
 /* 如果左孩子有富余 (>= T 个关键字)，用前驱替换 */
 if (node->children[index]->keyCount >= T) {

```

```

 int replacement = predecessor(node->children[index]);
 node->keys[index] = replacement; // 替换当前关键字
 removeFromNode(node->children[index],
 replacement); // 递归删除前驱
 }
 /* 否则如果右孩子有富余, 用后继替换 */
 else if (node->children[index+1]->keyCount >= T) {
 int replacement =
 successor(node->children[index + 1]);
 node->keys[index] = replacement;
 removeFromNode(node->children[index + 1],
 replacement);
 }
 /* 两个孩子都只有 T-1 个关键字, 则合并, 然后递归删除 */
 else {
 mergeChildren(node, index);
 removeFromNode(node->children[index], key);
 }
}
return; // 删除完成, 返回
}

```



```

/* ===== 情况 2: 未在当前节点找到 key ===== */
/* 如果是叶子节点, 说明树中不存在该 key */
if (node->isLeaf)
 return;
/* 否则进入孩子继续删除, 但先要保证该孩子至少有 T 个关键字 */
if (node->children[index]->keyCount == T - 1) {
 // 优先考虑从兄弟借一个关键字 (左兄弟或右兄弟)
 // 借位条件: 兄弟的关键字数 >= T (即有富余)
 if (index > 0 && node->children[index-1]->keyCount >= T)
 borrowFromPrevious(node, index); // 从左兄弟借
 else if (index < node->keyCount
 && node->children[index + 1]->keyCount >= T)
 borrowFromNext(node, index); // 从右兄弟借
 // 如果无法借位 (兄弟也都只有 T-1 个关键字), 则合并
 else if (index < node->keyCount)
 // 如果 index 在 keyCount 范围内, 则当前孩子有右兄弟,
 // 将当前孩子与右兄弟合并 (合并时父节点关键字 index 下移)
 mergeChildren(node, index);
}

```

```

else {
 // 如果 index == keyCount, 说明当前孩子是最右边的孩子,
 // 只能与左兄弟合并 (合并时父节点关键字 index-1 下移)
 mergeChildren(node, index - 1);
 --index;
 // 合并后, 原来 index 位置的孩子现在是合并后的左孩子
 // 所以递归时要调整索引
}
}

/* 递归进入孩子继续删除 */
removeFromNode(node->children[index], key);
}

```

# B树：删除相关：前驱和后继

- 前驱是子树的最大关键字，也是最幼子孙中的最大值

```
int predecessor(const BTreeNode *node) {
 while (!node->isLeaf)
 node = node->children[node->keyCount];
 return node->keys[node->keyCount - 1];
}
```

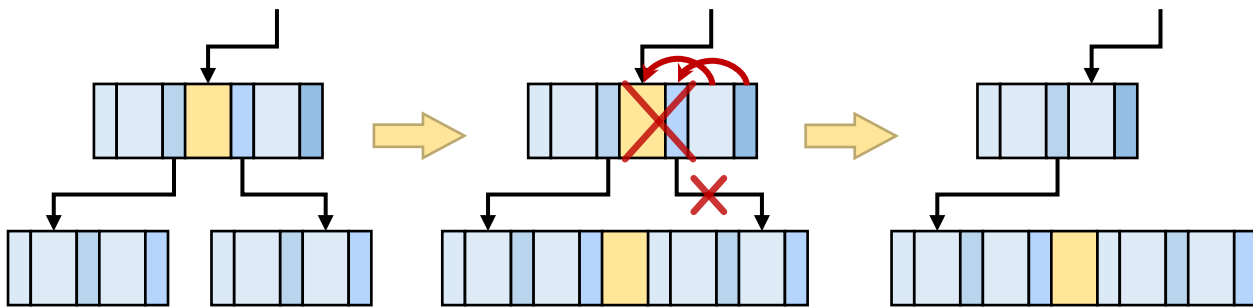
- 后继是子树的最小关键字，也是最长子孙中的最小值

```
int successor(const BTreeNode *node) {
 while (!node->isLeaf)
 node = node->children[0];
 return node->keys[0];
}
```

# B树：删除相关：合并

## • 算法

- 将父节点关键字下移到左子末尾



```
left->keys[leftCount] = parent->keys[index];
left->recptr[leftCount] = parent->recptr[index];
```

- 右孩子追加到左孩子后

```
for (i = 0; i < right->keyCount; ++i) {
 left->keys[leftCount + 1 + i] = right->keys[i];
 left->recptr[leftCount + 1 + i] = right->recptr[i];
}
```

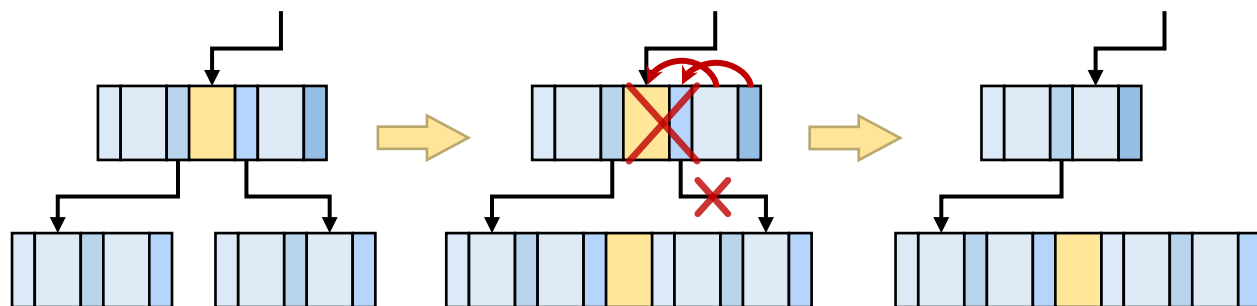
- 如果非叶子，还要复制指针

```
if (!left->isLeaf)
 for (i = 0; i <= right->keyCount; ++i)
 left->children[leftCount + 1 + i] = right->children[i];
left->keyCount += right->keyCount + 1;
```

# B树：删除相关：合并

- 算法

- 删除父节点  
关键字



```
for (i = index; i < parent->keyCount - 1; ++i) {
 parent->keys[i] = parent->keys[i + 1];
 parent->recptr[i] = parent->recptr[i + 1];
 parent->children[i + 1] = parent->children[i + 2];
}
--parent->keyCount;
```

- 释放右孩子

```
free(right);
```

```

void mergeChildren(BTreeNode* parent, int index) {
 BTreeNode* left = parent->children[index]; // 左孩子
 BTreeNode* right = parent->children[index + 1]; // 右孩子
 int leftCount = left->keyCount; // 记录左孩子原有关键字数
 int i;
 // 第一步：将父节点的分隔关键字下移到左孩子的末尾
 left->keys[leftCount] = parent->keys[index];
 left->recptr[leftCount] = parent->recptr[index];
 // 第二步：将右孩子的所有关键字和记录指针追加到左孩子后面
 for (i = 0; i < right->keyCount; ++i) {
 left->keys[leftCount + 1 + i] = right->keys[i];
 left->recptr[leftCount + 1 + i] = right->recptr[i];
 }
 // 第三步：如果是内部节点，还要将右孩子的孩子指针追加到左孩子后面
 if (!left->isLeaf)
 for (i = 0; i <= right->keyCount; ++i)
 left->children[leftCount + 1 + i]
 = right->children[i];
 // 左孩子的关键字总数：原有 + 1（下移的父关键字） + 右孩子的全部
 left->keyCount += right->keyCount + 1;
}

```

```

// 第四步：从父节点中删除索引为 index 的关键字以及右孩子指针
// 将父节点中 index 后面的所有关键字和孩子指针依次前移一位
for (i = index; i < parent->keyCount - 1; ++i) {
 parent->keys[i] = parent->keys[i + 1];
 parent->recptr[i] = parent->recptr[i + 1];
 parent->children[i + 1] = parent->children[i + 2];
}
--parent->keyCount; // 父节点关键字数减 1

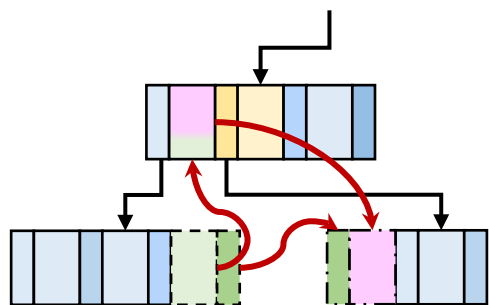
/* 释放右孩子节点 */
free(right);
}

```

# B树：删除相关：借子

## • 向左兄借子的算法

– 本人腾位置



– 接父关键字

```
for (i = child->keyCount - 1; i >= 0; --i) {
 child->keys[i + 1] = child->keys[i];
 child->recptr[i + 1] = child->recptr[i];
}
if (!child->isLeaf)
 for (i = child->keyCount; i >= 0; --i) {
 child->children[i + 1] = child->children[i];
 }
```

```
child->keys[0] = parent->keys[index - 1];
child->recptr[0] = parent->recptr[index - 1];
```

– 迎接左兄幼子

```
if (!child->isLeaf)
 child->children[0]
 = sibling->children[sibling->keyCount];
```

– 左兄关键字上提

```
parent->keys[index - 1] = sibling->keys[sibling->keyCount - 1];
parent->recptr[index - 1] = sibling->recptr[sibling->keyCount - 1];
```



# B树：删除相关：借子

## • 向右兄借子的算法

– 接父关键字

```
child->keys[child->keyCount] = parent->keys[index];
child->recptr[child->keyCount] = parent->recptr[index];
```

– 迎接右兄长子

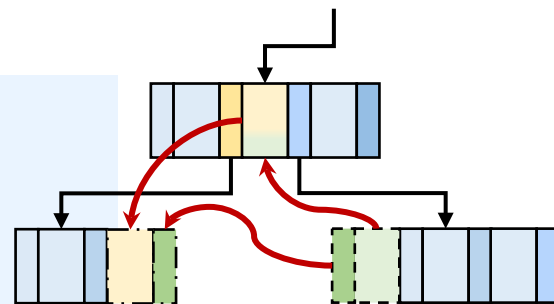
```
if (!child->isLeaf)
 child->children[child->keyCount + 1]
 = sibling->children[0];
```

– 右兄关键字上提

```
parent->keys[index] = sibling->keys[0];
parent->recptr[index] = sibling->recptr[0];
```

– 右兄剩余孩子补缺

```
for (i = 1; i < sibling->keyCount; ++i) {
 sibling->keys[i - 1] = sibling->keys[i];
 sibling->recptr[i - 1] = sibling->recptr[i];
}
if (!sibling->isLeaf)
 for (i = 1; i <= sibling->keyCount; ++i)
 sibling->children[i-1] = sibling->children[i];
```



```

void borrowFromPrevious(BTreeNode* parent, int index) {
 BTreeNode* child = parent->children[index]; // 当前孩子
 BTreeNode* sibling = parent->children[index - 1]; // 左兄弟
 int i;
 // 第一步：为当前孩子腾出第一个位置（所有关键字和记录后移一位）
 for (i = child->keyCount - 1; i >= 0; --i) {
 child->keys[i + 1] = child->keys[i];
 child->recptr[i + 1] = child->recptr[i];
 }
 // 如果是内部节点，孩子指针也要后移（为从兄弟过来的孩子指针腾位置）
 if (!child->isLeaf)
 for (i = child->keyCount; i >= 0; --i)
 child->children[i + 1] = child->children[i];
 // 第二步：将父节点中位于index-1的关键字下移到当前孩子的第一个位置
 child->keys[0] = parent->keys[index - 1];
 child->recptr[0] = parent->recptr[index - 1];
 // 第三步：将左兄弟的最后一个孩子过继给当前孩子作为第一个孩子
 // 因为左兄弟的关键字要上提，它的最后一个孩子就一起移动到当前孩子。
 if (!child->isLeaf)
 child->children[0] = sibling->children[sibling->keyCount];
}

```

```

// 第四步：将左兄弟的最后一个关键字上提到父节点，替代原来的分隔关键字
parent->keys[index-1] = sibling->keys[sibling->keyCount - 1];
parent->recptr[index-1] = sibling->recptr[sibling->keyCount-1];
/* 左兄弟关键字数减 1，当前孩子关键字数加 1 */
--sibling->keyCount;
++child->keyCount;
}

void borrowFromNext(BTreeNode* parent, int index) {
 BTreeNode* child = parent->children[index]; // 当前孩子
 BTreeNode* sibling = parent->children[index + 1]; // 右兄弟
 int i;
 // 第一步：将父节点的分隔关键字下移到当前孩子的末尾
 child->keys[child->keyCount] = parent->keys[index];
 child->recptr[child->keyCount] = parent->recptr[index];
 // 第二步：将右兄弟的第一个孩子过继给当前孩子作为最后一个孩子
 // 因为右兄弟的第一个关键字要上提，它的第一个孩子应该移到当前孩子。
 if (!child->isLeaf)
 child->children[child->keyCount+1] = sibling->children[0];
 // 第三步：将右兄弟的第一个关键字上提到父节点，替代原来的分隔关键字
 parent->keys[index] = sibling->keys[0];
 parent->recptr[index] = sibling->recptr[0];
}

```

// 第四步：将右兄弟中剩余的关键字和孩子指针整体前移一位（填补空缺）

```
for (i = 1; i < sibling->keyCount; ++i) {
 sibling->keys[i - 1] = sibling->keys[i];
 sibling->recptr[i - 1] = sibling->recptr[i];
}
if (!sibling->isLeaf) {
 for (i = 1; i <= sibling->keyCount; ++i) {
 sibling->children[i - 1] = sibling->children[i];
 }
}
```

/\* 更新关键字数：右兄弟减少，当前孩子增加 \*/

```
--sibling->keyCount;
++child->keyCount;
```

```
}
```

# B<sup>+</sup>树

- 在实际文件系统中，使用B树的变体，称为m阶B<sup>s</sup>树。
  - 它与B树的主要不同是：只在叶子结点中存储记录。
  - 在B<sup>+</sup>树中，所有的非叶子结点是索引，关键字是分界，用来界定某一关键字的记录所在的子树。
- 一棵m阶B<sup>+</sup>树与m阶B树的主要差异是：
  - 若一个结点有n棵子树，则必含有n个关键字；
  - 所有叶子结点中包含了全部记录的关键字信息及其记录的指针，而且叶子结点按关键字的大小从小到大顺序链接；
  - 所有的非叶子结点可以看成是索引的部分，结点中只含有其子树的根结点中的最大(或最小)关键字。

# B<sup>+</sup>树

## • B树的存储结构

```
#define ORDER 5
typedef struct BPlusNode {
 bool isLeaf;
 int keyCount;
 int keys[ORDER]; // 多留一个位置，供分裂前的暂存溢出使用
 void* recptr[ORDER]; // 仅叶子节点使用，与 keys 一一对应
 struct BPlusNode* children[ORDER + 1]; // 最多 ORDER+1 个孩子
 struct BPlusNode* parent;
 struct BPlusNode* next; // 仅叶子节点使用，连接下一片叶子
} BPlusNode;
```

# B<sup>+</sup>树

## • B树的存储结构（联合体版）

```
#define ORDER 5
typedef struct BPlusNode {
 bool isLeaf;
 int keyCount;
 int keys[ORDER]; // 两者共用
 union {
 struct { // 叶子节点专用的部分
 void* recptr[ORDER];
 struct BPlusNode* next;
 } leaf;
 struct { // 内部节点专用的部分
 struct BPlusNode* children[ORDER + 1];
 } internal;
 } u;
 struct BPlusNode* parent; // 两者共用，放在外面简化访问
} BPlusNode;
```

# B+树

- 在B+树上进行随机查找、插入、删除的过程基本上和B\_树类似。
- B+树查找：
  - 与B\_树相比，对B+树不仅可以从根结点开始按关键字随机查找，而且可以从最小关键字起，按叶子结点的链接顺序进行顺序查找。
  - 在B+树上进行随机查找时，若非叶子结点的关键字等于给定的K值，并不终止，而是继续向下直到叶子结点(只有叶子结点才存储记录)，即无论查找成功与否，都走了一条从根结点到叶子结点的路径。



# B<sup>+</sup>树

- B<sup>+</sup>树插入：

- B<sup>+</sup>树的插入仅仅在叶子结点上进行。当叶子结点中的关键字个数大于 $m$ 时，分裂为两个结点，含关键字个数分别是 $\lfloor (m+1)/2 \rfloor$ 和 $\lceil (m+1)/2 \rceil$ ，且将其中各自的最大关键字提升到父结点中，替代原结点在父结点中对应的关键字。提升后父结点又可能会分裂，依次类推。

- B<sup>+</sup>树删除：

- B<sup>+</sup>树的删除也仅在叶子结点进行，当叶子结点中的最大关键字被删除时，其在非叶子结点中的值可以作为一个“分界关键字”存在。若因删除而使结点中关键字的个数少于 $\lceil m/2 \rceil$ 时，其与兄弟结点的合并过程也和B树类似。

# 本节小结

- 介绍二叉排序树的基本概念、查找算法、插入算法和删除算法。
- 依据数据元素构造二叉排序树，其形状跟数据及其输入顺序都有关。
- 中序遍历二叉排序树可以得到排序序列。
- 介绍8种二叉排序树的基本平衡方法。
- 介绍B树和B<sup>+</sup>树的概念，查找、插入和删除操作等。

# 目录

|  |   |         |
|--|---|---------|
|  | 1 | 基本概念和分类 |
|  | 2 | 静态查找    |
|  | 3 | 动态查找    |
|  | 4 | 哈希表     |
|  |   |         |

# 哈希表的概念

- 哈希（ Hashing， 又称：散列）

- 一种将任意长度的输入，通过一个确定的函数，映射为固定长度的输出的过程。
- 输入，称为关键字  $k \in K$ ；输出，称为哈希值  $h(k)$ 。
- 哈希函数为  $h: K \rightarrow A$ ，其中：
- $K$  为关键字全域（可能为无限集或极大有限集），  
 $A = \{0, 1, \dots, m-1\}$  为一个有限地址空间（长度为  $m$ ）。
- 一般  $|K| \gg m$ （或  $|K| > m$ ）时，该映射必定不是单射。

- 哈希表（ Hash List，散列表）是计算直达的存储结构。

- 基于哈希函数建立的顺序表。

# 哈希表的概念

- 哈希的本质是压缩映射。
- 冲突 ( Collision )
  - 对于  $k_1 \neq k_2$ ，若  $h(k_1) = h(k_2)$ ，则称  $k_1, k_2$  为一对同义词。
  - 根据鸽巢原理，冲突在有限地址空间中不可避免。
- 装填因子  $\alpha$  ( 关键性能指标 ) :
  - 定义： $\alpha = n/m$ ，其中  $n$  为表中已存储的记录个数， $m$  为哈希表的总长度。
  - $\alpha$  越小，空间冗余度越大，时间效率越高。

# 哈希的构造方法

- 1、直接定址法

- 哈希函数： $H(k)=a \cdot k+b$
- 适合于：关键字分布连续且密集的场景
- 例：统计某班学生年龄，学号20262201—20262250，可令  $a=1, b=0$ ，直接保存。

- 2、数字分析法；

- 不直接拿关键字算，而是分析关键字的组成结构，提取出“随机性最强”的部分做地址。
- 例：上题，取  $a=1, b=-20262200$ 。
- 适合于：静态数据集，关键字有结构。

# 哈希的构造方法

## • 3、平方取中法

- 方法：关键字平方（自己乘以自己），取中间几位。
  - 中间几位的值受 Key 所有位的影响（高位和低位都会乘进去），比直接取后几位更随机。
- 例如： $k=1234$ ，取万、千位，得到22（15**22**756）

## • 4、折叠法

- 方法：把长数字切碎，再叠起来求和。
- 适用于：关键字位数特别长的情况，如：身份证号。
  - 移位折叠：123456789，按 $123+456+789=1368$ ，取368
  - 边界折叠：123456789，按 $123+654+789=1566$ ，取566

# 哈希的构造方法

## • 5、除留余数法

– 方法： $H(k) = \text{mod}(k, p)$ ,  $p \leq m$ ，建议 $p$ 为质数。

- 因为取模运算的本质是“去掉了 Key 中能被  $p$  整除的公因子”，素数能最大程度保留余数的随机性，避开底层数据可能存在的周期性规律（比如全是偶数或全是 5 的倍数）。

– 例如： $p=7$ 时， $10\%7=3$ ， $20\%7=6$ ， $30\%7=2\ldots\ldots$

## • 6、随机数法

– 方法：`srand(key); return rand()%m;`

– 缺点：方法慢，极少单独用于工程。



# 哈希的构造方法

- 选取哈希函数，考虑以下因素：
  - 计算哈希函数所需时间；
  - 关键字的长度；
  - 哈希表长度（哈希地址范围）；
  - 关键字分布情况；
  - 记录的查找频率。

# 处理冲突的方法

- 处理冲突：为产生冲突的哈希地址寻找新哈希地址。
- 1. 开放定址法
  - 为产生冲突的哈希地址 $H(k)$ 求得一个地址序列： $H_0, H_1, \dots, H_s, 1 \leq s \leq m-1$ ，其中， $H_0 = H(k)$ ， $H_i = \text{mod}(H_0 + d_i, m)$ ， $1 \leq i \leq s$ ， $d_i$ 称为增量
  - (1) 线性探测再散列： $d_i = c \cdot i$ ，其中 $c$ 为常数
  - (2) 二次探测再散列： $d_i = c_1 \cdot i^2 + c_2 \cdot i$
  - (3) 双重哈希再散列： $d_i = i \cdot h_2(k)$

# 处理冲突的方法

例 设定哈希函数  $H(\text{key}) = \text{key} \% 11$ ，采用线性探测再散列处理： $d_i = i$ ，则

|      |    |    |    |    |    |    |    |    |    |  |
|------|----|----|----|----|----|----|----|----|----|--|
| 关键字值 | 19 | 01 | 23 | 14 | 55 | 68 | 11 | 82 | 36 |  |
| 地址初值 | 8  | 1  | 1  | 3  | 0  | 2  | 0  | 5  | 3  |  |
| 哈希地址 | 8  | 1  | 2  | 3  | 0  | 4  | 5  | 6  | 7  |  |

$$\begin{aligned} \rightarrow \text{ASL} &= (1 + 1 + 2 + 1 + 1 + 3 + 6 + 2 + 5) / n \\ &= 22/9 \end{aligned}$$

# 处理冲突的方法

## • 2. 链地址法

- 将所有哈希地址相同的数据元素都存放在同一个链表中；
- 链地址法的平均查找长度较短；
- 链地址法更适合于：构造表前无法确定表长的情况(动态申请链结点)；
- 在用链地址法构造的哈希表中，删除结点的操作易于实现。

# 处理冲突的方法

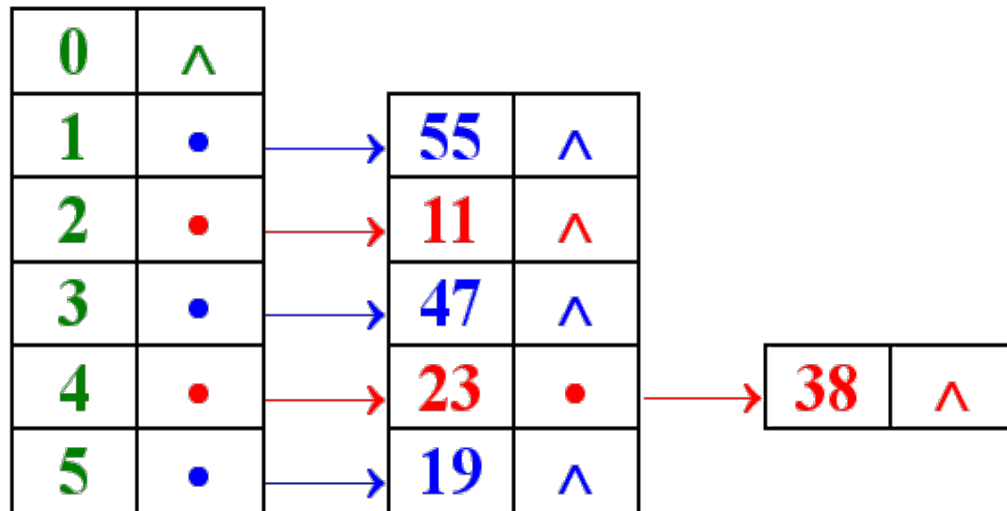
- 哈希表链表的存储结构

```
#define TABLE_SIZE 20
typedef struct HashNode {
 int key;
 void* data;
 struct HashNode* next;
} HashNode;
typedef struct {
 HashNode* heads[TABLE_SIZE];
 int count;
} HashTable;
```

// 链表节点结构（存储同义词）  
// 关键字  
// 指向实际数据记录的指针（泛型）  
// 指向下一个同义词的指针  
  
// 哈希表结构（表头数组）  
// 指针数组，每个元素指向链表头节点  
// 表中记录总数

# 处理冲突的方法

- 例 关键字集={19, 23, 55, 11, 47, 38}，表长m=6。
- 取哈希函数 $H(\text{key})=\text{key}\%5+1$ ，
- 则哈希地址={5, 4, 1, 2, 3, 4}



# 哈希表查找算法：开放地址法

- 算法

- 计算哈希值 `int start = hash(key);`

- 计算下一个哈希改变量（线性探测为例）直至遍历

```
for (step = 0; step < TABLE_SIZE; ++step) {
```

- 算出新哈希 `int index = (start + step) % TABLE_SIZE;`

- 如果该位置为空，找不到 `if (table[index].state == EMPTY)`  
`return -1;`

- 如果找到了，返回 `if (table[index].state == OCCUPIED`  
`&& table[index].key == key)`  
`return index;`

- 默认找不到

```
}
return -1;
```

```

typedef struct {
 int key;
 int state; /* EMPTY、OCCUPIED 或 DELETED */
} OpenAddressSlot;

int searchOpenAddress(const OpenAddressSlot table[], int key) {
 int start = hash(key);
 int step;
 for (step = 0; step < TABLE_SIZE; ++step) {
 int index = (start + step) % TABLE_SIZE;
 if (table[index].state == EMPTY)
 return -1;
 if (table[index].state == OCCUPIED
 && table[index].key == key)
 return index;
 }
 return -1;
}

```



# 哈希表查找算法：链地址法

- 算法

- 计算哈希值
- 进入链表，直到找到关键字，否则进入下一节点

```
ChainNode* searchChained(const ChainedHashTable* table, int key) {
 ChainNode* current = table->buckets[hash(key)];
 while (current != NULL) {
 if (current->key == key)
 return current;
 current = current->next;
 }
 return NULL;
}
```

# 哈希表查找算法：性能分析

- 哈希表的平均查找长度实际上并不等于1。
- 决定哈希表平均查找长度ASL的因素：
- 处理冲突的方法和装填因子。
- 例如，对于表长 $m=6$ ，关键字集
- $= \{ 19, 23, 55, 11, 47, 38 \}$ ，
- 如果采用链地址法， $ASL = (1 \times 5 + 2 \times 1) / 6 > 1$
- 一般情况下，装填因子 $\alpha$ 越小，发生冲突的可能性越小；装填因子 $\alpha$ 越大，填记录就越容易发生冲突。

# 哈希表查找算法：性能分析

- 哈希表的一个特点：

- 用哈希表构造列表时，可以选择一个适当的装填因子 $\alpha$ ，使得平均查找长度限定在某个范围内。一般情况下，哈希表的平均查找长度是 $\alpha$ 的函数。如利用线性探测再散列处理冲突法的平均查找长度  $ASL = \frac{1}{2} \left( 1 + \frac{1}{1 - \alpha} \right)$ 。

# 哈希表查找算法：性能分析

- 例 已知列表L中的1000个关键字各不相同。请给出列表L的一个哈希表设计方案，要求它在等概率情况下，查找成功时的平均查找长度不超过3。
  - (1) 选择处理冲突的方法，并求出 $\alpha$
  - “线性探测再散列”处理冲突法。由于要求 $ASL \leq 3$ ， $\alpha \leq 0.8$
  - (2) 确定哈希表的长度m
  - 一般取 $m \geq k/\alpha$ 中的最小素数。
  - $\because k/\alpha = 1000/0.8 = 1250$
  - $\Rightarrow m = 1259$  (根据给定的素数表找出满足要求的最小质数)。

# 本节小结

- 哈希表概念
- 哈希函数：确定关键字和存储位置对应关系的函数  $\text{Hash}(\text{key})$ ，其构造方法主要有直接定址法和除留余数法等。
- 处理冲突方法：当出现冲突时，为关键字寻找下一个存储位置的方法。主要方法有开放定址法和链地址法等。
- 哈希表的查找和性能分析。

8

谢谢观看

理论课程



廈門大學  
XIAMEN UNIVERSITY



信息学院 黄 焯  
(特色化示范性软件学院) 博士, 副教授  
School of Informatics Wei Huang

9

# 内部排序

理论课程



廈門大學  
XIAMEN UNIVERSITY



信息学院 黃 煒  
(特色化示范性软件学院) 博士, 副教授  
School of Informatics Wei Huang

# 内容要点

- 概述
- 基本排序
- 归并排序
- 快速排序
- 堆排序
- 计数排序
- 树形选择排序
- 基数排序



# 目录

|  |   |        |
|--|---|--------|
|  | 1 | 概述     |
|  | 2 | 基本排序   |
|  | 3 | 分治法排序  |
|  | 4 | 树形选择排序 |
|  | 5 | 非比较排序  |

# 排序的定义

- 对于数据表  $\mathcal{R} = \{r_1, r_2, \dots, r_n\}$ ，试确定  $1, 2, \dots, n$  的一种排列  $p_1, p_2, \dots, p_n$ ，使数据表  $\mathcal{R}$  满足非递减关系：

$$r_{p_1} \leq r_{p_2} \leq \dots \leq r_{p_n}$$

- 即数据表  $\{r_{p_1}, r_{p_2}, \dots, r_{p_n}\}$  递增有序。
- 对于数据表  $\mathcal{R}$ ，如果有  $r_i = r_j$  ( $i < j$ )，如果排序前  $r_i$  领先于  $r_j$ ，排序后  $r_i$  仍然领先于  $r_j$ ，则称该排序方法是稳定的。否则称该排序方法是不稳定的。

设  $\mathcal{R}$  排序前 (56, 34, 47, 23, 66, 18, 47)。

- ✓ 如果排序后得到 (18, 23, 34, 47, 47, 56, 66)，则称该排序方法是稳定的；
- ✓ 如果排序后得到 (18, 23, 34, 47, 47, 56, 66)，则称该排序方法是不稳定的。

# 排序的分类

## • 按策略流派分类

| 策略流派               | 包含的排序算法           | 核心思想（一句话概括）                               |
|--------------------|-------------------|-------------------------------------------|
| 插入/交换/选择<br>（朴素基础） | 基本排序（指冒泡、选择、插入）   | 利用相邻元素交换或不断选择极值，逐步缩小无序区间，时间复杂度 $O(n^2)$ 。 |
| 分治法                | 归并排序、快速排序         | 将大数组递归拆分为独立子问题，分别求解后再合并（归并），或先分区再递归（快排）。  |
| 树形选择<br>（选择排序的进化）  | 堆排序、树形选择排序（锦标赛排序） | 利用完全二叉树（堆）或锦标赛树的结构，快速找出极值，避免重复比较。         |
| 线性/分布排序<br>（非比较类）  | 计数排序、基数排序         | 不通过比较关键字大小，而是通过分类、计数或按位分配直接确定元素最终位置。      |

# 排序的分类

- 按是否基于比较分类（决定时间复杂度下界）

| 类别       | 包含算法                      | 时间复杂度下界                     | 核心特征                                   |
|----------|---------------------------|-----------------------------|----------------------------------------|
| 基于比较的排序  | 基本排序、归并排序、快速排序、堆排序、树形选择排序 | 存在下界 $\Omega(n\log n)$      | 只依赖元素间的大小比较（大于、小于、等于）来决策顺序。            |
| 非基于比较的排序 | 计数排序、基数排序                 | 可突破 $O(n\log n)$ ，达到 $O(n)$ | 依赖数据本身的数值范围（如整数范围 $k$ ）或位数（如 $d$ 位数字）。 |

# 排序的分类

## • 按“时间复杂度特性”分类（工程选型速查）

| 时间复杂度          | 包含算法                     | 备注                                    |
|----------------|--------------------------|---------------------------------------|
| $O(n^2)$       | 基本排序                     | 数据量小（ $< 1000$ ）时可用，简单且稳定性可控。         |
| $O(n\log n)$   | 归并排序、快速排序（平均）、堆排序、树形选择排序 | 工业界主力。快排最快（常数因子小），归并稳定，堆排序最省内存空间（原地）。 |
| $O(n)$<br>（线性） | 计数排序、基数排序                | 有苛刻前提：数据必须是整数且范围有限（计数），或位数已知（基数）。     |

# 目录

|  |   |        |
|--|---|--------|
|  | 1 | 概述     |
|  | 2 | 基本排序   |
|  | 3 | 分治法排序  |
|  | 4 | 树形选择排序 |
|  | 5 | 非比较排序  |

# 比较式排序

- 两种基本操作：
  - 比较两个关键字的大小
  - 交换两个记录的位置
- 两类数据区：

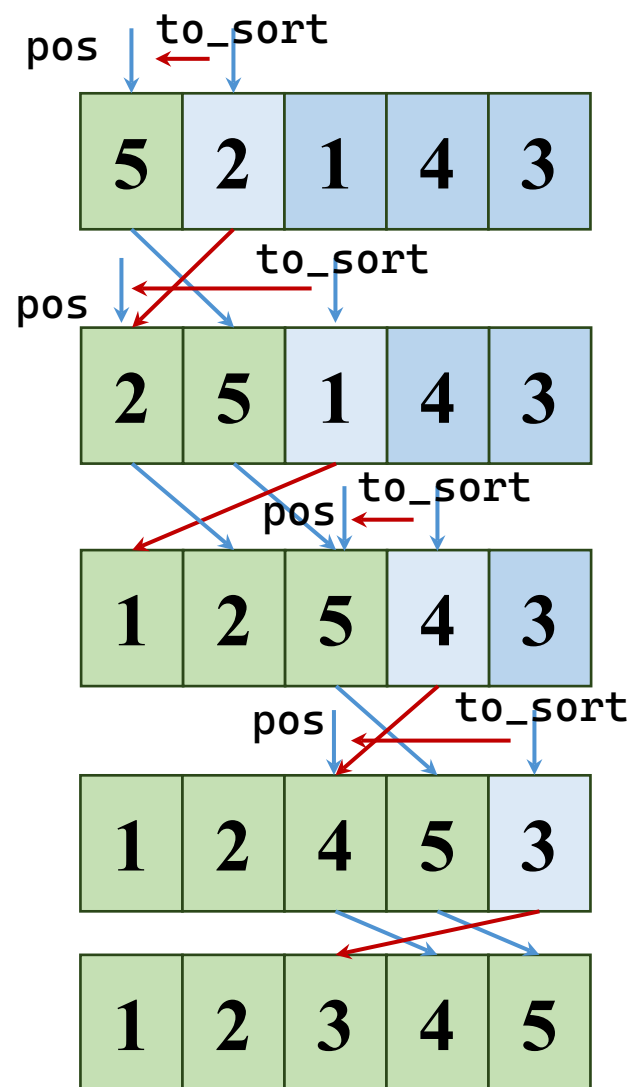


# 插入排序

- 算法

- 依次处理无序区首元素
- 从后往前找到位置
  - 越界或首次小于关键字的位置
- 有序区每个元素后移
- 关键字复制到指定位置

- 依次从无序区取首元素，在有序区寻找插入位置的方法，称插入排序





# 插入排序

- 算法

- 依次处理无序区首元素

```
for (i = 1; i < count; ++i) {
```

- 从后往前找到位置

```
int value = array[i], p = i - 1;
```

- 越界或首次小于关键字的位置

```
while (p >= 0 && array[p] > value) {
```

- 有序区每个元素后移

```
array[p + 1] = array[p];
--p;
```

- 关键字复制到指定位置

```
}
```

```
array[p + 1] = value;
```

```
}
```

```
void insertionSort(int array[], int count) {
 int index;

 for (index = 1; index < count; ++index) {
 int value = array[index];
 int position = index - 1;

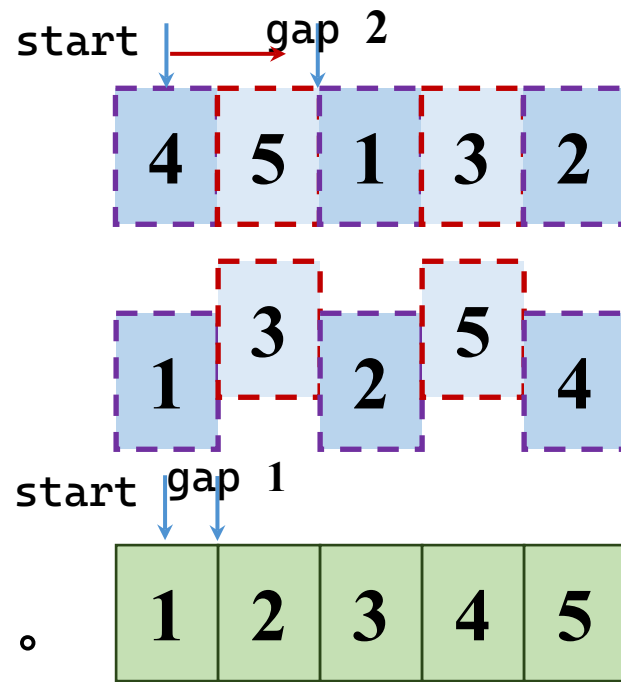
 while (position >= 0 && array[position] > value) {
 array[position + 1] = array[position];
 --position;
 }
 array[position + 1] = value;
 }
}
```

# 插入排序的时间分析

- 最好情况：关键字在数据表中有序
  - 比较次数： $n-1$
  - 移动次数： $0$
- 最坏情况：关键字在数据表中逆序有序
  - 比较次数： $\sum_{i=2}^n i = \frac{(n+2)(n-1)}{2}$
  - 移动次数： $\sum_{i=2}^n (i+1) = \frac{(n+4)(n-1)}{2}$

# 希尔排序

- 希尔 ( Shell ) 排序
  - 又称缩小增量排序
- 基本思想：
  - 将整个数据表分成 $m$ 个子序列，对每个子序列分别进行插入排序。
  - 逐步减少子序列数 $m$ ，直至 $m=1$ 为止。
- 时间复杂度： $O(n^2/m)$



# 希尔排序

```
for (gap = count / 2; gap > 0; gap /= 2) {
 for (int start = 0; start < gap; ++start) {
```

start + gap

i += gap

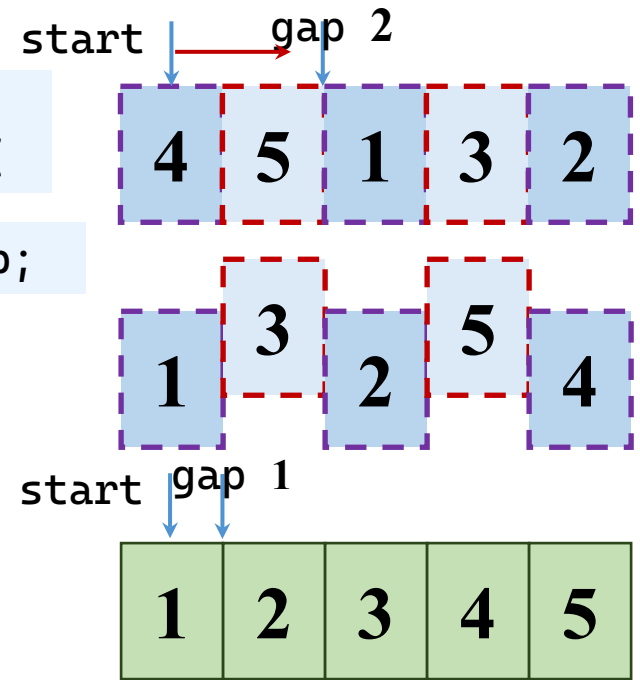
p = i - gap;

```
for (i = 1; i < count; ++i) {
 int value = array[i];
 int p = index - 1;
 while (p >= 0 && array[p] > value) {
 array[p + 1] = array[p];
 --p;
 }
 array[p + 1] = value;
}
```

array[p + gap] = value;

p -= gap;

```
}
}
```



array[p + gap] = array[p];

# 选择排序

- 算法；直接选数不插入

- 外层循环控制无序区的范围

```
for (begin = 0; begin < count - 1; ++begin) {
```

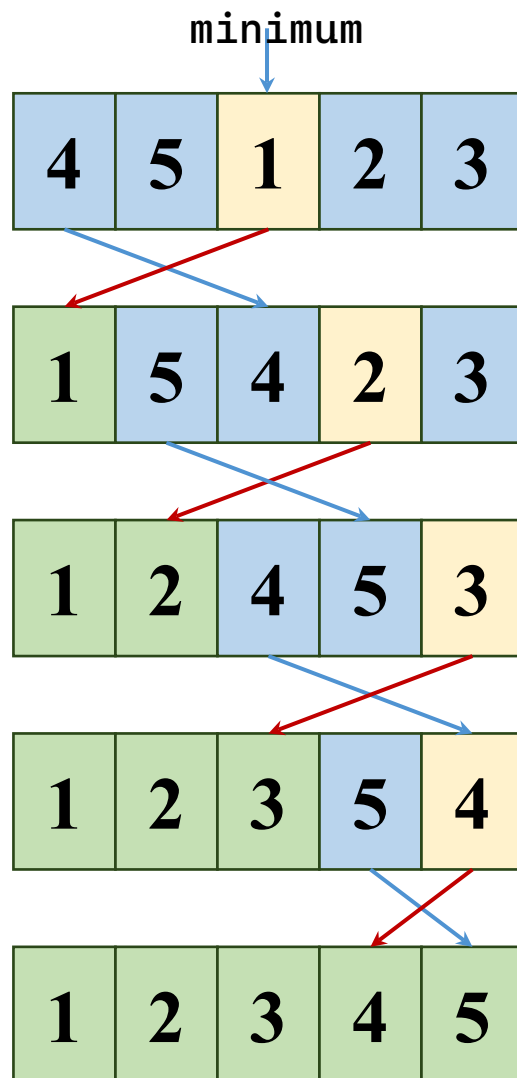
- 内层循环取得将最小值

```
 int minimum = begin, i;
 for (i = begin + 1; i < count; ++i) {
 if (array[i] < array[minimum])
 minimum = i;
```

- 与无序区首位置换

```
 if (minimum != begin)
 swap(&array[begin], &array[minimum]);
```

```
}
```



```

static void swap(int* left, int* right) {
 int temporary = *left;
 *left = *right;
 *right = temporary;
}

void selectionSort(int array[], int count) {
 int begin;
 for (begin = 0; begin < count - 1; ++begin) {
 int minimum = begin;
 int index;
 for (index = begin + 1; index < count; ++index) {
 if (array[index] < array[minimum]) {
 minimum = index;
 }
 }
 if (minimum != begin) {
 swap(&array[begin], &array[minimum]);
 }
 }
}

```

# 冒泡排序

- 算法：把轻的泡泡浮上来

- 外层循环控制无序区的范围

```
for (unsorted = count; unsorted > 1; --unsorted) {
```

- 内层循环比较轻重

```
 for (i = 1; i < unsorted; ++i) {
```

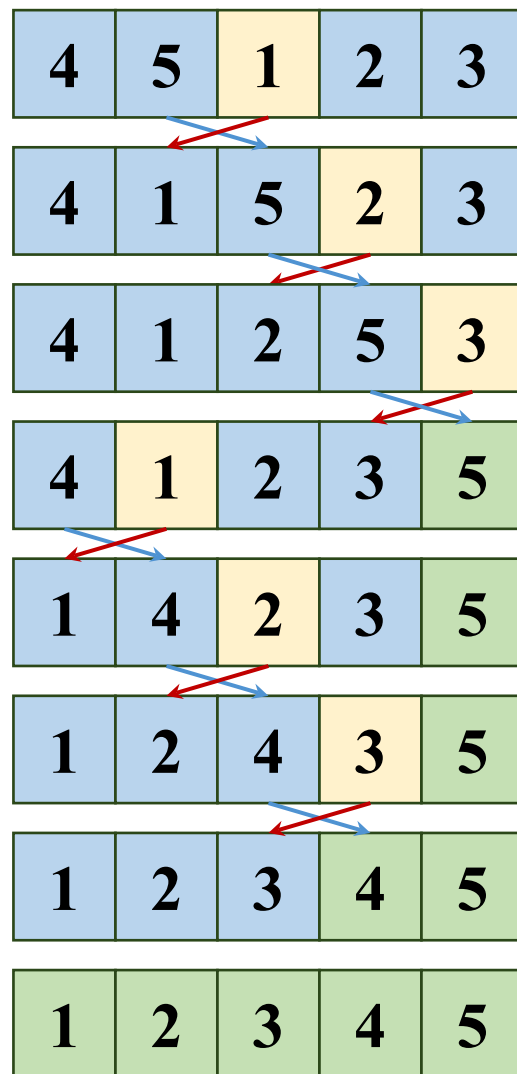
- 不对就交换

```
 if (array[i - 1] > array[i]) {
 swap(&array[i - 1], &array[i]);
```

```
 }
 }
```

- 加速

- 如果某一层没有交换，则算法已完成





```
void bubbleSort(int array[], int count) {
 int unsorted;

 for (unsorted = count; unsorted > 1; --unsorted) {
 int index;
 bool changed = false;

 for (index = 1; index < unsorted; ++index) {
 if (array[index - 1] > array[index]) {
 swap(&array[index - 1], &array[index]);
 changed = true;
 }
 }
 if (!changed) {
 return;
 }
 }
}
```

# 冒泡排序：性能分析

- 最好情况(关键字在数据表中递增有序)：

- 只需进行一趟冒泡。

- 比较的次数： $n-1$                       移动的次数：0

- 最坏情况(关键字在数据表中递减有序)：

- 需进行 $n-1$ 趟冒泡。

- 比较的次数：                                      移动的次数：

$$\sum_{i=1}^{n-1} (i-1) = \frac{n(n-1)}{2}$$

$$3 \cdot \sum_{i=1}^{n-1} (i-1) = \frac{3n(n-1)}{2}$$

```

void bidirectionalBubbleSort(int array[], int count) {
 int left = 0, right = count - 1;
 while (left < right) {
 int lastForward = left, lastBackward = right;
 for (int j = left; j < right; ++j) {
 /* 从左向右扫描, 将较大的数往后冒泡 */
 if (array[j] > array[j + 1]) {
 swap(&array[j], &array[j + 1]);
 lastForward = j;
 }
 /* 从右向左扫描, 将较小的数往前冒泡 */
 int k = right + left - j; /* 对称位置的索引 */
 if (array[k] < array[k - 1]) {
 swap(&array[k], &array[k - 1]);
 lastBackward = k;
 }
 }
 /* 收缩无序区间 */
 right = lastForward; left = lastBackward;
 }
}

```

# 地精排序

- 算法

- 遍历数组

```
while (pos < count) {
```

- 如果有序区未发现乱序

- 只有一个元素

```
if (pos == 0{
```

- 顺序是对的

```
|| array[pos] >= array[pos - 1])
```

- 则前进

```
++pos;
```

- 否则交换

```
else {
 swap(&array[pos], &array[pos - 1]);
```

- 有序区后撤

```
--pos;
```

```
}
```

```
}
```

```
void gnomeSort(int array[], int count) {
 int pos = 0;

 while (pos < count) {
 if (pos == 0 || array[pos] >= array[pos - 1]) {
 ++pos;
 } else {
 swap(&array[pos], &array[pos - 1]);
 --pos;
 }
 }
}
```

# 本节小结

- 基本排序：

- 插入排序， $O(n^2)$
- 希尔排序，时间复杂度优于插入排序
- 选择排序， $O(n^2)$
- 冒泡排序， $O(n^2)$ ，实际效率可能较高
- 地精排序， $O(n^2)$ ，单重循环的排序算法

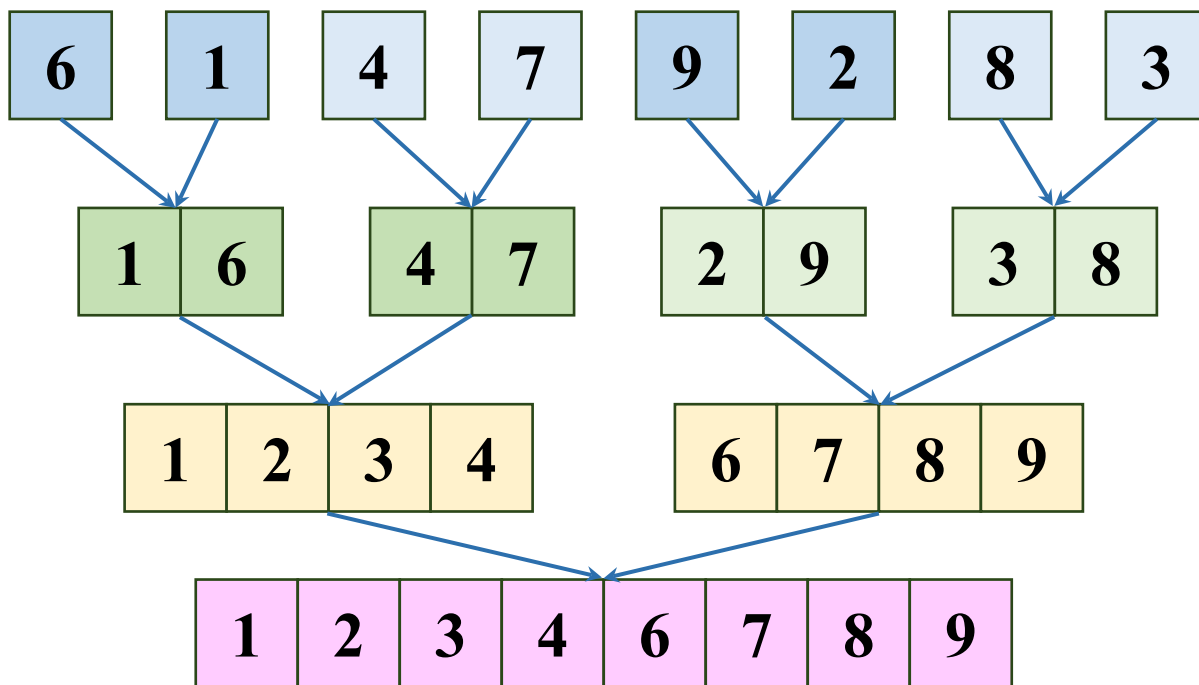
# 目录

|  |     |        |
|--|-----|--------|
|  | 3   | 分治法排序  |
|  | 3.1 | 归并排序   |
|  | 3.2 | 快速排序   |
|  | 4   | 树形选择排序 |
|  | 5   | 非比较排序  |

# 归并排序

- 归并排序

- 通过将两个(2路)或两个以上的有序(子)序列合并为一个有序序列的方法，对序列进行排序。

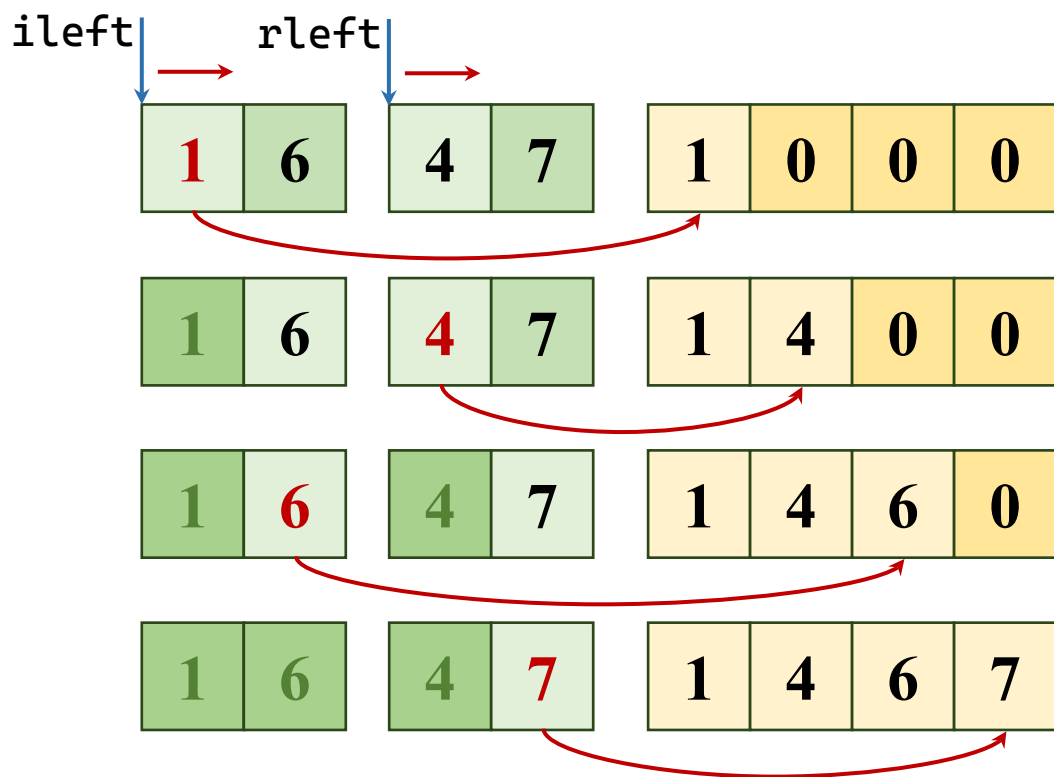




# 归并排序

## • 合并算法

- 左右半边从左到右遍历
- 当前位置左边小，存左
- 当前位置右边小，存右
- 左边完了全存右
- 右边完了全存左



- 为了简化，前面存在临时数组，后续复制回去

# 归并排序

## • 合并算法

– 左右半边从左到右遍历

– 当前位置左边小，存左

– 当前位置右边小，存右

– 左边完了全存右

– 右边完了全存左

– 为了简化，前面存在临时数组，后续复制回去

```
int ileft = left, iright = mid + 1, target = left;
```

```
while (ileft <= mid && iright <= right)
```

```
 if (array[ileft] <= array[iright])
 aux[target++] = array[ileft++];
 else
 aux[target++] = array[iright++];
```

```
while (ileft <= middle)
 auxiliary[target++] = array[ileft++];
```

```
while (iright <= middle)
 auxiliary[target++] = array[iright++];
```

```
for (target = left; target <= right; ++target)
 array[target] = auxiliary[target];
```

# 归并排序

- 总体算法

- 递归，每一层都有
- 先排序左半部，再排序右半部，最后合并两半部分

```
void mergeSortRange(int array[], int aux[], int left, int right) {
 int middle;
 if (left >= right)
 return;
 middle = left + (right - left) / 2;
 mergeSortRange(array, auxiliary, left, middle);
 mergeSortRange(array, auxiliary, middle + 1, right);
 merge(array, auxiliary, left, middle, right);
}
```

- 排序接口

```
mergeSortRange(array, auxiliary, 0, count - 1);
```

```
void merge(int array[], int auxiliary[], int left, int middle,
 int right) {
 int leftIndex = left, rightIndex = middle+1, target = left;
 while (leftIndex <= middle && rightIndex <= right)
 if (array[leftIndex] <= array[rightIndex])
 auxiliary[target++] = array[leftIndex++];
 else
 auxiliary[target++] = array[rightIndex++];
 while (leftIndex <= middle)
 auxiliary[target++] = array[leftIndex++];
 while (rightIndex <= right)
 auxiliary[target++] = array[rightIndex++];
 for (target = left; target <= right; ++target)
 array[target] = auxiliary[target];
}
```

```
void mergeSort(int array[], int count) {
 int* auxiliary;

 if (count < 2) {
 return;
 }
 auxiliary = (int*)malloc((size_t)count * sizeof(int));
 if (auxiliary == NULL) {
 perror("无法分配归并排序辅助数组");
 exit(EXIT_FAILURE);
 }
 mergeSortRange(array, auxiliary, 0, count - 1);
 free(auxiliary);
}
```

# 目录

|  |     |        |
|--|-----|--------|
|  | 3   | 分治法排序  |
|  | 3.1 | 归并排序   |
|  | 3.2 | 快速排序   |
|  | 4   | 树形选择排序 |
|  | 5   | 非比较排序  |

# 快速排序

- 基本思想

- 通过一趟快速排序将待排数据分割成独立的左中右三个部分，其中，中部分只含一个数，左边部分的所有关键字都比这个数小，右边部分的所有关键字都比这个数大。
- 然后再按此方法对这两部分记录分别进行快速排序，直到所有记录整理成有序序列。

- 枢轴的选取

- 选取末尾元素实现更简洁，选取首元素或随机位置也可以

# 快速排序

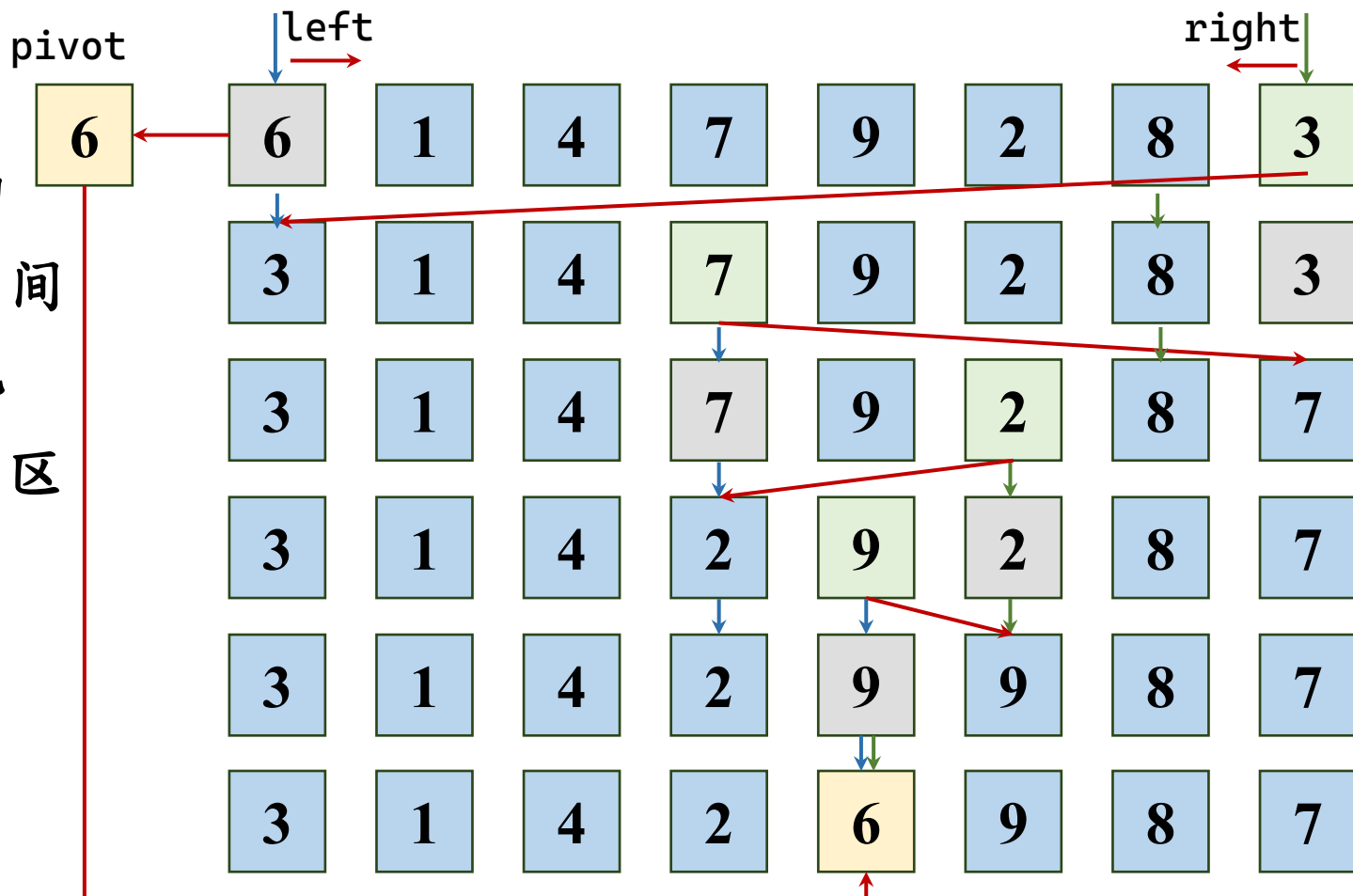
- 一趟排序

- 选出枢轴

- 两边往中间

- 收缩填坑

- 分出大小区





# 快速排序

## • 一趟排序算法

– 选出枢轴

```
int pivot = array[left]; // 枢轴值, 变成第一个坑
int low = left, high = right;
```

– 往中间收缩直到分出大小区

```
while (low < high) {
```

– 右线收缩找到第一个错

```
while (low < high && array[high] >= pivot)
 high--;
```

– 填坑

```
array[low] = array[high];
```

– 左线收缩找到第一个错

```
while (low < high && array[low] <= pivot)
 low++;
```

– 填坑

```
array[high] = array[low];
```

– 区分完毕，枢轴回填

```
}
```

```
array[low] = pivot;
```

– 返回枢轴的下标

```
return low;
```

# 快速排序

- 算法

```
int pivotIndex = partition(array, left, right);
```

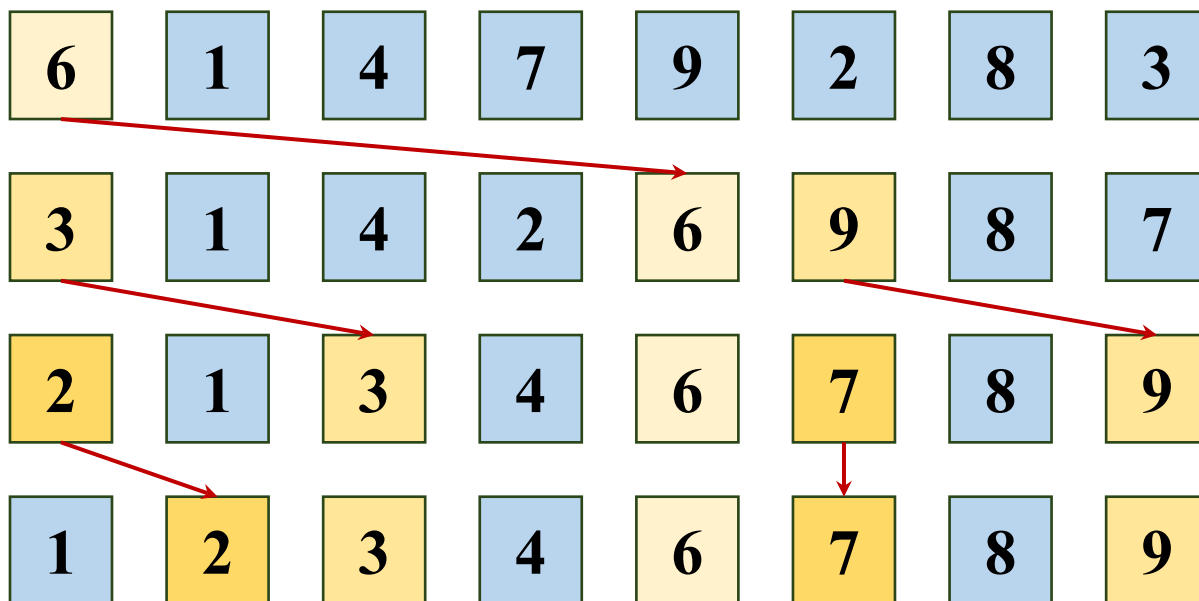
- 选出枢轴，将数组分为左右区

- 左区排序

```
quickSortRange(array, left, pivotIndex - 1);
```

- 右区排序

```
quickSortRange(array, pivotIndex + 1, right);
```



```

int partition(int array[], int left, int right) {
 int pivot = array[left]; // 枢轴值, arr[left] 变成第一个坑
 int low = left, high = right;
 while (low < high) {
 while (low < high && array[high] >= pivot)
 high--;
 array[low] = array[high]; // 右边小值填入左坑, high变成新坑
 // 从左向右找第一个 > key 的值, 填到右边的坑
 while (low < high && array[low] <= pivot)
 low++;
 array[high] = array[low]; // 左边大值填入右坑, low 变成新坑
 }
 // low == high, 这就是枢轴最终位置
 array[low] = pivot;
 return low;
}

```

```
void quickSortRange(int array[], int left, int right) {
 if (left < right) {
 int pivotIndex = partition(array, left, right);
 quickSortRange(array, left, pivotIndex - 1);
 quickSortRange(array, pivotIndex + 1, right);
 }
}

void quickSort(int array[], int count) {
 if (count > 1) {
 quickSortRange(array, 0, count - 1);
 }
}
```

# 快速排序

- 假设一次划分所得枢轴位置  $i=k$ ，则对  $n$  个记录进行快速排序所需时间为

$$T(n) = T_{\text{pass}}(n) + T(k-1) + T(n-k)$$

- 若待排序列中的关键字随机分布，则  $k$  取 1 至  $n$  中任意一个值是等概率的。

# 快速排序

- 由此可得快速排序所需时间的平均值为：

$$T(n) = cn + \frac{1}{n} \sum_{k=1}^n (T(k-1) + T(n-k))$$

$$T(n) = cn + \frac{2}{n} \sum_{k=0}^{n-1} T(k) \quad T(n-1) = c(n-1) + \frac{2}{n-1} \sum_{k=0}^{n-2} T(k)$$

$$T(n) = \frac{2n-1}{n} c + \frac{n+1}{n} T(n-1)$$

$$T(n) < \frac{n+1}{n} T(1) + 2(n+1) \left( \frac{1}{2} + \cdots + \frac{1}{n+1} \right) c$$

$$T(n) \leq \frac{n+1}{n} b + 2c(n+1) \ln(n+1) \leq 2d(n+1) \ln(n+1)$$

# 快速排序

- 若待排记录的初始状态按关键字有序，快速排序将蜕化为冒泡排序，其时间复杂度为 $O(n^2)$ 。
- 为避免出现这种情况，需在进行一次划分之前，进行“预处理”。例如，
- 先对 $L(s).key$ ,  $L(t).key$ 和 $L[(s+t)/2].key$ 进行相互比较，然后取关键字为“三者之中”的记录作为枢轴记录。

# 目录

|  |     |        |
|--|-----|--------|
|  | 3   | 分治法排序  |
|  | 4   | 树形选择排序 |
|  | 4.1 | 堆排序    |
|  | 4.2 | 树形选择排序 |
|  | 5   | 非比较排序  |



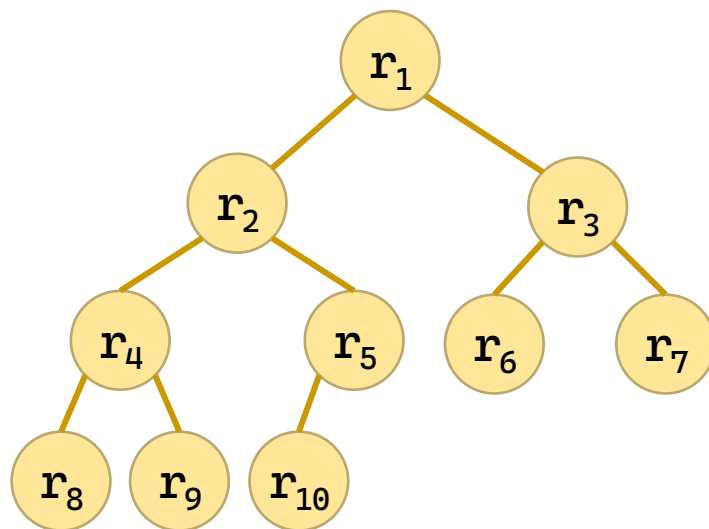
# 堆排序

- 堆是一棵完全二叉树。

- 结点 $i$ 的孩子编号为 $2i$ 和 $2i+1$ ，父亲编号 $\lfloor i/2 \rfloor$
- 最大分支编号为 $n/2$ 。
- 树高为 $\lfloor \log_2 n \rfloor + 1$ 。

- 小顶堆和大顶堆

- 小顶堆  $r_i \leq r_{2i}, r_i \leq r_{2i+1}$
- 大顶堆  $r_i \geq r_{2i}, r_i \geq r_{2i+1}$

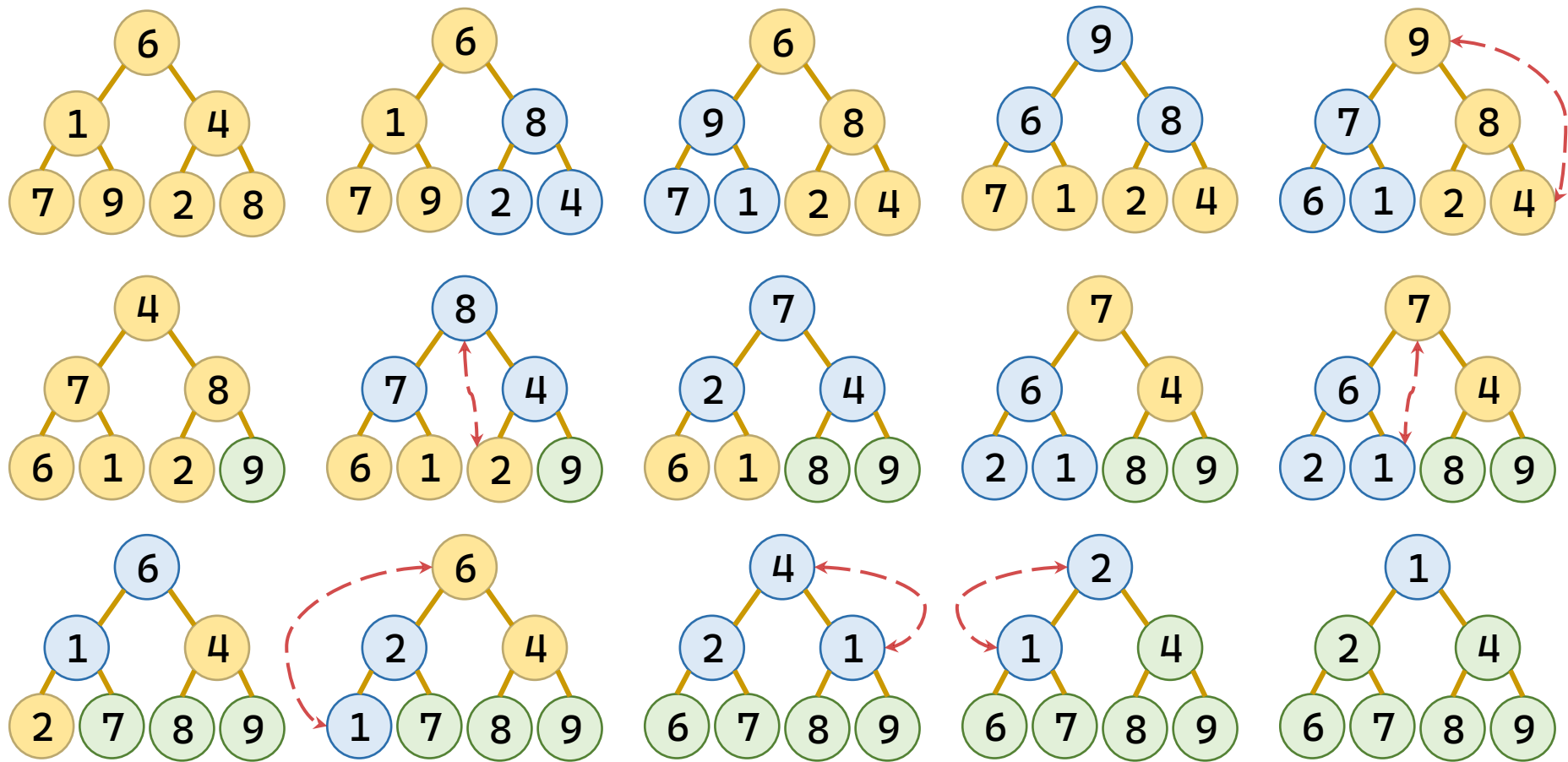


小顶堆: {12, 36, 27, 65, 40, 34, 98, 81, 73, 55}

不是堆: {12, 36, 27, 65, 40, 14, 98, 81, 73, 55}

# 堆排序

- 保持大顶堆（下滤操作）+提取最大值



# 堆排序

## • 保持大顶堆（下滤操作）算法

– 自顶向下

– 找到三角的叶子最大值

▪ 如果有右子，比较左右

– 如果父子合规，停止

– 交换父子

– 向下过滤

```
while (root * 2 + 1 < count) {
```

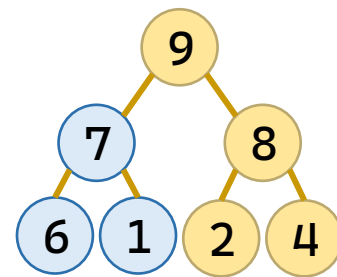
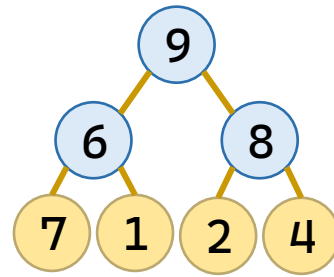
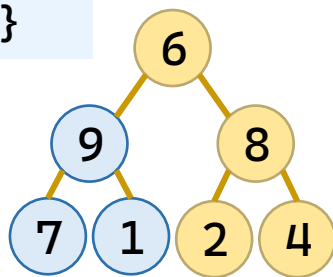
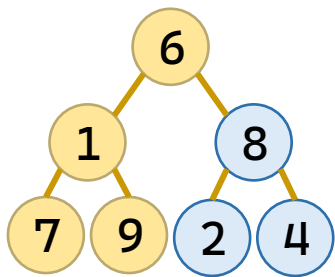
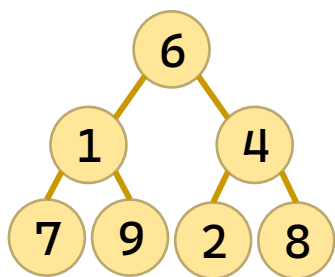
```
 int child = root * 2 + 1;
 if (child + 1 < count
 && array[child] < array[child + 1])
 ++child;
```

```
 if (array[root] >= array[child])
 return;
```

```
 swap(&array[root], &array[child]);
```

```
 root = child;
```

```
}
```



# 堆排序

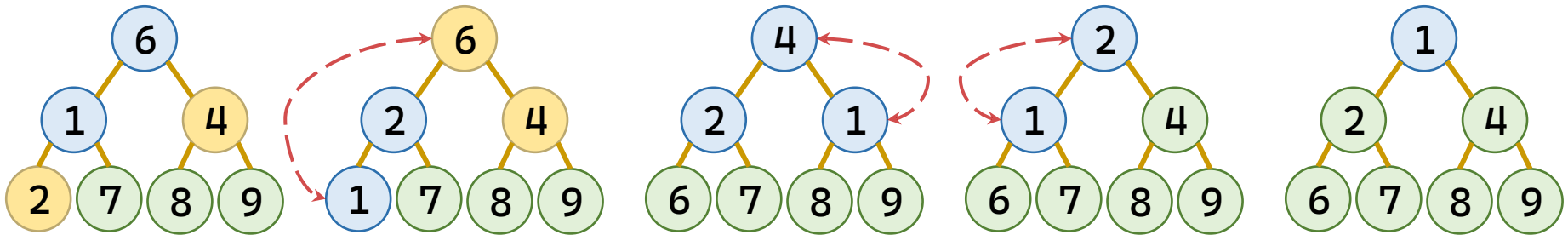
- 算法

- 第一趟：非叶子自底向上，依次下滤

```
for (index = count / 2 - 1; index >= 0; --index)
 siftDown(array, index, count);
```

- 第二趟：提取最顶（大）值，继续下滤

```
for (index = count - 1; index > 0; --index) {
 swap(&array[0], &array[index]);
 siftDown(array, 0, index);
}
```



```

void siftDown(int array[], int root, int count) {
 while (root * 2 + 1 < count) {
 int child = root * 2 + 1;
 if (child + 1 < count && array[child] < array[child+1])
 ++child;
 if (array[root] >= array[child])
 return;
 swap(&array[root], &array[child]);
 root = child;
 }
}

void heapSort(int array[], int count) {
 int index;
 for (index = count / 2 - 1; index >= 0; --index)
 siftDown(array, index, count);
 for (index = count - 1; index > 0; --index) {
 swap(&array[0], &array[index]);
 siftDown(array, 0, index);
 }
}

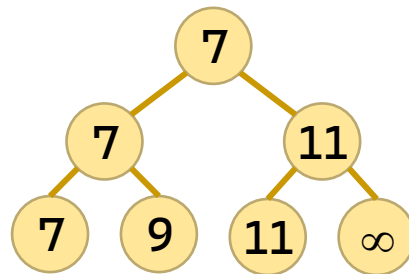
```

# 目录

|  |     |        |
|--|-----|--------|
|  | 3   | 分治法排序  |
|  | 4   | 树形选择排序 |
|  | 4.1 | 堆排序    |
|  | 4.2 | 树形选择排序 |
|  | 5   | 非比较排序  |

# 树形选择排序

- 树形选择排序又称锦标赛排序，是一种按照锦标赛的思想进行选择排序的方法。首先对 $n$ 个记录的关键字进行两两比较，然后在 $n/2$ 个较小者之间再进行两两比较，如此重复，直至选出最小的记录为止。
- 树形选择排序的思路来自于锦标赛/淘汰赛/选拔赛。
- 树形选择排序是一种在“完全二叉树”上完成的排序算法。
- 难点：如何生成和保存这棵完全二叉树？

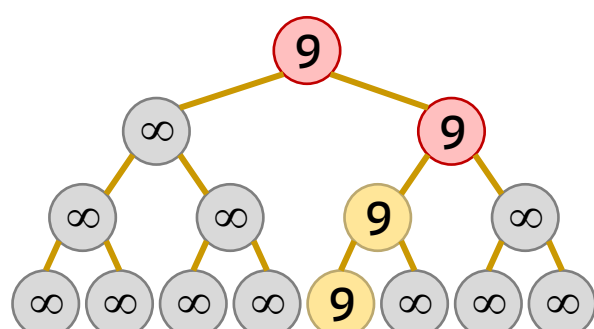
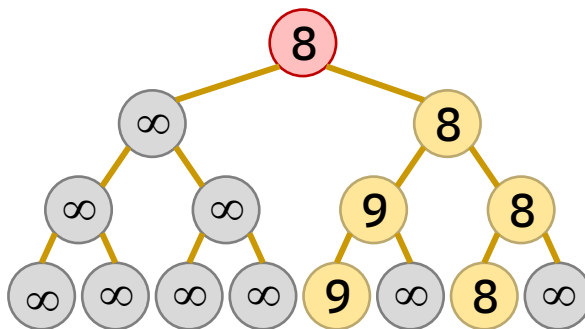
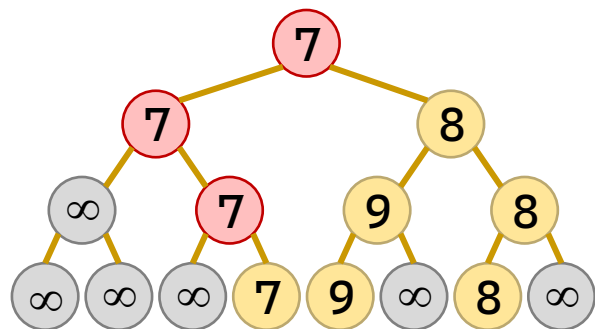
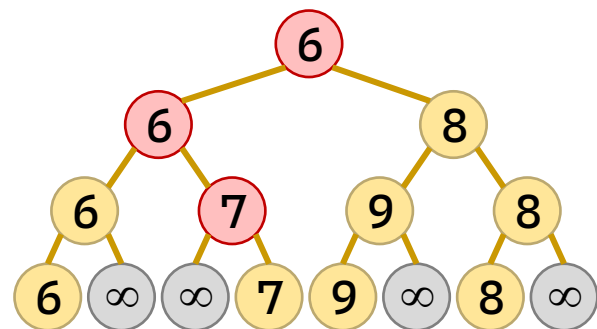
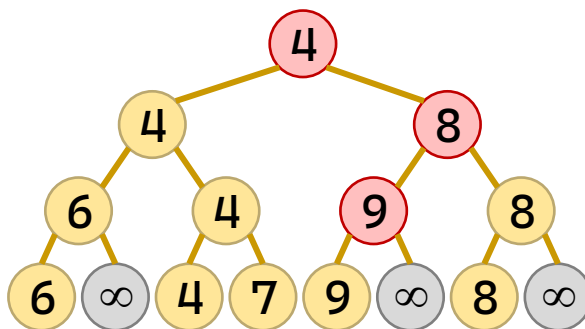
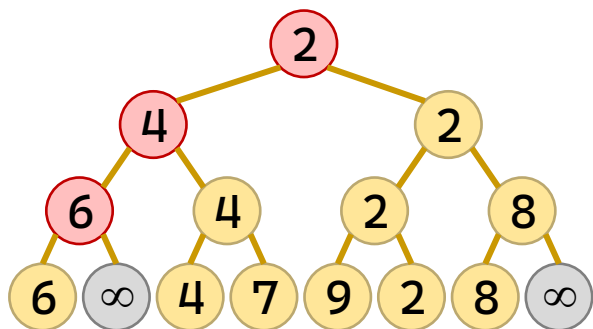
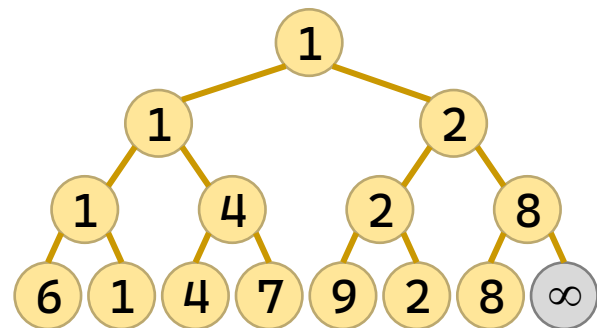


# 树形选择排序

## • 算法

– 建树

– 提取根（最小值），维护树





# 树形选择排序

- 算法：建树

- 计算叶子数

```
int leafCount = 1;
while (leafCount < count)
 leafCount *= 2;
```

- 开辟空间

```
tree = (int*)malloc((size_t)(leafCount * 2) * sizeof(int));
candidates = (int*)malloc((size_t)leafCount * sizeof(int));
if (tree == NULL || candidates == NULL) {
 free(tree); free(candidates);
 perror("无法分配胜者树"); exit(EXIT_FAILURE);
}
```

- 叶子赋值

```
for (index = 0; index < leafCount; ++index) {
 candidates[index] = index < count ? array[index] : INT_MAX;
 tree[leafCount + index] = index;
}
```

- 父亲计算

```
for (index = leafCount - 1; index > 0; --index) {
 int left = tree[index * 2];
 int right = tree[index * 2 + 1];
 tree[index] = candidates[left] <= candidates[right] ? left : right;
}
```

# 树形选择排序

### • 算法：提取根

## — 提取根结点

## — 从索引删去根结点

### - 计算根结点对应的叶子

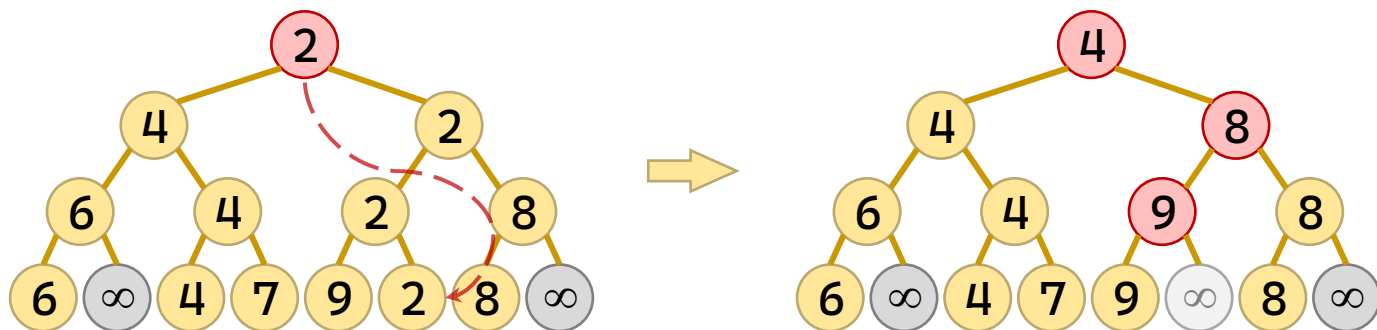
## — 不断更新父亲

```
for (iOut = 0; iOut < count; ++iOut) {
 int winner = tree[1];
 array[outputIndex] = candidates[winner];
```

```
candidates[winner] = INT_MAX;
```

```
node = (leafCount + winner) / 2;
```

```
while (node > 0) {
 int l = tree[node*2], r = tree[node*2+1];
 tree[node] = cand[l] <= cand[r] ? l : r;
 node /= 2;
}
```



```

void tournamentSort(int array[], int count) {
 int leafCount = 1, outIndex, index;
 int* tree, * candidates;
 while (leafCount < count)
 leafCount *= 2;
 tree = (int*)malloc((size_t)(leafCount * 2) * sizeof(int));
 candidates = (int*)malloc((size_t)leafCount * sizeof(int));
 memset(tree, 0, (size_t)(leafCount * 2) * sizeof(int));
 memset(candidates, 0, (size_t)leafCount * sizeof(int));
 if (tree == NULL || candidates == NULL) {
 free(tree);
 free(candidates);
 perror("无法分配胜者树");
 exit(EXIT_FAILURE);
 }
 for (index = 0; index < leafCount; ++index) {
 candidates[index] = index < count ? array[index] : INT_MAX;
 tree[leafCount + index] = index;
 }
 for (index = leafCount - 1; index > 0; --index) {
 int left = tree[index * 2];
 int right = tree[index * 2 + 1];
 }
}

```

```

 tree[index] = candidates[left] <= candidates[right] ?
left : right;
 }
 for (outIndex = 0; outIndex < count; ++outIndex) {
 int winner = tree[1], node;
 array[outIndex] = candidates[winner];
 candidates[winner] = INT_MAX;
 node = (leafCount + winner) / 2;
 while (node > 0) {
 int left = tree[node*2], right = tree[node*2+1];
 tree[node] = candidates[left] <= candidates[right]
 ? left : right;
 node /= 2;
 }
 }
 free(tree);
 free(candidates);
}

```

# 目录

|  |     |        |
|--|-----|--------|
|  | 3   | 分治法排序  |
|  | 4   | 树形选择排序 |
|  | 5   | 非比较排序  |
|  | 5.1 | 计数排序   |
|  | 5.2 | 基数排序   |

# 计数排序

- 计数排序是一种计算式算法，要求数据元素为整型。
- 设数据元素(整数)的个数为  $n$ ，数据元素最大值和最小值的差值为  $m-1$ 。
- 如果  $m < n \log_2 n$ ，计数排序算法的时间复杂度小于比较式排序算法。

# 计数排序

## • 算法

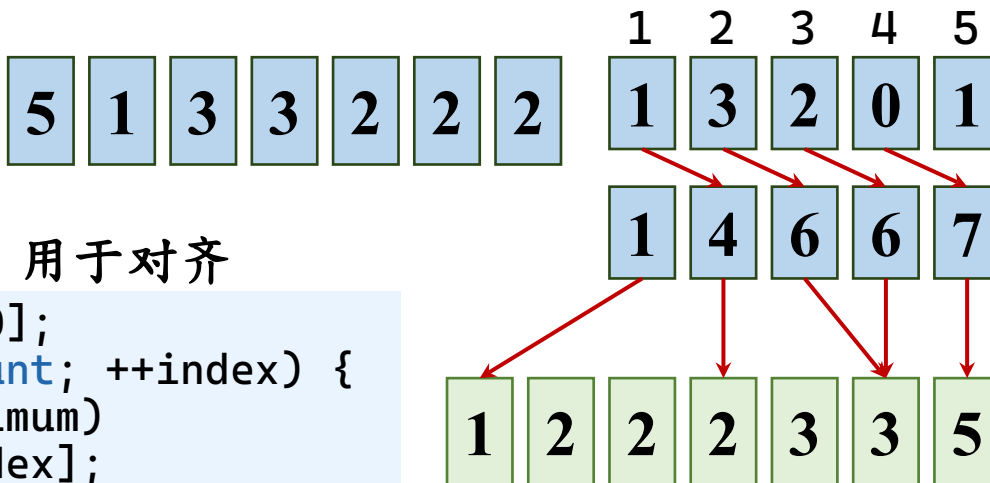
### – 预备工作 ( 对齐 )

- 算出最大最小值范围，用于对齐

```
minimum = maximum = array[0];
for (index = 1; index < count; ++index) {
 if (array[index] < minimum)
 minimum = array[index];
 if (array[index] > maximum)
 maximum = array[index];
}
range = maximum - minimum + 1;
```

- 开辟空间

```
counts = (int*)calloc((size_t)range, sizeof(int));
output = (int*)malloc((size_t)count * sizeof(int));
if (counts == NULL || output == NULL) {
 free(counts); free(output);
 perror("无法分配计数排序辅助数组");
 exit(EXIT_FAILURE);
}
```



# 计数排序

## • 算法

### – 计算统计直方图

```
for (index = 0; index < count; ++index)
 ++counts[array[index] - minimum];
```

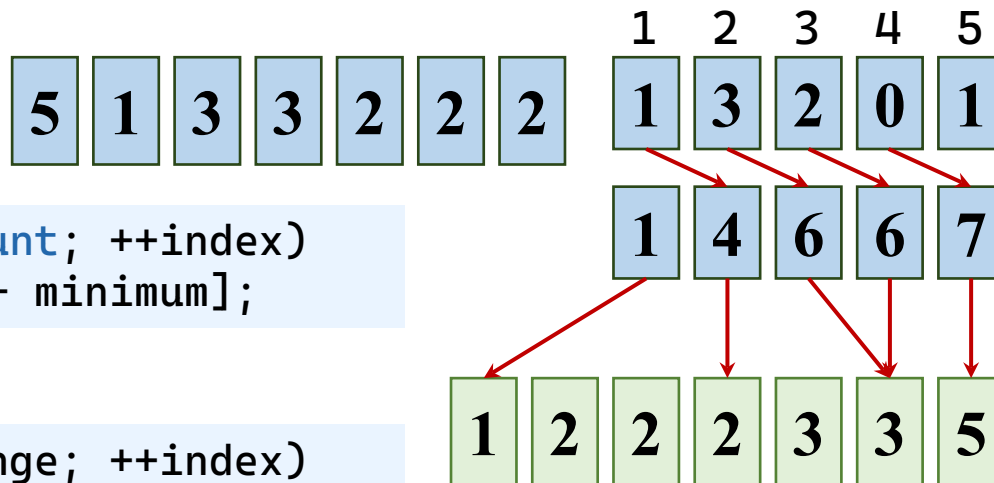
### – 计算累计计数直方图

```
for (index = 1; index < range; ++index)
 counts[index] += counts[index - 1];
```

### – 填入输出位置

- 数组：counts[]，偏移量：array[index] - minimum
- 填一个就回退一格

```
for (index = count - 1; index >= 0; --index)
 output[--counts[array[index] - minimum]] = array[index];
memcpy(array, output, (size_t)count * sizeof(int));
```





```
void countingSort(int array[], int count) {
 int minimum, maximum, range, index;
 int* counts, * output;
 if (count < 2)
 return;
 minimum = maximum = array[0];
 for (index = 1; index < count; ++index) {
 if (array[index] < minimum)
 minimum = array[index];
 if (array[index] > maximum)
 maximum = array[index];
 }
 range = maximum - minimum + 1;
 counts = (int*)calloc((size_t)range, sizeof(int));
 output = (int*)malloc((size_t)count * sizeof(int));
 if (counts == NULL || output == NULL) {
 free(counts);
 free(output);
 perror("无法分配计数排序辅助数组");
 exit(EXIT_FAILURE);
 }
}
```

```
for (index = 0; index < count; ++index)
 ++counts[array[index] - minimum];
for (index = 1; index < range; ++index)
 counts[index] += counts[index - 1];
for (index = count - 1; index >= 0; --index)
 output[--counts[array[index] - minimum]] = array[index];
memcpy(array, output, (size_t)count * sizeof(int));
free(counts);
free(output);
}
```

# 目录

|  |     |        |
|--|-----|--------|
|  | 3   | 分治法排序  |
|  | 4   | 树形选择排序 |
|  | 5   | 非比较排序  |
|  | 5.1 | 计数排序   |
|  | 5.2 | 基数排序   |

# 基数排序

- 假如多关键字的记录序列中，每个关键字的取值范围相同(如十进制数)，则按 LSD (Least Significant Digit first，最低位优先法) 法进行排序时，可以采用“分配-收集”法，该方法不需要进行关键字之间的比较。
- 基数排序方法的基本思路是，将单关键字看成是由多个数位(或多个字符)构成的多关键字，并采用“分配-收集”的方法进行排序。

# 链式基数排序

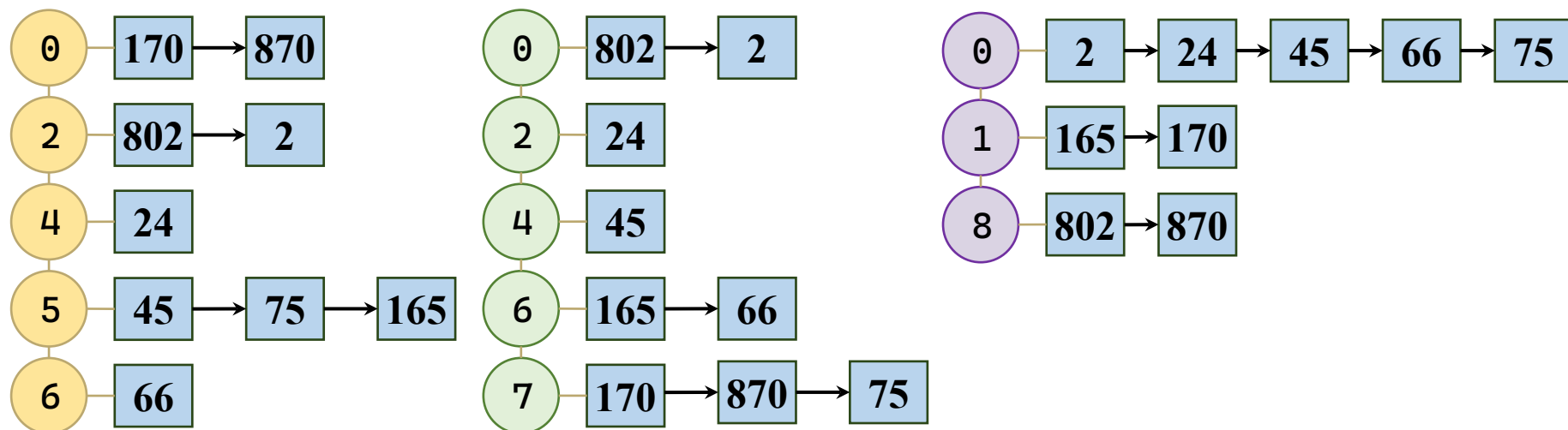
- 基数排序一般采用链表存储结构，即链式基数排序。  
操作：
  - (1)将待排记录建成一个链表；
  - (2)分配时，将当前关键字位值相同的记录分配到同一个链队列中；
  - (3)收集时，按当前关键字位取值从小到大顺序将各链队列链成一个链表；
  - (4)对每个关键字位重复(2)和(3)两步。

# 链式基数排序

## • 算法

|     |    |    |     |     |    |   |    |     |
|-----|----|----|-----|-----|----|---|----|-----|
| 170 | 45 | 75 | 870 | 802 | 24 | 2 | 66 | 165 |
|-----|----|----|-----|-----|----|---|----|-----|

- 按照数位遍历，把各数位的值相同的节点串联起来
- 第一遍，个位数小的到在前；第二遍，十位数小的排到前；第三遍，百位数小的排到前，十位数、个位数次序不变



# 链式基数排序

- 算法

- 按数位从小到大排序
- 初始化
- 遍历链表
- 找到对应的桶  
进行尾插
- 将桶按顺序接续起来

```
int max = getMaxValue(*head);
for (exp = 1; max / exp > 0; exp *= 10) {
```

```
RadixNode* bucketHead[10] = { NULL };
RadixNode* bucketTail[10] = { NULL };
RadixNode* current = *head;
```

```
while (current != NULL) {
 int digit = (current->data / exp) % 10;
 RadixNode* nextNode = current->next;
```

```
 current->next = NULL;
 if (bucketTail[digit] == NULL) {
 bucketHead[digit] = current;
 bucketTail[digit] = current;
 } else {
 bucketTail[digit]->next = current;
 bucketTail[digit] = current;
 }
}
```

```
 current = nextNode;
```

```
}
```

# 链式基数排序

- 算法

- 按数位从小到大排序
- 初始化
- 遍历链表
- 找到对应的桶  
进行尾插
- 将桶按顺序接续起来
- 更新头指针

```
int max = getMaxValue(*head);
for (exp = 1; max / exp > 0; exp *= 10) {
```

```
 for (int i = 0; i < 10; ++i)
 if (bucketHead[i] != NULL)
 if (newHead == NULL) {
 newHead = bucketHead[i];
 newTail = bucketTail[i];
 } else {
 newTail->next = bucketHead[i];
 newTail = bucketTail[i];
 }
}
```

```
}
```

```
*head = newHead;
```



```

void radixSort(RadixNode** head) {
 if (head == NULL || *head == NULL)
 return;
 int max = getMaxValue(*head), exp;
 for (exp = 1; max / exp > 0; exp *= 10) { // 从个位到最高位
 /* 定义 10 个桶的头尾指针 */
 RadixNode* bucketHead[10] = {}, bucketTail[10] = {};
 RadixNode* current = *head;
 /* 分配阶段：遍历原链表，根据当前位数字放入对应的桶 */
 while (current != NULL) {
 int digit = (current->data / exp) % 10;
 RadixNode* nextNode = current->next; /* 下一个节点 */
 current->next = NULL; /* 断开当前节点 */
 /* 若桶为空，则头尾均指向该节点；否则追加到尾部 */
 if (bucketTail[digit] == NULL) {
 bucketHead[digit] = current;
 bucketTail[digit] = current;
 } else {
 bucketTail[digit]->next = current;
 bucketTail[digit] = current;
 }
 current = nextNode;
 }
 }
}

```

```

/* 收集阶段：按 0~9 的顺序将所有桶重新链接为一个链表 */
RadixNode* newHead = NULL;
RadixNode* newTail = NULL;
for (int i = 0; i < 10; ++i) {
 if (bucketHead[i] != NULL) {
 if (newHead == NULL) {
 newHead = bucketHead[i];
 newTail = bucketTail[i];
 } else {
 newTail->next = bucketHead[i];
 newTail = bucketTail[i];
 }
 }
}
head = newHead; / 更新头指针，进入下一次位处理 */
}
}

```

# 链式基数排序

- 基数排序算法的基本思路：将单关键字看成是由多个数位 (或多个字符)构成的多关键字，并采用 多趟 “分配-收集” 的方法进行排序。
- 算法时间复杂度为  $O(k(n+r))$ 。

9

谢谢观看

理论课程



廈門大學  
XIAMEN UNIVERSITY



信息学院 黃 煒  
(特色化示范性软件学院) 博士, 副教授  
School of Informatics Wei Huang